UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

SISTEMA DE INFORMACIÓN PARA LA GESTIÓN INTEGRAL DEL TALLER DE CERÁMICA MITSÜ

Informe final · Ciclos I, II y III (PUDS)

Materia: Ingeniería en Requisitos y Modelado (INF342)

Docente: M.Sc. Ing. Angélica Garzón Cuéllar

Grupo: Grupo 4

Integrantes:

- Baqueros Hurtado Marco Antonio Junior
- Estaca Paco Sergio
- Miranda Soria Mauricio
- Santeyana Durán Sarah Nicol

Santa Cruz de la Sierra – Bolivia
Octubre de 2026

Índice

1. Perfil....5
1.1 Introducción....5
1.2 Antecedente....5
1.3 Justificación....6
1.4 Descripción del problema....7
1.4.1 Desafíos identificados....7
1.5 Formulación del problema....8
1.6 Objetivos....8
1.6.1 Objetivo general....8
1.6.2 Objetivos específicos....8
1.7 Alcance....9
1.8 Entrevista....9
2. Elementos del sistema en computadoras....11
2.1 Hardware....11
2.1.1 Servidor....11
2.1.2 Cliente....11
2.1.3 Medios de comunicación....11
2.1.4 Otros dispositivos....12
2.2 Software....12
2.2.1 Servidor....12
2.2.2 Cliente....12
2.2.3 Otro software adicional....13
2.3 Datos....13
2.4 Procesos....13
2.5 Gente / usuarios....14
2.6 Documento....14
3. Tecnología para el desarrollo del software....15
3.1 Estrategia para el desarrollo del software....15
3.2 Metodología para el desarrollo del software....15
3.2.1 Características del PUDS....15
3.2.2 Características del UML....15
3.3 Herramientas de desarrollo....16
3.3.1 Software....16
3.3.2 Hardware....16
4. Posibles costos....17
4.1 Costos de desarrollo....17
4.2 Costos de software....17
4.3 Costos de infraestructura....17
5. Posibles beneficios....18
5.1 Tiempo....18
5.2 Esfuerzo....18
5.3 Costos....18
6. Posibles clientes....19
7. Modelo de dominio....20
8. Modelo de negocio....21

9. Marco teórico....22
Capítulo 1. Método de Ishikawa....24
1. Identificar problemas....24
1.1 Lista de problemas....24
1.2 Depurar problemas....25
1.3 Lista final de problemas....25
1.4 Lista de propietarios de problemas....26
1.5 Análisis de problemas....26
1.6 Estimación y cuantificación....28
1.7 Alternativas de cambio....28
1.8 Conclusión y recomendación....28
1.9 Diseñar el diagrama de Ishikawa....29
2. Identificar las principales categorías....29
3. Identificar las causas....31
4. Analizar y discutir el diagrama....33
5. Modelo de negocio....36
5.1 Diagrama de actividad....36
5.1.1 Reserva de clases y registro de asistencia....38
5.1.2 Consumo de materiales y compra a proveedores....39
5.1.3 Venta en caja y cierre de caja....40
5.1.4 Compra en la tienda en línea y pago....41
Capítulo 2. Flujo de trabajo: captura de requisitos....42
2.1 Identificar actores y casos de uso....42
2.2 Priorizar los casos de uso....44
2.3 Detallar un caso de uso....46
2.4 Prototipar interfaz de usuario....69
2.5 Estructurar modelo de casos de uso....91
Capítulo 3. Flujo de trabajo: análisis....95
3.1 Análisis de la arquitectura....95
3.1.1 Identificar paquetes....95
3.1.2 Relacionar paquetes y casos de uso....97
3.1.3 Vista de paquetes....99
3.2 Analizar un caso de uso....103
3.3 Análisis de clase....124
3.4 Análisis de paquetes....147
Capítulo 4. Flujo de trabajo: diseño....148
4.1 Diseño de arquitectura....148
4.1.1 Diseño de datos lógico (diagrama de despliegue)....148
4.1.2 Diseño de datos físico (diagrama organizado en capas)....148
Capítulo 5. Diseño de datos....150
5.1 Diseño de datos lógico....150
Diagrama de clase....150
Mapeo....153
Normalización....155
5.2 Diseño de datos físico....156
Tabla de volumen....168

Script....169
Diagrama relacional....170
Actualización de tuplas (población de datos)....173
Consultas (30 SQL)....174
Procedimientos almacenados....183
Triggers (disparadores)....193
5.3 Diseñar casos de uso (diagrama de secuencia)....195
Capítulo 6. Flujo de trabajo: implementación....218
6.1 Elección de la plataforma de desarrollo del software....218
Lenguaje de programación....218
Base de datos....218
Sistemas operativos....218
Otros....218
6.2 Implementación de la arquitectura del sistema....219
Diagrama de componentes principal del sistema....219
6.3 Implementación de la arquitectura del subsistema....220
Diagrama de componentes de cada paquete....220
7. Conclusiones....225
8. Recomendaciones....226
9. Bibliografía....227
Anexos....229
Anexo A. Entrevista a la gerente....229
Anexo B. Cuentas de prueba....230
Anexo C. Pruebas del sistema....231
Anexo D. Manual de uso....231
Anexo E. Script completo de la base de datos....232
Anexo F. Enlaces del proyecto....265

1. Perfil

1.1 Introducción

La cerámica artesanal es una forma de expresión cultural que une técnicas tradicionales con diseño contemporáneo. Detrás de cada pieza hay un proceso que involucra materiales, horarios de clases, maestras, inventario y atención a clientes. Cuando todo eso se lleva en cuadernos y hojas de cálculo separadas, el trabajo administrativo crece y la información se pierde.

El Taller de Cerámica Mitsü, de Santa Cruz de la Sierra, crea piezas artesanales (vajillas, tazas, platos y piezas artísticas) y enseña modelado a mano y torno a niños, jóvenes y adultos. Hoy organiza sus horarios en Excel y sus finanzas en un software pequeño que no se comunica con lo demás (entrevista a la gerente, Anexo A).

Este informe documenta el desarrollo del **Sistema de Información para la Gestión Integral del Taller de Cerámica Mitsü**, una aplicación web construida con el Proceso Unificado de Desarrollo de Software (PUDS) en cuatro ciclos y modelada con UML. Este documento cubre los **ciclos I, II y III**, que ya están implementados y publicados en internet: seguridad y configuración, clases y reservas, inventario, ventas en caja y tienda en línea con pasarela de pago. El ciclo IV (cafetería, cobro de mensualidades y reportes) queda planificado.

1.2 Antecedente

El taller nació cuando su fundadora volvió de estudiar Artes Plásticas en Argentina y, junto con una amiga, decidió crear un espacio para producir y enseñar cerámica, porque conseguir arcilla y hornear localmente era difícil (Anexo A). Su estética fusiona el diseño contemporáneo con la esencia de los pueblos originarios de Bolivia, de la región andina y de la Amazonía.

Según la entrevista, el taller tiene **4 maestras** y personal administrativo; la cafetería tiene personal propio. La organización es la siguiente: gerencia general; área de producción y enseñanza (maestras ceramistas); área de administración y ventas (atención al cliente, caja y finanzas); y área de cafetería. La materia prima (arcilla, cuarzo y otros minerales) se compra a empresas mineras de Cochabamba y La Paz.

Análisis FODA

Tabla 1
Análisis FODA del Taller de Cerámica Mitsü

Fortalezas	Debilidades
Maestras con formación en artes plásticas y experiencia en el oficio.	Horarios y reservas organizados a mano en hojas de Excel.
Propuesta artística propia: diseño contemporáneo con raíces originarias bolivianas.	Sin control sistematizado del inventario de arcilla, esmaltes y minerales.
Proveedores fijos de arcilla y minerales (Cochabamba y La Paz).	Ventas y caja sin registro centralizado ni arqueo.
Servicios diversificados: clases, venta de piezas y cafetería.	Herramientas separadas (Excel y software de finanzas) que no se comunican.

Tabla 2
Análisis FODA (continuación)

Oportunidades	Amenazas
Interés creciente por la cerámica como actividad recreativa y terapéutica.	Situación económica que impide contratar más personal.
Ampliar talleres y horarios (las clases son lo más solicitado).	Competencia de otros talleres de cerámica de la ciudad.
Vender piezas en línea y cobrar con QR, tarjeta o PayPal.	Dependencia de dos proveedores externos de materia prima.
Cafetería como espacio de encuentro para alumnos y visitantes.	Variación del precio de materiales y minerales.

Nota. Basado en la entrevista a la gerente (Anexo A) y en el informe de perfil anterior del grupo.

1.3 Justificación

En la entrevista se identificó que la organización de horarios, el control de materiales y el registro de ventas se hacen a mano o con herramientas que no se integran. Esto provoca pérdida

de tiempo al responder reservas una por una, compras urgentes por falta de material y la imposibilidad de saber cuánto se vendió o cuánto efectivo debería haber en caja.

Un sistema web integrado permite que el alumno vea los cupos y reserve solo; que la maestra marque la asistencia desde el celular; que el stock de materiales y piezas se actualice automáticamente con cada compra, consumo o venta; que la caja se cierre con arqueo; y que cualquier persona compre piezas en línea. Además, como proyecto académico, permite aplicar el PUDS y UML a un caso real, desde la captura de requisitos hasta un sistema publicado.

1.4 Descripción del problema

El taller gestiona sus operaciones con medios manuales y herramientas aisladas. Los horarios están en Excel, por lo que no se ve la disponibilidad en tiempo real y se producen dobles reservas y confusiones en la asignación de maestras. No existe un registro del inventario de materiales, lo que provoca faltantes en momentos críticos y compras urgentes. Las ventas de piezas no tienen un registro centralizado ni comprobante numerado, y no hay cierre de caja que compare el efectivo esperado con el contado. Tampoco existe un canal para que los clientes vean y compren piezas en línea.

1.4.1 Desafíos identificados

- Evitar que dos alumnos ocupen el último lugar de una clase al mismo tiempo (control de cupo concurrente).
- Impedir que una maestra tenga dos horarios que se crucen.
- Que el stock de materiales y piezas no pueda modificarse “a mano”, solo con movimientos registrados (kardex).
- Conservar el precio con que se vendió cada pieza aunque después cambie (historial de precios).

- Cobrar en línea sin guardar datos sensibles de tarjetas y reservar el stock mientras el cliente paga.
- Controlar el acceso por roles y dejar registro (bitácora) de las acciones importantes.

1.5 Formulación del problema

¿Cómo mejorar la gestión de las clases, el inventario y las ventas del Taller de Cerámica Mitsü mediante un sistema de información web que centralice los datos, controle las reglas del negocio y permita a alumnos y clientes reservar y comprar en línea?

1.6 Objetivos

1.6.1 Objetivo general

Desarrollar un sistema de información web para la gestión integral del Taller de Cerámica Mitsü que administre la seguridad, las clases y reservas, el inventario, las ventas y la tienda en línea, aplicando el PUDS y UML, y publicarlo en internet.

1.6.2 Objetivos específicos

1. Capturar y priorizar los requisitos del taller mediante entrevista, diagrama de Ishikawa y modelo de casos de uso.
2. Analizar y diseñar la arquitectura del sistema en paquetes, capas y componentes con diagramas UML.
3. Diseñar una base de datos relacional normalizada en PostgreSQL con reglas de negocio en triggers y procedimientos almacenados.
4. Implementar el sistema en PHP 8.3 con el patrón MVC y controles de seguridad (roles, permisos, CSRF, contraseñas cifradas, bitácora).
5. Publicar el sistema en Vercel con la base de datos en Supabase y probarlo con scripts automáticos.

1.7 Alcance

El sistema se construye en cuatro ciclos del PUDS. Los tres primeros están terminados y publicados; el cuarto queda planificado.

Tabla 3
Alcance por ciclo

Ciclo	Paquetes	Casos de uso	Estado
I	P1 Seguridad y configuración	CU01 Iniciar y cerrar sesión · CU02 Recuperar contraseña · CU03 Usuarios · CU04 Roles y permisos · CU05 Bitácora · CU06 Configuración y catálogos	Terminado
II	P2 Clases y reservas	CU07 Maestras · CU08 Talleres · CU09 Horarios y calendario · CU10 Reservar clase · CU11 Asistencia · CU12 Alumnos y portal	Terminado
III	P3 Inventario y compras · P4 Ventas, caja y tienda	CU13 Materiales · CU14 Proveedores · CU15 Compras · CU16 Consumo, merma y alertas · CU17 Piezas · CU18 Venta en caja · CU19 Caja y arqueo · CU20 Tienda en línea · CU21 Pago en línea · CU22 Pedidos web	Terminado
IV	P5 Cafetería · P6 Cobros · P7 Reportes	CU23 Menú · CU24 Comandas · CU25 Mensualidades y morosidad · CU26 Pedidos personalizados · CU27 Notificaciones · CU28 Reportes	Planificado

Nota. El alcance incluye el despliegue en internet (Vercel y Supabase). La pasarela de pago funciona en modo de demostración: no realiza cobros reales.

No incluye en esta entrega: facturación electrónica ante Impuestos Nacionales, cobros reales con un banco o con PayPal (requieren contrato comercial), ni aplicaciones móviles nativas; el sistema es web y se adapta al celular.

1.8 Entrevista

Se realizó una entrevista a la gerente del taller, Alessandra Civardi Soria, en las instalaciones del taller (duración: 22 minutos), conducida por Mauricio Miranda Soria. Sus respuestas principales se resumen en la Tabla 4 y la transcripción completa está en el Anexo A.

Tabla 4
Resumen de la entrevista a la gerente

Tema	Respuesta resumida
Productos	Vajillas, tazas y platos, con enfoque utilitario o artístico según el pedido.

Tema	Respuesta resumida
Maestras y horarios	4 maestras. Martes a viernes 10:00–12:00; martes y miércoles 18:00–20:00; sábado adultos 10:00–12:00, niños 16:00–18:00 y modelado y torno 18:00–20:00.
Lo más solicitado	Las clases, porque las piezas terminadas son más costosas.
Personal	No se ampliará por ahora por la situación económica.
Materiales	Arcilla, cuarzo y minerales comprados a empresas mineras de Cochabamba y La Paz.
Herramientas actuales	Excel para horarios y un pequeño software para finanzas.
Otros servicios	Cafetería con personal propio e ingredientes naturales (ciclo IV).

Nota. Basado en la entrevista del Anexo A.

2. Elementos del sistema en computadoras

2.1 Hardware

2.1.1 Servidor

El sistema no necesita un servidor propio en el taller: se ejecuta en la nube. La aplicación PHP corre en Vercel como función sin servidor (región gru1, São Paulo) y la base de datos PostgreSQL está en Supabase (región sa-east-1). Ambos servicios escalan solos y se pagan por uso; en su plan gratuito alcanzan para el volumen del taller.

2.1.2 Cliente

Cualquier computadora, tableta o celular con un navegador actualizado. Para la recepción y el punto de venta se recomienda la siguiente configuración:

Tabla 5
Configuración recomendada del equipo cliente

Nombre	Versión	Características
Procesador	Intel Core i3 o AMD Ryzen 3 (o superior)	2 núcleos o más
Memoria RAM	DDR4	4 GB o más
Disco	SSD	128 GB o más
Monitor	13 pulgadas o más	Resolución 1366 × 768 o mayor
Celular (maestras y alumnos)	Android 9 o iOS 14 en adelante	Navegador actualizado

Nota. Configuración mínima estimada por el equipo; el sistema se probó también en celular.

2.1.3 Medios de comunicación

Conexión a internet de banda ancha (se recomienda 10 Mbps o más) por Wi-Fi o cable. Todo el tráfico viaja cifrado por HTTPS; la conexión de la aplicación a la base de datos usa TLS.

2.1.4 Otros dispositivos

Tabla 6
Otros dispositivos

Nombre	Características
Impresora térmica o común	Imprime la nota de venta y los comprobantes.
Celular de la maestra	Registra la asistencia desde la clase.
Celular del cliente	Escanea el QR del taller para pagar los pedidos en línea.
Cámara o celular	Toma las fotos de las piezas para la tienda.

2.2 Software

2.2.1 Servidor

Tabla 7
Software del lado del servidor

Componente	Elección	Uso
Lenguaje	PHP 8.3 (sin framework, MVC propio)	Lógica de la aplicación
Plataforma	Vercel con vercel-php 0.7.4	Publicación y ejecución en la nube
Base de datos	PostgreSQL 17 en Supabase, esquema propio “mitsu”	Datos, triggers y procedimientos
Conexión	PDO con pdo_pgsql por el pooler Supavisor (puerto 6543)	Acceso seguro y con parámetros
Correo	SMTP de Gmail (se configura con contraseña de aplicación)	Comprobantes y recuperación de contraseña

2.2.2 Cliente

Tabla 8
Software del lado del cliente

Nombre	Versión
Sistema operativo	Windows 10/11, macOS, Android o iOS
Navegador	Chrome, Edge, Firefox o Safari (versión actual)
Interfaz	HTML5, CSS3 y JavaScript propio, modo claro y oscuro, adaptable al celular

2.2.3 Otro software adicional

Tabla 9
Otro software adicional

Nombre	Descripción
Git y GitHub	Control de versiones y repositorio público.
Visual Studio Code	Editor de código.
Generador UML propio (Python)	Diagramas UML en notación de Enterprise Architect, generados desde la especificación de los casos de uso.
LibreOffice y librería docx	Armado del informe y exportación a PDF.
Puppeteer	Capturas de pantalla y pruebas en celular.

2.3 Datos

La información se guarda en 33 tablas relacionales: usuarios, roles y permisos; maestras, alumnos, talleres, horarios, clases y reservas; proveedores, materiales, compras y movimientos (kardex); piezas, clientes, cajas, ventas, pedidos en línea y pagos. Las contraseñas se guardan cifradas con bcrypt y de las tarjetas solo se guarda la marca y los cuatro últimos dígitos. El volumen es bajo (miles de registros por año; ver la tabla de volumen en el capítulo 5).

2.4 Procesos

- **Seguridad:** ingreso con correo y contraseña, recuperación por código, usuarios, roles, permisos y bitácora.
- **Clases:** talleres, horarios, generación del calendario, reservas con control de cupo y asistencia.
- **Inventario:** materiales, proveedores, compras, consumo, merma, ajustes y alertas de stock mínimo.
- **Ventas:** piezas con foto, venta en caja con nota de venta, anulación con motivo, apertura y cierre de caja.
- **Tienda en línea:** catálogo público, carrito, pedido con reserva de stock, pago con QR, tarjeta o PayPal y gestión de pedidos.

2.5 Gente / usuarios

Tabla 10
Usuarios del sistema

Rol	Quién es	Qué hace en el sistema
Administrador	Gerente del taller	Todo: usuarios, roles, bitácora, configuración, talleres, maestras, inventario, ventas y anulaciones.
Administrativo	Personal de recepción y ventas	Reservas, alumnos, asistencia, materiales, compras, piezas, ventas, caja y pedidos web.
Maestra	Las 4 maestras ceramistas	Ve sus clases y registra la asistencia de sus alumnos.
Alumno	Persona inscrita en talleres	Portal: reserva y cancela clases, ve su historial y compra en la tienda.
Cliente	Comprador en línea	Carrito, pago con QR, tarjeta o PayPal, sus pedidos y comprobantes.
Visitante	Cualquier persona	Ve la tienda y las clases sin iniciar sesión.

2.6 Documento

Documentos que produce el sistema: nota de venta numerada (NV-000000), comprobante de compra (CP-000000), comprobante de pedido en línea (PW-000000) con el método de pago, resumen del arqueo de caja y kardex de cada material y pieza. Documentación del proyecto: este informe, el README del repositorio, el guion de presentación y el manual de uso (Anexo D).

3. Tecnología para el desarrollo del software

3.1 Estrategia para el desarrollo del software

Se eligió una estrategia **iterativa e incremental**: el sistema se divide en cuatro ciclos y cada ciclo entrega casos de uso terminados, probados y publicados. Así el taller puede usar la seguridad y las reservas mientras se construye el inventario y las ventas, y los riesgos más altos (control de cupo, stock y pagos) se resuelven temprano (Jacobson et al., 2000).

3.2 Metodología para el desarrollo del software

Se usa el **Proceso Unificado de Desarrollo de Software (PUDS)** con la notación **UML**. En cada ciclo se recorren los flujos de trabajo de requisitos, análisis, diseño, implementación y prueba.

3.2.1 Características del PUDS

- **Dirigido por casos de uso:** los casos de uso guían el análisis, el diseño y las pruebas.
- **Centrado en la arquitectura:** se define temprano la arquitectura en paquetes y capas.
- **Iterativo e incremental:** cada ciclo agrega funcionalidad sobre la anterior.
- **Fases:** inicio, elaboración, construcción y transición, con hitos en cada una.

3.2.2 Características del UML

UML es un lenguaje gráfico estándar para especificar, visualizar y documentar sistemas (Booch et al., 2006). En este proyecto se usan diagramas de casos de uso, actividad, comunicación, clases, paquetes, secuencia, despliegue y componentes, con los estereotipos de análisis «frm» (interfaz), «ctrl» (control) y «entidad».

3.3 Herramientas de desarrollo

3.3.1 Software

Tabla 11
Herramientas de software

Herramienta	Uso	Costo
Visual Studio Code	Editor de código	Gratuito
PHP 8.3 y PostgreSQL 17	Lenguaje y base de datos	Gratuito (código abierto)
Git y GitHub	Control de versiones y repositorio público	Gratuito
Vercel y Supabase (planes gratuitos)	Publicación y base en la nube	Gratuito
Python, Node.js y Puppeteer	Diagramas, informe y capturas de pantalla	Gratuito
LibreOffice	Índice y exportación a PDF	Gratuito

3.3.2 Hardware

Cuatro computadoras portátiles de los integrantes (procesador de 4 núcleos, 8 GB de RAM o más) y conexión a internet. No se compró hardware.

4. Posibles costos

Los montos de esta sección son **estimados** por el equipo; no provienen de cotizaciones formales. Se usa el tipo de cambio oficial de Bs 6,96 por dólar.

4.1 Costos de desarrollo

Tabla 12
Costo estimado de desarrollo

Concepto	Cálculo	Monto (Bs)
Análisis y diseño (requisitos, UML, base de datos)	5 personas × 50 h × Bs 25/h	6.250
Programación y pruebas (ciclos I a III)	5 personas × 90 h × Bs 25/h	11.250
Documentación e informe	5 personas × 20 h × Bs 25/h	2.500
Total estimado		20.000

Nota. Estimado. Bs 25 por hora corresponde a una referencia de pago a practicantes; las horas son aproximadas.

4.2 Costos de software

Todo el software usado es libre o gratuito (ver la tabla de herramientas de software en 3.3.1), por lo que el costo de licencias es **Bs 0**.

4.3 Costos de infraestructura

Tabla 13
Costo estimado de infraestructura

Concepto	Plan	Costo mensual (Bs)
Publicación de la aplicación	Vercel Hobby (actual)	0
Base de datos	Supabase Free (actual)	0
Base de datos recomendada en producción (sin pausa por inactividad, respaldos diarios)	Supabase Pro, USD 25/mes	174
Dominio propio .com (opcional)	USD 12/año	7
Correo SMTP	Gmail con contraseña de aplicación	0

Nota. Estimado según los precios públicos de los proveedores consultados por el equipo; deben verificarse al contratar.

5. Posibles beneficios

Los valores de esta sección son **estimaciones** del equipo a partir de la entrevista y de las pruebas del sistema; se deben medir con el uso real.

5.1 Tiempo

- Reservar una clase pasa de una conversación por WhatsApp (estimado: 5 a 10 minutos) a menos de 1 minuto desde el portal del alumno.
- El cierre de caja con arqueo se calcula solo; antes había que sumar a mano las ventas del día.
- El stock de cada material se consulta en segundos en lugar de revisar los estantes.

5.2 Esfuerzo

- La recepción ya no responde cupos uno por uno: el calendario muestra los lugares libres.
- La maestra registra la asistencia con un toque por alumno desde el celular.
- Las compras actualizan stock y costo promedio de forma automática.

5.3 Costos

- Menos compras urgentes de material gracias a las alertas de stock mínimo (estimado: se evitan 1 o 2 envíos urgentes al mes desde Cochabamba o La Paz).
- Menos diferencias de caja no explicadas, porque cada venta queda registrada y la anulación exige motivo.
- Ventas adicionales por la tienda en línea, que funciona a toda hora.

6. Posibles clientes

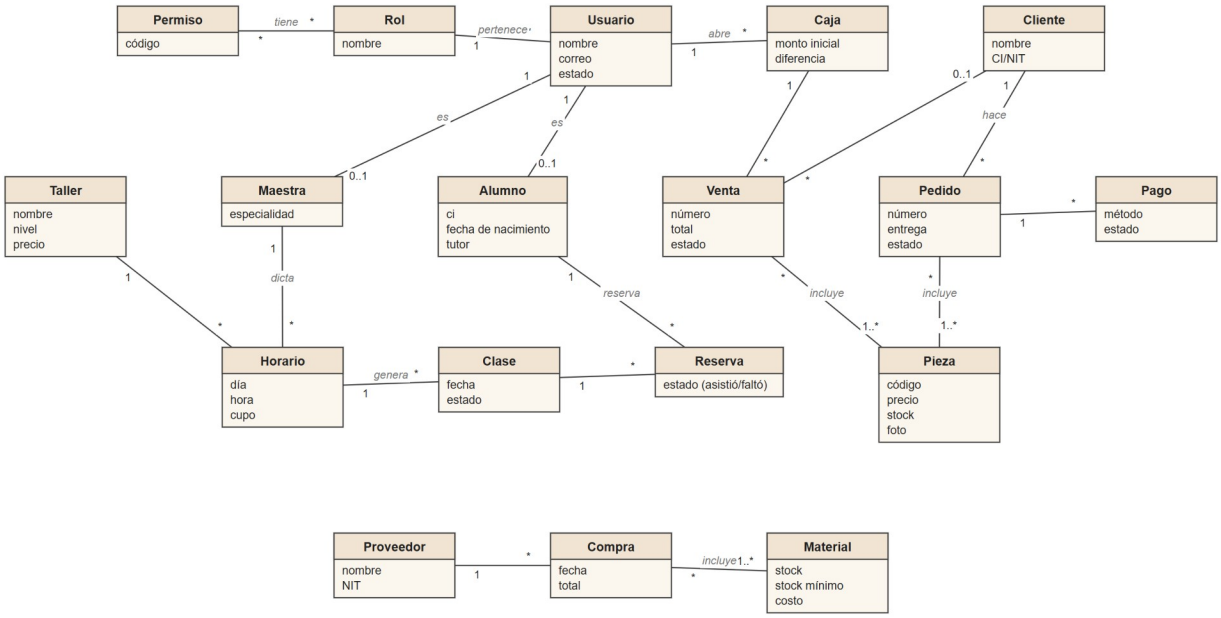
El cliente principal es el Taller de Cerámica Mitsü. Por estar organizado en paquetes y roles configurables, el sistema puede adaptarse a otros negocios que **enseñan y venden**: talleres de cerámica, pintura o carpintería; academias de arte, música o idiomas con venta de materiales; estudios de yoga o danza con tienda; y emprendimientos artesanales que necesitan inventario de insumos y venta en línea.

7. Modelo de dominio

El modelo de dominio muestra los conceptos del negocio y cómo se relacionan, sin detalles técnicos. Un usuario tiene un rol con permisos y puede ser maestra o alumno. Un taller tiene horarios dictados por una maestra; cada horario genera clases y los alumnos reservan lugares en ellas. Las compras a proveedores incluyen materiales. Las ventas en caja y los pedidos en línea incluyen piezas; el pedido se paga con uno o más intentos de pago.

Figura 1

Modelo de dominio del Taller Mitsü



Nota. Elaboración propia.

8. Modelo de negocio

El Taller de Cerámica Mitsü genera ingresos con tres líneas de negocio. La principal son las **clases** de modelado a mano y torno para niños y adultos, que dictan cuatro maestras de martes a sábado; según la gerente, es lo más solicitado porque las piezas terminadas cuestan más por el trabajo artesanal. La segunda es la **venta de piezas** de cerámica hechas a mano (vajillas, tazas y platos), de uso utilitario o artístico según el pedido del cliente. La tercera es la **cafetería** para alumnos y visitantes, atendida por personal propio y con ingredientes mayormente de cultivo propio.

Para producir, el taller compra arcilla y minerales como el cuarzo a empresas mineras de Cochabamba y La Paz, que son sus únicos proveedores. Sus clientes son los alumnos que se inscriben a los talleres, las personas que compran piezas y quienes visitan la cafetería. La propuesta de valor es una cerámica artesanal con materiales del país, que fusiona el diseño contemporáneo con la estética de los pueblos originarios de Bolivia.

Hoy la gestión es manual: los horarios se llevan en hojas de Excel y las finanzas en un pequeño software aparte, sin conexión entre sí. El flujo de trabajo actual de reservas, clases, ventas y cierre se modela con el diagrama de actividad del capítulo 1 (punto 5.1), donde se analiza junto con el diagrama de Ishikawa.

9. Marco teórico

Sistema de información

Conjunto de personas, datos, procesos y tecnología que recoge, guarda, procesa y distribuye información para apoyar las operaciones y la toma de decisiones de una organización (Laudon y Laudon, 2020).

Proceso Unificado (PUDS)

Proceso de desarrollo dirigido por casos de uso, centrado en la arquitectura, iterativo e incremental, organizado en fases e iteraciones (Jacobson et al., 2000).

UML

Lenguaje de modelado estándar del Object Management Group para especificar y documentar sistemas con diagramas de estructura y de comportamiento (Booch et al., 2006).

Patrón Modelo-Vista-Controlador (MVC)

Separa los datos y reglas (modelo), la presentación (vista) y la coordinación de las peticiones (controlador), lo que facilita mantener y probar el sistema (Larman, 2004).

Base de datos relacional y normalización

Los datos se organizan en tablas relacionadas por llaves; la normalización (1FN, 2FN, 3FN) elimina redundancias y anomalías de actualización (Elmasri y Navathe, 2016).

Triggers y procedimientos almacenados

Código que se ejecuta dentro del gestor de base de datos; los triggers reaccionan a inserciones o cambios y garantizan reglas del negocio aunque falle la aplicación (PostgreSQL Global Development Group, 2024).

Seguridad en aplicaciones web

Las amenazas más comunes (inyección SQL, XSS, CSRF, control de acceso roto) se mitigan con consultas parametrizadas, escape de salida, tokens CSRF, contraseñas con hash y permisos verificados en el servidor (OWASP Foundation, 2021).

Pasarela de pago

Servicio que autoriza cobros con tarjeta u otros medios. El número de tarjeta se valida con el algoritmo de Luhn (Luhn, 1960) y las normas PCI DSS prohíben guardar el número completo o el CVV (PCI Security Standards Council, 2022).

Computación en la nube

Modelo en que la infraestructura se usa como servicio: plataforma como servicio para la aplicación (Vercel) y base de datos como servicio (Supabase), sin administrar servidores propios (Mell y Grance, 2011).

Diagrama de Ishikawa

Diagrama causa-efecto que organiza las causas de un problema en categorías para analizarlas (Ishikawa, 1985).

Capítulo 1. Método de Ishikawa

1. Identificar problemas

1.1 Lista de problemas

Tabla 14
Lista inicial de problemas

N.º	Problema	Proceso
1	Gestión de horarios de clases realizada manualmente con hojas de Excel.	Comercial
2	No existe un medio para que los alumnos consulten la disponibilidad en tiempo real.	Comercial
3	Confusión en la asignación de maestras por falta de un registro centralizado.	Comercial
4	No existe un registro sistematizado del inventario de materiales.	Inventario
5	Falta de materiales en momentos críticos.	Inventario
6	Compras urgentes o innecesarias sin planificación.	Compras
7	Pérdida de tiempo en la verificación manual de existencias.	Inventario
8	Las ventas de piezas no tienen registro centralizado.	Ventas
9	Las ventas de la cafetería no se integran con el resto del taller.	Ventas
10	Dificultad para generar reportes de ventas.	Ventas
11	Falta de indicadores para decidir.	Gerencia
12	Descoordinación entre producción, administración y cafetería.	Comercial
13	Retrasos en la atención a clientes.	Comercial
14	Dependencia de herramientas externas no integradas.	Todos
15	Dificultad para organizar la demanda de clases.	Comercial
16	No existe historial de asistencia de los alumnos.	Comercial
17	No hay control de pagos y morosidad de los alumnos.	Ventas
18	La información de clientes y alumnos está dispersa.	Comercial
19	No hay registro de pedidos personalizados.	Ventas
20	Duplicidad de información y errores de registro.	Todos
21	Riesgo de pérdida de datos sin respaldos.	Todos
22	No existe un canal para mostrar y vender las piezas en línea.	Ventas

Nota. Tomado del informe de perfil anterior del grupo, construido a partir de la entrevista (Anexo A).

1.2 Depurar problemas

Se agruparon los problemas que eran consecuencia de otro (por ejemplo, la falta de materiales es consecuencia de no tener inventario) y se separaron los que corresponden al ciclo IV (cafetería, mensualidades, pedidos personalizados y reportes).

Tabla 15
Depuración de problemas

Problemas	Acción	Justificación
1, 2, 3, 15	Unir	Todos nacen de llevar horarios y reservas en Excel.
4, 5, 6, 7	Unir	Son consecuencias de no tener inventario ni kardex.
8, 10, 11	Unir	Sin registro de ventas no hay reportes ni indicadores.
12, 13, 14, 18, 20	Unir	Información dispersa en herramientas que no se comunican.
16	Mantener	Problema propio del proceso de clases.
21	Mantener	Riesgo técnico que el sistema resuelve con la nube y respaldos.
22	Mantener	Problema propio de la venta de piezas: los clientes solo pueden comprar en el local.
9, 17, 19	Postergar al ciclo IV	Cafetería, cobro de mensualidades y pedidos personalizados.

1.3 Lista final de problemas

- P1. Horarios y reservas manuales: sin cupos en tiempo real y con dobles reservas.
- P2. Sin historial de asistencia de los alumnos.
- P3. Inventario de materiales sin registro: faltantes y compras urgentes.
- P4. Ventas de piezas sin registro, sin nota numerada y sin cierre de caja.
- P5. Información dispersa en herramientas no integradas y sin control de acceso.
- P6. Clientes que no pueden ver ni comprar piezas en línea.
- P7. Riesgo de pérdida de datos sin respaldos.

1.4 Lista de propietarios de problemas

Tabla 16
Matriz problema – propietario

Problema	Gerencia	Recepción	Maestras	Alumnos / clientes
P1 Horarios y reservas	X	X	X	X
P2 Asistencia	X		X	
P3 Inventario	X	X	X	
P4 Ventas y caja	X	X		X
P5 Información dispersa	X	X		
P6 Venta en línea	X			X
P7 Respaldos	X			

Nota. X indica que el problema afecta a ese propietario.

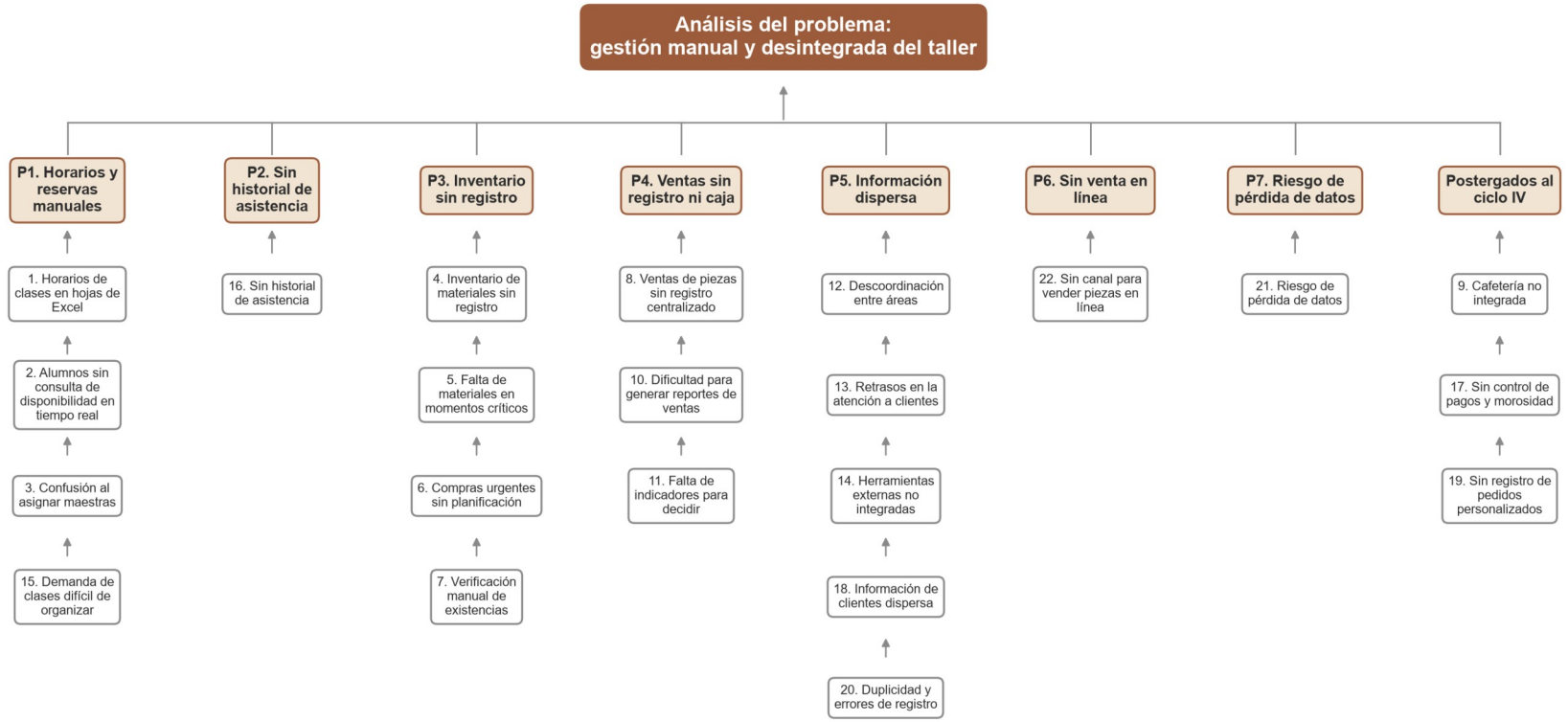
1.5 Análisis de problemas

Tabla 17
Análisis causa – efecto de cada problema

Problema	Causa principal	Efecto
P1	Excel sin acceso para alumnos ni control de cupo.	Dobles reservas, tiempo de recepción, maestras mal asignadas.
P2	Asistencia anotada en papel.	No se sabe qué alumnos faltan seguido.
P3	No hay kardex ni stock mínimo.	Faltantes en plena producción y compras urgentes más caras.
P4	Ventas anotadas a mano.	No se puede comparar el efectivo con lo vendido.
P5	Herramientas separadas.	Datos repetidos y errores.
P6	No existe tienda en línea.	Se pierden ventas fuera del horario del local.
P7	Archivos en computadoras personales.	Riesgo de perder la información.

El árbol de problemas muestra cómo los veintidós problemas de la lista inicial se agrupan en los siete problemas de la lista final y todos llevan al problema central: la gestión manual y desintegrada del taller. Los problemas de la cafetería, las mensualidades y los pedidos personalizados quedan aparte, postergados al ciclo IV.

Figura 2
Árbol de problemas del Taller Mitsü



Fuente: elaboración propia a partir de la lista inicial de problemas (1.1) y su depuración (1.2).

Nota. Elaboración propia a partir de la lista inicial de problemas y su depuración.

1.6 Estimación y cuantificación

Tabla 18
Estimación del tiempo perdido por semana

Problema	Actividad	Tiempo semanal estimado
P1	Responder consultas y anotar reservas	4 a 6 horas
P2	Pasar la asistencia del papel	1 hora
P3	Revisar existencias y gestionar compras urgentes	2 horas
P4	Sumar ventas y cuadrar la caja	2 a 3 horas
P5	Buscar datos en distintos archivos	1 a 2 horas
Total		10 a 14 horas

Nota. Estimado por el equipo a partir de la entrevista; no fue medido en el taller.

1.7 Alternativas de cambio

Tabla 19
Comparación de alternativas

Criterio	A. Mejorar las hojas de Excel	B. Software comercial genérico	C. Sistema web a medida
Reservas con cupo en línea	No	Parcial	Sí
Inventario con kardex	Manual	Sí	Sí
Venta en caja y tienda en línea	No	Por separado	Integradas
Reglas del taller (talleres, maestras)	No	Difícil de adaptar	Sí
Costo inicial	Bajo	Licencias mensuales	Desarrollo académico
Acceso desde el celular	Limitado	Depende	Sí

1.8 Conclusión y recomendación

La alternativa C, un sistema web a medida, es la única que resuelve los siete problemas en una sola herramienta y se adapta a la forma de trabajo del taller. Se recomienda desarrollarla en ciclos, empezando por la seguridad y las reservas (P1, P2, P5), luego el inventario y las ventas (P3, P4, P6), y publicarla en la nube para eliminar el riesgo de pérdida de datos (P7).

1.9 Diseñar el diagrama de Ishikawa

El diagrama se arma sobre un eje horizontal que termina en el efecto, es decir, el problema central que se quiere explicar: la gestión manual y desintegrada de las clases, el inventario y las ventas del Taller Mitsü. Sobre este eje se agregan después las categorías y las causas.

Figura 3
Eje y efecto del diagrama de Ishikawa



Fuente: elaboración propia a partir de la entrevista a la gerente y de la lista final de problemas.

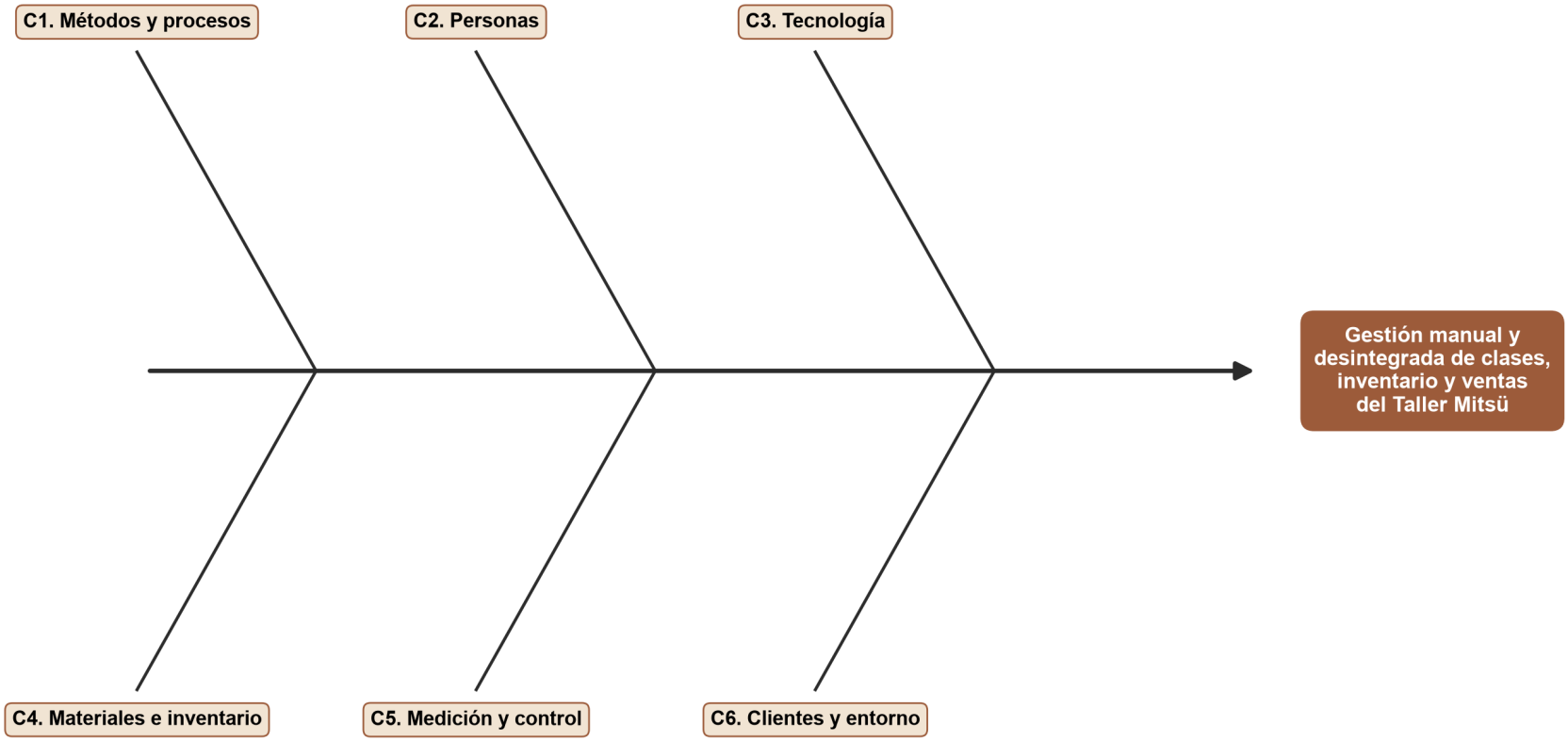
Nota. Elaboración propia.

2. Identificar las principales categorías

Para el análisis del problema se identificaron seis categorías principales, que representan las áreas críticas del funcionamiento del taller y forman las espinas del diagrama.

Figura 4

Categorías principales del diagrama de Ishikawa



Nota. Elaboración propia.

C1. Métodos y procesos: Agrupa la forma en que hoy se hacen las tareas del día a día: llevar los horarios y las reservas de las clases, anotar las ventas de piezas y cerrar la caja. Los procesos relacionados son la reserva de clases, la venta en caja y el arqueo diario.

C2. Personas: Se refiere al personal del taller y a cómo se reparten las tareas. El taller no puede contratar más personal, así que recepción y las maestras cargan con el trabajo manual. Los procesos relacionados son la asignación de maestras a los horarios y la atención de las reservas.

C3. Tecnología: Abarca las herramientas que usa el taller para registrar su información: hojas de Excel y un pequeño software de finanzas, sin conexión entre sí. Los procesos relacionados son el registro de datos, el control de acceso y el respaldo de la información.

C4. Materiales e inventario: Se centra en la arcilla, los esmaltes y los minerales que se usan para producir las piezas y dictar las clases. Los procesos relacionados son el control de existencias, las compras a proveedores y el registro del consumo.

C5. Medición y control: Reúne la información que la gerencia necesita para controlar el taller y decidir: historial de asistencia, ventas, precios y costos. Los procesos relacionados son el registro de asistencia, el cierre de caja y la consulta de historiales.

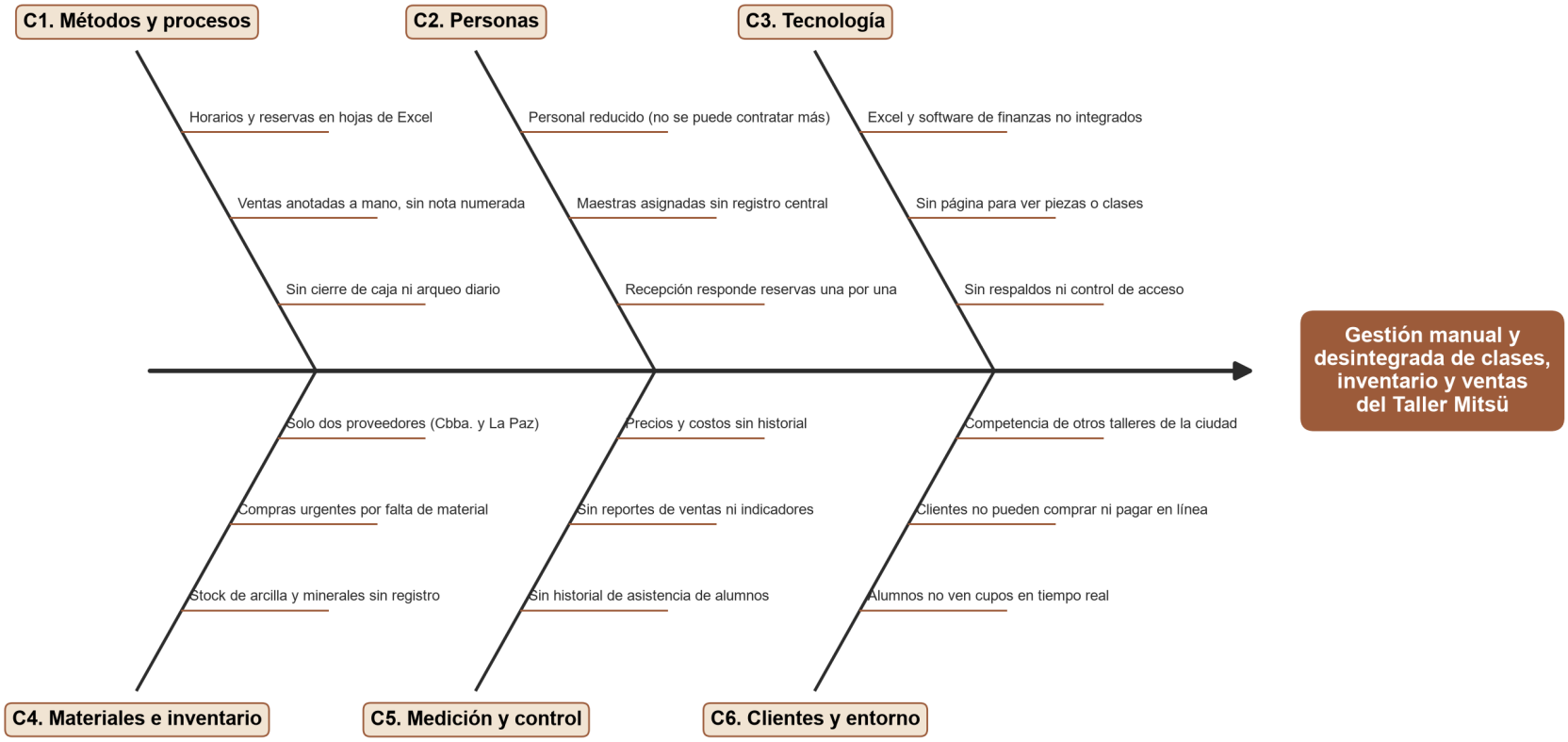
C6. Clientes y entorno: Trata la relación con los alumnos y los compradores, y la competencia de otros talleres. Los procesos relacionados son la consulta de clases disponibles, la venta de piezas y el pago.

3. Identificar las causas

A partir de las categorías definidas y de la lista final de problemas se identificaron tres causas por categoría, que se ubican como espinas secundarias del diagrama.

Figura 5

Diagrama de Ishikawa del Taller Mitsü



Fuente: elaboración propia a partir de la entrevista a la gerente y de la lista final de problemas.

Nota. Elaboración propia a partir de la entrevista (Anexo A) y de la lista final de problemas.

C1. Métodos y procesos
Los horarios y las reservas se llevan en hojas de Excel, las ventas se anotan a mano sin nota numerada y no hay cierre de caja ni arqueo diario.

C2. Personas
El personal es reducido y no se puede contratar más, las maestras se asignan sin un registro central y recepción responde las reservas una por una.

C3. Tecnología
El Excel y el software de finanzas no están integrados, no existe una página para ver las piezas o las clases y no hay respaldos ni control de acceso.

C4. Materiales e inventario
El stock de arcilla y minerales no tiene registro, se hacen compras urgentes por falta de material y el taller depende de solo dos proveedores, de Cochabamba y La Paz.

C5. Medición y control
No hay historial de asistencia de los alumnos, no existen reportes de ventas ni indicadores y los precios y costos no guardan historial.

C6. Clientes y entorno
Los alumnos no ven los cupos en tiempo real, los clientes no pueden comprar ni pagar en línea y existe competencia de otros talleres de la ciudad.

4. Analizar y discutir el diagrama

Las categorías con más peso son **métodos y procesos** y **tecnología**: casi todos los problemas nacen de anotar en Excel o en papel lo que debería registrarse una sola vez. Para cada categoría se resumen los problemas que reúne y la solución que propone el sistema, indicando los casos de uso que la implementan.

C1. Métodos y procesos
Problemas identificados:

- P1. Horarios y reservas manuales, sin cupos en tiempo real.
- P4. Ventas sin registro, sin nota numerada y sin cierre de caja.

Solución propuesta:
Registrar los horarios y generar el calendario de clases (CU09), reservar con control de cupo (CU10), vender en caja con nota de venta numerada (CU18) y abrir y cerrar la caja con arqueo (CU19). Cada dato se ingresa una sola vez y el resto se calcula.

C2. Personas
Problemas identificados:

- P1. Maestras asignadas sin registro central y reservas atendidas una por una.

Solución propuesta:
Registrar las maestras y asignarlas a los horarios sin cruces (CU07 y CU09) y dar a los alumnos un portal para reservar por su cuenta (CU12), de modo que el personal reducido no tenga que atender cada reserva.

C3. Tecnología
Problemas identificados:

- P5. Información dispersa en herramientas no integradas y sin control de acceso.
- P6. Sin página para ver o comprar las piezas.
- P7. Riesgo de pérdida de datos sin respaldos.

Solución propuesta:
Un solo sistema web en la nube con usuarios, roles y permisos (CU01 a CU04), bitácora de operaciones (CU05) y configuración centralizada (CU06), más la tienda en línea para ver y comprar las piezas (CU20). La base de datos en la nube elimina los archivos en computadoras personales.

C4. Materiales e inventario
Problemas identificados:

- P3. Inventario de materiales sin registro, faltantes y compras urgentes.

Solución propuesta:
Registrar los materiales con stock mínimo (CU13), los proveedores (CU14) y las compras (CU15), y el consumo y la merma en el kardex con alertas de stock bajo (CU16). La dependencia de dos proveedores es una condición del entorno que el sistema no cambia, pero el registro de compras permite planificarlas.

C5. Medición y control
Problemas identificados:

- P2. Sin historial de asistencia de los alumnos.
- P4. Sin reportes de ventas ni historial de precios.

Solución propuesta:
Registrar la asistencia de cada clase (CU11), guardar el historial de precios de las piezas (CU17) y el listado de ventas con su cierre de caja (CU18 y CU19). Los reportes con indicadores se completan en el ciclo IV (CU28).

C6. Clientes y entorno
Problemas identificados:

- P1. Los alumnos no ven los cupos en tiempo real.
- P6. Los clientes no pueden comprar ni pagar en línea.

Solución propuesta:
Portal del alumno con los cupos libres de cada clase (CU10 y CU12) y tienda en línea con carrito, pago por QR, tarjeta o PayPal y gestión de los pedidos web (CU20 a CU22), que además diferencia al taller de su competencia.

5. Modelo de negocio

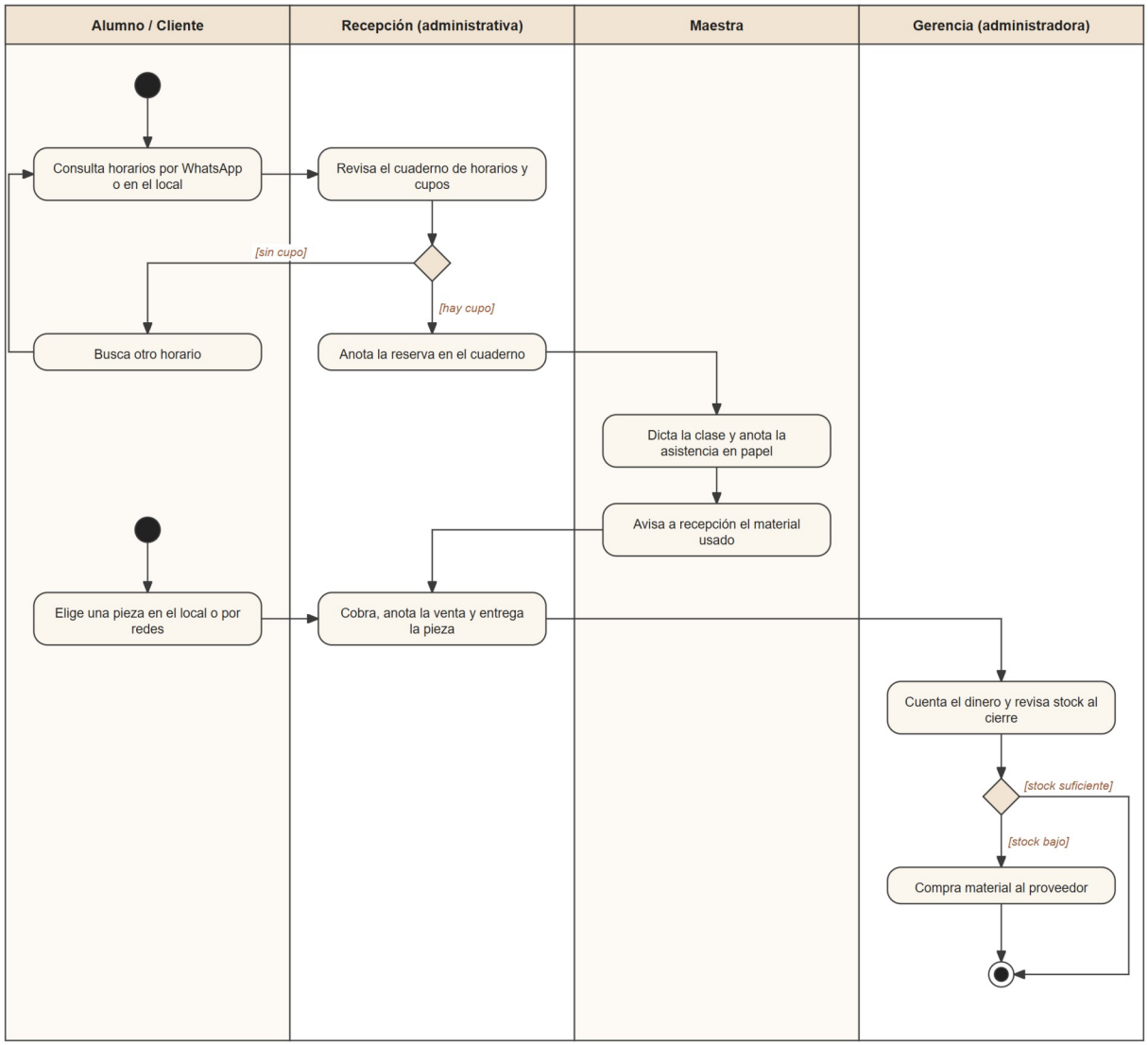
El modelo de negocio describe cómo trabaja el taller y conecta el análisis de Ishikawa con la captura de requisitos. Primero se muestra el proceso actual, donde aparecen las causas identificadas (anotar en papel, revisar el cuaderno, contar el dinero a mano). Luego se detallan los cuatro procesos principales tal como funcionan con el sistema: cada carril es un actor del capítulo 2 y cada actividad del carril «Sistema Mitsü» corresponde a un caso de uso.

5.1 Diagrama de actividad

El alumno consulta los horarios por WhatsApp y recepción revisa el cuaderno para ver si hay lugar y anotar la reserva. La maestra dicta la clase y anota la asistencia en papel. Las ventas de piezas se anotan a mano y, al cierre, la gerencia cuenta el dinero y revisa el stock para decidir las compras. Estas actividades manuales son las que el sistema reemplaza en los ciclos I a III.

Figura 6

Diagrama de actividad del proceso actual (modelo de negocio)



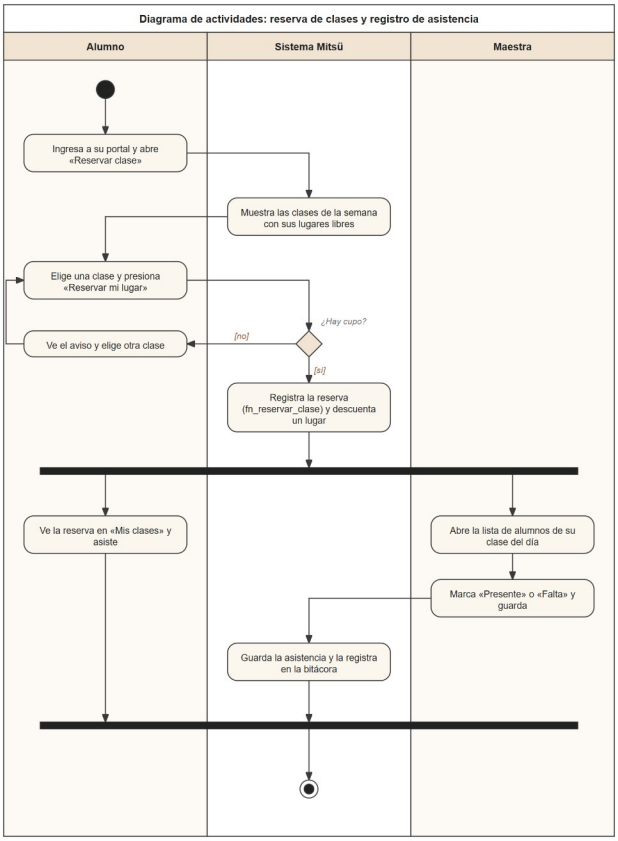
Nota. Elaboración propia a partir de la entrevista (Anexo A).

5.1.1 Reserva de clases y registro de asistencia

El alumno ingresa a su portal y ve las clases de la semana con sus lugares libres. Al pedir un lugar, el sistema verifica que haya cupo y que la clase sea futura; si no se cumple, el alumno ve el aviso y elige otra clase. Si se cumple, el procedimiento fn_reservar_clase registra la reserva. Desde ahí el proceso sigue en paralelo: el alumno asiste a la clase y la maestra abre la lista de su clase del día, marca presente o falta, y el sistema guarda la asistencia en la bitácora. Reemplaza el cuaderno de reservas y la asistencia en papel (causas C1 y C5) y corresponde a los casos de uso CU10, CU11 y CU12.

Figura 7

Diagrama de actividad: reserva de clases y registro de asistencia



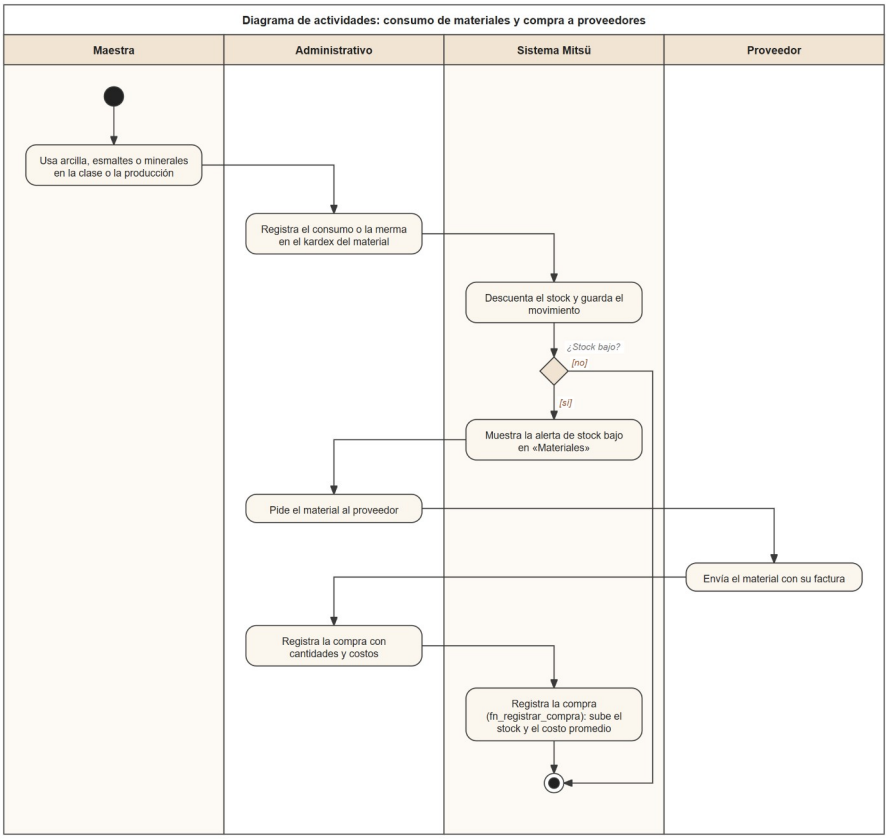
Nota. Elaboración propia a partir del funcionamiento del sistema.

5.1.2 Consumo de materiales y compra a proveedores

Cuando la maestra usa arcilla, esmaltes o minerales, el administrativo registra el consumo o la merma en el kardex del material y el sistema descuenta el stock. Si el stock queda bajo el mínimo, el sistema muestra la alerta en la pantalla de materiales; el administrativo pide el material al proveedor de Cochabamba o La Paz y, al recibirlo con su factura, registra la compra. El procedimiento fn_registrar_compra sube el stock y recalcula el costo promedio. Resuelve el inventario sin registro y las compras urgentes (causa C4) y corresponde a los casos de uso CU13 a CU16.

Figura 8

Diagrama de actividad: consumo de materiales y compra a proveedores



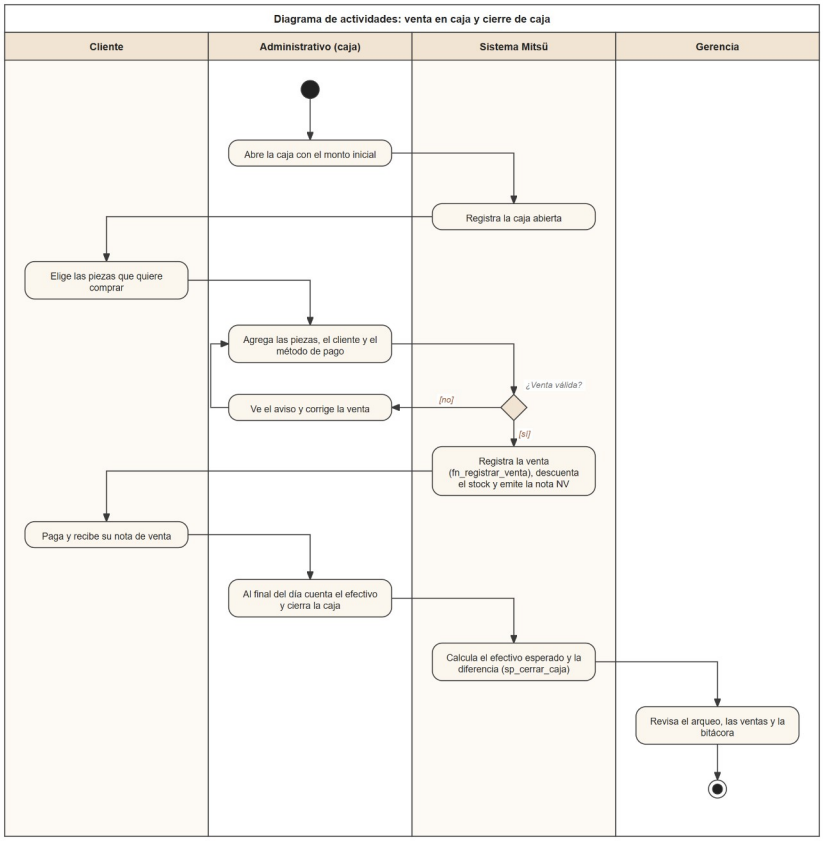
Nota. Elaboración propia a partir del funcionamiento del sistema.

5.1.3 Venta en caja y cierre de caja

Al iniciar el día el administrativo abre la caja con el monto inicial. Cuando un cliente elige piezas en el local, el administrativo las agrega con el cliente y el método de pago; el sistema verifica que la caja esté abierta y que haya stock suficiente. Si la venta es válida, fn_registrar_venta la registra, descuenta el stock y emite la nota de venta numerada (NV). Al cierre, el administrativo cuenta el efectivo y sp_cerrar_caja calcula el efectivo esperado y la diferencia, que la gerencia revisa junto con las ventas y la bitácora. Reemplaza las ventas anotadas a mano y el cierre sin arqueo (causas C1 y C5) y corresponde a los casos de uso CU17, CU18 y CU19.

Figura 9

Diagrama de actividad: venta en caja y cierre de caja



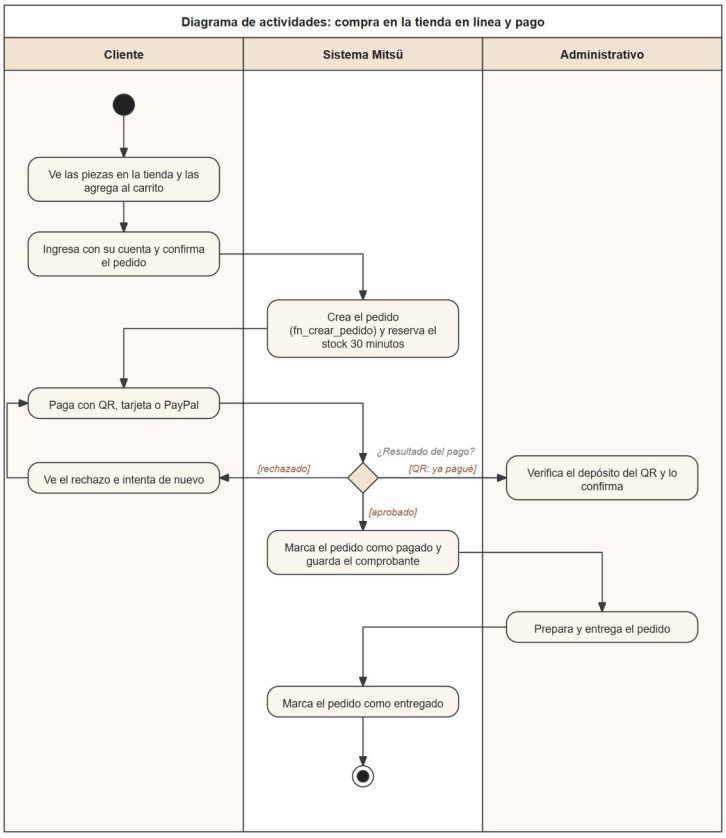
Nota. Elaboración propia a partir del funcionamiento del sistema.

5.1.4 Compra en la tienda en línea y pago

El cliente ve las piezas en la tienda pública, las agrega al carrito y confirma el pedido después de ingresar con su cuenta. El procedimiento fn_crear_pedido crea el pedido y reserva el stock por 30 minutos. El cliente paga con QR, tarjeta o PayPal: si el pago se rechaza puede intentar de nuevo; si se aprueba, el pedido queda pagado con su comprobante; y si avisa que pagó por QR, el administrativo verifica el depósito y lo confirma. Finalmente el administrativo entrega el pedido y el sistema lo marca como entregado. Resuelve la falta de venta en línea (causa C6) y corresponde a los casos de uso CU20, CU21 y CU22.

Figura 10

Diagrama de actividad: compra en la tienda en línea y pago



Nota. Elaboración propia a partir del funcionamiento del sistema.

Capítulo 2. Flujo de trabajo: captura de requisitos

2.1 Identificar actores y casos de uso

Los actores son las personas que interactúan con el sistema. Se identificaron a partir de la entrevista y del modelo de negocio: cada carril de los diagramas de actividad del capítulo 1 es un actor. Para cada uno se describe quién es, qué hace en el sistema y hasta dónde llega su acceso.

Identificar actores y describir roles

A1: Administrador

Gerencia del taller, con control total del sistema.
Funcionalidades principales: Gestionar usuarios, roles y permisos; consultar la bitácora; configurar el taller y los catálogos; registrar maestras, talleres y horarios; anular ventas.
Rol en el sistema: Acceso completo a todos los paquetes (P1 a P4).

A2: Administrativo

Personal de recepción y ventas que atiende el día a día.
Funcionalidades principales: Reservar clases en recepción, registrar alumnos, materiales, proveedores, compras y consumo; vender en caja, abrir y cerrar la caja; verificar los pagos por QR y entregar los pedidos web.
Rol en el sistema: Acceso a clases, inventario, ventas, caja y pedidos web; sin acceso a usuarios, roles ni configuración.

A3: Maestra

Dicta los talleres de modelado y torno.
Funcionalidades principales: Ver sus clases de la semana y registrar la asistencia de sus alumnos.
Rol en el sistema: Acceso solo a sus propias clases; si abre la clase de otra maestra el sistema responde 403.

A4: Alumno

Persona inscrita en los talleres (o su tutor, si es menor de edad).
Funcionalidades principales: Crear su cuenta, reservar y cancelar clases desde su portal, ver su historial de asistencia y comprar en la tienda.
Rol en el sistema: Acceso a su portal y a la tienda en línea.

A5: Cliente

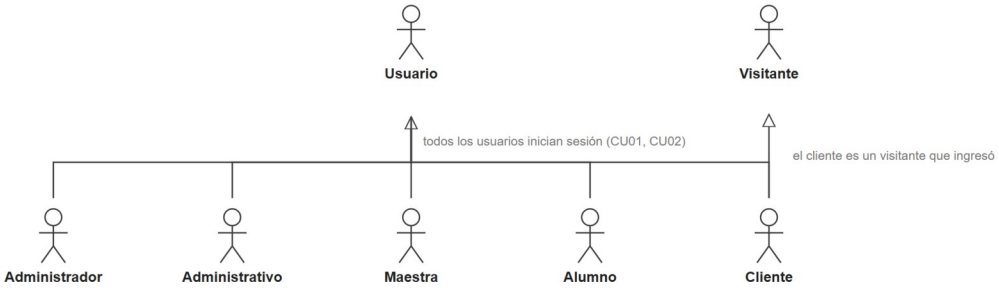
Comprador que creó su cuenta en la tienda en línea.
Funcionalidades principales: Armar el carrito, confirmar pedidos, pagar con QR, tarjeta o PayPal y ver sus pedidos y comprobantes.
Rol en el sistema: Acceso a la tienda y a «Mis pedidos»; sin acceso al panel interno.

A6: Visitante

Persona sin cuenta que entra a la página principal.
Funcionalidades principales: Ver las piezas y las clases, armar el carrito y crear su cuenta.
Rol en el sistema: Acceso al catálogo público sin iniciar sesión; para pagar debe ingresar o registrarse.

Todos los actores con cuenta heredan de Usuario el ingreso, el cierre de sesión y la recuperación de contraseña. El Cliente y el Alumno pueden comprar en la tienda; el Administrador puede hacer todo lo del Administrativo.

Figura 11
Generalización de actores



Nota. Elaboración propia.

Identificar casos de uso

Cada caso de uso es un objetivo completo que un actor logra con el sistema. Se identificaron 22 casos de uso para los ciclos I a III, agrupados en cuatro paquetes; los del ciclo IV quedan planificados.

Tabla 20
Casos de uso identificados

ID	Caso de uso	Actores	Ciclo	Paquete
CU01	Iniciar y cerrar sesión	Usuario	I	P1
CU02	Recuperar contraseña	Usuario	I	P1
CU03	Gestionar usuarios	Administrador	I	P1
CU04	Gestionar roles y permisos	Administrador	I	P1
CU05	Consultar bitácora	Administrador	I	P1
CU06	Gestionar configuración y catálogos	Administrador	I	P1
CU07	Gestionar maestras	Administrador	II	P2

ID	Caso de uso	Actores	Ciclo	Paquete
CU08	Gestionar talleres	Administrador	II	P2
CU09	Gestionar horarios y calendario	Administrador, Administrativo	II	P2
CU10	Reservar clase	Alumno, Administrativo	II	P2
CU11	Registrar asistencia	Maestra, Administrativo	II	P2
CU12	Gestionar alumnos y portal	Administrativo, Alumno	II	P2
CU13	Gestionar materiales	Administrativo	III	P3
CU14	Gestionar proveedores	Administrativo	III	P3
CU15	Registrar compras	Administrativo	III	P3
CU16	Registrar consumo, merma y alertas de stock	Administrativo	III	P3
CU17	Gestionar piezas de cerámica	Administrativo	III	P4
CU18	Registrar venta en caja	Administrativo, Administrador	III	P4
CU19	Abrir y cerrar caja (arqueo)	Administrativo	III	P4
CU20	Comprar en la tienda en línea	Visitante, Cliente, Alumno	III	P4
CU21	Pagar en línea (QR, tarjeta o PayPal)	Cliente	III	P4
CU22	Gestionar pedidos web	Administrativo	III	P4

Tabla 21

Casos de uso planificados para el ciclo IV

ID	Caso de uso	Paquete
CU23	Gestionar menú de cafetería	P5
CU24	Registrar comandas	P5
CU25	Cobrar mensualidades y controlar morosidad	P6
CU26	Gestionar pedidos personalizados	P6
CU27	Enviar notificaciones	P7
CU28	Generar reportes e indicadores	P7

Nota. No forman parte de esta entrega.

2.2 Priorizar los casos de uso

La prioridad indica qué tan necesario es el caso de uso para que el taller funcione, y el riesgo, qué tan delicada es su construcción. Dentro de cada ciclo se construyeron primero los de prioridad alta y riesgo alto, porque concentran las reglas más delicadas: cupo, stock y dinero.

Ciclo I: seguridad y configuración

Tabla 22
Priorización del ciclo i

Caso de uso	Prioridad	Riesgo	Justificación
CU01: Iniciar y cerrar sesión	Alta	Alto	Sin ingreso no se puede usar ninguna otra función; concentra la seguridad del sistema.
CU02: Recuperar contraseña	Media	Medio	Es necesaria, pero el administrador puede restablecer una contraseña mientras tanto.
CU03: Gestionar usuarios	Alta	Medio	Sin cuentas no hay quién use el sistema; además protege que siempre exista un administrador.
CU04: Gestionar roles y permisos	Alta	Alto	Define qué puede hacer cada persona; un error aquí expone funciones sensibles.
CU05: Consultar bitácora	Media	Bajo	Apoya el control, pero el sistema funciona sin consultarla; solo es de lectura.
CU06: Gestionar configuración y catálogos	Media	Bajo	Los catálogos se cargan al instalar; se modifican pocas veces.

Ciclo II: clases y reservas

Tabla 23
Priorización del ciclo ii

Caso de uso	Prioridad	Riesgo	Justificación
CU07: Gestionar maestras	Alta	Medio	Sin maestras no se pueden armar los horarios ni registrar la asistencia.
CU08: Gestionar talleres	Alta	Medio	Los talleres son la base del calendario y de las reservas.
CU09: Gestionar horarios y calendario	Alta	Alto	Resuelve la causa principal (horarios en Excel); el cruce de maestras es una regla delicada.
CU10: Reservar clase	Alta	Alto	Es el proceso más usado y el que causaba dobles reservas; el cupo debe controlarse aun con pedidos simultáneos.
CU11: Registrar asistencia	Media	Medio	Da el historial de asistencia, pero no bloquea otras funciones del taller.
CU12: Gestionar alumnos y portal	Alta	Medio	Sin alumnos registrados no hay reservas ni asistencia.

Ciclo III: inventario, ventas y tienda en línea

Tabla 24
Priorización del ciclo iii

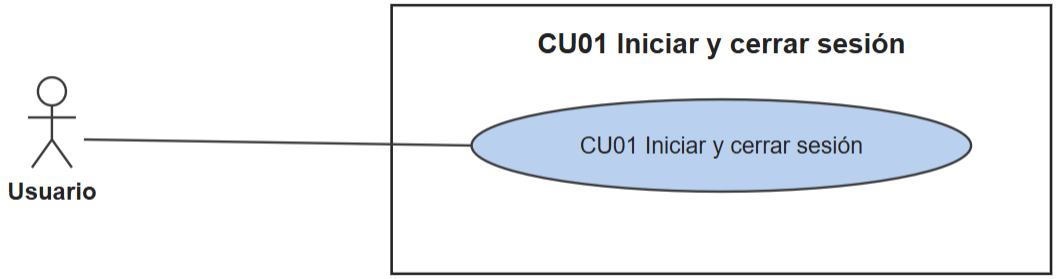
Caso de uso	Prioridad	Riesgo	Justificación
CU13: Gestionar materiales	Alta	Medio	Es la base del inventario, que antes no tenía registro.
CU14: Gestionar proveedores	Media	Bajo	Son solo dos proveedores y cambian poco.
CU15: Registrar compras	Alta	Alto	Mueve stock y dinero en varias tablas a la vez; se resuelve con un procedimiento en una sola transacción.
CU16: Registrar consumo, merma y alertas de stock	Media	Medio	Completa el kardex, pero depende de que primero existan los materiales.
CU17: Gestionar piezas de cerámica	Alta	Medio	Las piezas son lo que se vende en caja y en la tienda en línea.
CU18: Registrar venta en caja	Alta	Alto	Maneja dinero y stock; el precio real lo fija la base de datos aunque se manipule el formulario.
CU19: Abrir y cerrar caja (arqueo)	Alta	Medio	Sin caja abierta no se puede vender; el arqueo controla el efectivo del día.
CU20: Comprar en la tienda en línea	Alta	Alto	Abre un canal de venta nuevo; la reserva temporal de stock es una regla delicada.
CU21: Pagar en línea (QR, tarjeta o PayPal)	Alta	Alto	Maneja datos de pago: solo se guardan la marca y los 4 últimos dígitos de la tarjeta.
CU22: Gestionar pedidos web	Alta	Medio	Cierra el ciclo de la venta en línea; sin él los pagos por QR quedan en verificación.

2.3 Detallar un caso de uso

Cada caso de uso se detalla con su diagrama y una ficha: objetivo, resumen, actores, precondición y postcondición, flujo principal paso a paso, flujos secundarios (caminos alternativos dentro del caso) y excepciones (errores que interrumpen el caso). Los nombres de las rutas son los del sistema publicado.

Ciclo I: seguridad y configuración

Figura 12
Diagrama de casos de uso CU01 Iniciar y cerrar sesión



Nota. Elaboración propia.

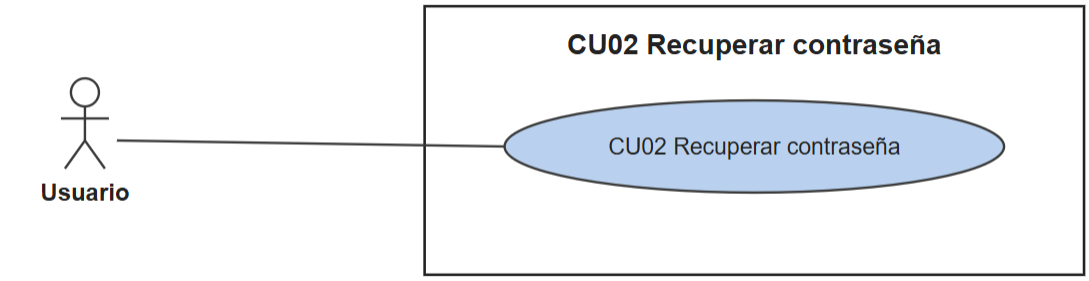
Tabla 25
Ficha del caso de uso CU01 Iniciar y cerrar sesión

Elemento	Descripción
Id. caso de uso	CU01: Iniciar y cerrar sesión
Objetivo	Permitir que cada usuario entre al sistema con su correo y contraseña, y vea solo las opciones de su rol.
Resumen	Con su correo y contraseña el usuario entra al sistema y ve solo las opciones de su rol; al terminar cierra su sesión.
Actores	Usuario
Actor iniciador	Usuario
Ciclo / paquete	Ciclo I · P1
Precondición	El usuario está registrado y activo.
Postcondición	El usuario queda autenticado con los permisos de su rol, o el intento queda registrado.
Flujo principal	1. El usuario abre la página de ingreso. 2. Escribe su correo y su contraseña y presiona “Ingresar”. 3. El sistema verifica el token CSRF y que la IP no esté bloqueada por intentos fallidos. 4. El sistema busca el usuario por correo y compara la contraseña con su hash bcrypt. 5. El sistema crea la sesión (guardada en la tabla sesion), registra LOGIN en la bitácora y lo lleva a su inicio: panel, portal del alumno o tienda. 6. Para salir, el usuario presiona “Cerrar sesión”; el sistema destruye la sesión y registra LOGOUT.
Flujo secundario	4a. Correo o contraseña incorrectos: se registra LOGIN_FALLIDO y se muestra un mensaje genérico. 3a. Cinco intentos fallidos desde la misma IP en 10 minutos: se bloquea el ingreso temporalmente. 4b. Usuario inactivo: no puede ingresar.

Elemento	Descripción
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/login · /salir

Figura 13

Diagrama de casos de uso CU02 Recuperar contraseña



Nota. Elaboración propia.

Tabla 26

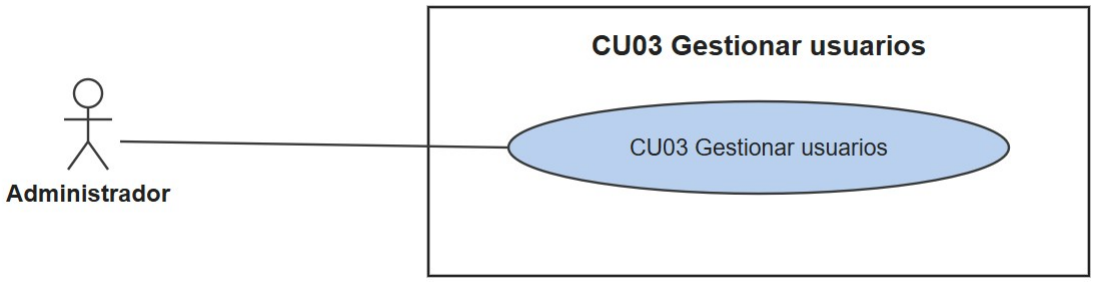
Ficha del caso de uso CU02 Recuperar contraseña

Elemento	Descripción
Id. caso de uso	CU02: Recuperar contraseña
Objetivo	Permitir que un usuario que olvidó su contraseña la cambie con un código de un solo uso.
Resumen	El usuario que olvidó su contraseña pide un código de 6 dígitos y con él define una contraseña nueva.
Actores	Usuario
Actor iniciador	Usuario
Ciclo / paquete	Ciclo I · P1
Precondición	El usuario tiene un correo registrado.
Postcondición	La contraseña queda cambiada y el código no puede volver a usarse.
Flujo principal	1. El usuario elige “¿Olvidaste tu contraseña?” y escribe su correo. 2. El sistema genera un código de 6 dígitos con vencimiento de 15 minutos y guarda solo su hash. 3. El sistema envía el código por correo (en modo demostración lo muestra en pantalla). 4. El usuario escribe el código y la nueva contraseña dos veces. 5. El sistema valida el código y la fortaleza de la contraseña, la actualiza y anula el código.

Elemento	Descripción
Flujo secundario	1a. El correo no existe: se muestra el mismo mensaje para no revelar qué correos están registrados. 4a. Código incorrecto: se suma un intento; al quinto el código se anula. 2a. Demasiadas solicitudes desde la misma IP: se pide esperar.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • Sin servidor de correo configurado: en modo demostración el código se muestra en la pantalla para copiarlo.
Ruta en el sistema	/recuperar · /recuperar/codigo

Figura 14

Diagrama de casos de uso CU03 Gestionar usuarios



Nota. Elaboración propia.

Tabla 27

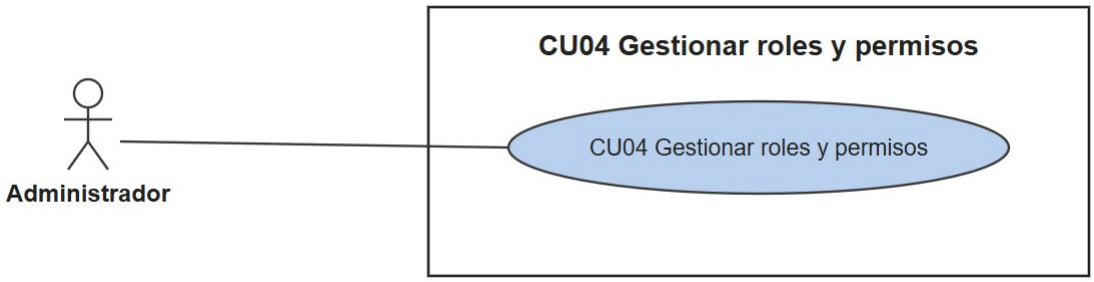
Ficha del caso de uso CU03 Gestionar usuarios

Elemento	Descripción
Id. caso de uso	CU03: Gestionar usuarios
Objetivo	Registrar, editar, activar y desactivar las cuentas del personal y asignarles un rol.
Resumen	El administrador registra las cuentas del personal, les asigna un rol y las activa o desactiva.
Actores	Administrador
Actor iniciador	Administrador
Ciclo / paquete	Ciclo I · P1
Precondición	El administrador inició sesión y tiene el permiso usuarios.gestionar.
Postcondición	El usuario queda registrado o actualizado y la acción queda en la bitácora.
Flujo principal	1. El administrador abre “Usuarios” y ve la lista con búsqueda, filtros y paginación. 2. Presiona “Nuevo usuario” y llena nombre, apellido, correo, teléfono, rol y contraseña. 3. El sistema valida los datos (formato, correo único sin importar mayúsculas o tildes, contraseña segura). 4. El sistema guarda el usuario con la contraseña cifrada y lo registra en la bitácora. 5. Para dar de baja, el administrador cambia el estado a INACTIVO (no se borra).

Elemento	Descripción
Flujo secundario	3a. Correo repetido: se muestra “Ese correo ya está registrado”. 5a. Intento de desactivar al último administrador activo: lo impide un trigger de la base de datos. 5b. Intento de desactivarse a sí mismo: el sistema lo impide.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/usuarios

Figura 15

Diagrama de casos de uso CU04 Gestionar roles y permisos



Nota. Elaboración propia.

Tabla 28

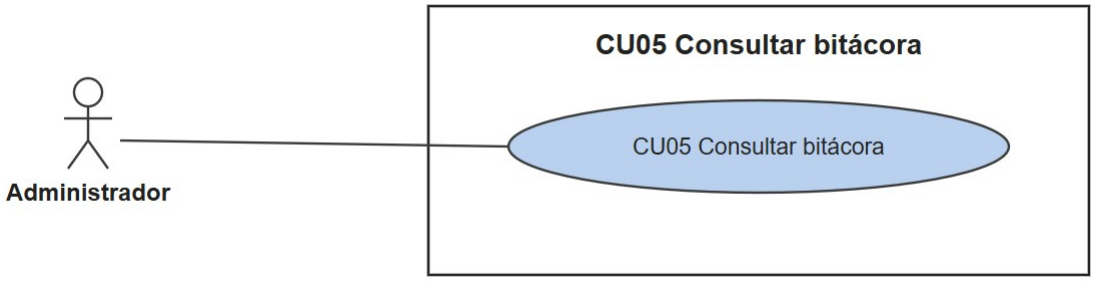
Ficha del caso de uso CU04 Gestionar roles y permisos

Elemento	Descripción
Id. caso de uso	CU04: Gestionar roles y permisos
Objetivo	Crear roles y marcar qué permisos tiene cada uno, para controlar lo que puede hacer cada persona.
Resumen	El administrador crea roles y marca en una matriz qué permisos tiene cada uno.
Actores	Administrador
Actor iniciador	Administrador
Ciclo / paquete	Ciclo I · P1
Precondición	El administrador tiene el permiso roles.gestionar.
Postcondición	El rol queda con los permisos elegidos; los usuarios de ese rol los obtienen en su siguiente petición.
Flujo principal	1. El administrador abre “Roles y permisos”. 2. Elige un rol y ve la matriz de permisos agrupada por paquete. 3. Marca o desmarca permisos y presiona “Guardar permisos”. 4. El sistema reemplaza los permisos del rol en una transacción y registra el cambio. 5. También puede crear un rol nuevo o desactivarlo.

Elemento	Descripción
Flujo secundario	3a. Se intenta quitar a ADMINISTRADOR un permiso de seguridad: el sistema lo impide. 5a. Se intenta desactivar un rol del sistema: el sistema lo impide.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/roles

Figura 16

Diagrama de casos de uso CU05 Consultar bitácora



Nota. Elaboración propia.

Tabla 29

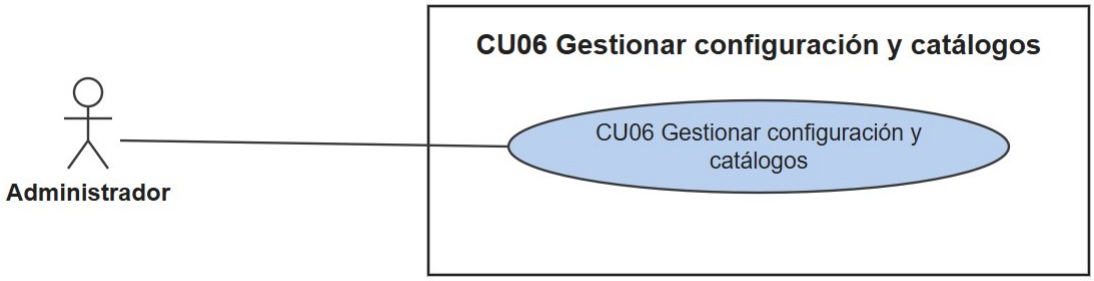
Ficha del caso de uso CU05 Consultar bitácora

Elemento	Descripción
Id. caso de uso	CU05: Consultar bitácora
Objetivo	Revisar quién hizo cada acción importante, cuándo y desde qué IP.
Resumen	El administrador revisa quién hizo cada acción importante, cuándo y desde qué IP, con filtros.
Actores	Administrador
Actor iniciador	Administrador
Ciclo / paquete	Ciclo I · P1
Precondición	El administrador tiene el permiso bitacora.ver.
Postcondición	Ninguna: la bitácora es de solo lectura (no se edita ni se borra).
Flujo principal	1. El administrador abre “Bitácora”. 2. Filtra por usuario, módulo, acción y rango de fechas. 3. El sistema muestra los registros paginados, del más reciente al más antiguo.
Flujo secundario	2a. No hay registros con esos filtros: se muestra un mensaje vacío.

Elemento	Descripción
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido).
Ruta en el sistema	/bitacora

Figura 17

Diagrama de casos de uso CU06 Gestionar configuración y catálogos



Nota. Elaboración propia.

Tabla 30

Ficha del caso de uso CU06 Gestionar configuración y catálogos

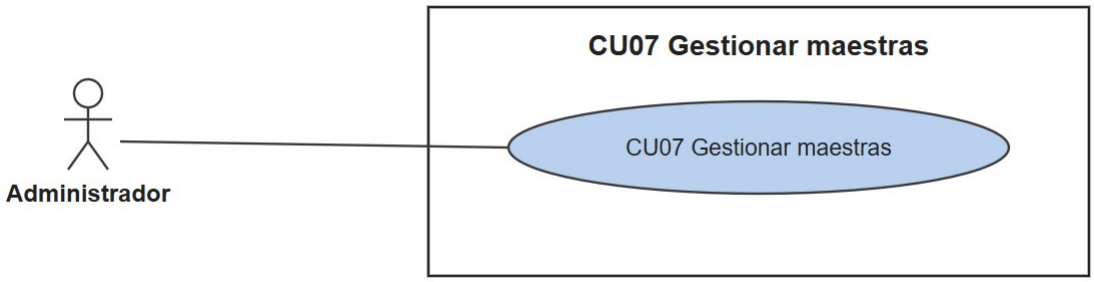
Elemento	Descripción
Id. caso de uso	CU06: Gestionar configuración y catálogos
Objetivo	Mantener los datos del taller y los catálogos base: categorías, unidades de medida y métodos de pago.
Resumen	El administrador mantiene los datos del taller y los catálogos de categorías, unidades y métodos de pago.
Actores	Administrador
Actor iniciador	Administrador
Ciclo / paquete	Ciclo I · P1
Precondición	El administrador tiene el permiso configuracion.gestionar.
Postcondición	Los valores quedan disponibles para el resto del sistema.
Flujo principal	1. El administrador abre “Configuración”. 2. Edita los datos del taller (nombre, ciudad, horas de cancelación, etc.) y guarda. 3. En la pestaña de catálogos agrega o edita una categoría, unidad o método de pago. 4. El sistema valida que el nombre no se repita y guarda el cambio en la bitácora.
Flujo secundario	4a. Nombre repetido (sin importar mayúsculas o tildes): se avisa. 4b. Desactivar un elemento usado: queda inactivo y deja de ofrecerse, sin borrar el historial.

Elemento	Descripción
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/configuracion

Ciclo II: clases y reservas

Figura 18

Diagrama de casos de uso CU07 Gestionar maestras



Nota. Elaboración propia.

Tabla 31

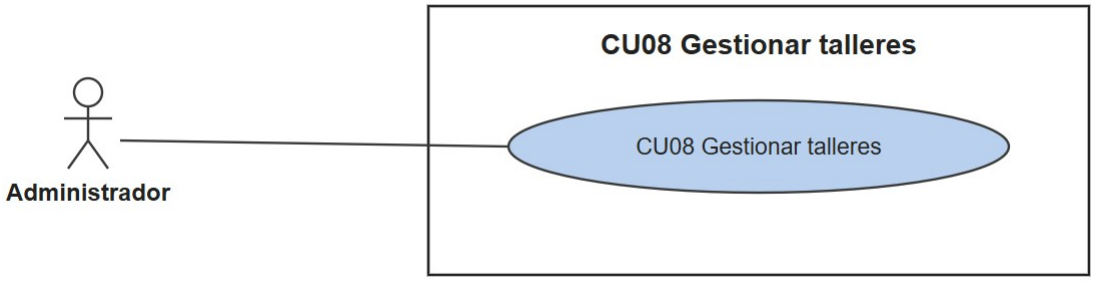
Ficha del caso de uso CU07 Gestionar maestras

Elemento	Descripción
Id. caso de uso	CU07: Gestionar maestras
Objetivo	Registrar a las maestras del taller con su cuenta de acceso y su especialidad.
Resumen	El administrador registra a cada maestra con su especialidad y le crea su cuenta de acceso.
Actores	Administrador
Actor iniciador	Administrador
Ciclo / paquete	Ciclo II · P2
Precondición	El administrador tiene el permiso maestras.gestionar.
Postcondición	La maestra queda registrada con su cuenta.
Flujo principal	1. El administrador abre “Maestras” y presiona “Nueva maestra”. 2. Llena los datos personales, el correo de acceso, la especialidad y una breve reseña. 3. El sistema crea en una sola transacción el usuario con rol MAESTRA y su ficha de maestra. 4. La maestra ya puede ingresar y ver sus clases.

Elemento	Descripción
Flujo secundario	3a. Correo repetido: no se guarda nada (la transacción se revierte). 4a. Desactivar una maestra con horarios activos: el sistema avisa que primero se reasignen.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/maestras

Figura 19

Diagrama de casos de uso CU08 Gestionar talleres



Nota. Elaboración propia.

Tabla 32

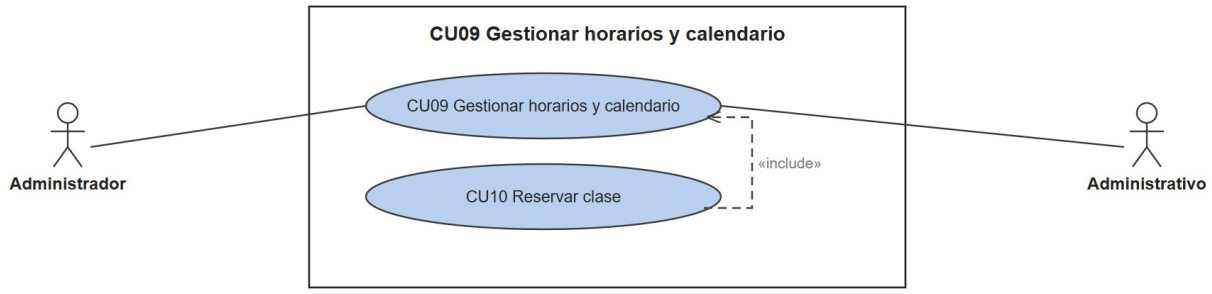
Ficha del caso de uso CU08 Gestionar talleres

Elemento	Descripción
Id. caso de uso	CU08: Gestionar talleres
Objetivo	Registrar los talleres que ofrece Mitsü con su nivel, público, duración, cupo y precios.
Resumen	El administrador registra los talleres con su nivel, público, duración, cupo y precios.
Actores	Administrador
Actor iniciador	Administrador
Ciclo / paquete	Ciclo II · P2
Precondición	El administrador tiene el permiso talleres.gestionar.
Postcondición	El taller queda disponible para crear horarios.
Flujo principal	1. El administrador abre “Talleres”. 2. Llena nombre, nivel, público (niños, jóvenes, adultos), cupo, precio por clase y mensual. 3. El sistema valida rangos (cupo de 1 a 30, precios positivos) y guarda.
Flujo secundario	3a. Nombre repetido: se avisa. 3b. Cupo o precio fuera de rango: se marca el campo.

Elemento	Descripción
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/talleres

Figura 20

Diagrama de casos de uso CU09 Gestionar horarios y calendario



Nota. Elaboración propia.

Tabla 33

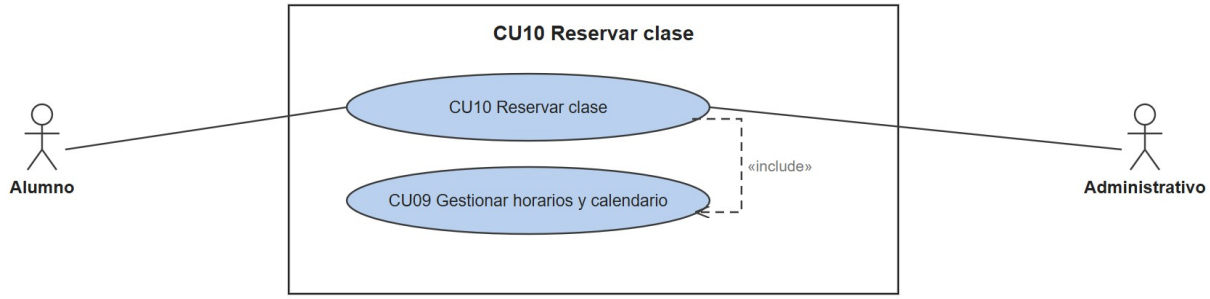
Ficha del caso de uso CU09 Gestionar horarios y calendario

Elemento	Descripción
Id. caso de uso	CU09: Gestionar horarios y calendario
Objetivo	Definir los horarios semanales de cada taller y generar las clases concretas del calendario.
Resumen	Se definen los horarios semanales de cada taller y el sistema genera las clases del calendario sin duplicarlas.
Actores	Administrador, Administrativo
Actor iniciador	Administrador
Ciclo / paquete	Ciclo II · P2
Precondición	Existen talleres y maestras activos.
Postcondición	El calendario muestra las clases con su cupo disponible.
Flujo principal	1. El administrador crea un horario: taller, maestra, día, hora de inicio y fin, cupo. 2. La base de datos impide que la misma maestra tenga dos horarios que se crucen (restricción EXCLUDE). 3. El administrativo abre el calendario y presiona “Generar clases” para un rango de fechas. 4. El sistema crea una sesión de clase por cada horario y fecha que coincida, sin duplicar. 5. Puede cancelar una clase (se cancelan sus reservas) o cambiar la maestra de una clase.

Elemento	Descripción
Flujo secundario	2a. Horario cruzado: “La maestra ya tiene otro horario que se cruza ese día”. 4a. Las clases ya existían: no se duplican y se informa cuántas se crearon.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/horarios · /clases

Figura 21

Diagrama de casos de uso CU10 Reservar clase



Nota. Elaboración propia.

Tabla 34

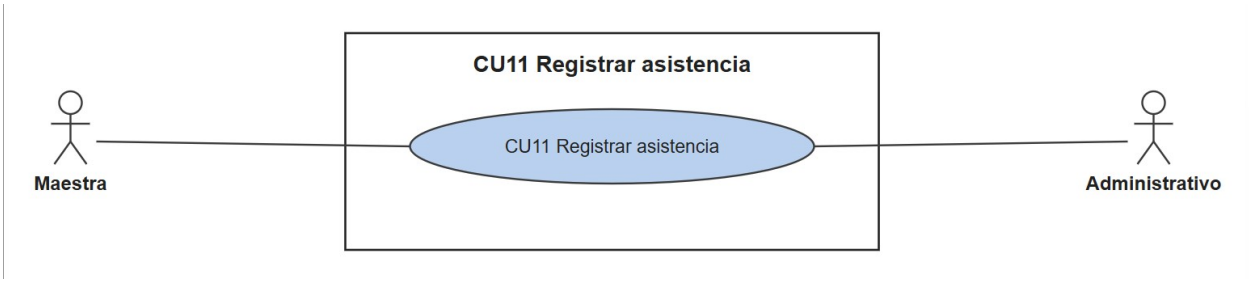
Ficha del caso de uso CU10 Reservar clase

Elemento	Descripción
Id. caso de uso	CU10: Reservar clase
Objetivo	Que el alumno (desde su portal) o el administrativo (en recepción) reserve un lugar en una clase.
Resumen	El alumno desde su portal, o el administrativo en recepción, reserva un lugar en una clase con cupo.
Actores	Alumno, Administrativo
Actor iniciador	Alumno
Ciclo / paquete	Ciclo II · P2
Precondición	Hay una clase programada con cupo y el alumno está activo.
Postcondición	La reserva queda registrada y el cupo disponible baja en uno.
Flujo principal	1. El alumno abre “Reservar” y ve las clases próximas con los lugares libres. 2. Elige una clase y confirma. 3. El sistema bloquea la clase (FOR UPDATE), verifica cupo, fecha y que no esté repetida. 4. El sistema guarda la reserva y muestra el mensaje de confirmación. 5. El alumno puede cancelar su reserva hasta 12 horas antes.

Elemento	Descripción
Flujo secundario	3a. Sin cupo: “La clase ya no tiene lugares”. 3b. Ya reservada: se avisa. 5a. Fuera del plazo de cancelación: solo el administrativo puede cancelarla.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite. • Dos alumnos piden el último lugar a la vez: el trigger bloquea la clase y solo uno lo obtiene.
Ruta en el sistema	/portal/reservar · /clases/reservar

Figura 22

Diagrama de casos de uso CU11 Registrar asistencia



Nota. Elaboración propia.

Tabla 35

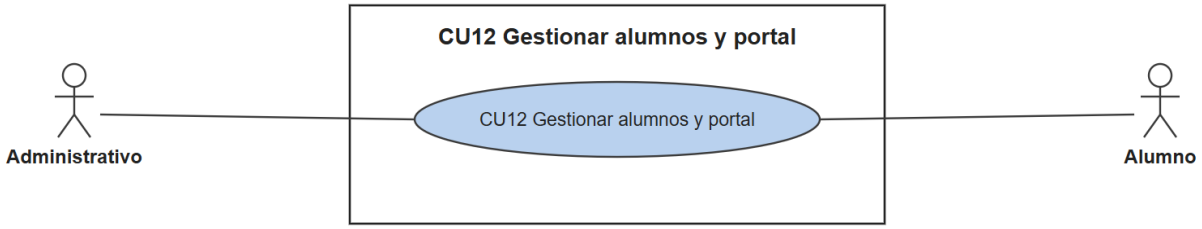
Ficha del caso de uso CU11 Registrar asistencia

Elemento	Descripción
Id. caso de uso	CU11: Registrar asistencia
Objetivo	Que la maestra marque quién asistió y quién faltó en su clase.
Resumen	La maestra abre su clase del día y marca quién asistió y quién faltó.
Actores	Maestra, Administrativo
Actor iniciador	Maestra
Ciclo / paquete	Ciclo II · P2
Precondición	La clase es del día o anterior y la maestra es la asignada (o es personal con permiso).
Postcondición	La asistencia queda registrada y alimenta el porcentaje de asistencia del alumno.
Flujo principal	1. La maestra abre su panel y elige una clase. 2. Ve la lista de alumnos con reserva. 3. Marca “Asistió” o “Faltó” en cada uno y guarda. 4. El sistema guarda todo en una transacción y marca la clase como realizada.
Flujo secundario	1a. Intenta abrir la clase de otra maestra: el sistema responde 403 (prohibido). 3a. Clase futura: no se permite registrar asistencia.

Elemento	Descripción
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/clases/ver · /clases/asistencia

Figura 23

Diagrama de casos de uso CU12 Gestionar alumnos y portal



Nota. Elaboración propia.

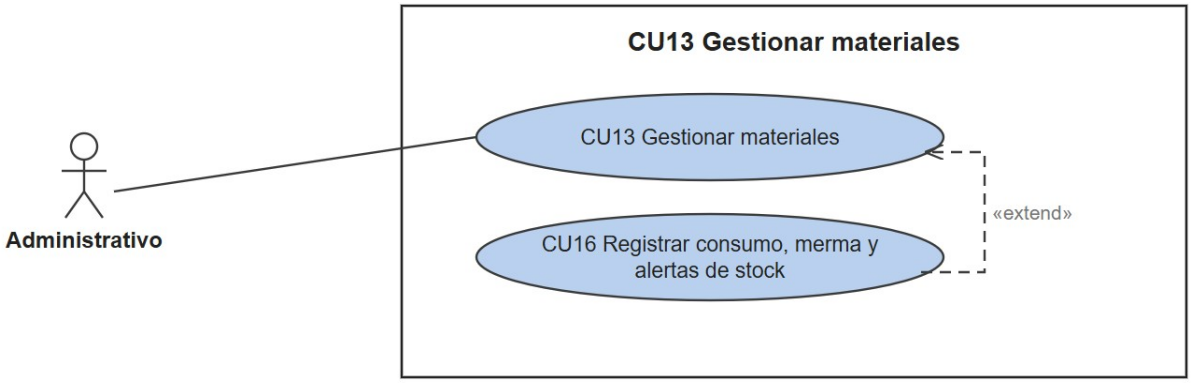
Tabla 36

Ficha del caso de uso CU12 Gestionar alumnos y portal

Elemento	Descripción
Id. caso de uso	CU12: Gestionar alumnos y portal
Objetivo	Registrar alumnos (en recepción o por autorregistro) y darles un portal con sus reservas e historial.
Resumen	Se registran los alumnos en recepción o por autorregistro, y cada uno tiene un portal con sus reservas e historial.
Actores	Administrativo, Alumno
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo II · P2
Precondición	Ninguna para el autorregistro; permiso alumnos.gestionar para el personal.
Postcondición	El alumno queda registrado y puede reservar clases.
Flujo principal	1. La persona abre “Crear cuenta” y llena sus datos, CI, fecha de nacimiento y contraseña. 2. Si es menor de edad, el sistema exige nombre y teléfono del tutor. 3. El sistema crea el usuario con rol ALUMNO y su ficha en una transacción. 4. El alumno entra a su portal: próximas clases, historial de asistencia y su perfil.
Flujo secundario	2a. Menor sin tutor: se marca el campo. 3a. CI o correo repetido: se avisa.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/registro · /alumnos · /portal

Ciclo III: inventario, ventas y tienda en línea

Figura 24
Diagrama de casos de uso CU13 Gestionar materiales

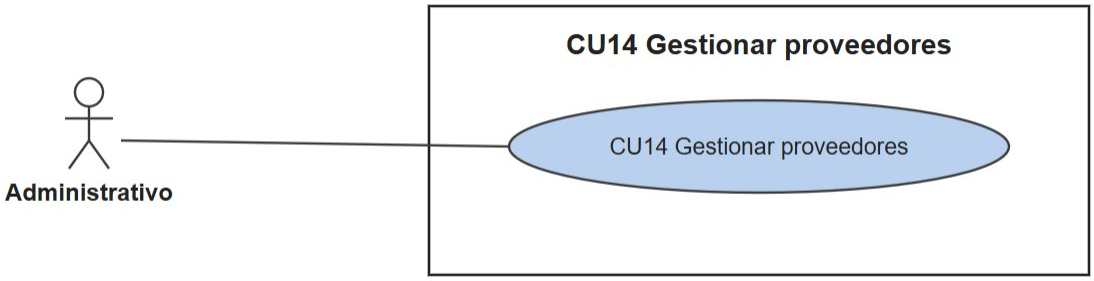


Nota. Elaboración propia.

Tabla 37
Ficha del caso de uso CU13 Gestionar materiales

Elemento	Descripción
Id. caso de uso	CU13: Gestionar materiales
Objetivo	Registrar los insumos del taller (arcillas, esmaltes, herramientas) con su unidad, stock mínimo y costo.
Resumen	El administrativo registra los insumos con su unidad, stock mínimo y costo.
Actores	Administrativo
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo III · P3
Precondición	Permiso materiales.gestionar.
Postcondición	El material queda disponible para compras y consumos.
Flujo principal	1. El administrativo abre “Materiales” y ve el stock con alertas de color. 2. Registra un material: nombre, categoría, unidad y stock mínimo. 3. El sistema guarda el material con stock 0; el stock solo cambia por compras o movimientos (kardex).
Flujo secundario	2a. Nombre repetido: se avisa. 3a. Se intenta modificar el stock directamente: lo impide un trigger.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/materiales

Figura 25
Diagrama de casos de uso CU14 Gestionar proveedores

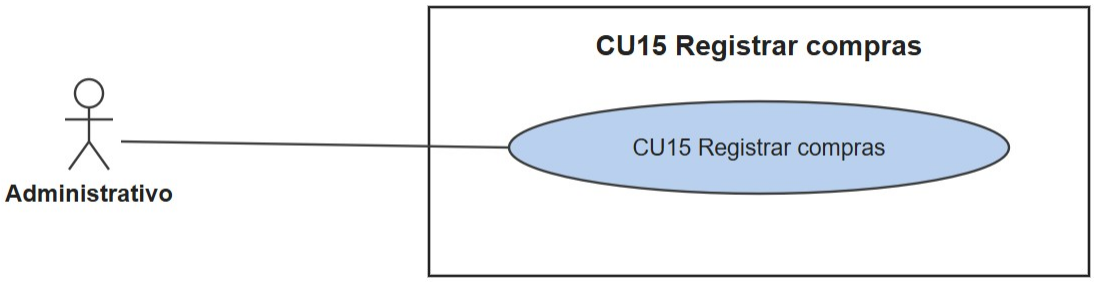


Nota. Elaboración propia.

Tabla 38
Ficha del caso de uso CU14 Gestionar proveedores

Elemento	Descripción
Id. caso de uso	CU14: Gestionar proveedores
Objetivo	Mantener la lista de proveedores con NIT, contacto, teléfono y ciudad.
Resumen	El administrativo mantiene la lista de proveedores con su NIT, contacto y ciudad.
Actores	Administrativo
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo III · P3
Precondición	Permiso proveedores.gestionar.
Postcondición	El proveedor queda disponible para registrar compras.
Flujo principal	1. El administrativo abre “Proveedores”. 2. Registra o edita nombre, NIT, contacto, teléfono y ciudad. 3. El sistema valida que el nombre y el NIT no se repitan y guarda.
Flujo secundario	3a. NIT repetido: se avisa. 3b. Desactivar un proveedor con compras: queda inactivo, sus compras se conservan.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/proveedores

Figura 26
Diagrama de casos de uso CU15 Registrar compras



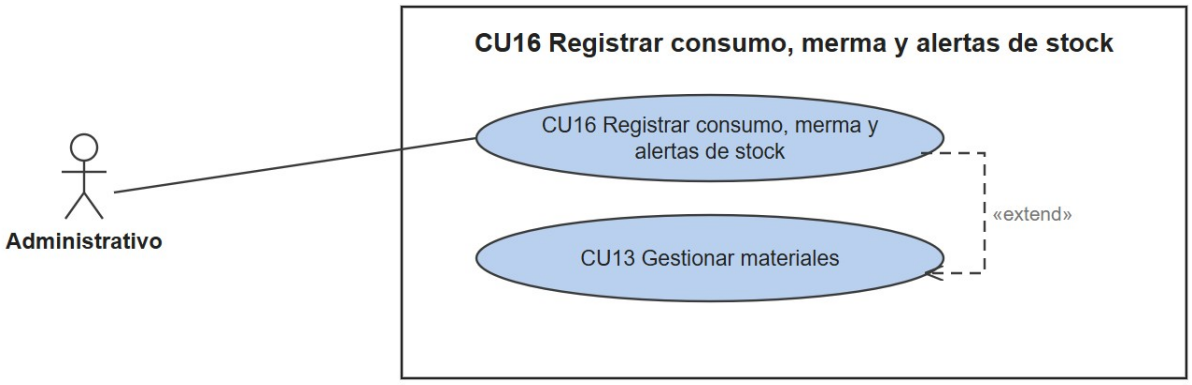
Nota. Elaboración propia.

Tabla 39
Ficha del caso de uso CU15 Registrar compras

Elemento	Descripción
Id. caso de uso	CU15: Registrar compras
Objetivo	Registrar la compra de materiales a un proveedor y que el stock y el costo promedio se actualicen solos.
Resumen	El administrativo registra una compra a un proveedor y el stock y el costo promedio se actualizan solos.
Actores	Administrativo
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo III · P3
Precondición	Hay proveedores y materiales activos.
Postcondición	La compra queda registrada, el kardex tiene los movimientos y el stock está actualizado.
Flujo principal	1. El administrativo abre “Nueva compra” y elige proveedor y fecha. 2. Agrega líneas: material, cantidad y costo unitario. 3. Presiona “Registrar compra”. 4. El sistema llama a fn_registrar_compra: guarda la compra, el detalle y un movimiento de entrada por línea. 5. Los triggers suben el stock y recalculan el costo promedio de cada material. 6. Si hubo un error, puede anular la compra con motivo; el stock vuelve atrás.
Flujo secundario	2a. Material repetido en el detalle: se avisa. 6a. Anular dejaría stock negativo: la base de datos lo impide.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/compras/nueva · /compras/anular

Figura 27

Diagrama de casos de uso CU16 Registrar consumo, merma y alertas de stock



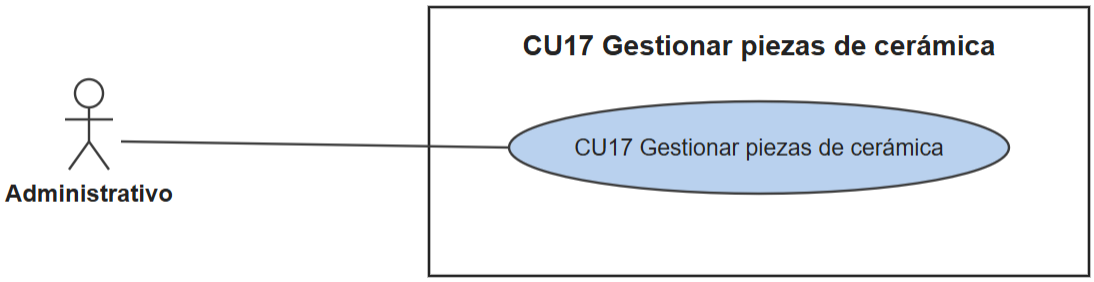
Nota. Elaboración propia.

Tabla 40

Ficha del caso de uso CU16 Registrar consumo, merma y alertas de stock

Elemento	Descripción
Id. caso de uso	CU16: Registrar consumo, merma y alertas de stock
Objetivo	Registrar la salida de material por uso en clases, merma o ajuste, y avisar cuando el stock está bajo.
Resumen	Se registra la salida de material por uso, merma o ajuste, y el sistema avisa cuando el stock baja del mínimo.
Actores	Administrativo
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo III · P3
Precondición	El material existe y está activo.
Postcondición	El kardex refleja el movimiento con usuario, fecha y saldo.
Flujo principal	1. El administrativo abre el kardex de un material. 2. Elige el tipo (consumo, merma, ajuste + o ajuste −), cantidad y motivo. 3. El sistema registra el movimiento; el trigger actualiza el stock y guarda el saldo resultante. 4. Si el stock queda en o bajo el mínimo, el material aparece en alertas del panel.
Flujo secundario	3a. La salida dejaría stock negativo: “Stock insuficiente”.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/materiales/kardex · /materiales/movimiento

Figura 28
Diagrama de casos de uso CU17 Gestionar piezas de cerámica



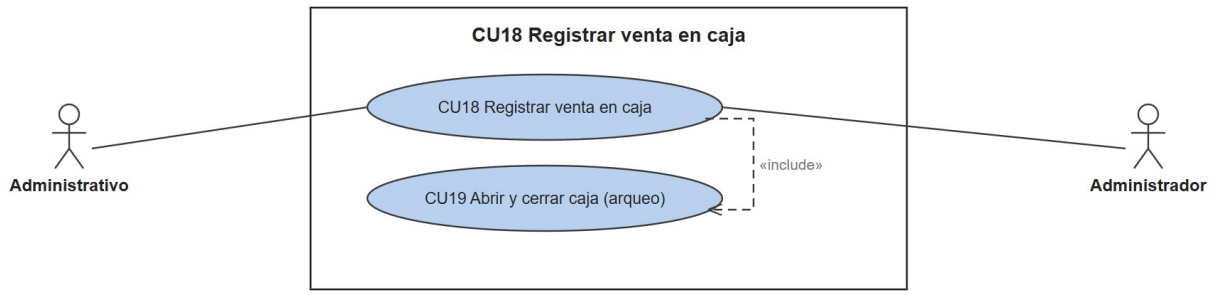
Nota. Elaboración propia.

Tabla 41
Ficha del caso de uso CU17 Gestionar piezas de cerámica

Elemento	Descripción
Id. caso de uso	CU17: Gestionar piezas de cerámica
Objetivo	Registrar las piezas que se venden con foto, precio y stock, y llevar su producción e historial de precios.
Resumen	El administrativo registra las piezas con foto, precio y stock, y el sistema guarda su historial de precios.
Actores	Administrativo
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo III · P4
Precondición	Permiso piezas.gestionar.
Postcondición	La pieza queda en el catálogo y aparece en la tienda en línea si tiene stock.
Flujo principal	1. El administrativo abre “Piezas” y presiona “Nueva pieza”. 2. Llena nombre, categoría, precio, stock mínimo y sube una foto (JPG/PNG/WebP hasta 2 MB). 3. El sistema valida que el archivo sea realmente una imagen y guarda la pieza con un código automático. 4. Registra la producción (horneada) desde el kardex; el stock sube. 5. Si cambia el precio, un trigger lo guarda en el historial de precios.
Flujo secundario	2a. Archivo que no es imagen o es muy grande: se rechaza. 2b. Precio menor o igual a cero: se marca el campo.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/piezas

Figura 29

Diagrama de casos de uso CU18 Registrar venta en caja



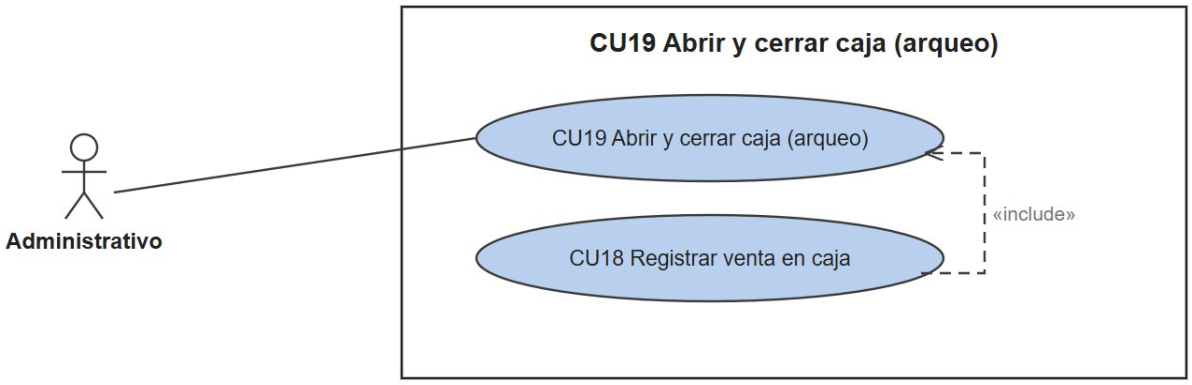
Nota. Elaboración propia.

Tabla 42

Ficha del caso de uso CU18 Registrar venta en caja

Elemento	Descripción
Id. caso de uso	CU18: Registrar venta en caja
Objetivo	Vender piezas en el local, emitir la nota de venta y, si hace falta, anularla con motivo.
Resumen	El administrativo vende piezas en el local, emite la nota de venta numerada y, si hace falta, la anula con motivo.
Actores	Administrativo, Administrador
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo III · P4
Precondición	Hay una caja abierta y piezas con stock.
Postcondición	La venta queda registrada, el stock descontado y la nota disponible.
Flujo principal	1. El administrativo abre “Nueva venta” (punto de venta). 2. Busca piezas, las agrega al carrito y elige cliente y método de pago. 3. Escribe el monto recibido; el sistema calcula el cambio. 4. Presiona “Cobrar”: fn_registrar_venta guarda la venta, copia el precio vigente y descuenta el stock. 5. El sistema muestra la nota de venta lista para imprimir. 6. El administrador puede anular una venta con motivo; el stock se devuelve.
Flujo secundario	4a. No hay caja abierta: “Abra la caja antes de vender”. 4b. Stock insuficiente: se indica la pieza. 3a. Monto recibido menor al total en efectivo: se avisa. 6a. Una venta anulada no se puede modificar (trigger).
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/ventas/nueva · /ventas/anular

Figura 30
Diagrama de casos de uso CU19 Abrir y cerrar caja (arqueo)



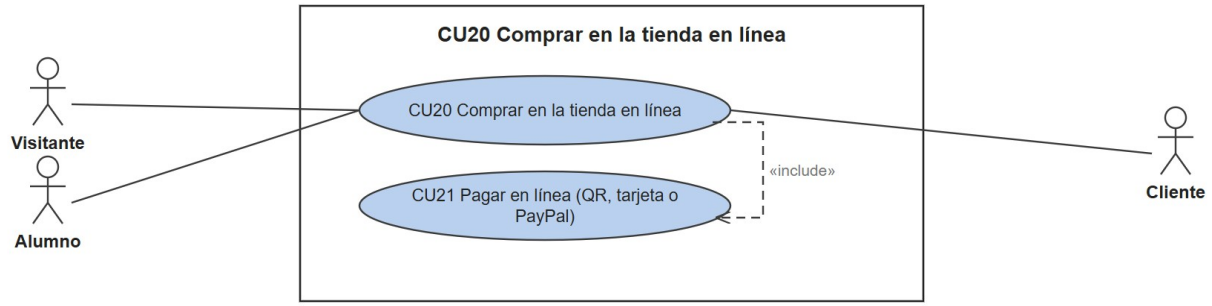
Nota. Elaboración propia.

Tabla 43
Ficha del caso de uso CU19 Abrir y cerrar caja (arqueo)

Elemento	Descripción
Id. caso de uso	CU19: Abrir y cerrar caja (arqueo)
Objetivo	Controlar el efectivo del día: abrir caja con un monto inicial y cerrarla comparando lo contado con lo esperado.
Resumen	El administrativo abre la caja con un monto inicial y al cierre compara el efectivo contado con el esperado.
Actores	Administrativo
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo III · P4
Precondición	Permiso caja.gestionar.
Postcondición	La caja queda cerrada con su arqueo.
Flujo principal	1. El administrativo abre la caja con el monto inicial. 2. Durante el día, cada venta queda ligada a esa caja. 3. Al cierre, cuenta el efectivo y escribe el monto declarado. 4. El procedimiento sp_cerrar_caja calcula efectivo esperado, otros métodos y la diferencia. 5. La caja queda CERRADA y no admite más ventas.
Flujo secundario	1a. Ya hay una caja abierta: el índice único lo impide. 4a. Diferencia distinta de cero: se registra con observación.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/caja

Figura 31

Diagrama de casos de uso CU20 Comprar en la tienda en línea



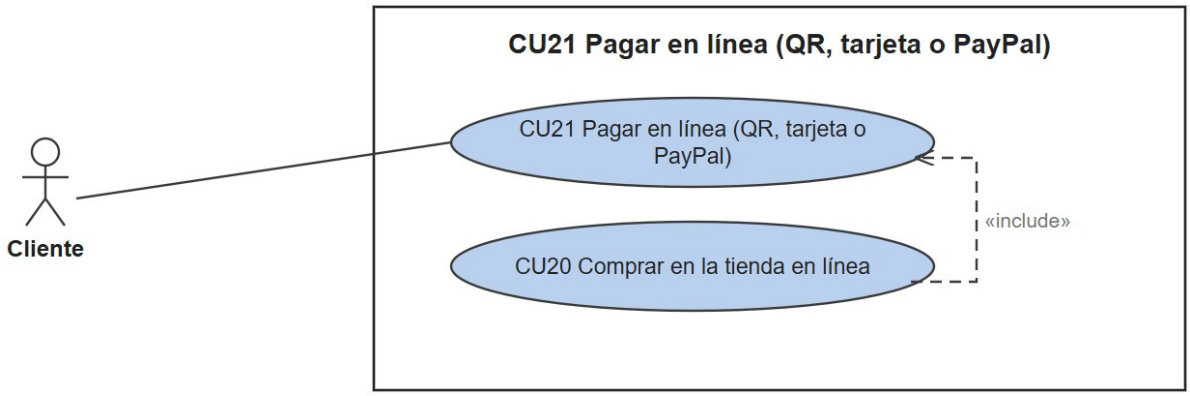
Nota. Elaboración propia.

Tabla 44

Ficha del caso de uso CU20 Comprar en la tienda en línea

Elemento	Descripción
Id. caso de uso	CU20: Comprar en la tienda en línea
Objetivo	Que cualquier visitante vea las piezas y arme su carrito, y que al pagar ingrese con su cuenta para recibir el comprobante.
Resumen	Cualquier visitante ve las piezas y arma su carrito; para confirmar el pedido ingresa con su cuenta y el stock queda reservado.
Actores	Visitante, Cliente, Alumno
Actor iniciador	Visitante
Ciclo / paquete	Ciclo III · P4
Precondición	Ninguna para ver la tienda; para confirmar el pedido, tener cuenta de cliente o alumno.
Postcondición	El pedido queda PENDIENTE_PAGO con el stock reservado.
Flujo principal	1. El visitante entra a la página principal: ve las piezas disponibles y las clases. 2. Abre una pieza y la agrega al carrito (se guarda en la sesión). 3. Presiona “Pagar”: si no ingresó, el sistema le pide ingresar o crear su cuenta y luego lo devuelve al carrito. 4. Elige retiro en el taller o envío, su teléfono y una nota. 5. El sistema llama a fn_crear_pedido: guarda el pedido y reserva el stock por 30 minutos. 6. El sistema lo lleva a la pasarela de pago (CU21). 7. El cliente puede ver “Mis pedidos” y cancelar un pedido impago.
Flujo secundario	3a. No tiene cuenta: se registra en /tienda/registro y vuelve al checkout. 5a. Stock insuficiente: se avisa qué pieza. 7a. Pedido no pagado en 30 minutos: se cancela solo y el stock vuelve (fn_vencer_pedidos).
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/ · /tienda · /tienda/carrito · /tienda/checkout

Figura 32
Diagrama de casos de uso CU21 Pagar en línea (QR, tarjeta o PayPal)



Nota. Elaboración propia.

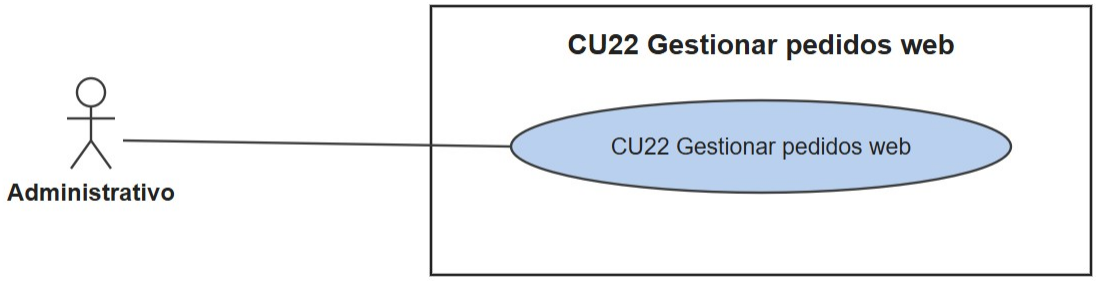
Tabla 45
Ficha del caso de uso CU21 Pagar en línea (QR, tarjeta o PayPal)

Elemento	Descripción
Id. caso de uso	CU21: Pagar en línea (QR, tarjeta o PayPal)
Objetivo	Pagar el pedido con el QR del taller, con tarjeta o con PayPal y recibir el comprobante.
Resumen	El cliente paga su pedido con el QR del taller, con tarjeta o con PayPal y recibe su comprobante.
Actores	Cliente
Actor iniciador	Cliente
Ciclo / paquete	Ciclo III · P4
Precondición	El pedido está PENDIENTE_PAGO y es del cliente que ingresó. (Modo demostración: no hay cobro real).
Postcondición	El pago queda registrado y el pedido PAGADO o EN_VERIFICACION.
Flujo principal	1. El cliente ve la pasarela con el total y elige el método. 2. Tarjeta: escribe número, titular, vencimiento y CVV; el sistema valida con el algoritmo de Luhn y la fecha. 3. El sistema simula el cobro; si se aprueba, fn_registrar_pago guarda el pago (solo marca y 4 últimos dígitos) y el pedido pasa a PAGADO. 4. QR: el cliente escanea el QR del taller, paga y escribe el número de comprobante; el pedido pasa a EN_VERIFICACION. 5. PayPal: el cliente aprueba el pago en la ventana de PayPal (sandbox) y el pedido pasa a PAGADO. 6. El sistema muestra el comprobante y lo envía por correo cuando el correo está configurado.

Elemento	Descripción
Flujo secundario	2a. Número inválido o tarjeta vencida: se avisa sin guardar nada. 3a. Pago rechazado (fondos insuficientes): se registra el intento RECHAZADO. 1a. El pedido ya fue pagado: “El pedido no está esperando pago”. 4a. Modo demostración del QR: el cliente toca «Pago realizado» y el pedido pasa a PAGADO, o «Pago no realizado» y se registra el intento RECHAZADO.
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite. • El pedido venció (30 minutos sin pagar): ya no admite pagos y el cliente arma uno nuevo.
Ruta en el sistema	/pago · /pago/tarjeta · /pago/qr · /pago/paypal

Figura 33

Diagrama de casos de uso CU22 Gestionar pedidos web



Nota. Elaboración propia.

Tabla 46

Ficha del caso de uso CU22 Gestionar pedidos web

Elemento	Descripción
Id. caso de uso	CU22: Gestionar pedidos web
Objetivo	Que el taller confirme los pagos por QR, entregue los pedidos y cancele los que corresponda.
Resumen	El administrativo confirma los pagos por QR, marca los pedidos como entregados y cancela los que corresponda.
Actores	Administrativo
Actor iniciador	Administrativo
Ciclo / paquete	Ciclo III · P4
Precondición	Permiso pedidos.gestionar.
Postcondición	El pedido avanza de estado y el kardex de piezas lo refleja.

Elemento	Descripción
Flujo principal	1. El administrativo abre “Pedidos web” y filtra por estado. 2. Abre un pedido EN_VERIFICACION y revisa el número de comprobante del QR. 3. Confirma o rechaza el depósito; fn_verificar_pago_qr deja el pedido PAGADO o lo devuelve a PENDIENTE_PAGO. 4. Cuando el cliente retira o recibe el pedido, lo marca como ENTREGADO. 5. Puede cancelar un pedido no entregado con motivo; fn_cancelar_pedido devuelve el stock.
Flujo secundario	3a. El pago ya fue verificado: se avisa. 5a. Un pedido entregado no se puede cancelar (trigger de transiciones).
Excepción	• Error de conexión con la base de datos: se muestra la página de error 500 sin detalles internos y no se guarda nada. • El usuario no tiene el permiso del rol para esta acción: el sistema responde 403 (prohibido). • La sesión venció o el formulario quedó abierto mucho tiempo: el token CSRF no coincide (error 419) y el usuario recarga y repite.
Ruta en el sistema	/pedidos

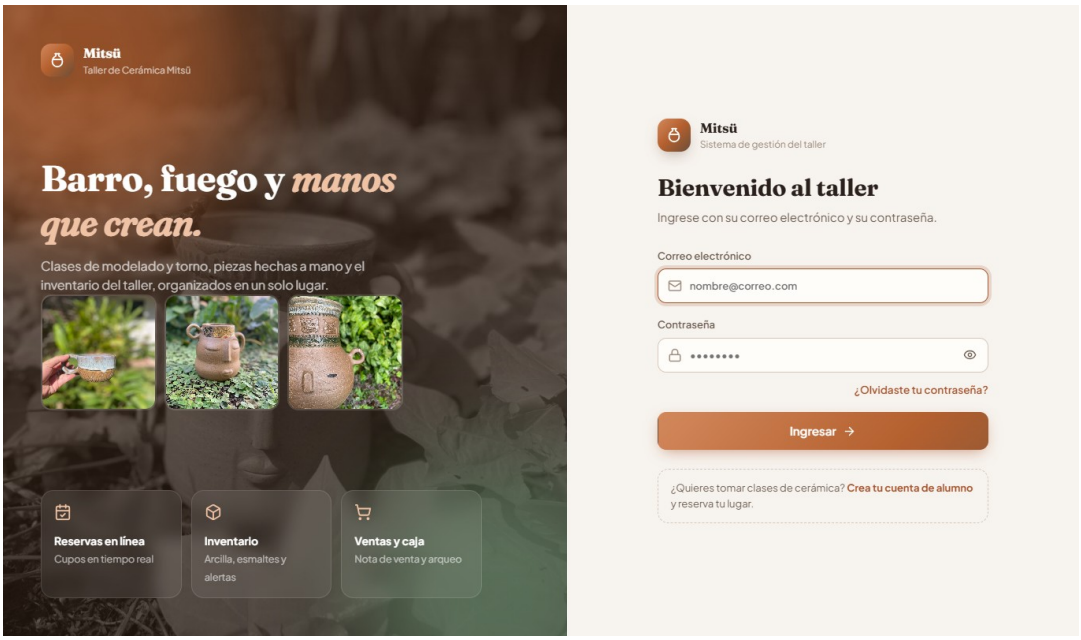
2.4 Prototipar interfaz de usuario

Los prototipos son las pantallas reales del sistema publicado, capturadas en computadora (1366 × 800) y en celular. La interfaz usa la paleta de colores arcilla y crema del taller.

Ciclo I: seguridad y configuración

Figura 34

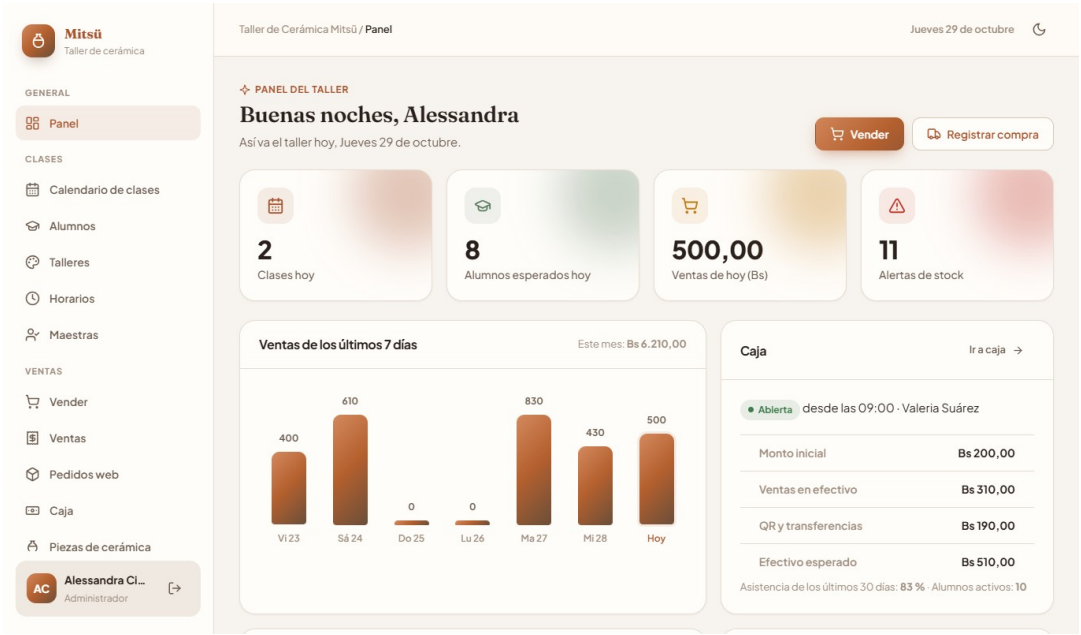
CU01 Pantalla de ingreso



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

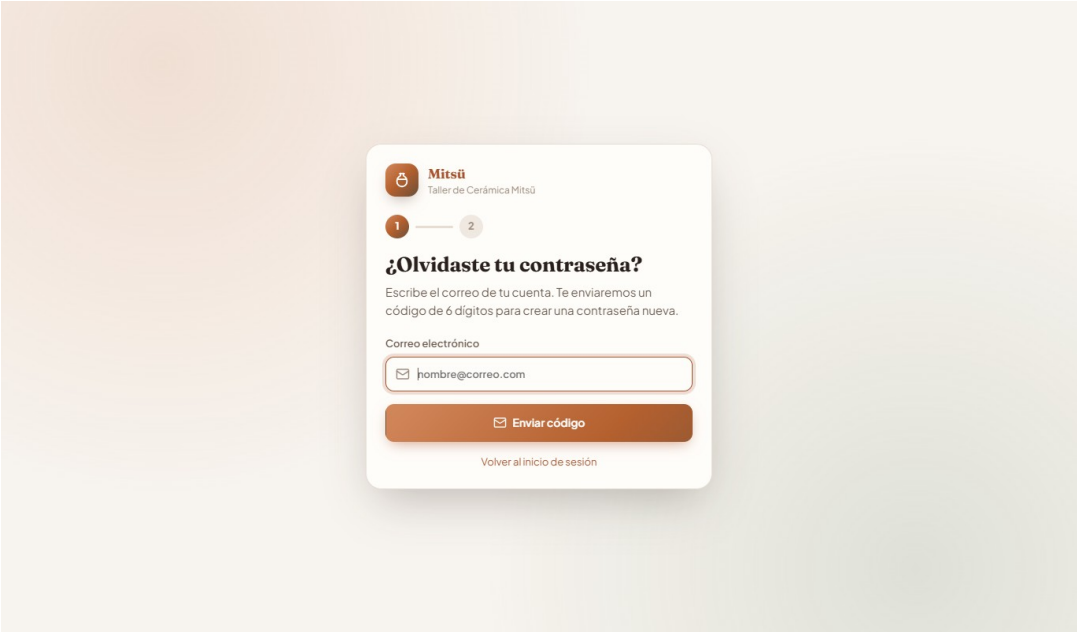
Figura 35

CU01 Panel del administrador después de ingresar



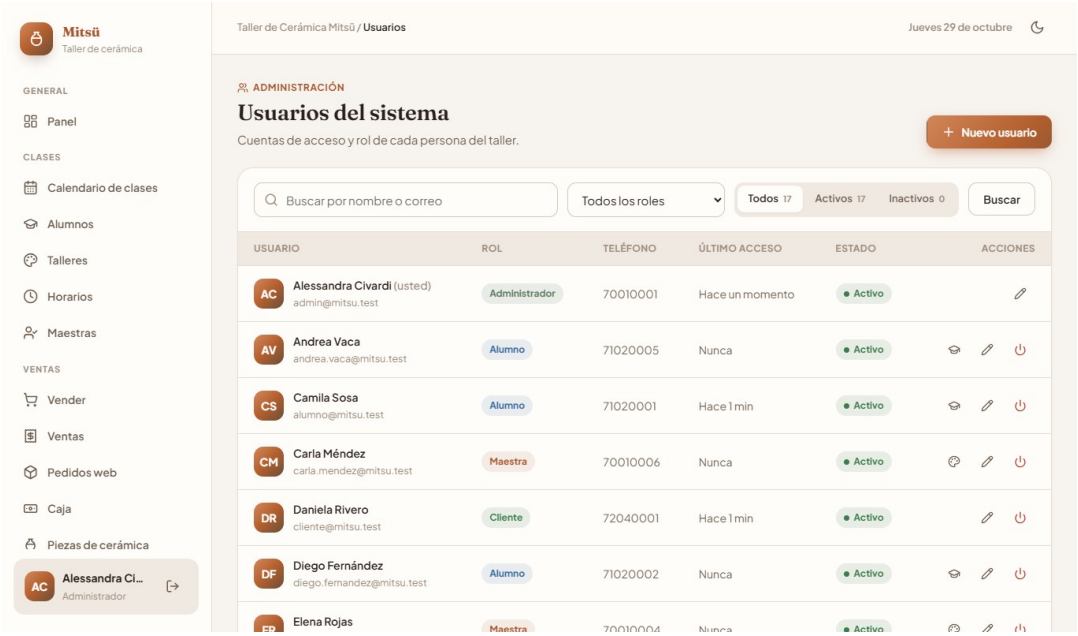
Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 36
CU02 Recuperar contraseña



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

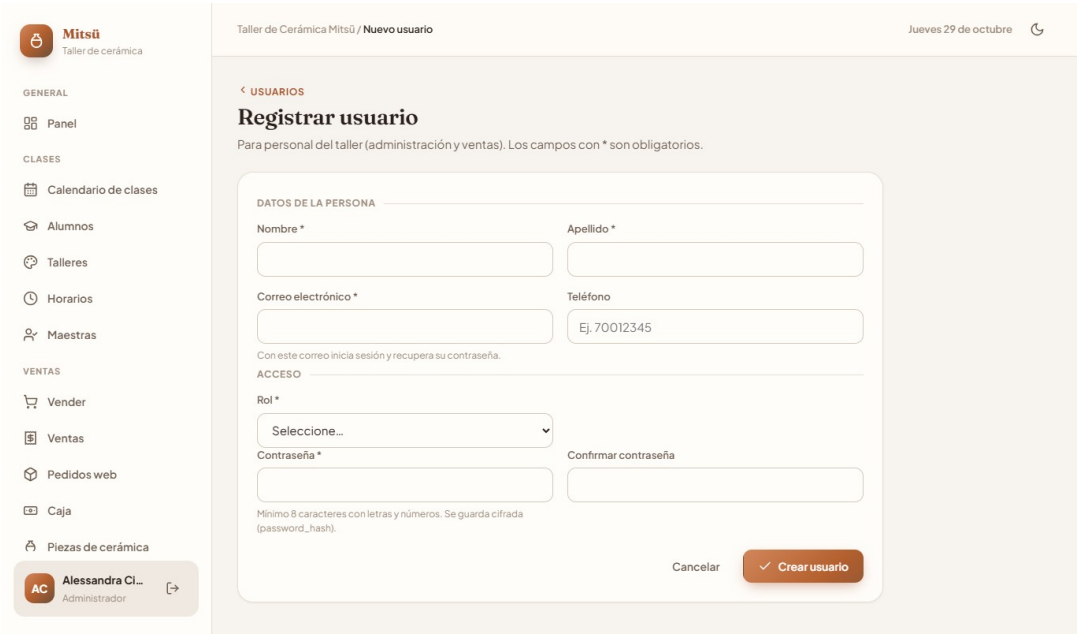
Figura 37
CU03 Lista de usuarios



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 38

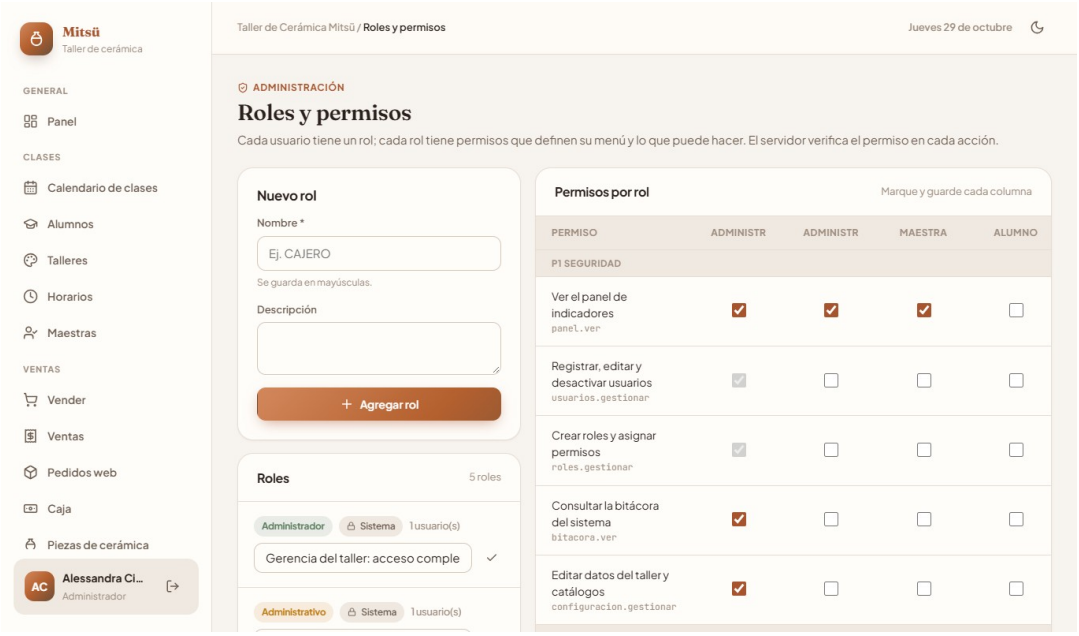
CU03 Formulario de nuevo usuario



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 39

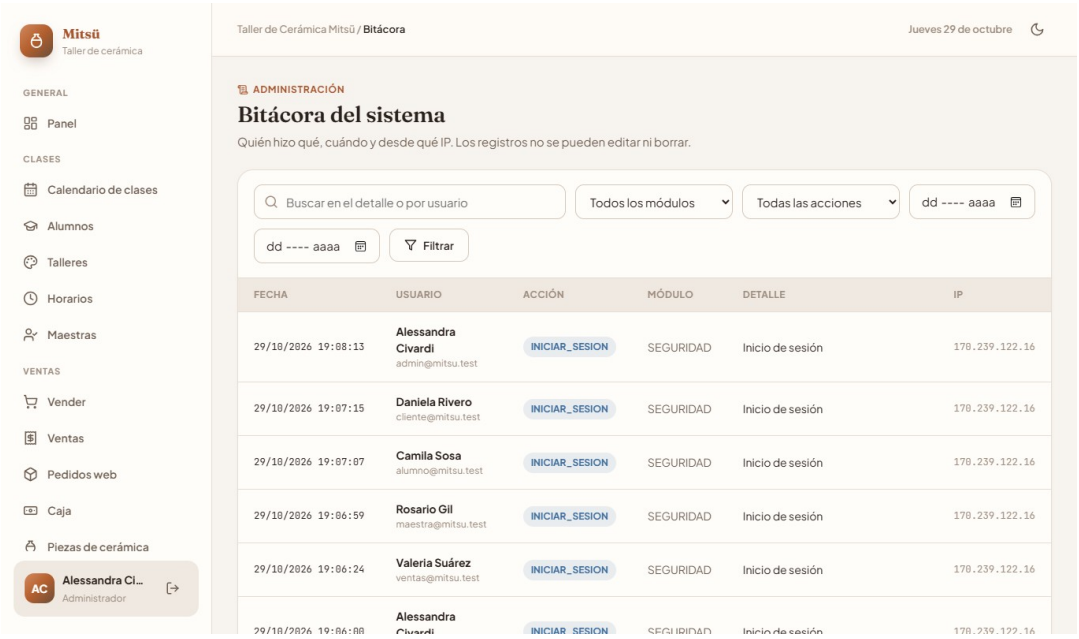
CU04 Roles y matriz de permisos



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 40

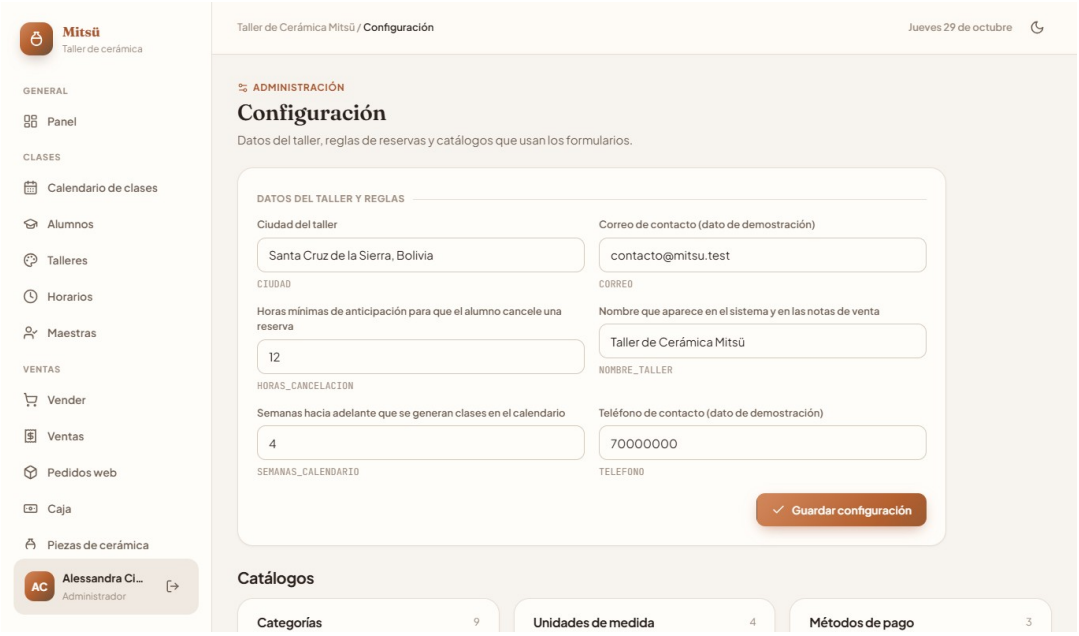
CU05 Bitácora con filtros



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 41

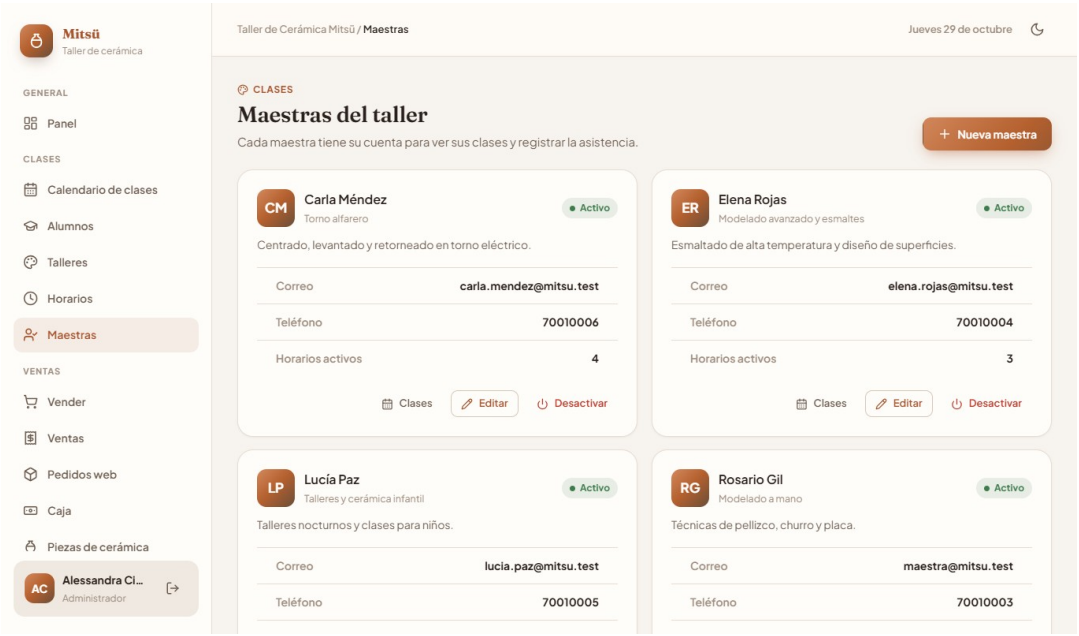
CU06 Configuración y catálogos



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

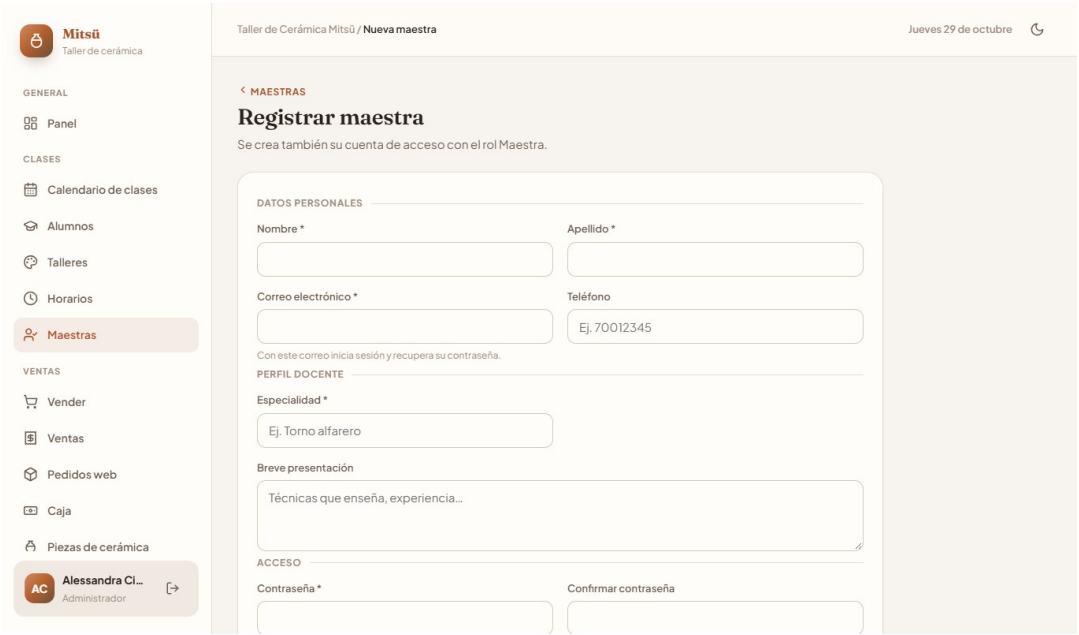
Ciclo II: clases y reservas

Figura 42
CU07 Lista de maestras



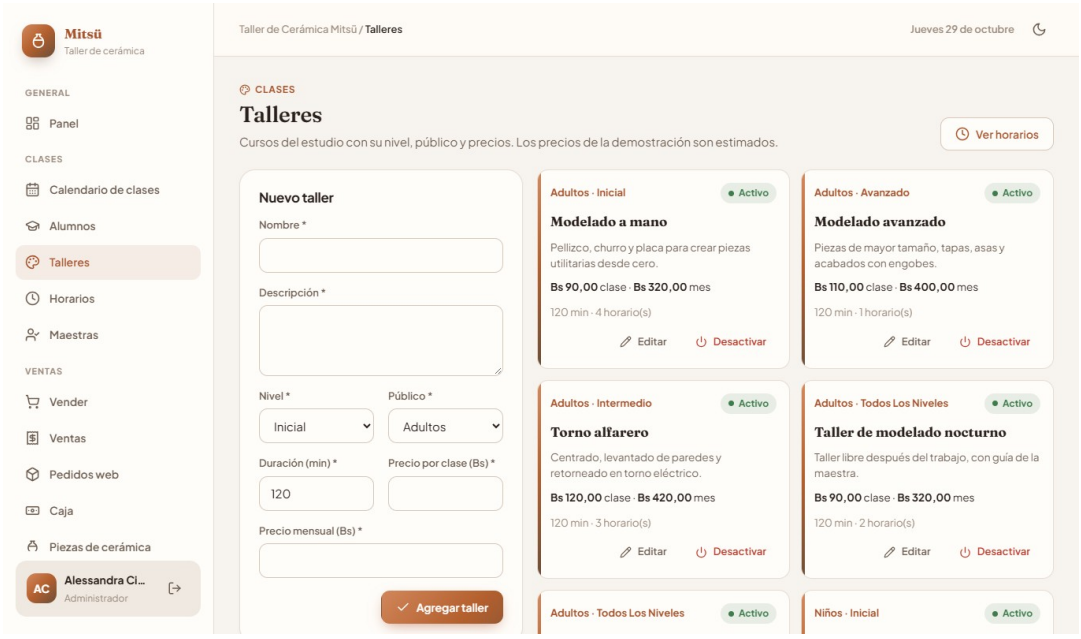
Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 43
CU07 Registro de maestra



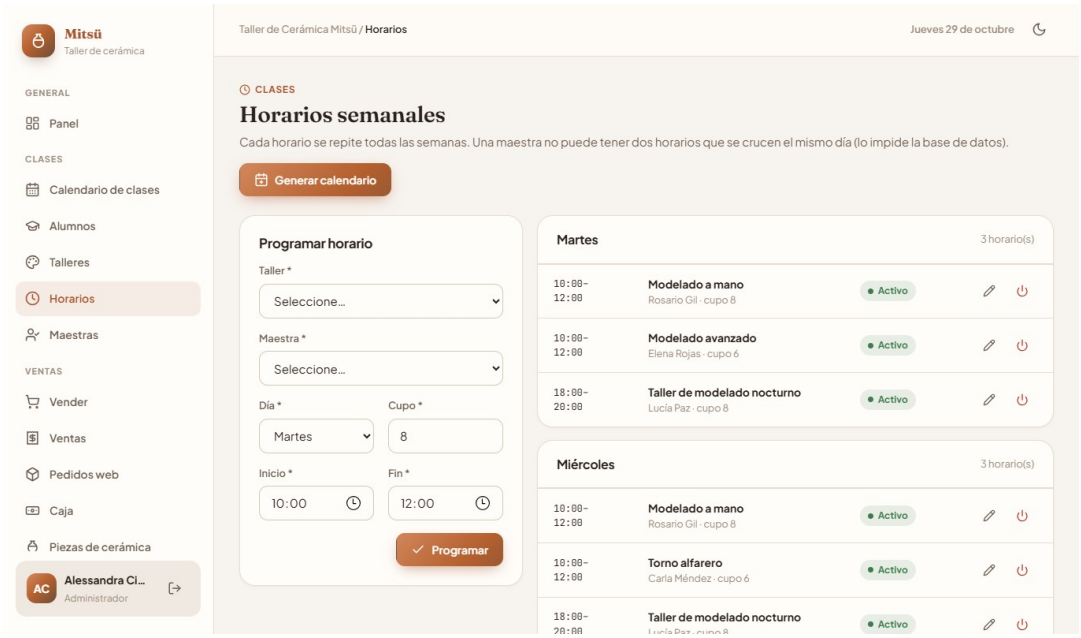
Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 44
CU08 Talleres



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 45
CU09 Horarios semanales



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 46

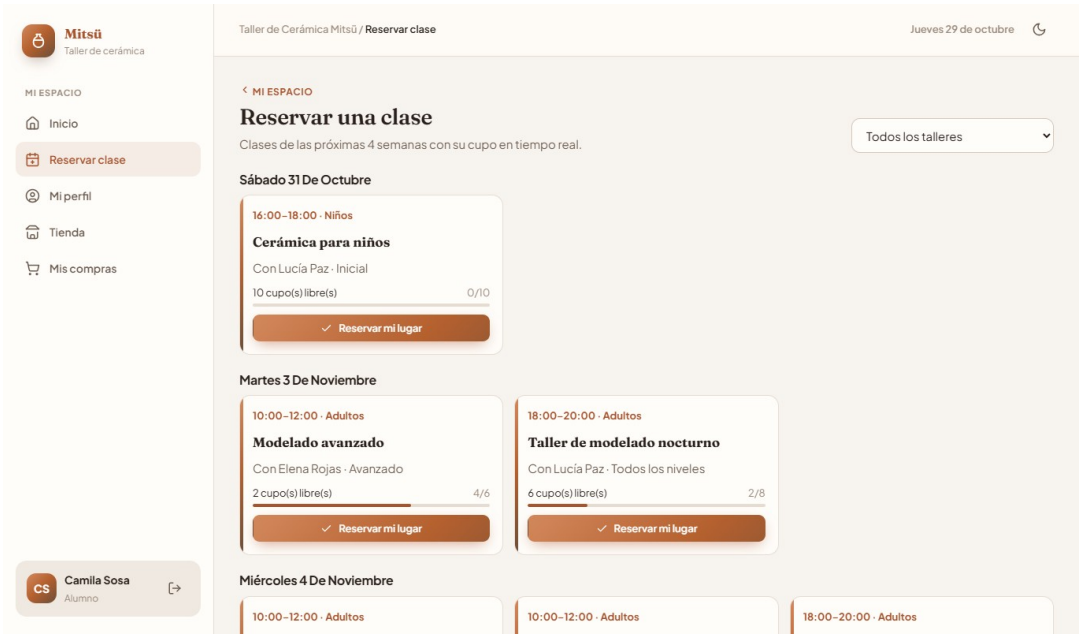
CU09 Calendario de clases



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 47

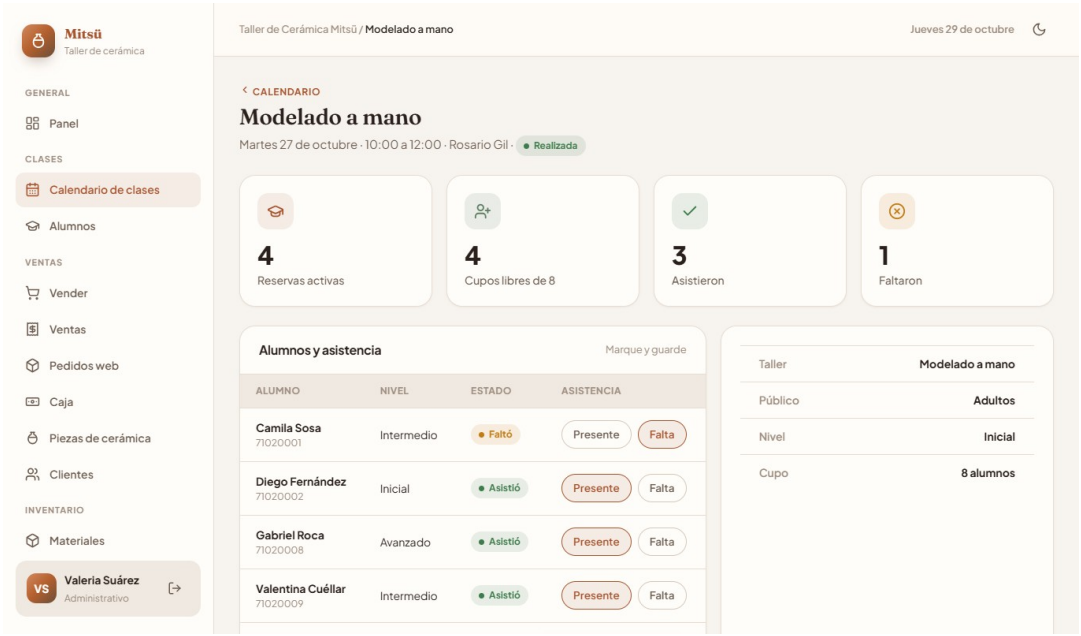
CU10 Portal del alumno: reservar clase



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 48

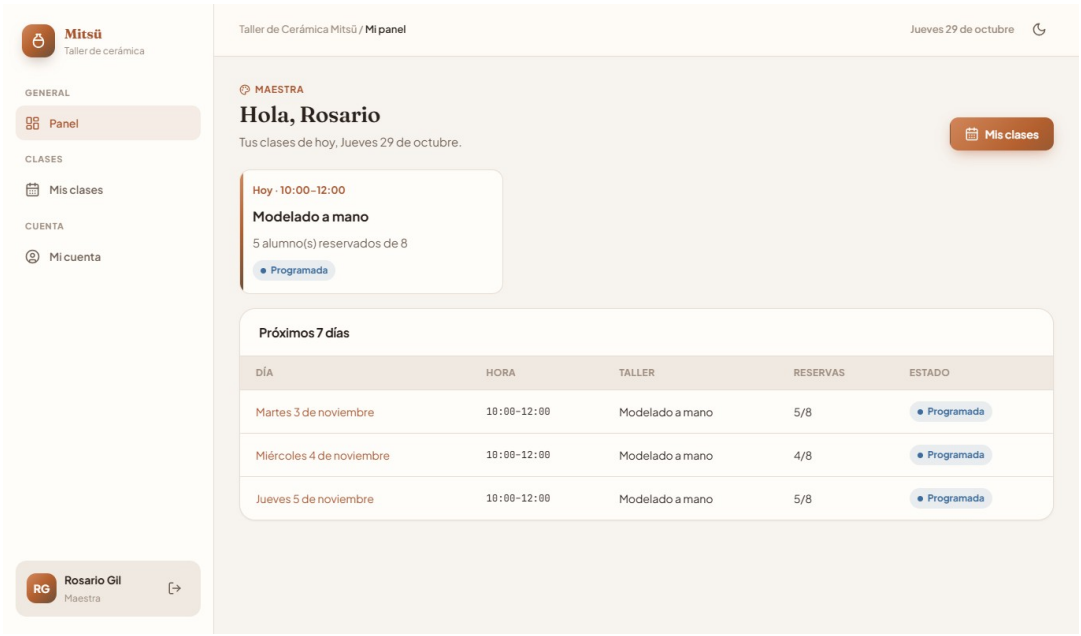
CU10 Detalle de una clase con sus reservas (recepción)



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 49

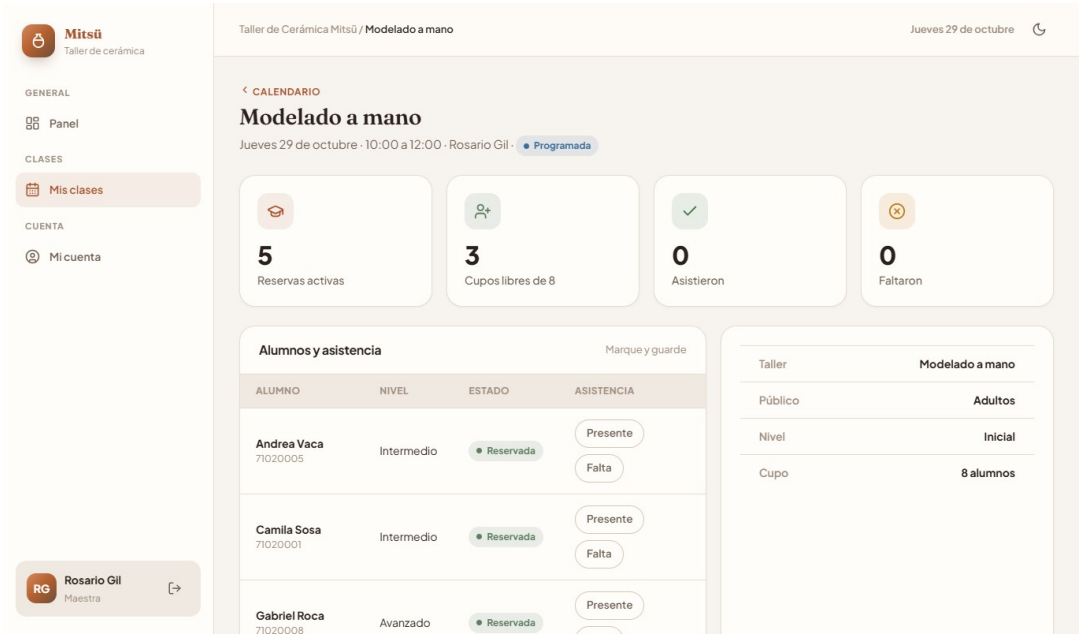
CU11 Panel de la maestra



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 50

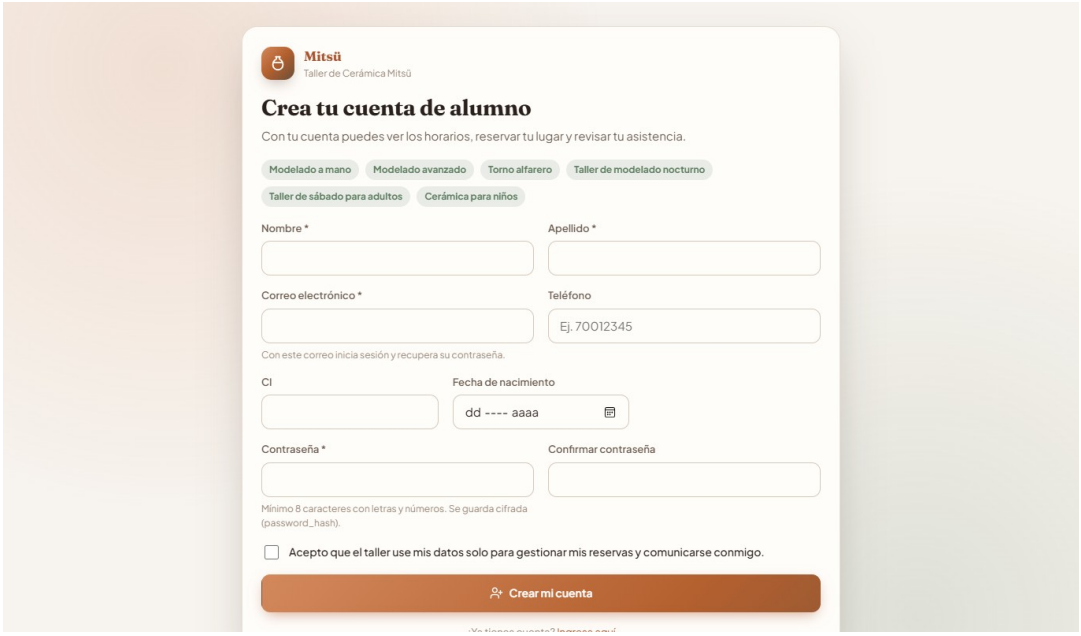
CU11 Registro de asistencia



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

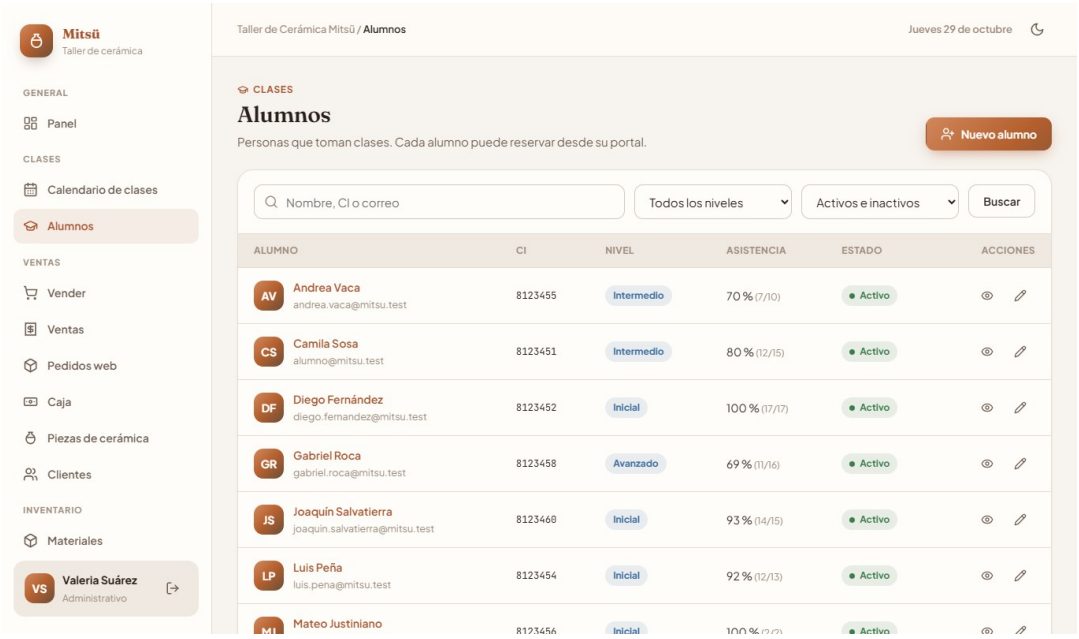
Figura 51

CU12 Autorregistro de alumno



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 52
CU12 Lista de alumnos



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 53
CU12 Portal del alumno

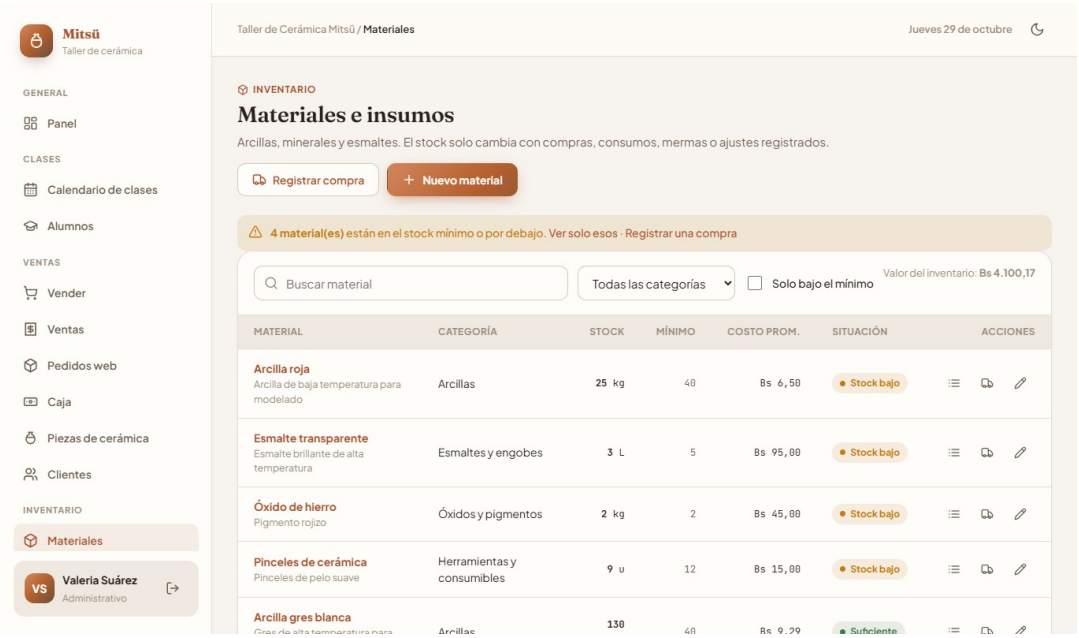


Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Ciclo III: inventario, ventas y tienda en línea

Figura 54

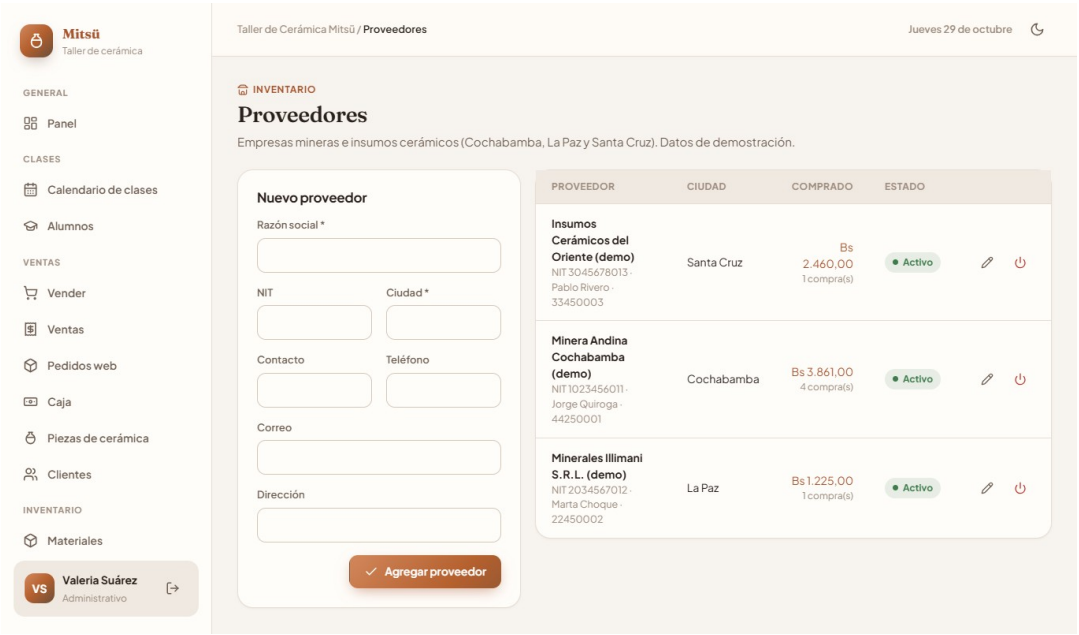
CU13 Materiales con alertas de stock



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

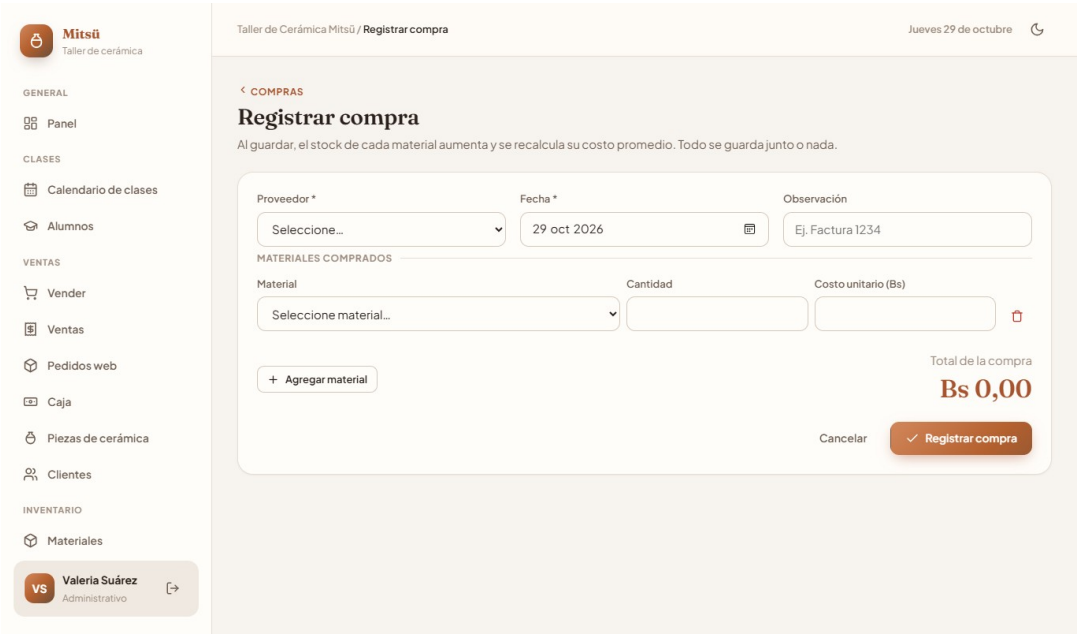
Figura 55

CU14 Proveedores



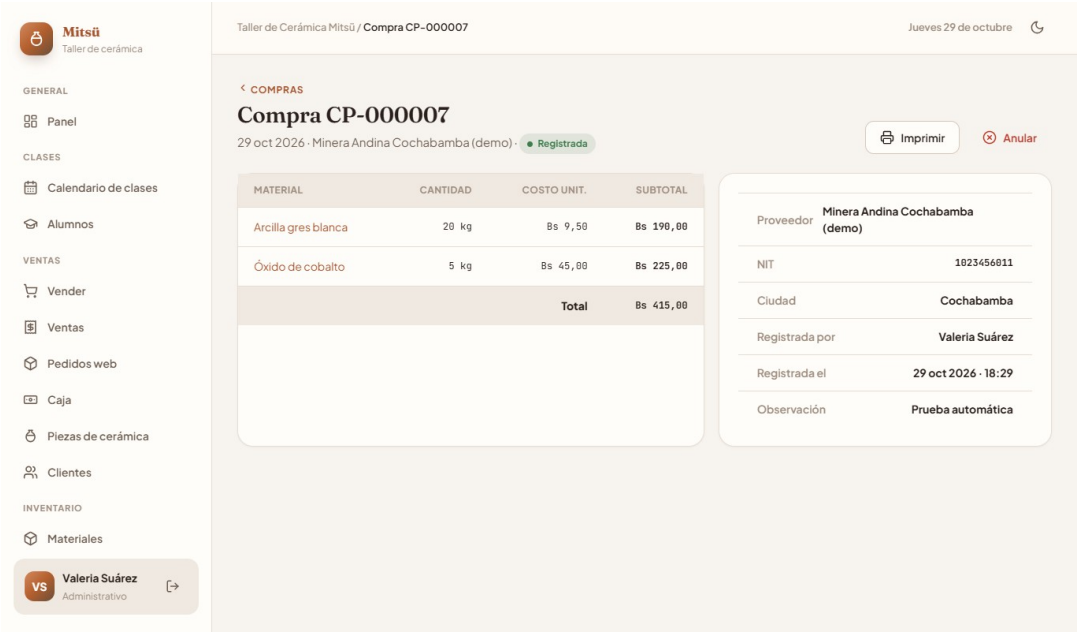
Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 56
CU15 Nueva compra



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

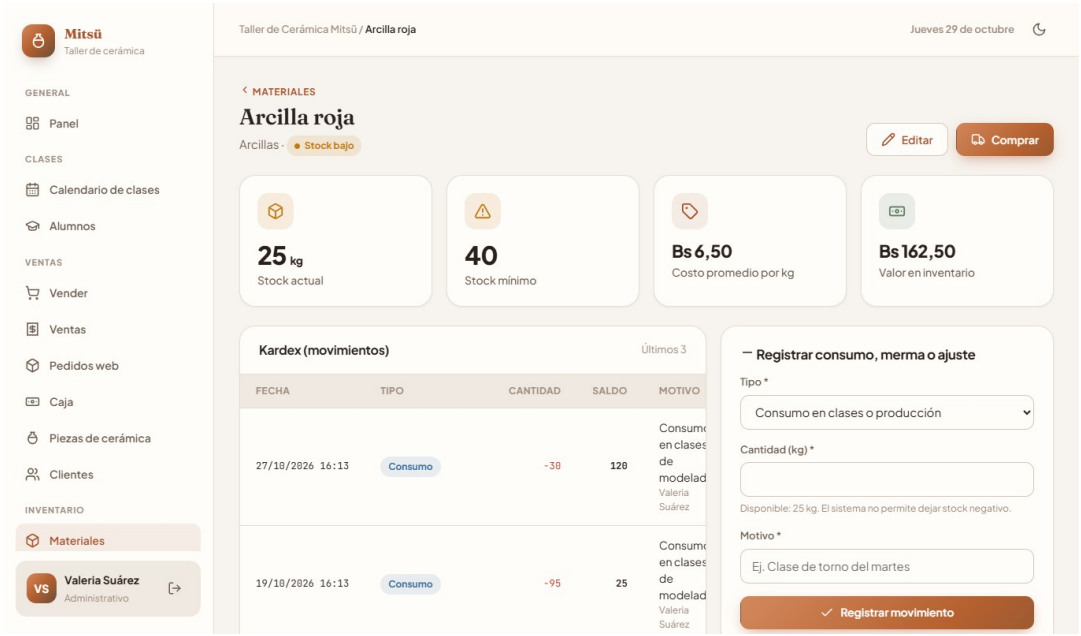
Figura 57
CU15 Comprobante de compra



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 58

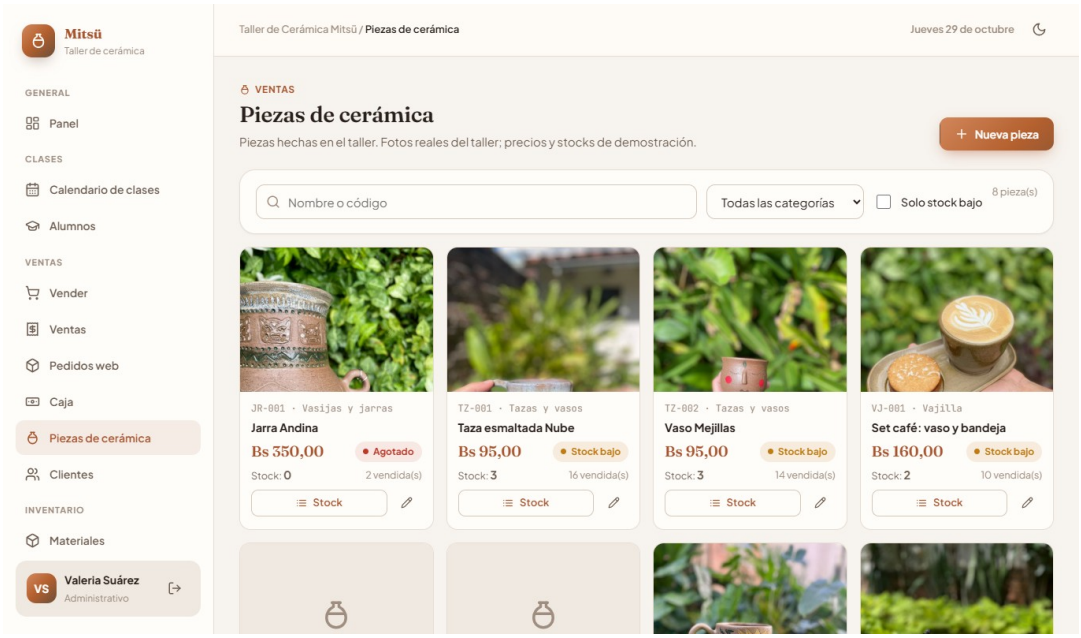
CU16 Kardex de un material



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 59

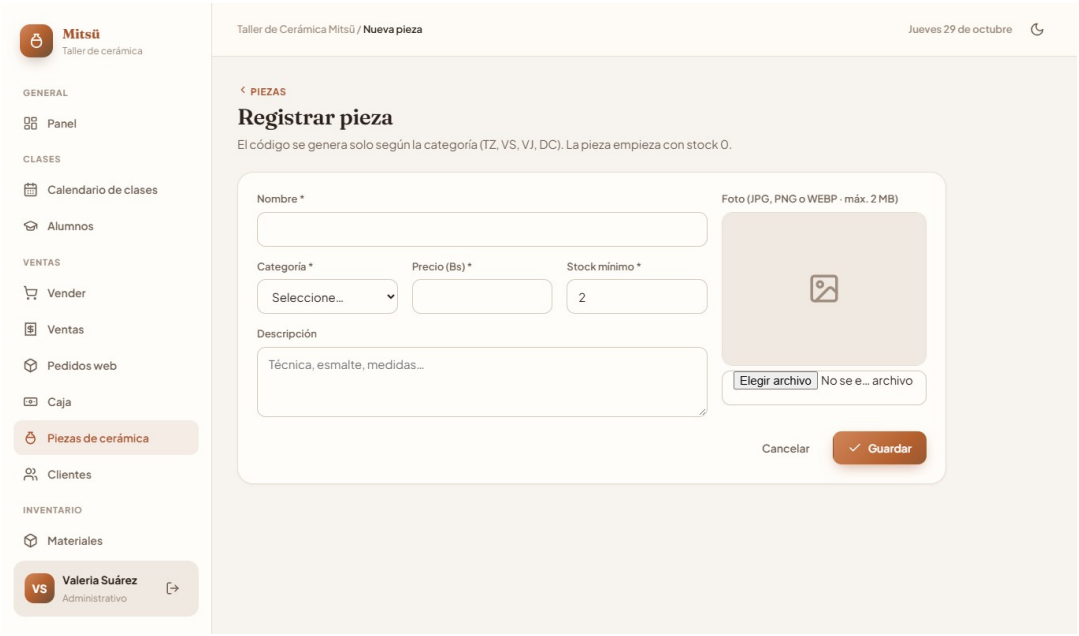
CU17 Piezas de cerámica



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 60

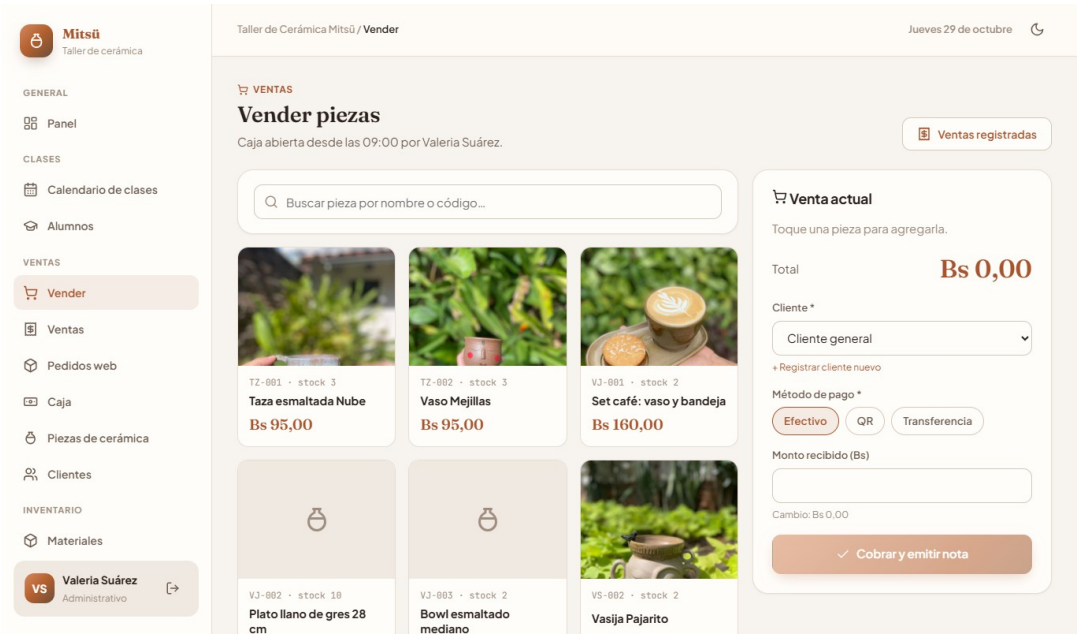
CU17 Formulario de pieza con foto



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

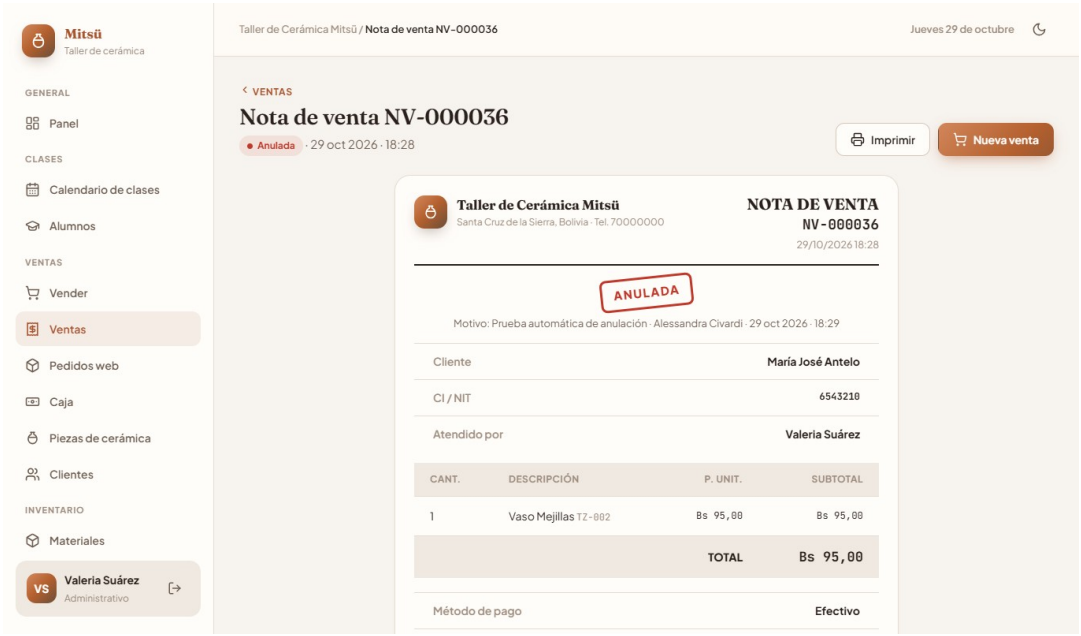
Figura 61

CU18 Punto de venta



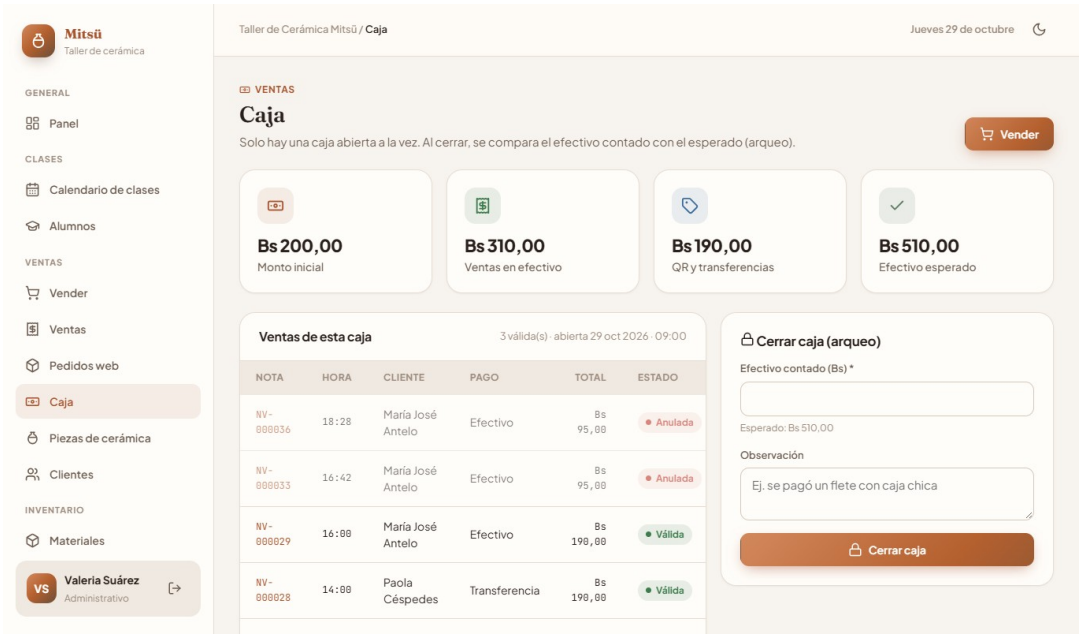
Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 62
CU18 Nota de venta



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 63
CU19 Apertura y cierre de caja



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 64

CU20 Página principal: la tienda se ve primero



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

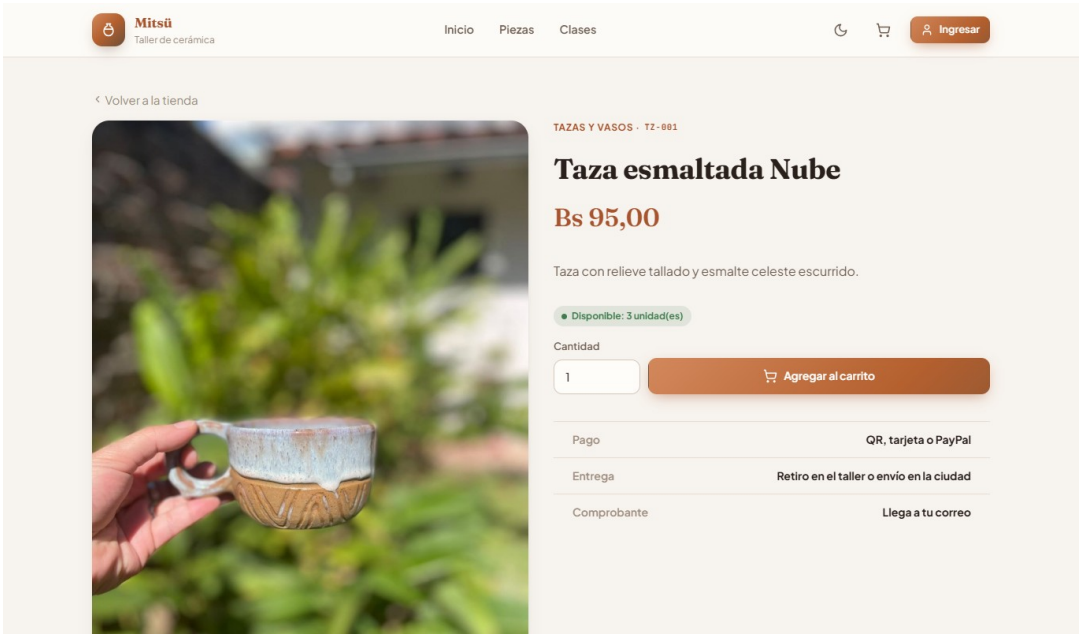
Figura 65
CU20 Tienda en el celular



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 66

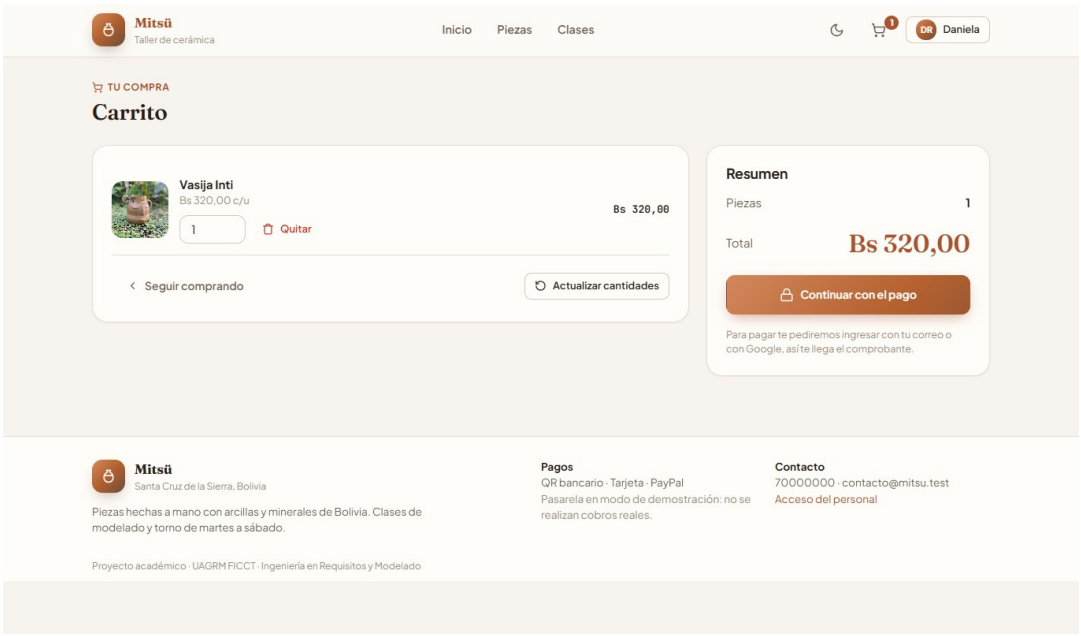
CU20 Detalle de una pieza



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 67

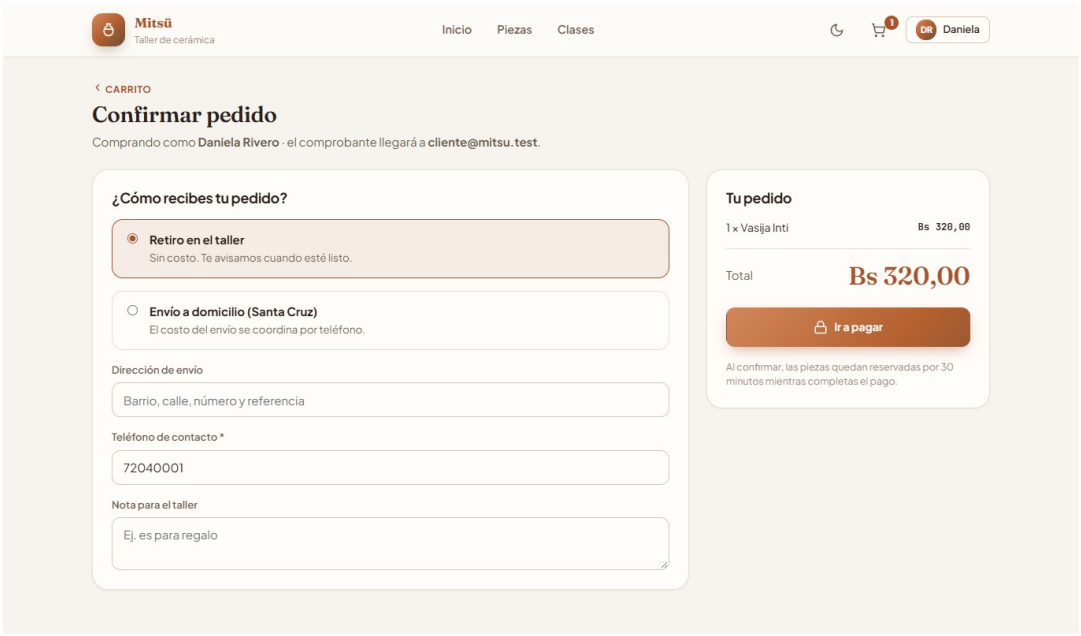
CU20 Carrito



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 68

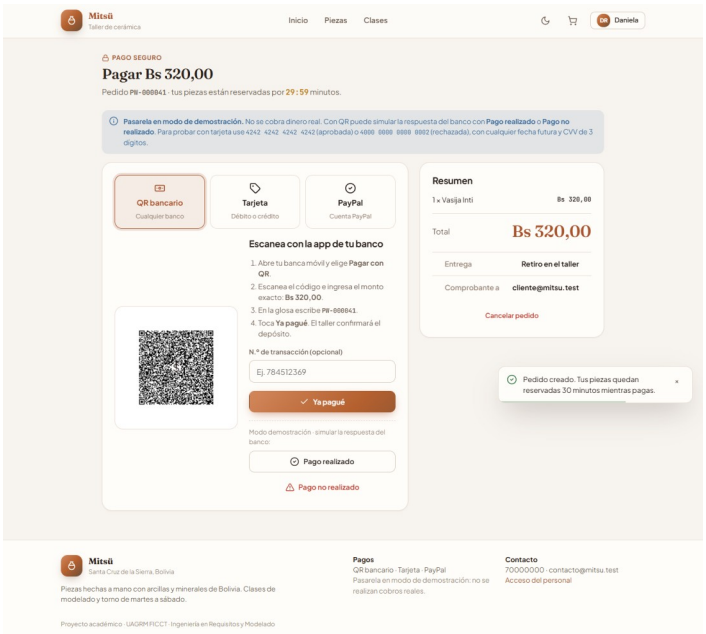
CU20 Confirmar pedido



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 69

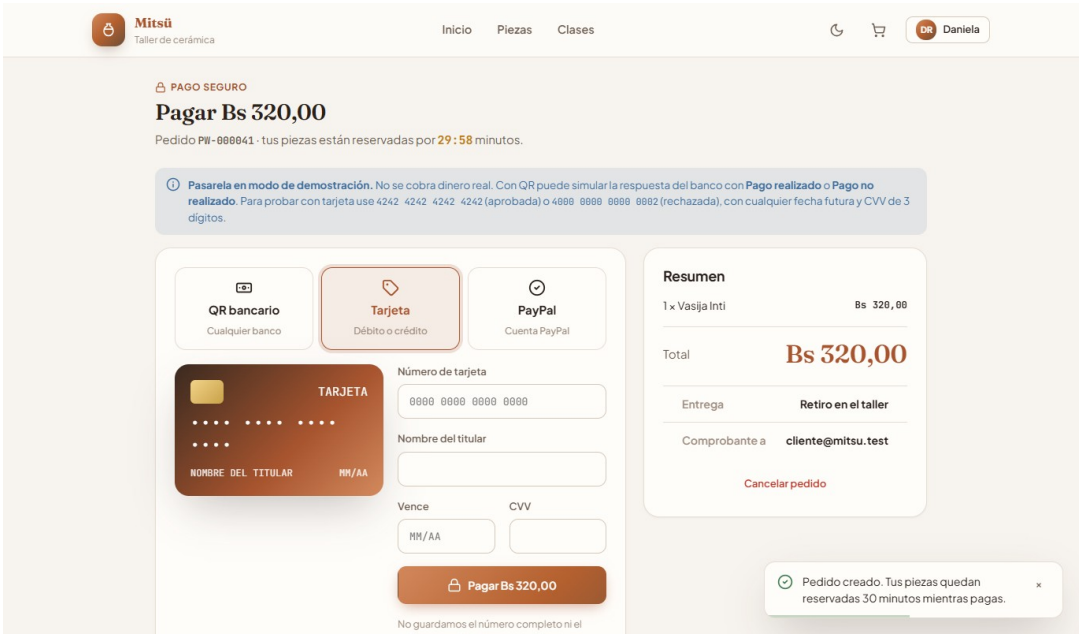
CU21 Pasarela: pago con el QR del taller



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 70

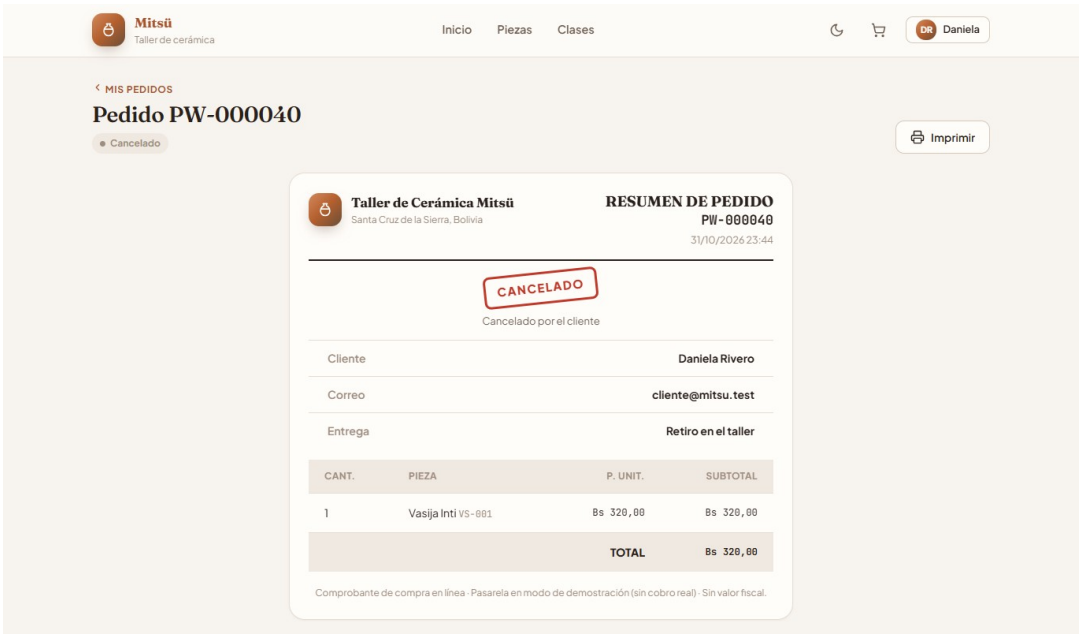
CU21 Pasarela: pago con tarjeta



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

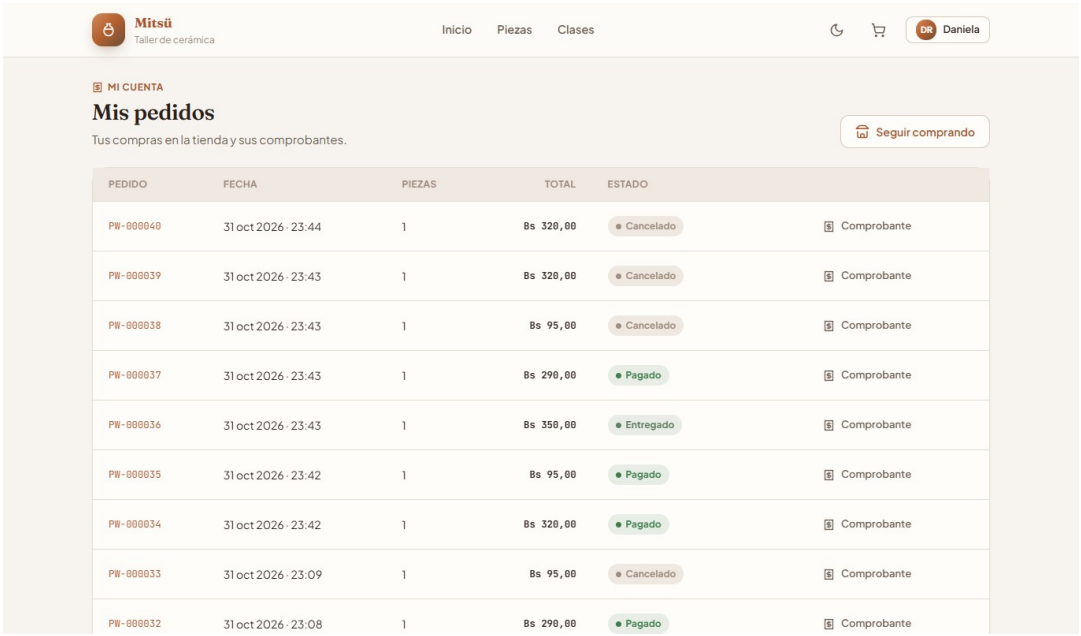
Figura 71

CU21 Comprobante del pedido



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 72
CU20 Mis pedidos



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

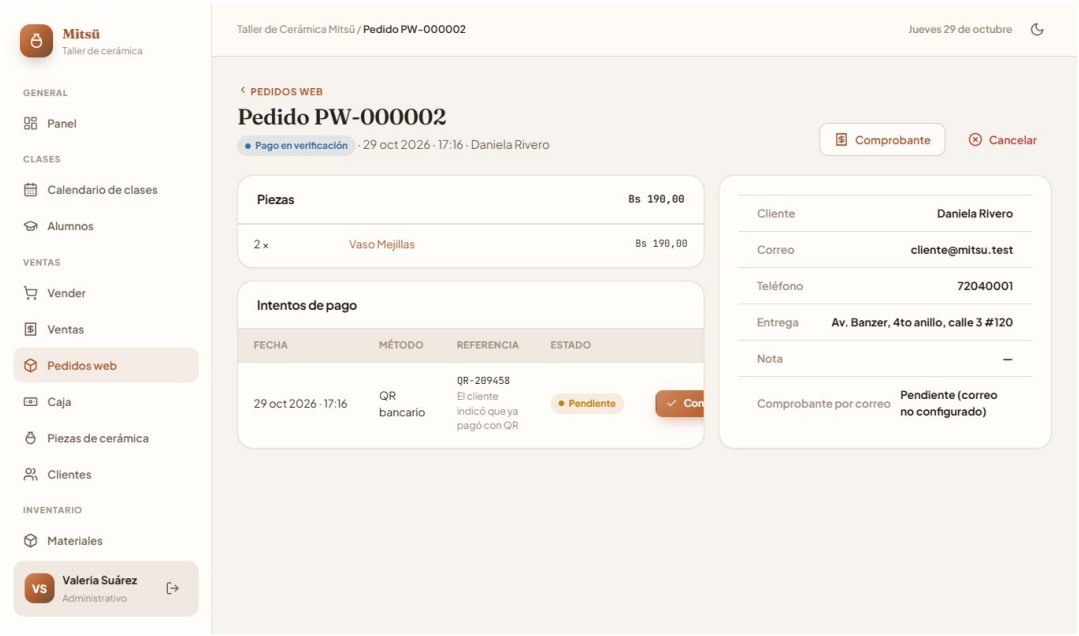
Figura 73
CU22 Pedidos web (taller)



Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

Figura 74

CU22 Detalle del pedido y verificación del QR

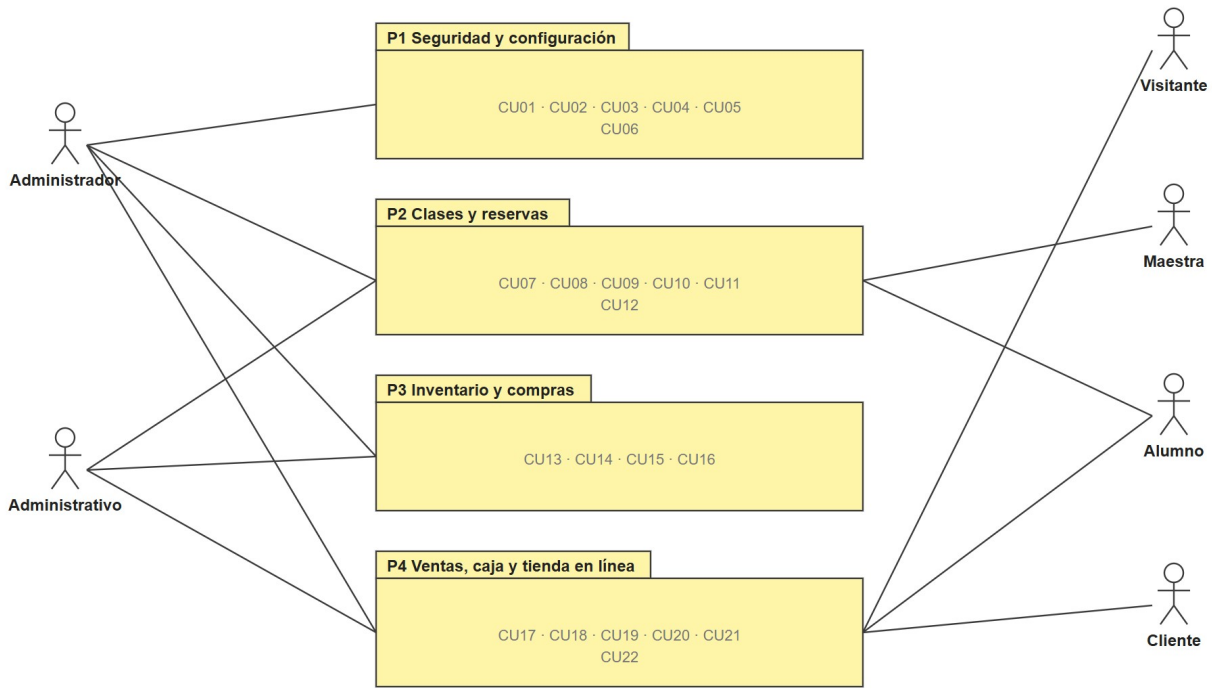


Nota. Captura del sistema publicado en https://taller-mitsu-ceramica.vercel.app.

2.5 Estructurar modelo de casos de uso

Figura 75

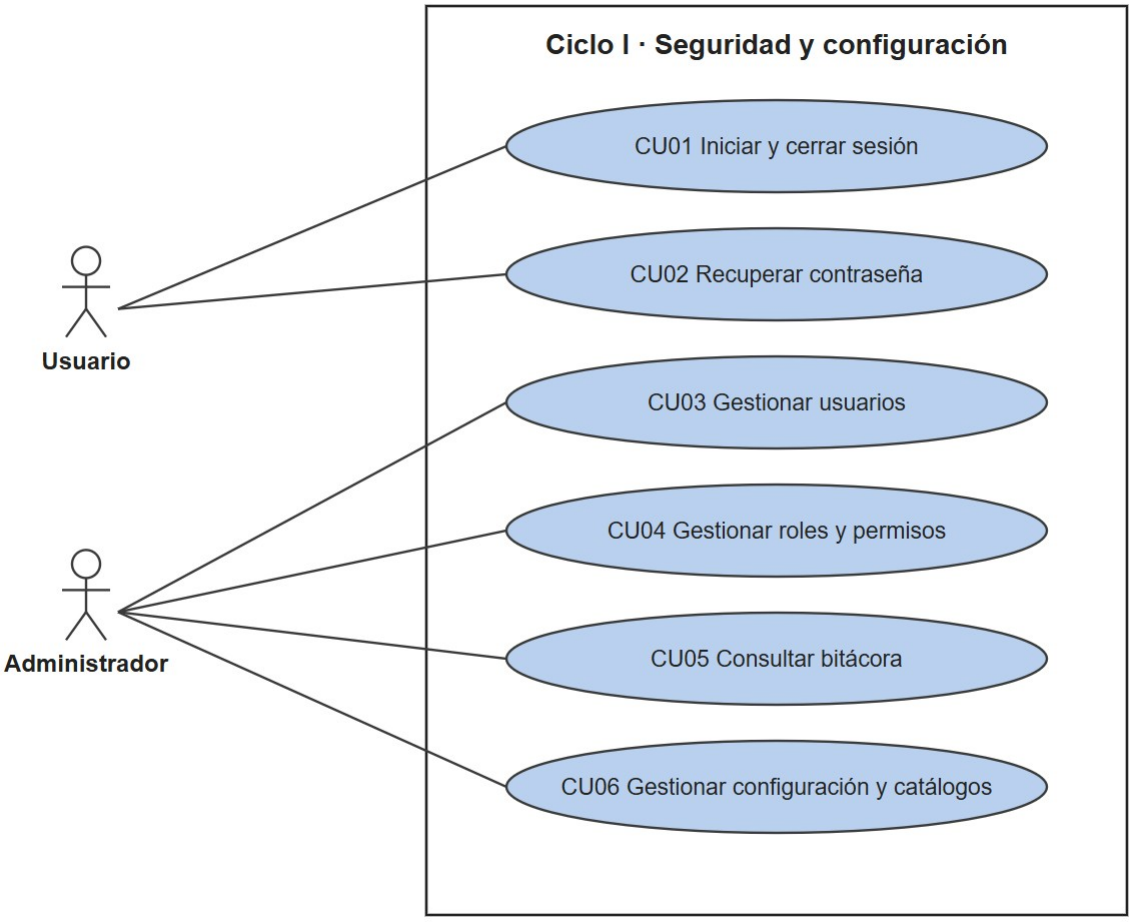
Diagrama general: actores y paquetes



Nota. Elaboración propia.

Ciclo I: seguridad y configuración

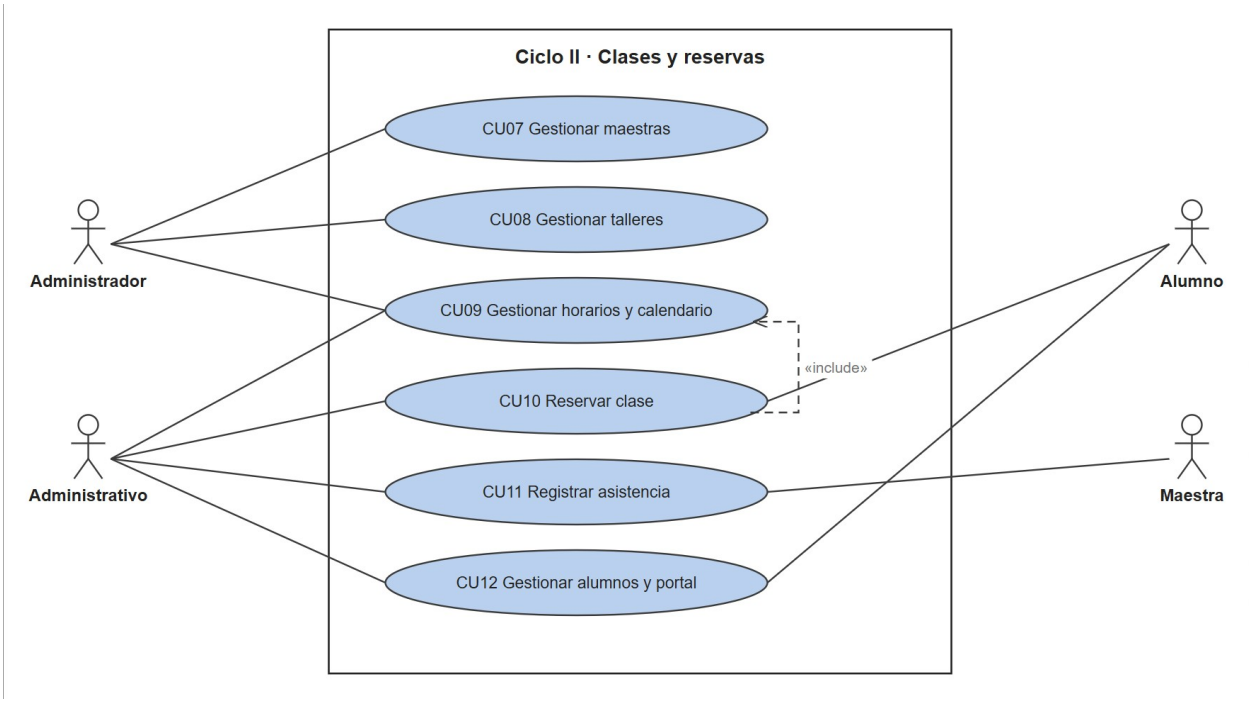
Figura 76
Diagrama de casos de uso del ciclo i



Nota. Elaboración propia.

Ciclo II: clases y reservas

Figura 77
Diagrama de casos de uso del ciclo ii

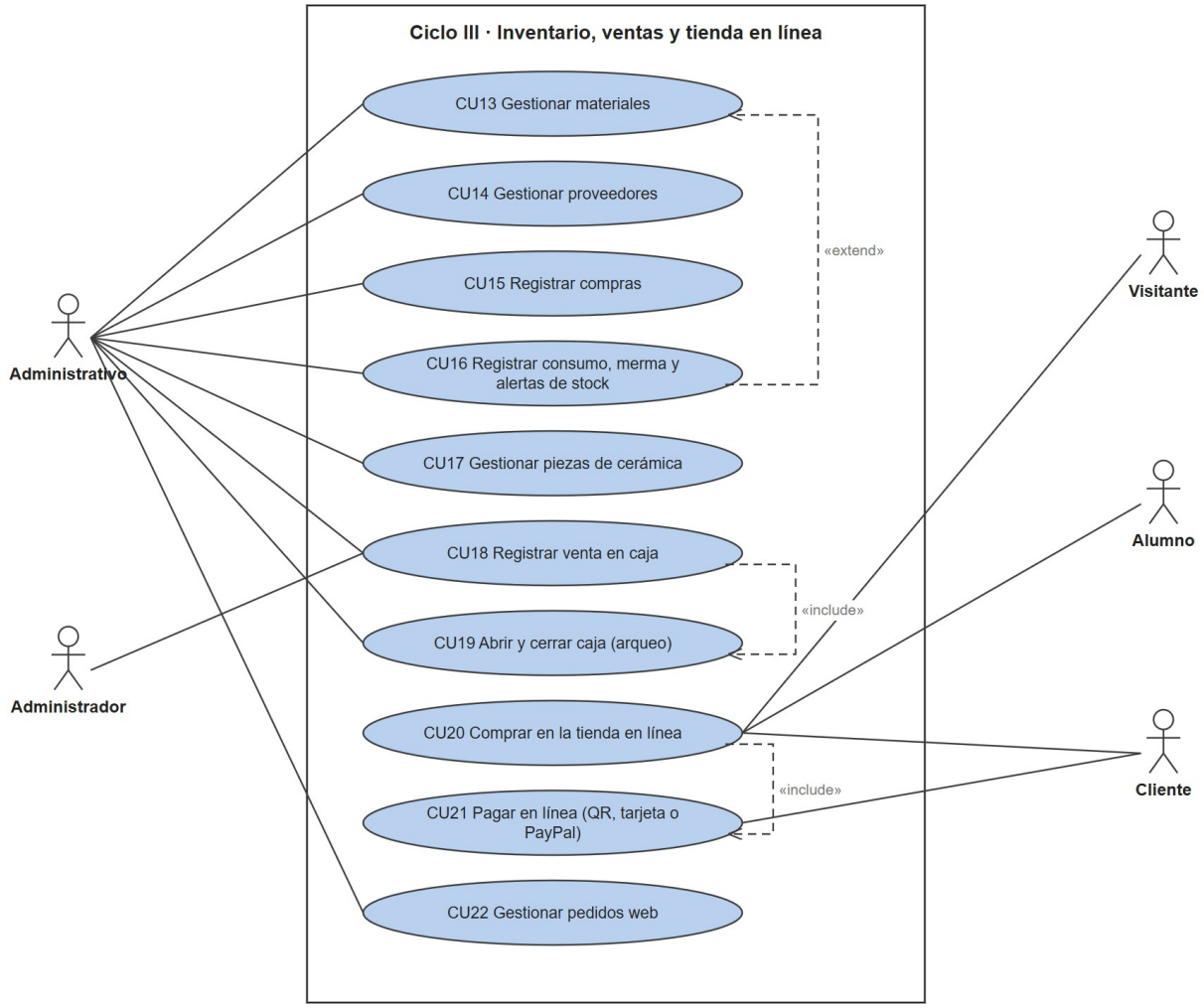


Nota. Elaboración propia.

Ciclo III: inventario, ventas y tienda en línea

Figura 78

Diagrama de casos de uso del ciclo iii



Nota. Elaboración propia.

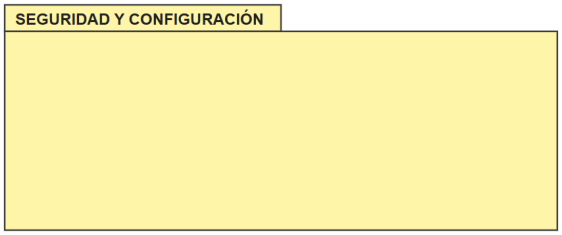
Capítulo 3. Flujo de trabajo: análisis

3.1 Análisis de la arquitectura

3.1.1 Identificar paquetes

Los casos de uso se agrupan en paquetes según el área del taller que atienden. Cada paquete se construye en un ciclo; los paquetes del ciclo IV (cafetería, cobros y reportes) quedan planificados.

Figura 79
Paquete P1 Seguridad y configuración

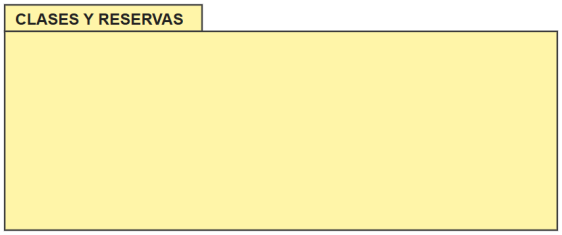


Nota. Elaboración propia.

P1 SEGURIDAD Y CONFIGURACIÓN

Gestiona el acceso al sistema y su configuración: inicio y cierre de sesión, recuperación de contraseña, usuarios, roles con su matriz de permisos, bitácora de auditoría y catálogos generales (categorías, unidades y métodos de pago). Garantiza que cada persona vea y haga solo lo que su rol permite.

Figura 80
Paquete P2 Clases y reservas



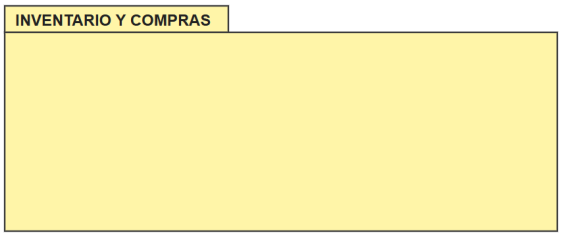
Nota. Elaboración propia.

P2 CLASES Y RESERVAS

Administra la enseñanza: maestras, talleres, horarios semanales, generación del calendario de clases, reservas con control de cupo, registro de asistencia y alumnos con su portal. Reemplaza la hoja de Excel de horarios y el cuaderno de reservas.

Figura 81

Paquete P3 Inventario y compras



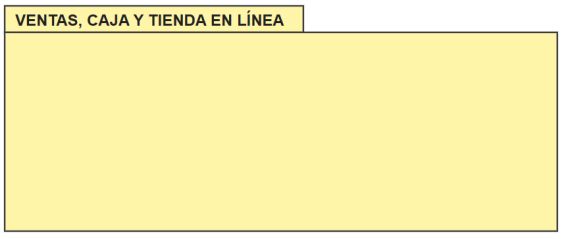
Nota. Elaboración propia.

P3 INVENTARIO Y COMPRAS

Controla los insumos de producción: materiales con stock mínimo, proveedores, compras con costo promedio y el kardex de entradas, consumos, mermas y ajustes. Evita quedarse sin arcilla o esmalte en plena producción.

Figura 82

Paquete P4 Ventas, caja y tienda en línea



Nota. Elaboración propia.

P4 VENTAS, CAJA Y TIENDA EN LÍNEA

Reúne todo lo que genera ingresos: piezas de cerámica con foto, venta en caja con nota numerada, apertura y cierre de caja con arqueo, tienda en línea con carrito, pasarela de pago (QR, tarjeta o PayPal) y gestión de pedidos web.

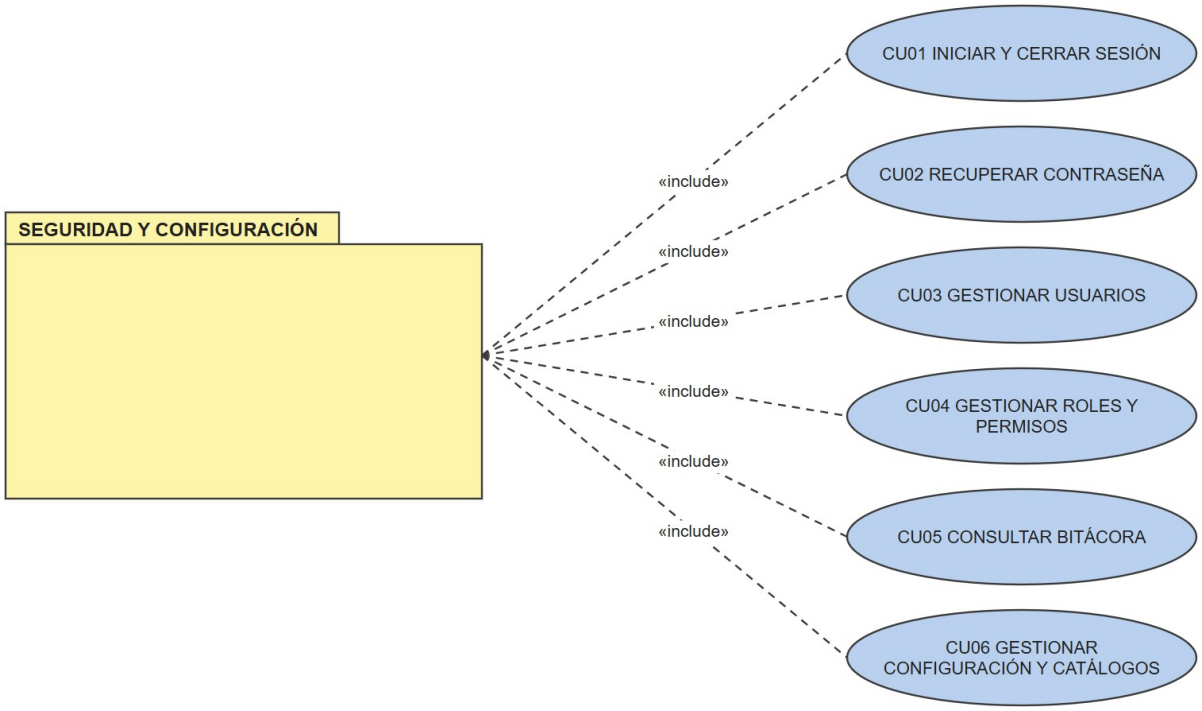
Tabla 47
Resumen de paquetes de análisis

Paquete	Responsabilidad	Ciclo
P1 Seguridad y configuración	Acceso, usuarios, roles, permisos, bitácora, catálogos	I
P2 Clases y reservas	Maestras, talleres, horarios, calendario, reservas, asistencia, alumnos	II
P3 Inventario y compras	Materiales, proveedores, compras, kardex y alertas	III
P4 Ventas, caja y tienda en línea	Piezas, clientes, ventas, caja, tienda, pedidos y pagos	III
P5 Cafetería · P6 Cobros · P7 Reportes	Menú, comandas, mensualidades, reportes	IV (planificado)

3.1.2 Relacionar paquetes y casos de uso

Cada paquete incluye los casos de uso que le pertenecen. Los diagramas siguientes muestran, para cada paquete, sus casos de uso unidos con la relación «include».

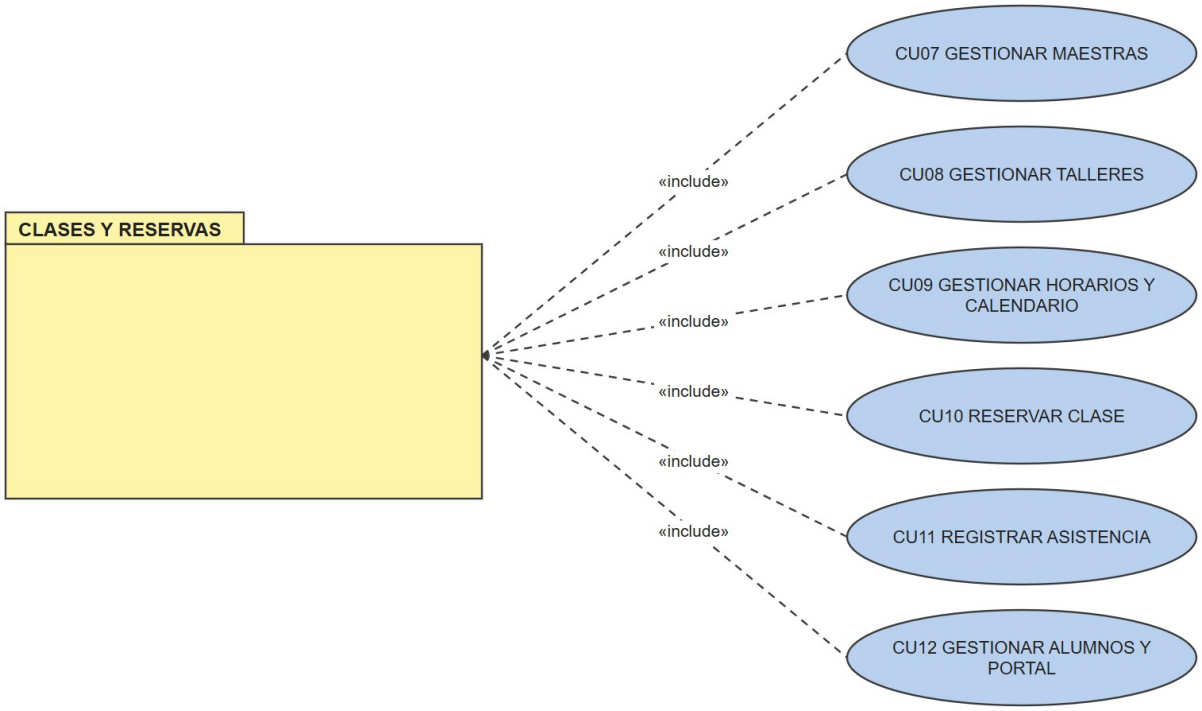
Figura 83
Paquete P1 Seguridad y configuración y sus casos de uso



Nota. Elaboración propia.

Figura 84

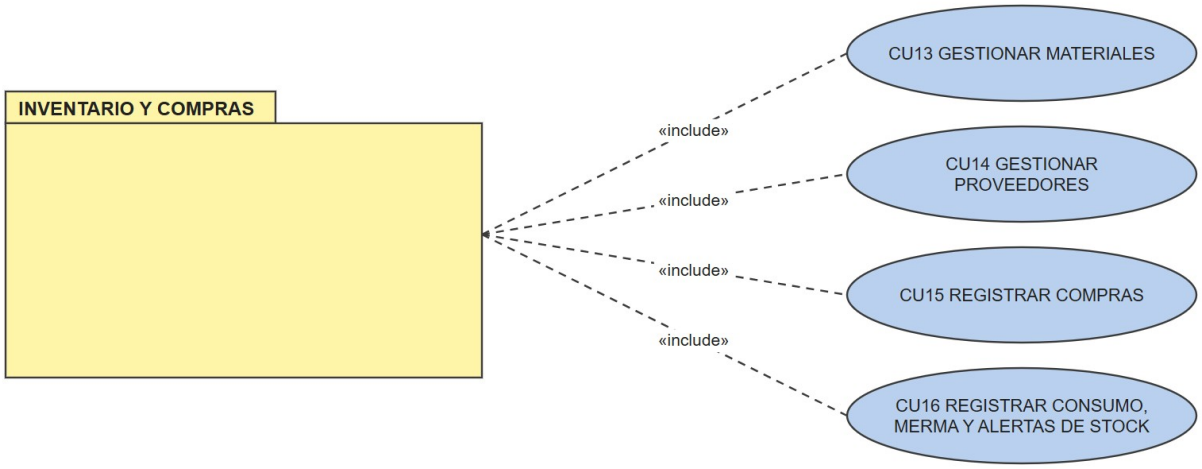
Paquete P2 Clases y reservas y sus casos de uso



Nota. Elaboración propia.

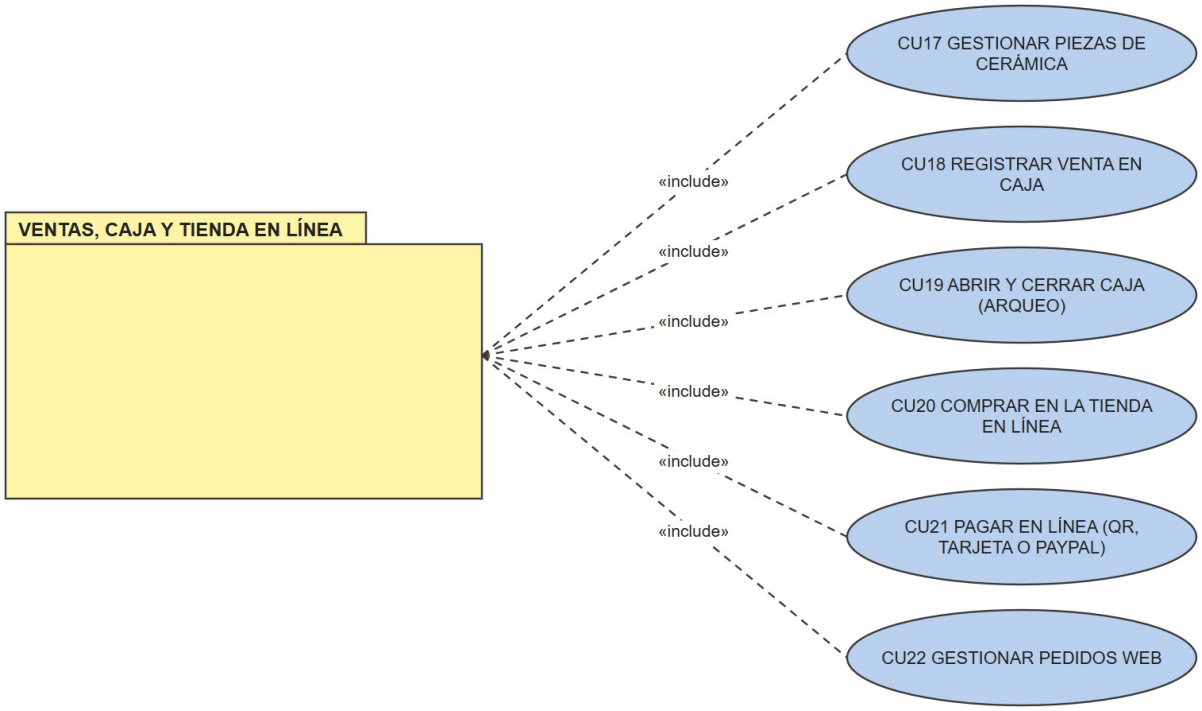
Figura 85

Paquete P3 Inventario y compras y sus casos de uso



Nota. Elaboración propia.

Figura 86
Paquete P4 Ventas, caja y tienda en línea y sus casos de uso



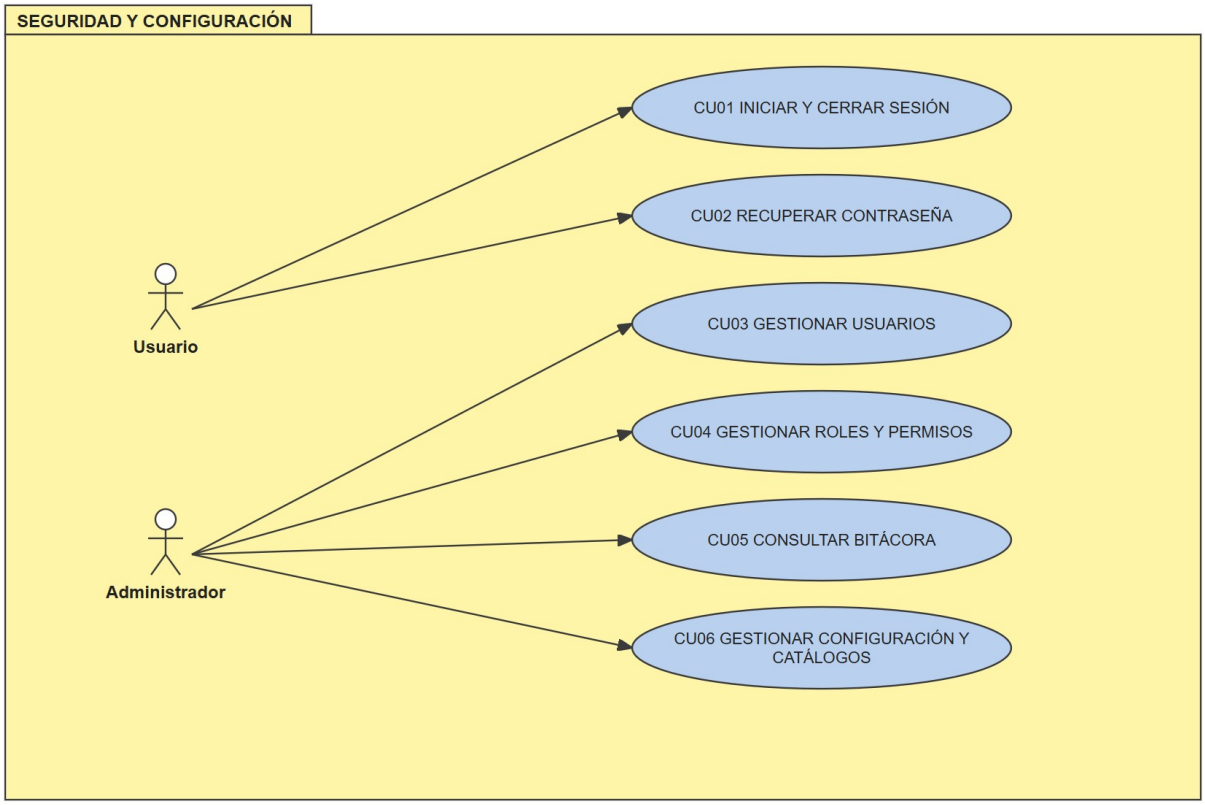
Nota. Elaboración propia.

3.1.3 Vista de paquetes

La vista de cada paquete muestra, dentro del paquete, los actores que participan y los casos de uso que ejecutan, incluidas las relaciones «include» y «extend» entre casos de uso del mismo paquete.

Figura 87

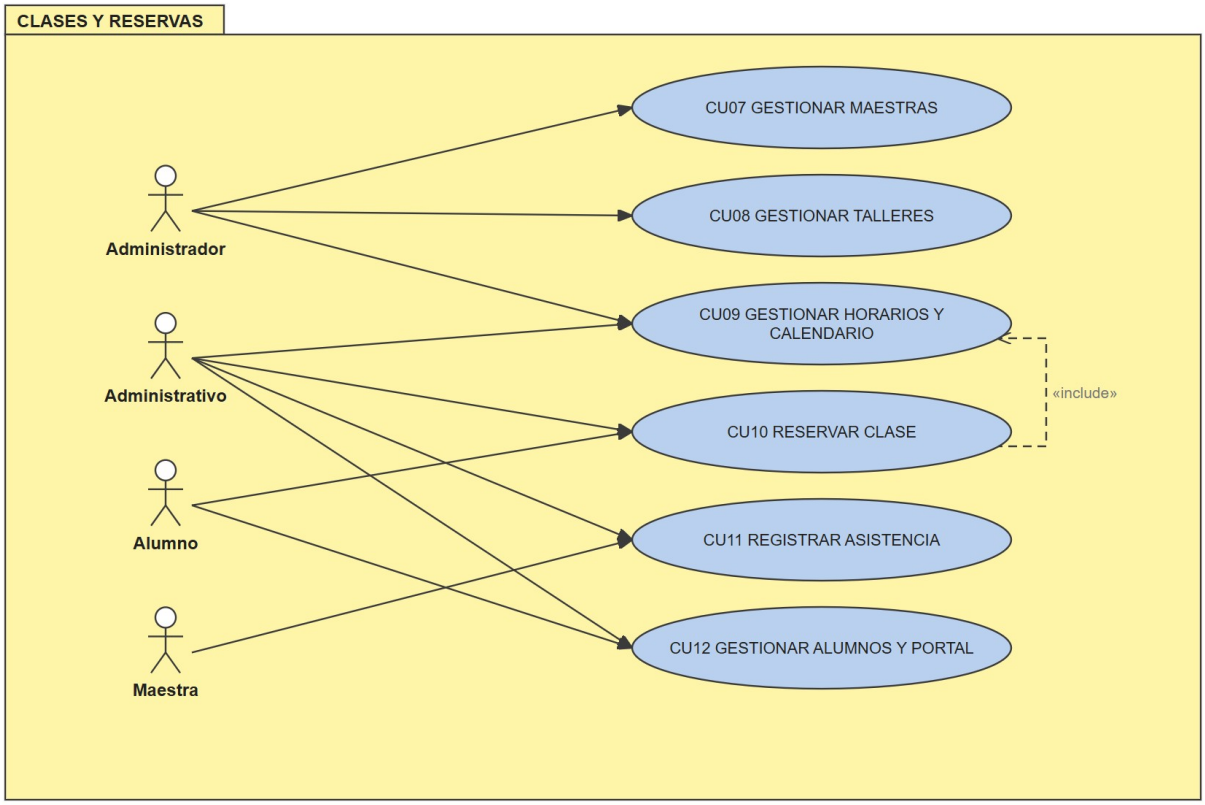
Vista del paquete P1 Seguridad y configuración



Nota. Elaboración propia.

Figura 88

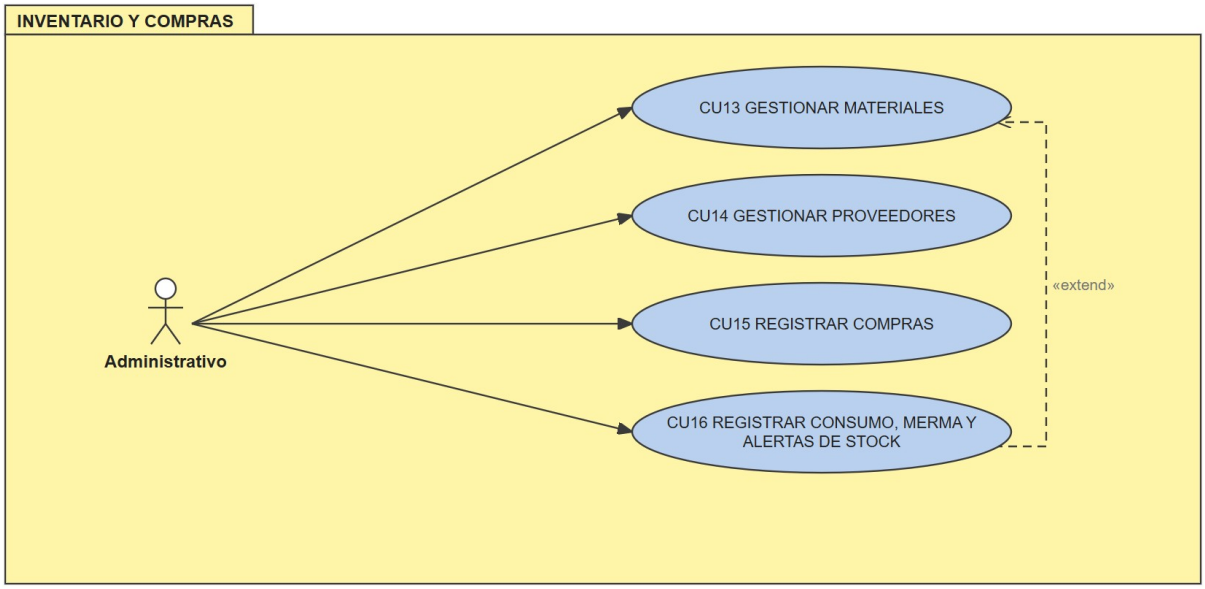
Vista del paquete P2 Clases y reservas



Nota. Elaboración propia.

Figura 89

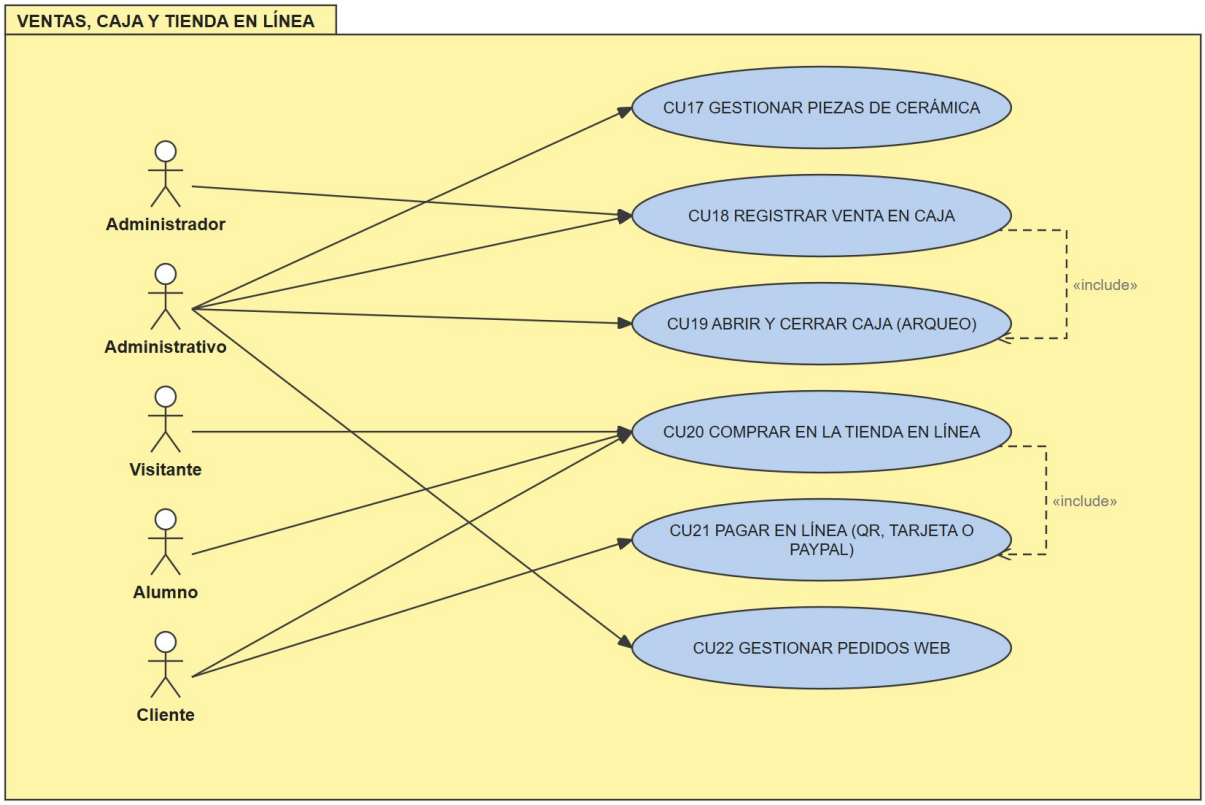
Vista del paquete P3 Inventario y compras



Nota. Elaboración propia.

Figura 90

Vista del paquete P4 Ventas, caja y tienda en línea



Nota. Elaboración propia.

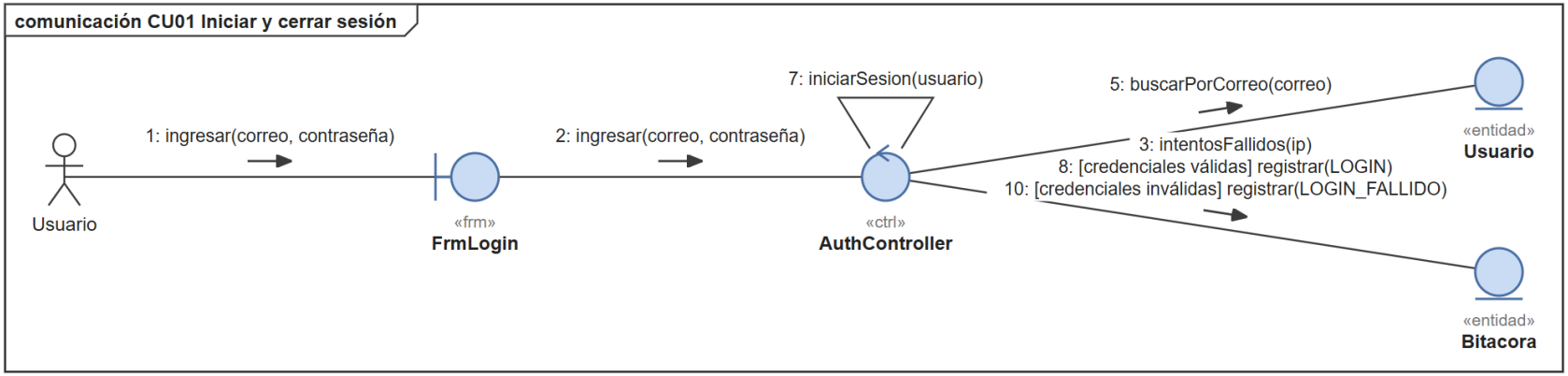
3.2 Analizar un caso de uso

Para cada caso de uso se elaboró un diagrama de comunicación con las clases de análisis: el actor, la interfaz «frm», el controlador «ctrl» y las entidades «entidad». Los mensajes están numerados en el orden en que ocurren; los que dependen de una condición llevan la guarda entre corchetes y los que se repiten llevan un asterisco. Por su ancho, los diagramas se presentan en páginas horizontales.

Ciclo I: seguridad y configuración

Figura 91

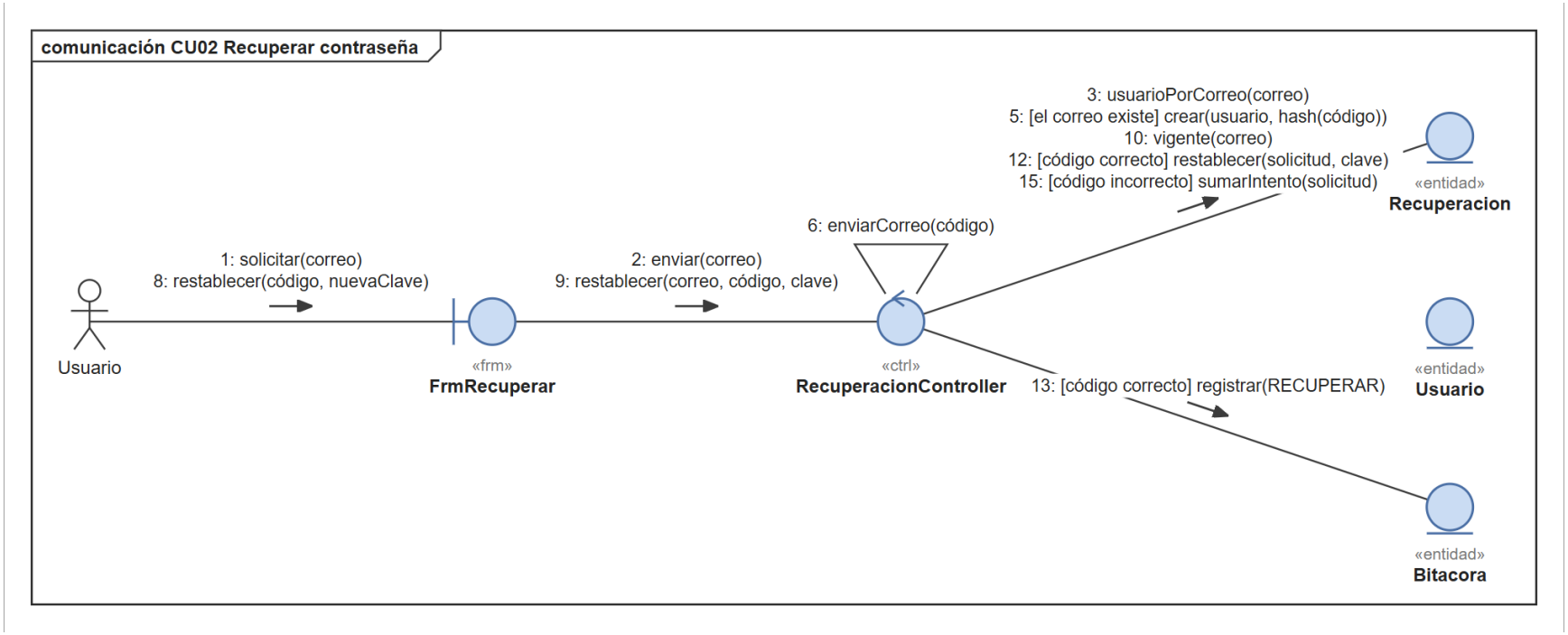
Diagrama de comunicación CU01 Iniciar y cerrar sesión



Nota. Elaboración propia.

Figura 92

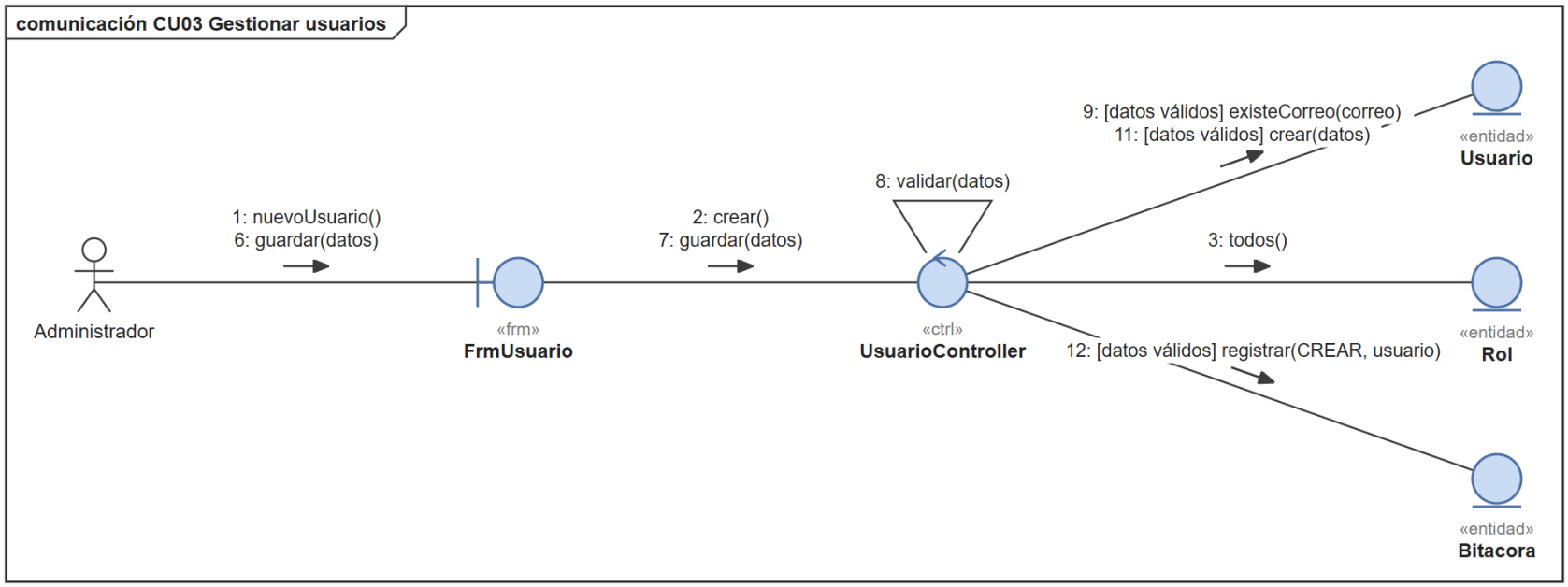
Diagrama de comunicación CU02 Recuperar contraseña



Nota. Elaboración propia.

Figura 93

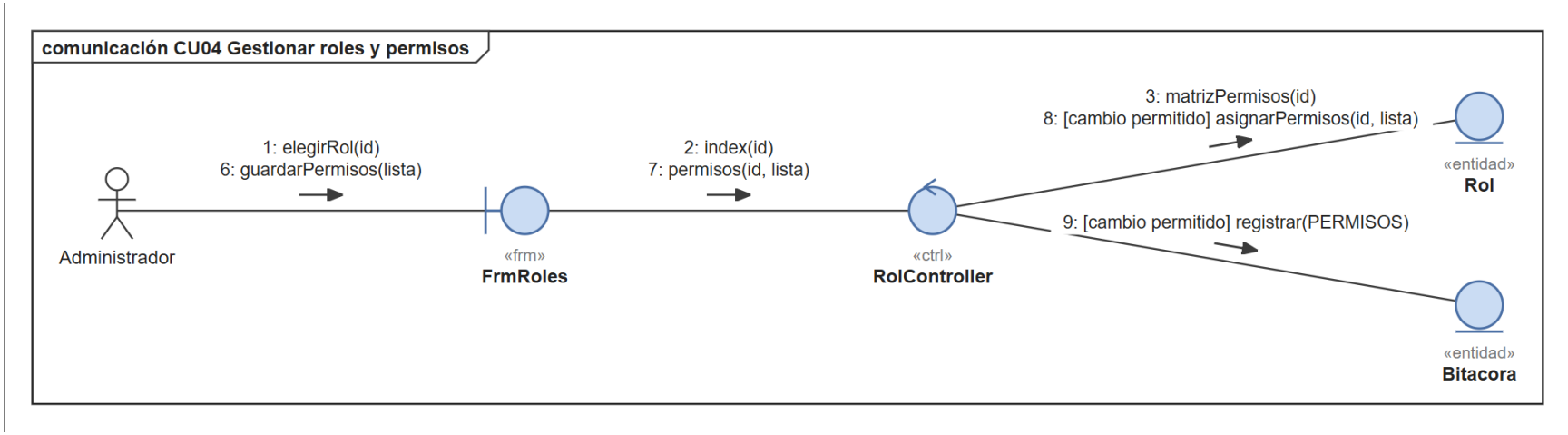
Diagrama de comunicación CU03 Gestionar usuarios



Nota. Elaboración propia.

Figura 94

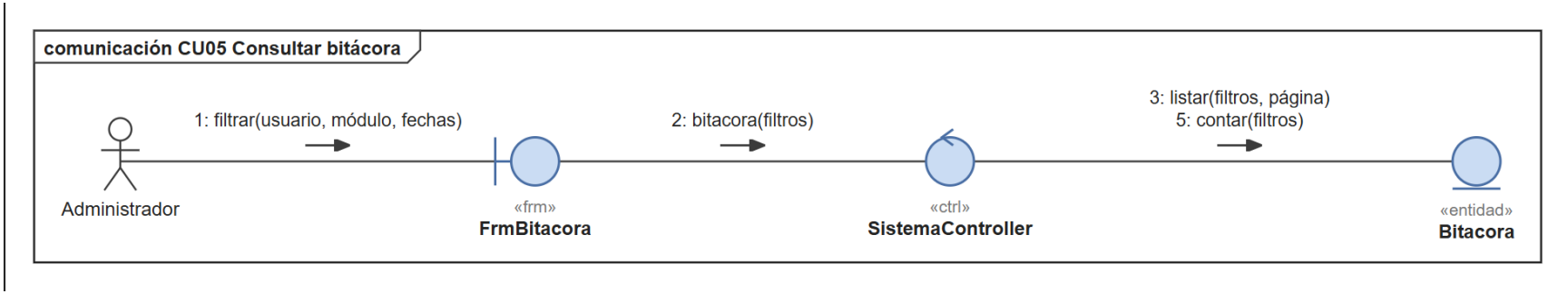
Diagrama de comunicación CU04 Gestionar roles y permisos



Nota. Elaboración propia.

Figura 95

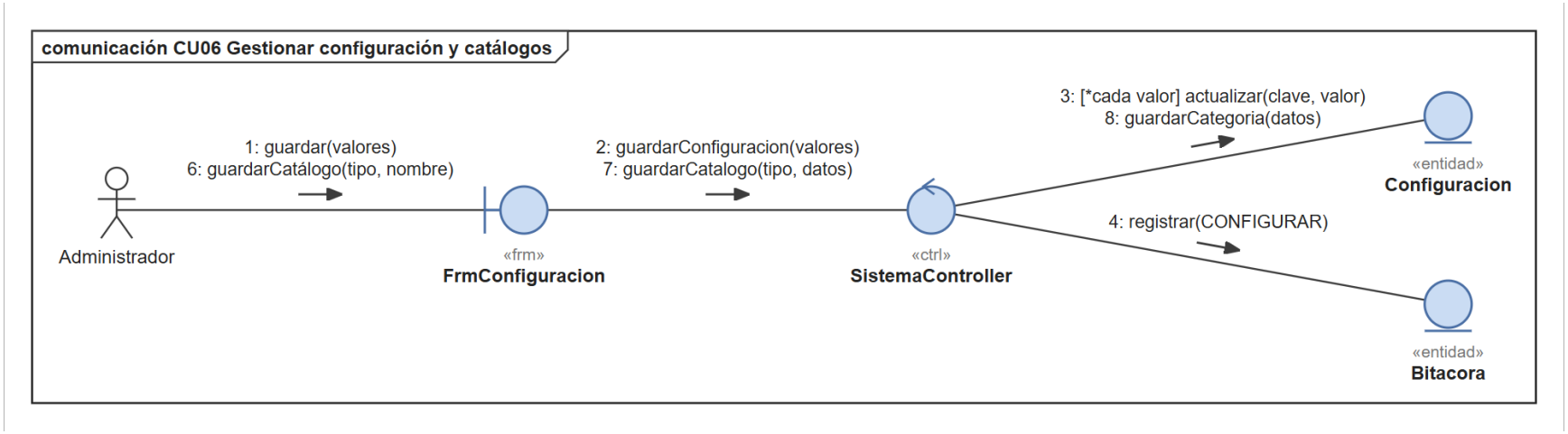
Diagrama de comunicación CU05 Consultar bitácora



Nota. Elaboración propia.

Figura 96

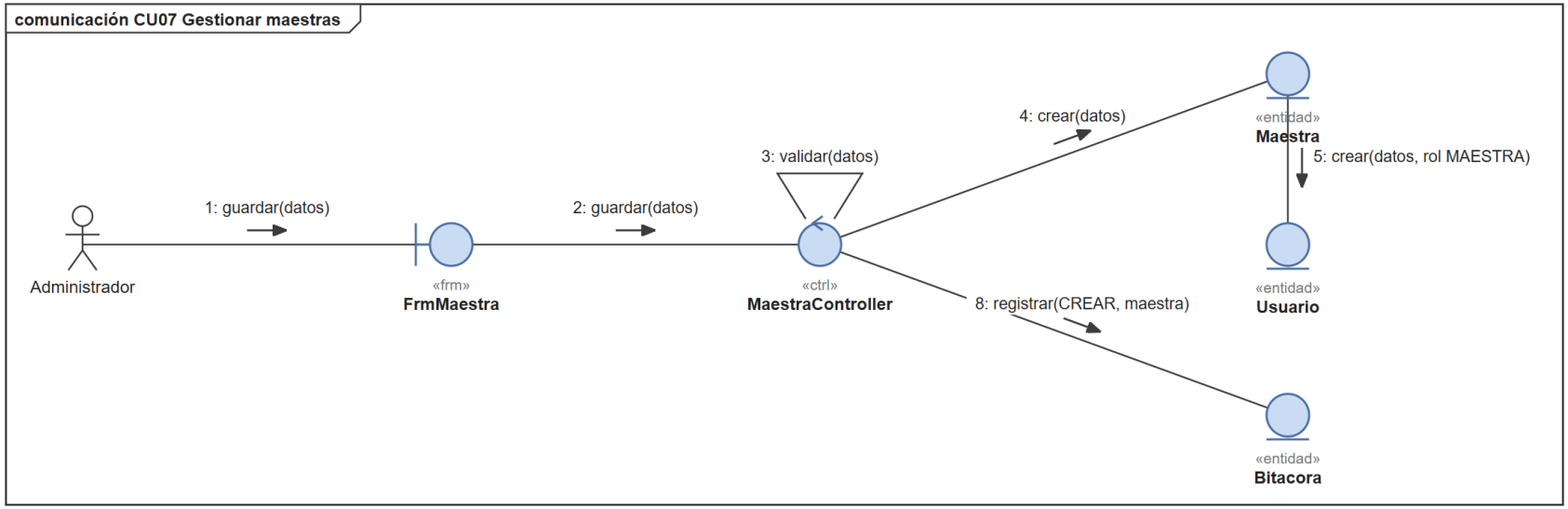
Diagrama de comunicación CU06 Gestionar configuración y catálogos



Nota. Elaboración propia.

Ciclo II: clases y reservas

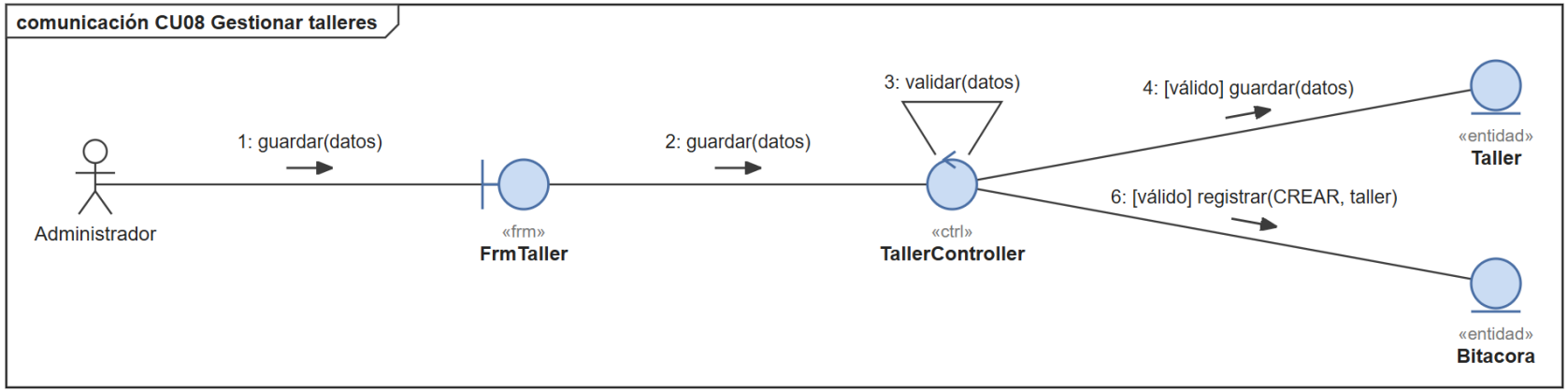
Figura 97
Diagrama de comunicación CU07 Gestionar maestras



Nota. Elaboración propia.

Figura 98

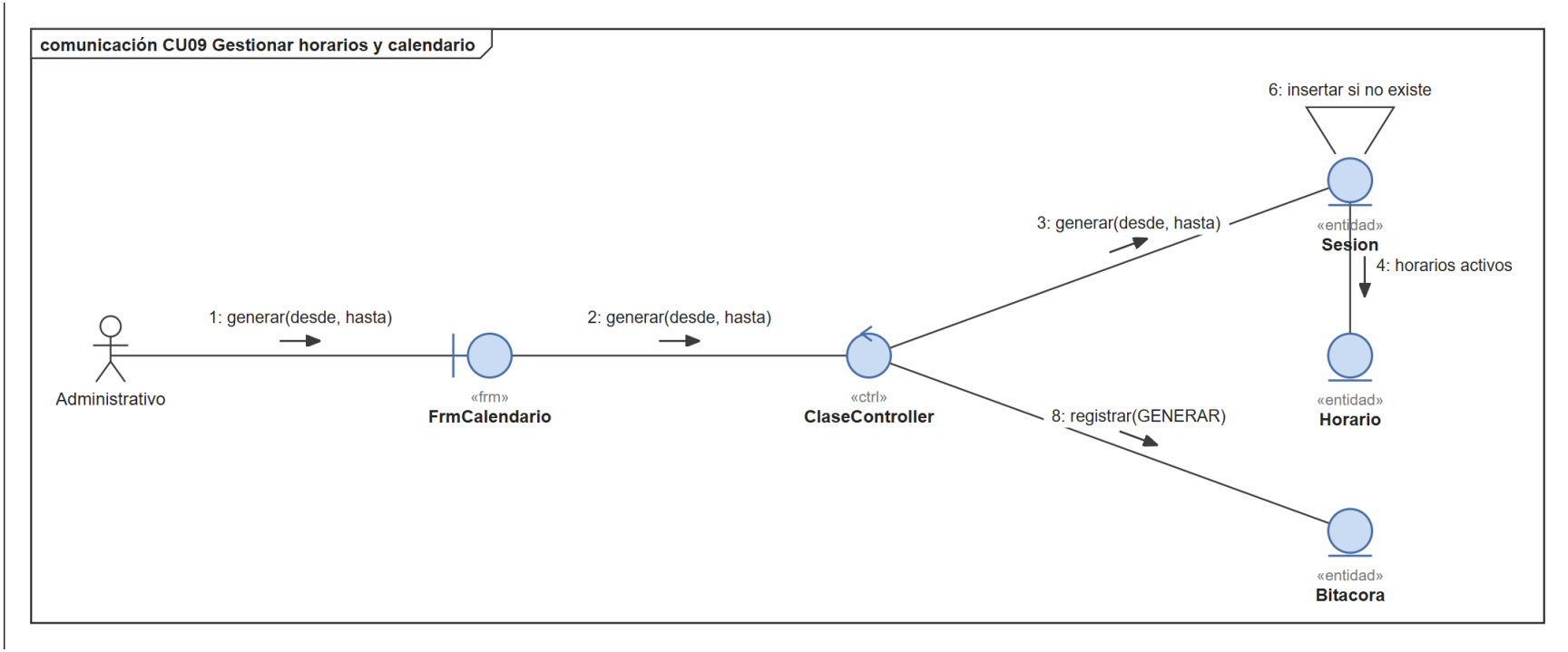
Diagrama de comunicación CU08 Gestionar talleres



Nota. Elaboración propia.

Figura 99

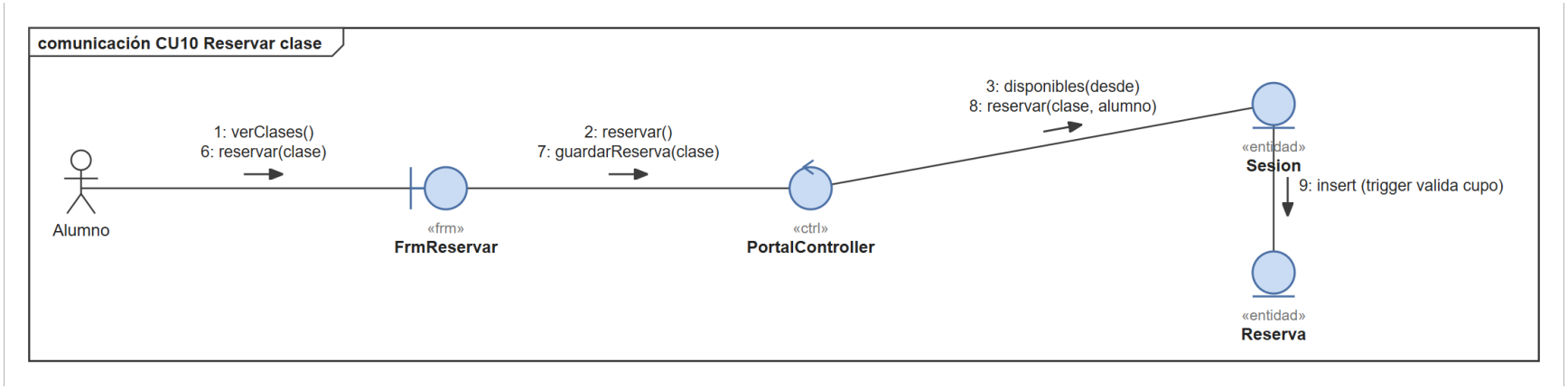
Diagrama de comunicación CU09 Gestionar horarios y calendario



Nota. Elaboración propia.

Figura 100

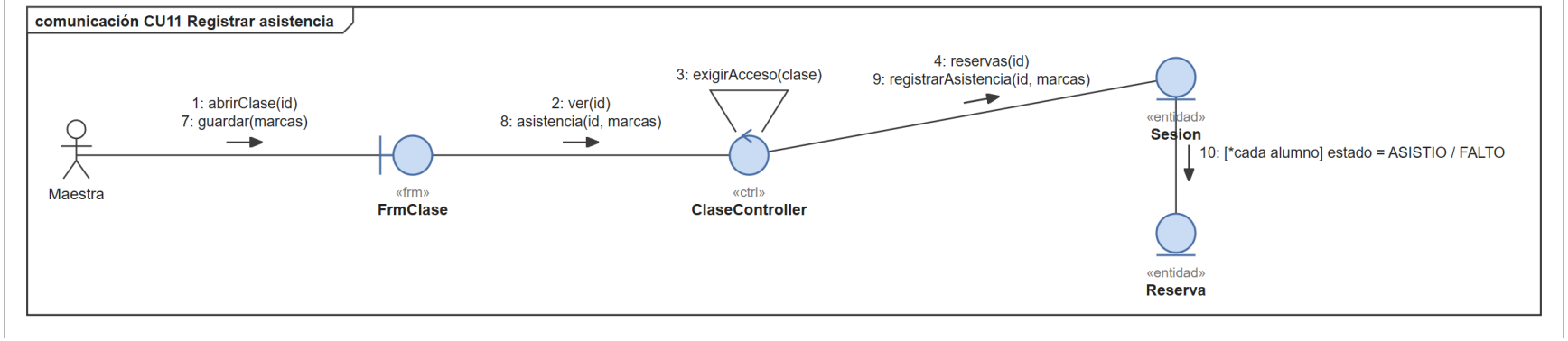
Diagrama de comunicación CU10 Reservar clase



Nota. Elaboración propia.

Figura 101

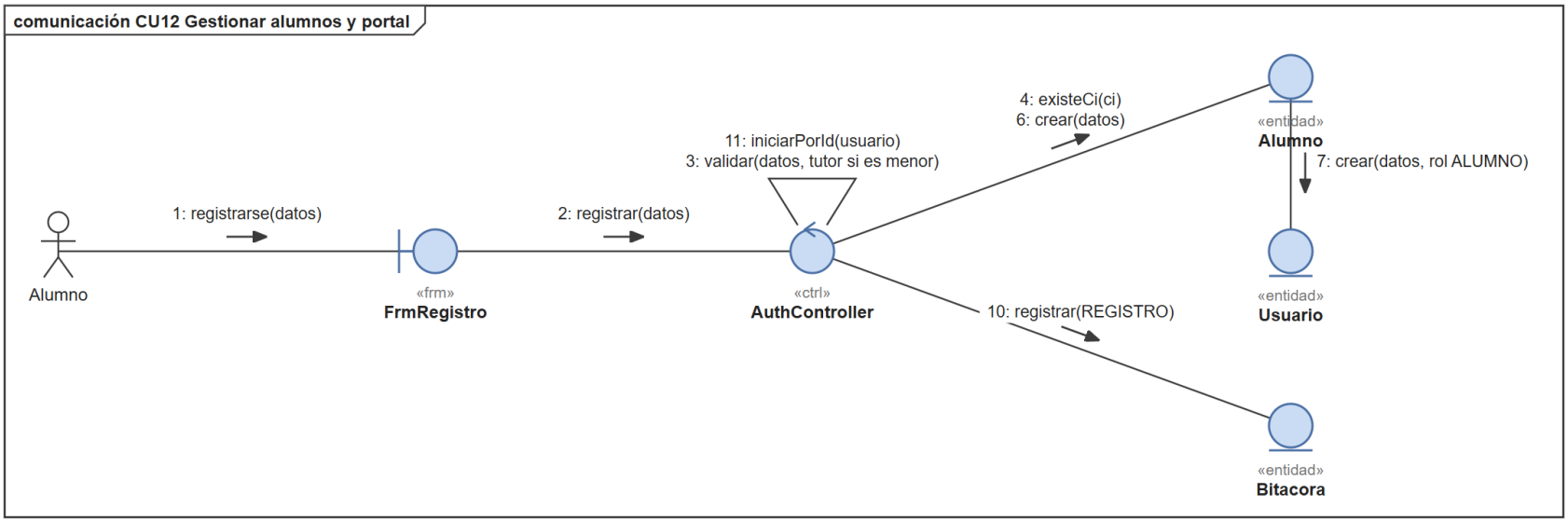
Diagrama de comunicación CU11 Registrar asistencia



Nota. Elaboración propia.

Figura 102

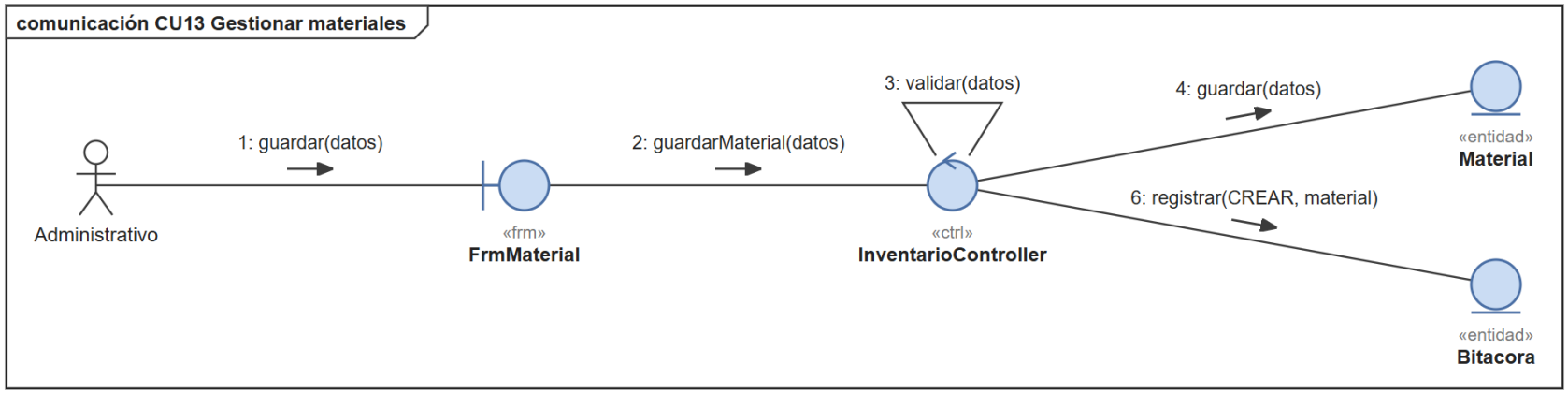
Diagrama de comunicación CU12 Gestionar alumnos y portal



Nota. Elaboración propia.

Ciclo III: inventario, ventas y tienda en línea

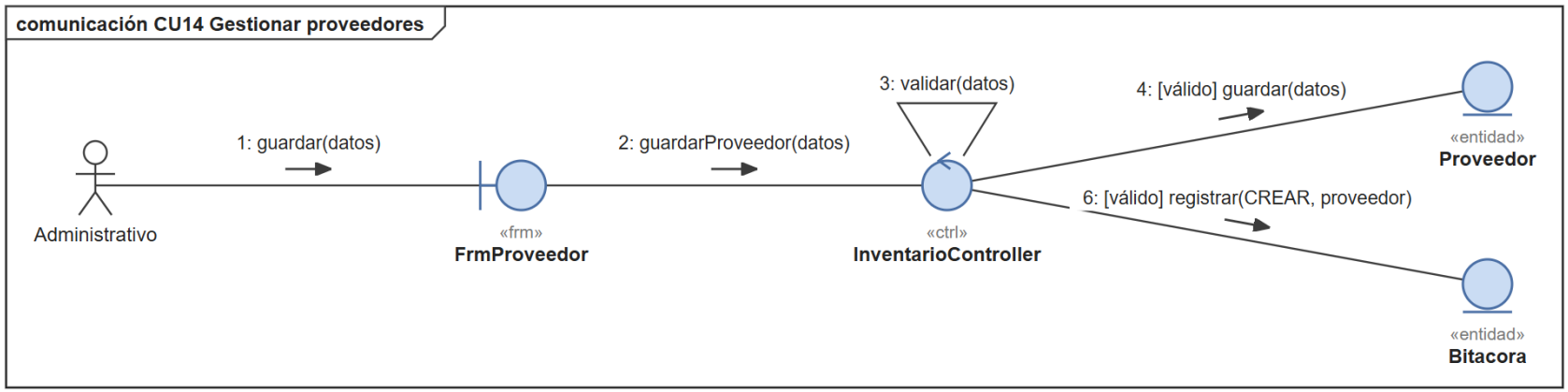
Figura 103
Diagrama de comunicación CU13 Gestionar materiales



Nota. Elaboración propia.

Figura 104

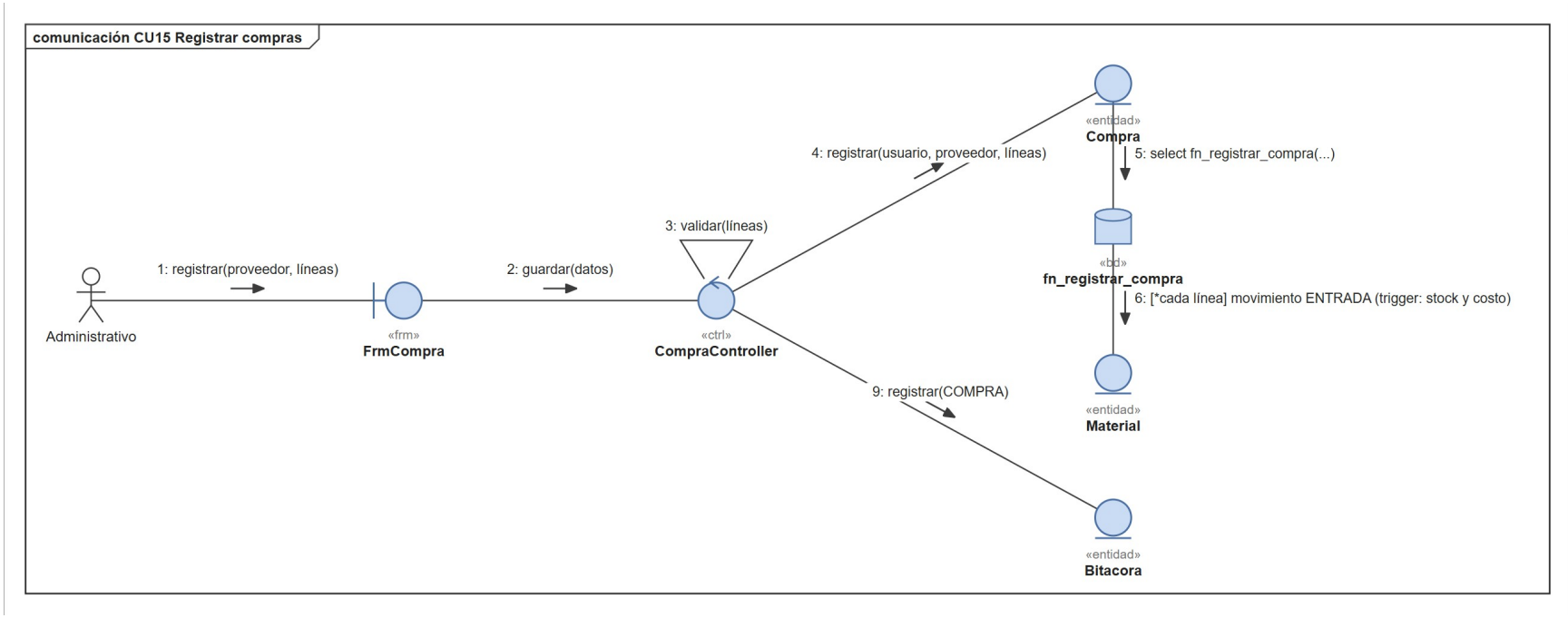
Diagrama de comunicación CU14 Gestionar proveedores



Nota. Elaboración propia.

Figura 105

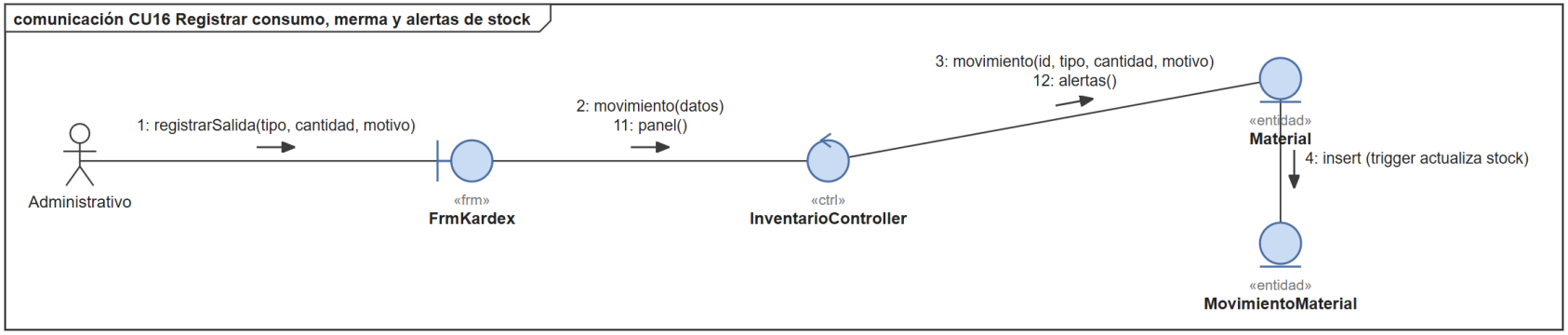
Diagrama de comunicación CU15 Registrar compras



Nota. Elaboración propia.

Figura 106

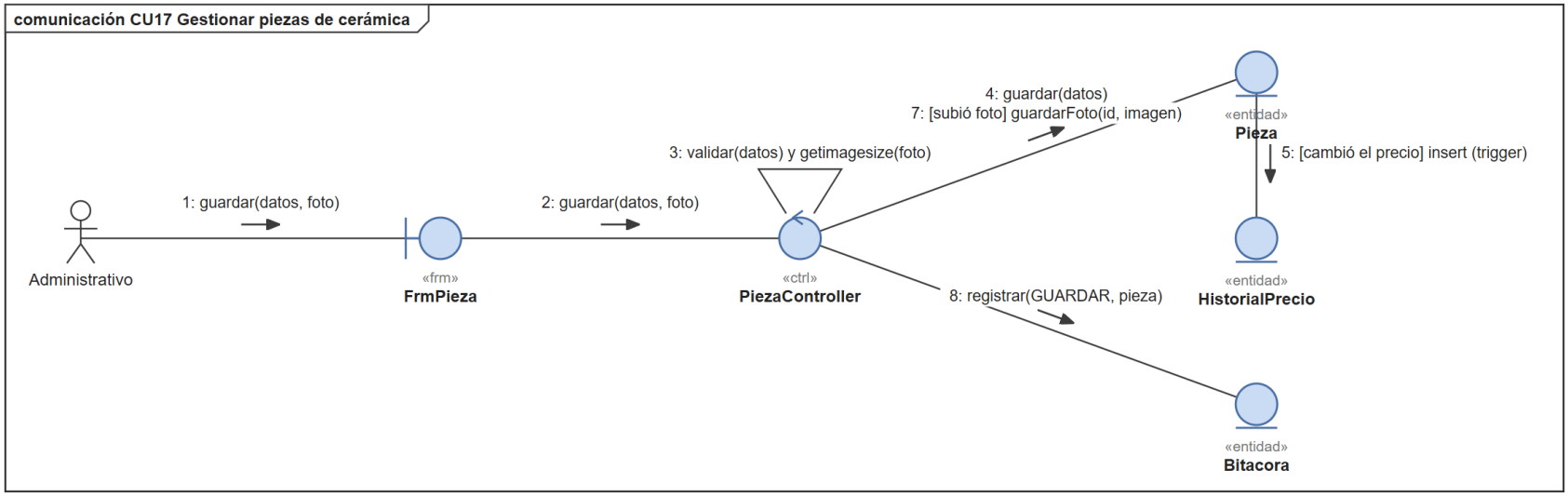
Diagrama de comunicación CU16 Registrar consumo, merma y alertas de stock



Nota. Elaboración propia.

Figura 107

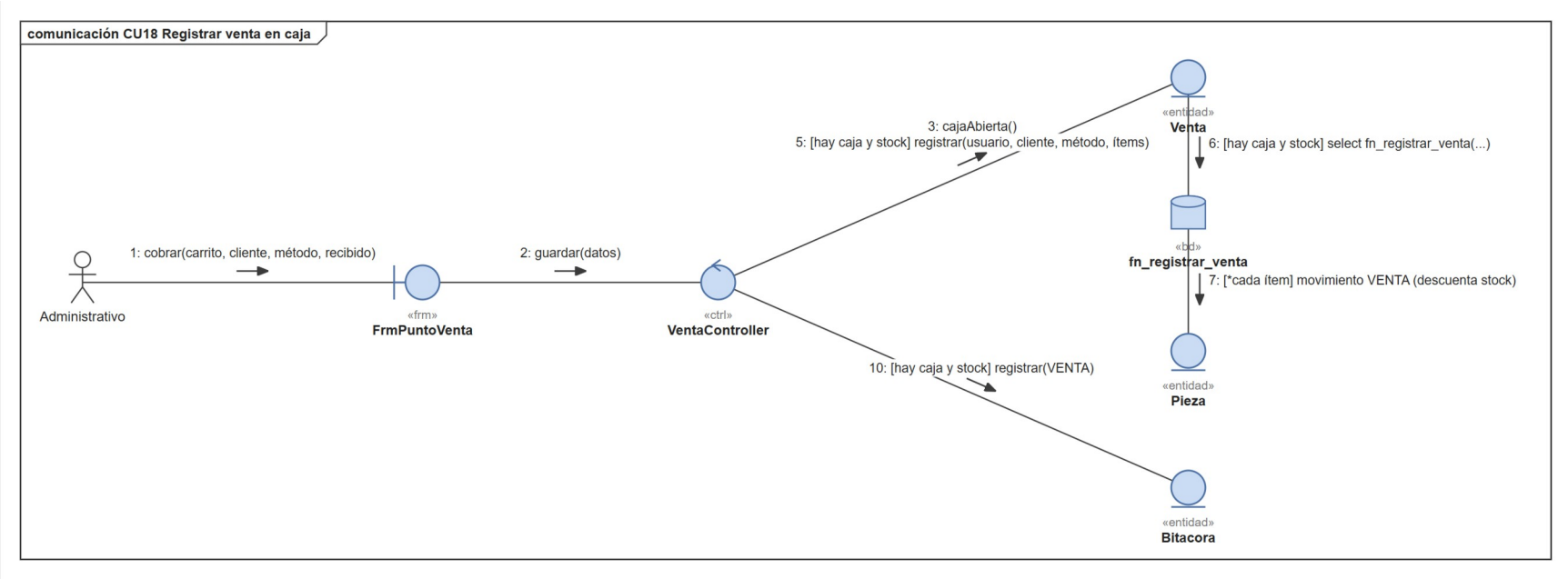
Diagrama de comunicación CU17 Gestionar piezas de cerámica



Nota. Elaboración propia.

Figura 108

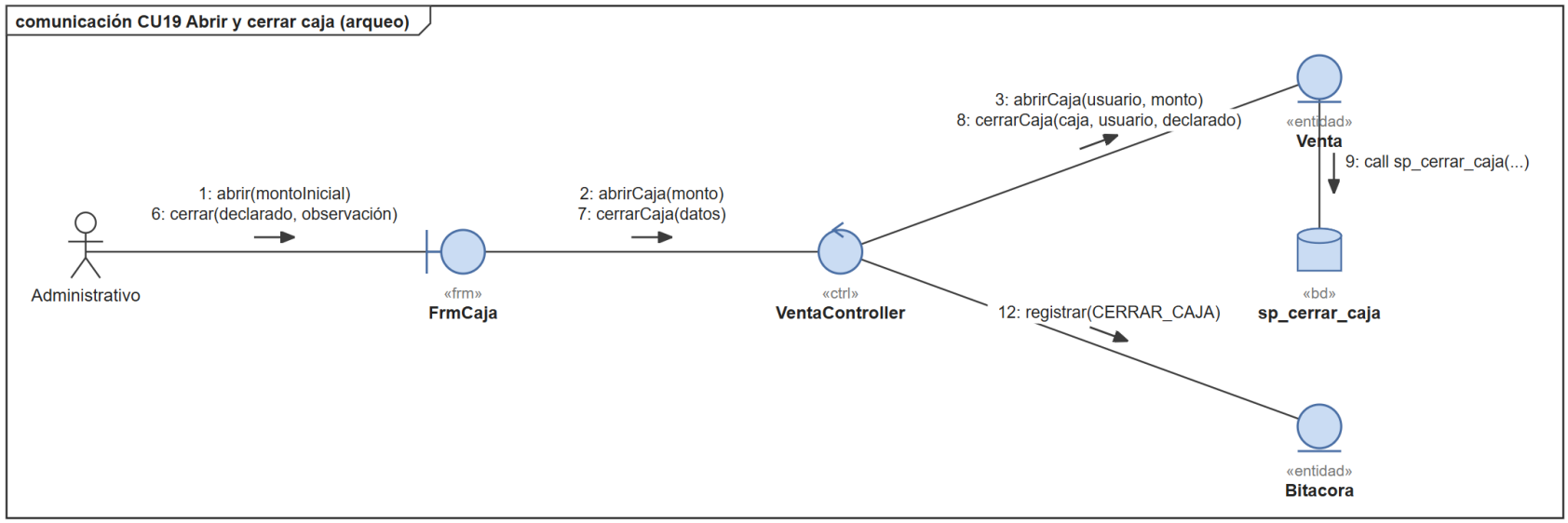
Diagrama de comunicación CU18 Registrar venta en caja



Nota. Elaboración propia.

Figura 109

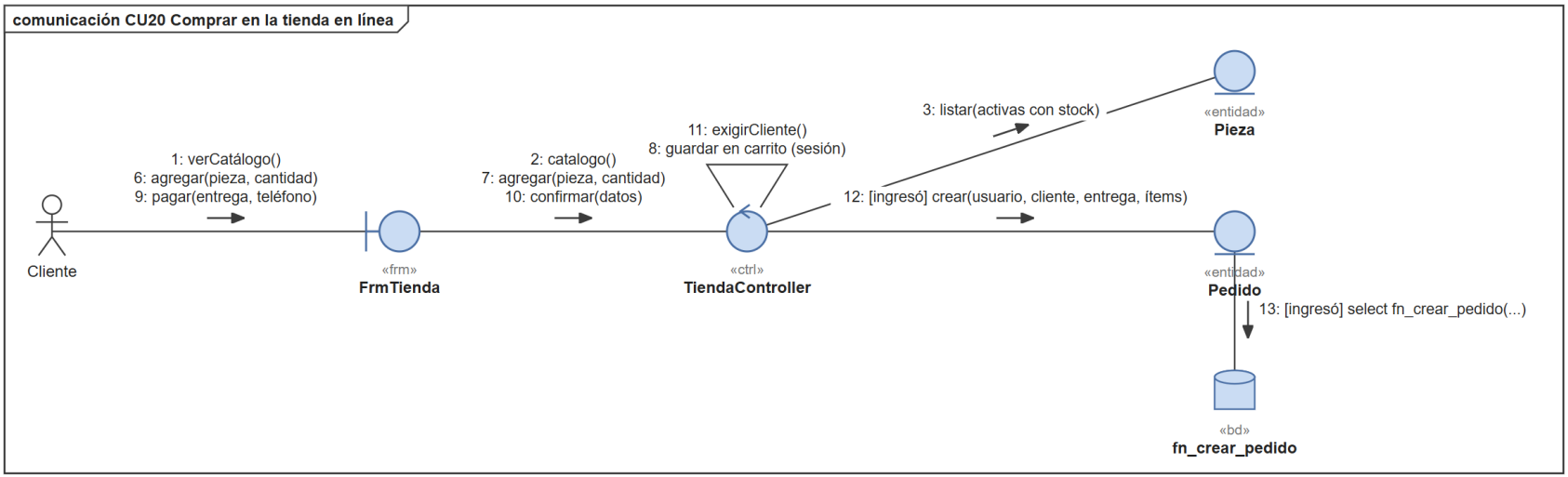
Diagrama de comunicación CU19 Abrir y cerrar caja (arqueo)



Nota. Elaboración propia.

Figura 110

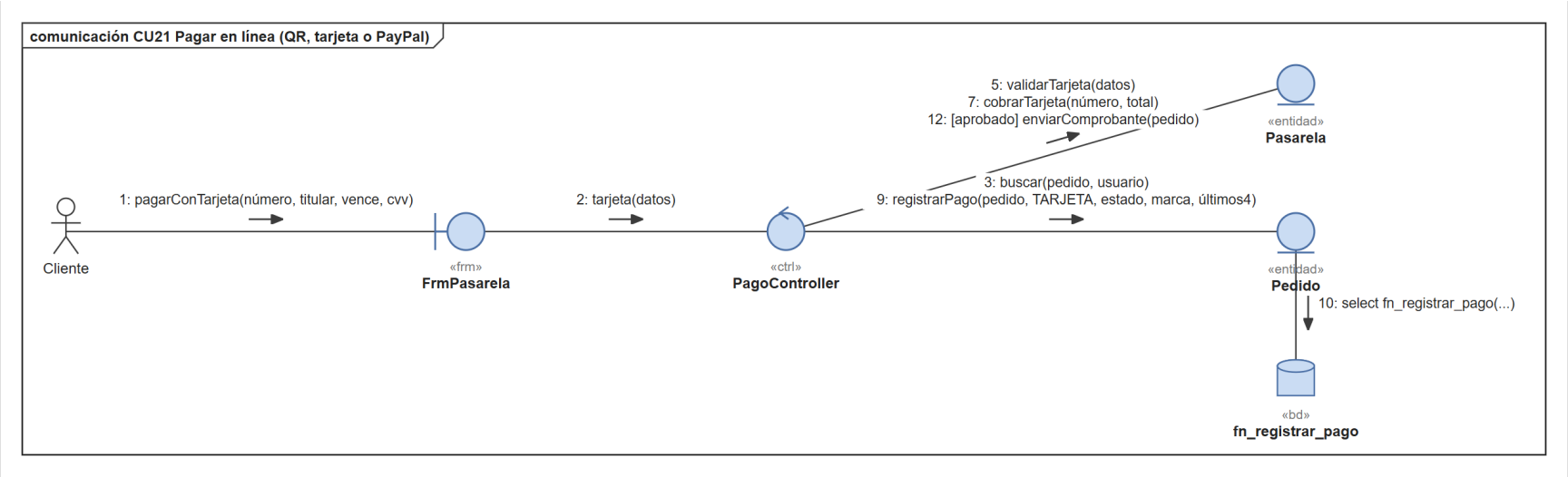
Diagrama de comunicación CU20 Comprar en la tienda en línea



Nota. Elaboración propia.

Figura 111

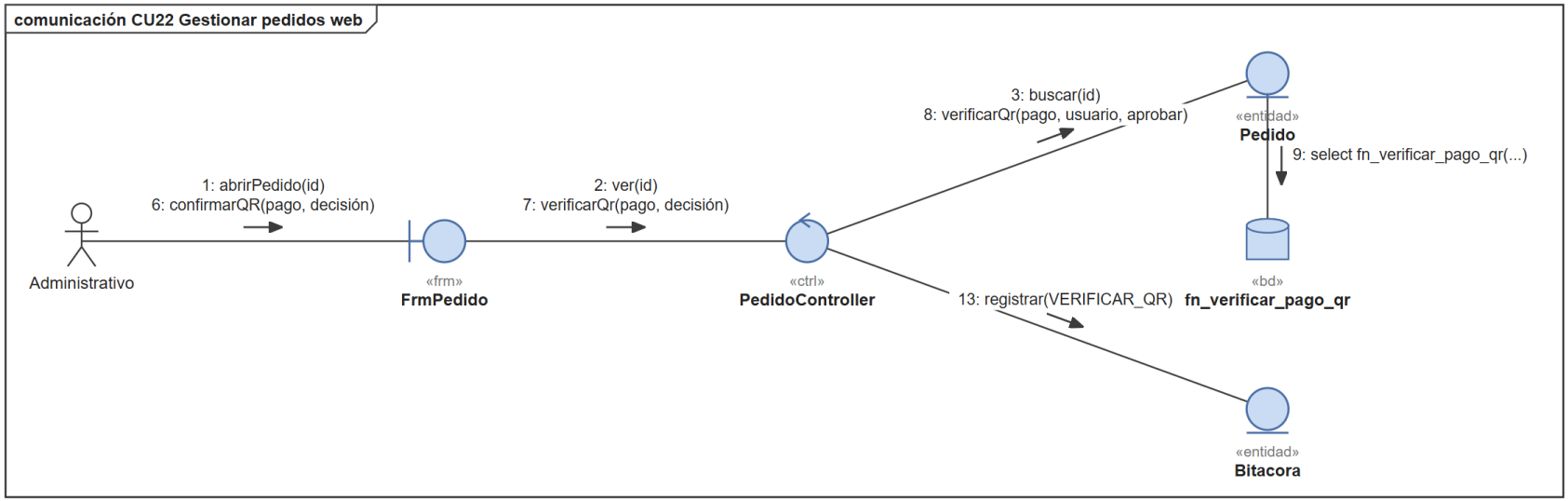
Diagrama de comunicación CU21 Pagar en línea (QR, tarjeta o PayPal)



Nota. Elaboración propia.

Figura 112

Diagrama de comunicación CU22 Gestionar pedidos web



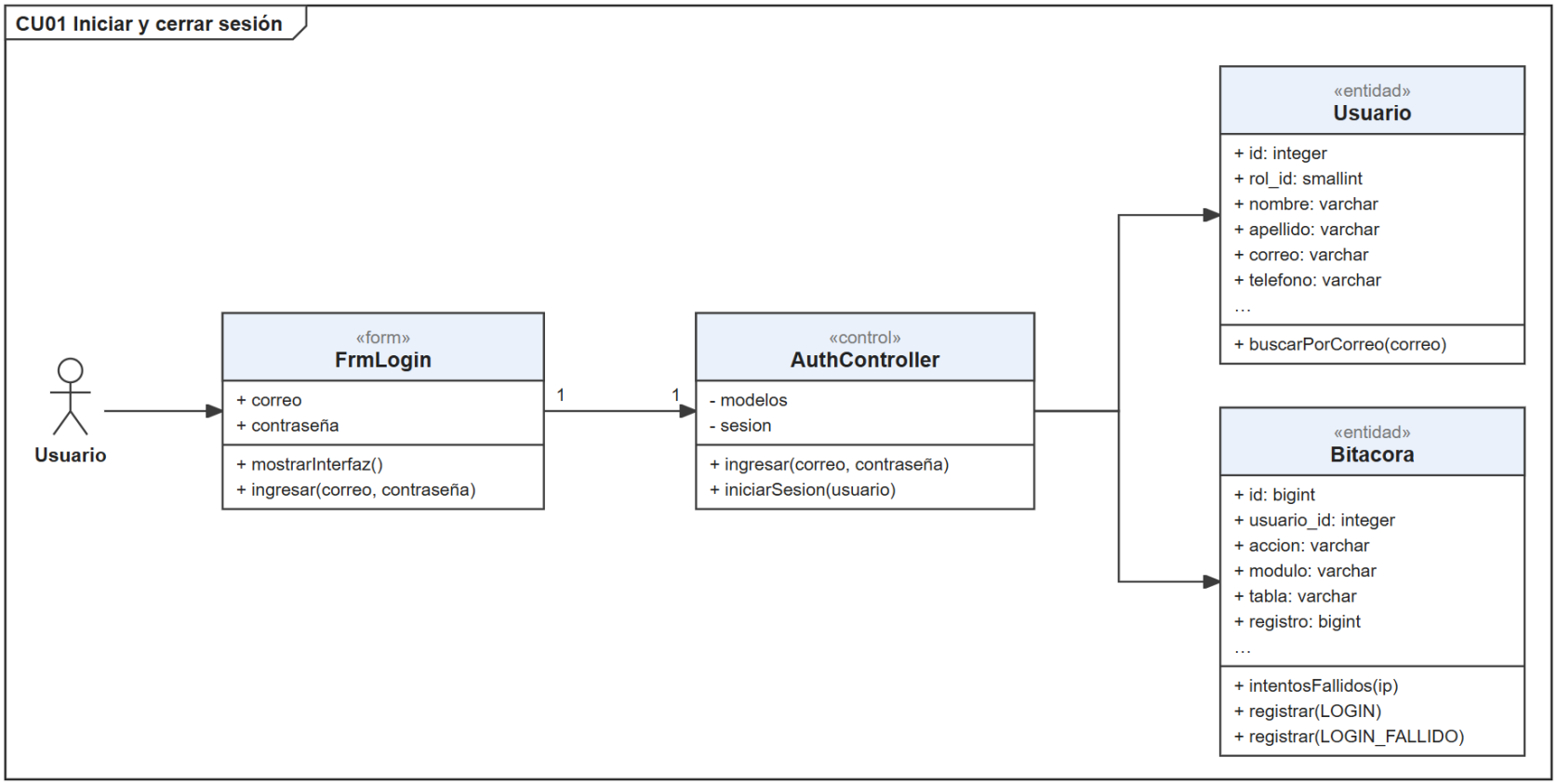
Nota. Elaboración propia.

3.3 Análisis de clase

Para cada caso de uso se elaboró un diagrama de clases de análisis con tres tipos de clase: la interfaz «form» (lo que ve el usuario, con sus campos y acciones), el controlador «control» (coordina el caso de uso) y las entidades «entidad» (datos que persisten, con los atributos reales de su tabla). Las clases «procedimiento» son funciones de PostgreSQL que ejecutan una operación completa en una transacción. Los métodos son los mensajes del diagrama de comunicación del mismo caso de uso.

Ciclo I: seguridad y configuración

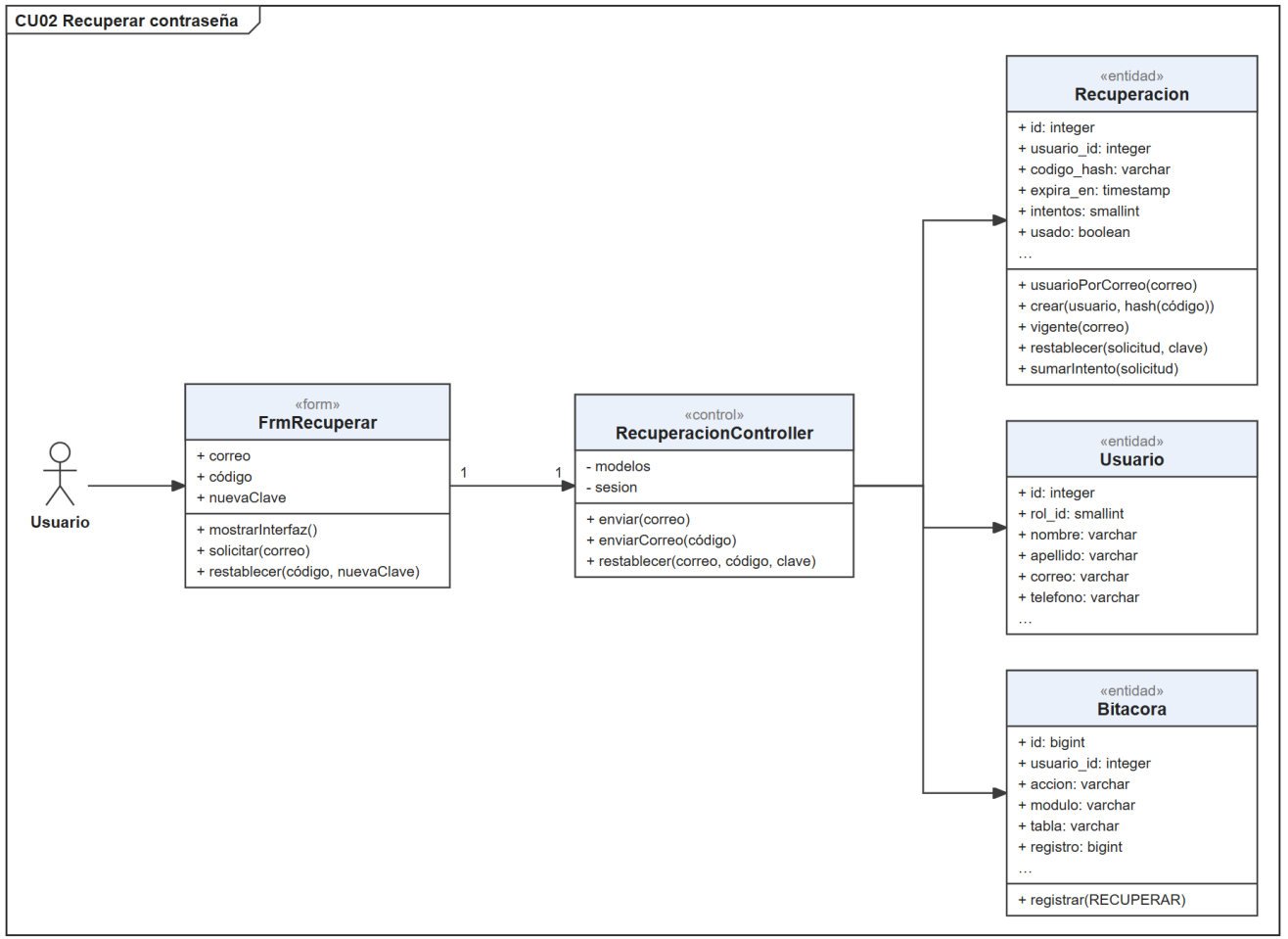
Figura 113
Clases de análisis CU01 Iniciar y cerrar sesión



Nota. Elaboración propia.

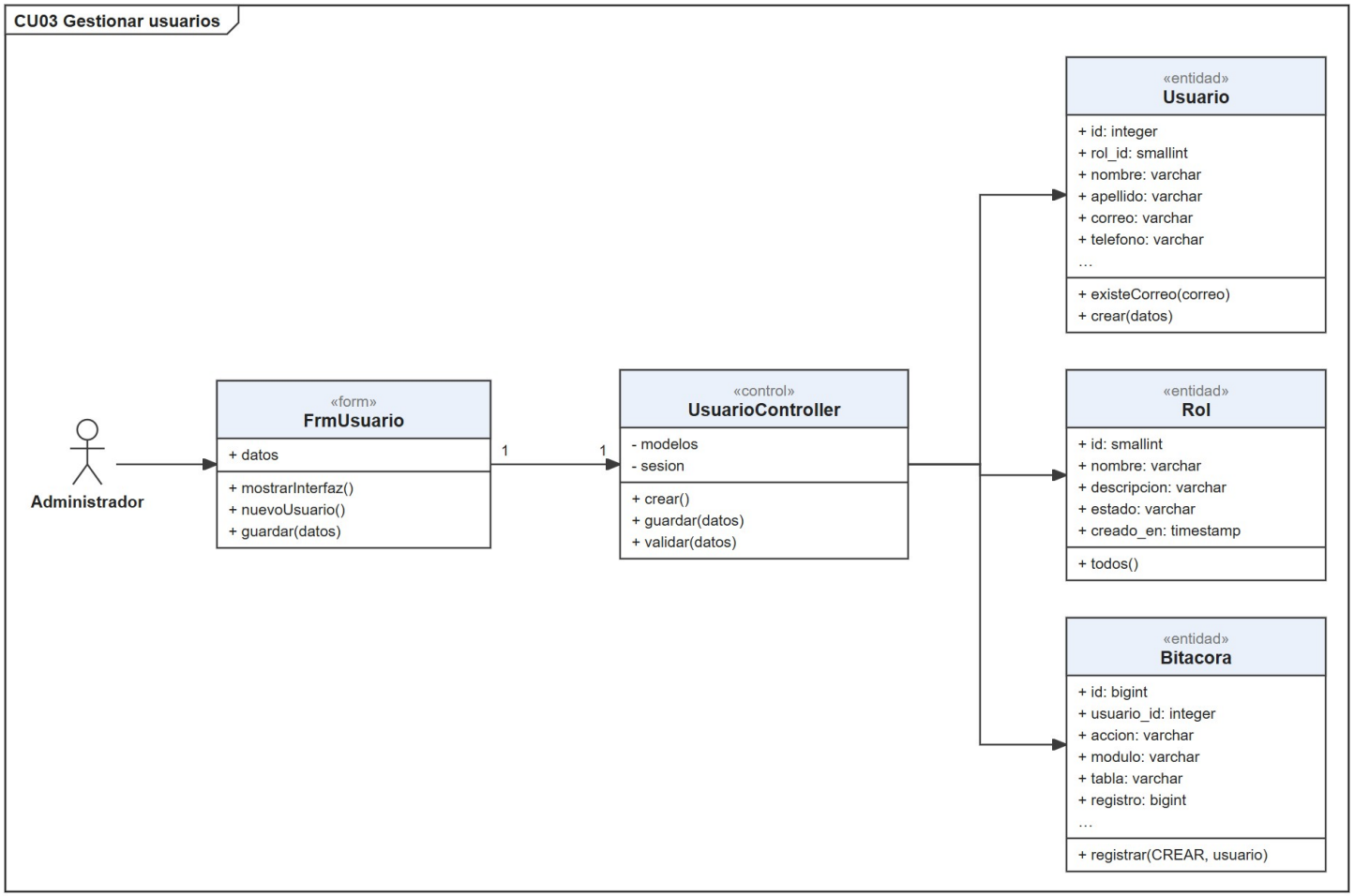
Figura 114

Clases de análisis CU02 Recuperar contraseña



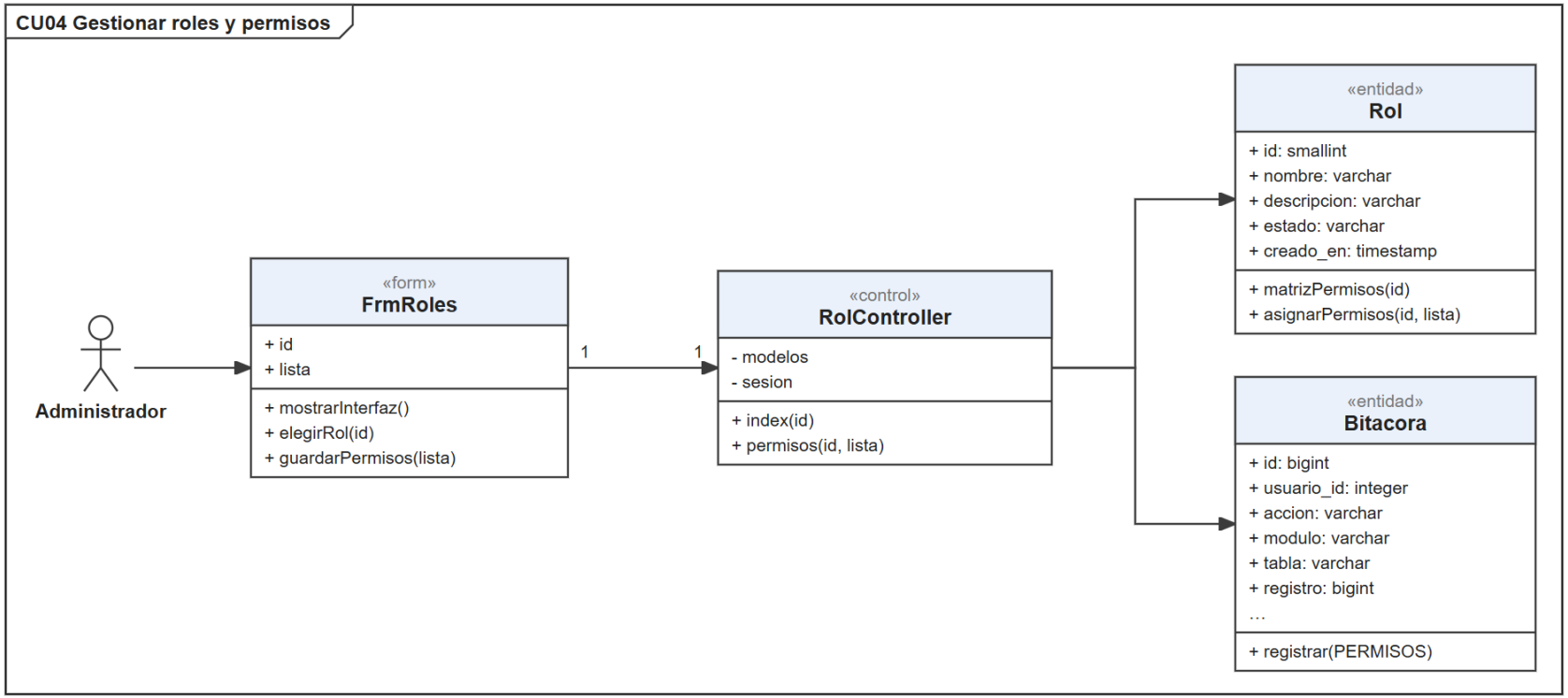
Nota. Elaboración propia.

Figura 115
Clases de análisis CU03 Gestionar usuarios



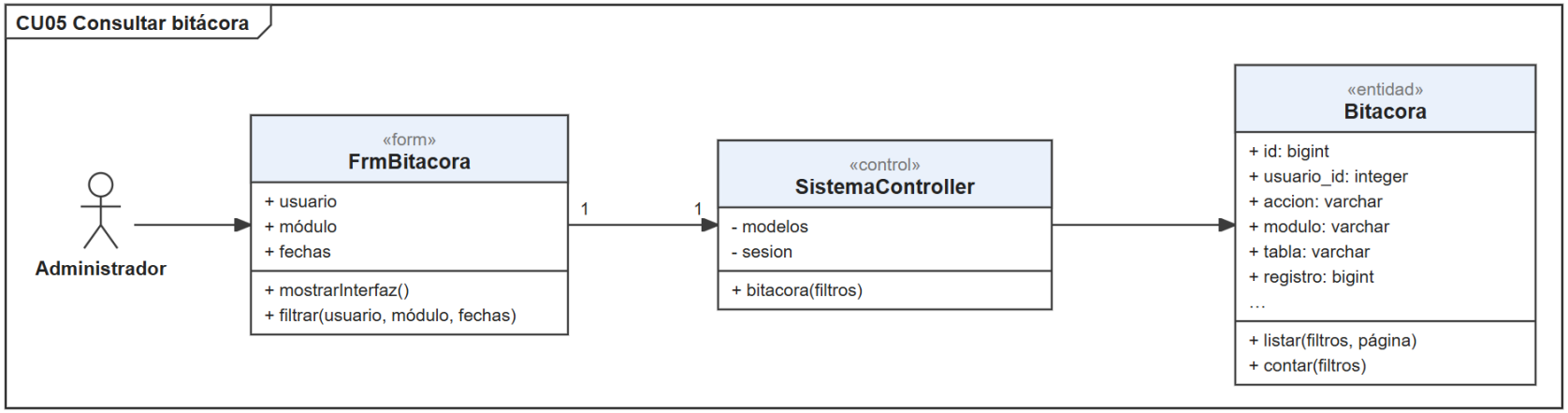
Nota. Elaboración propia.

Figura 116
Clases de análisis CU04 Gestionar roles y permisos



Nota. Elaboración propia.

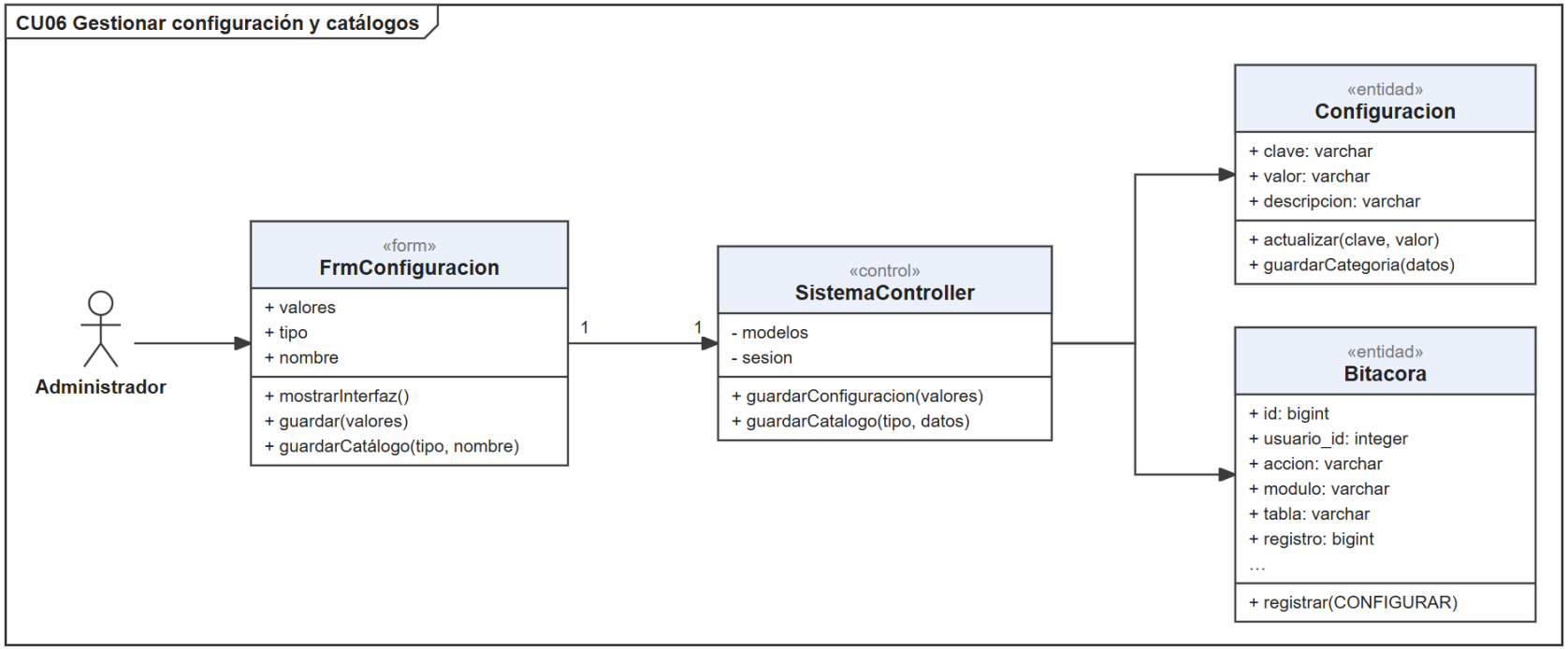
Figura 117
Clases de análisis CU05 Consultar bitácora



Nota. Elaboración propia.

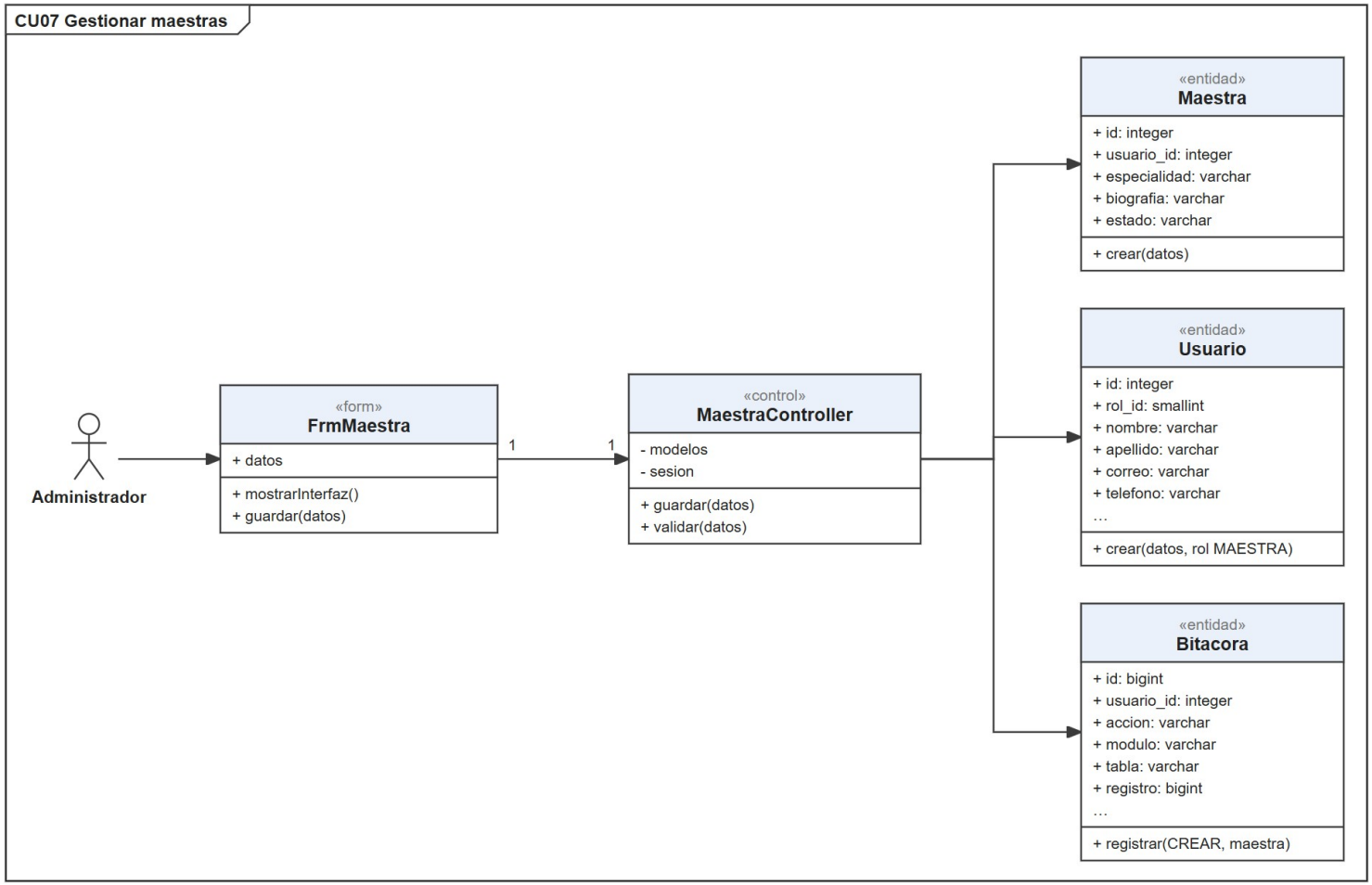
Figura 118

Clases de análisis CU06 Gestionar configuración y catálogos



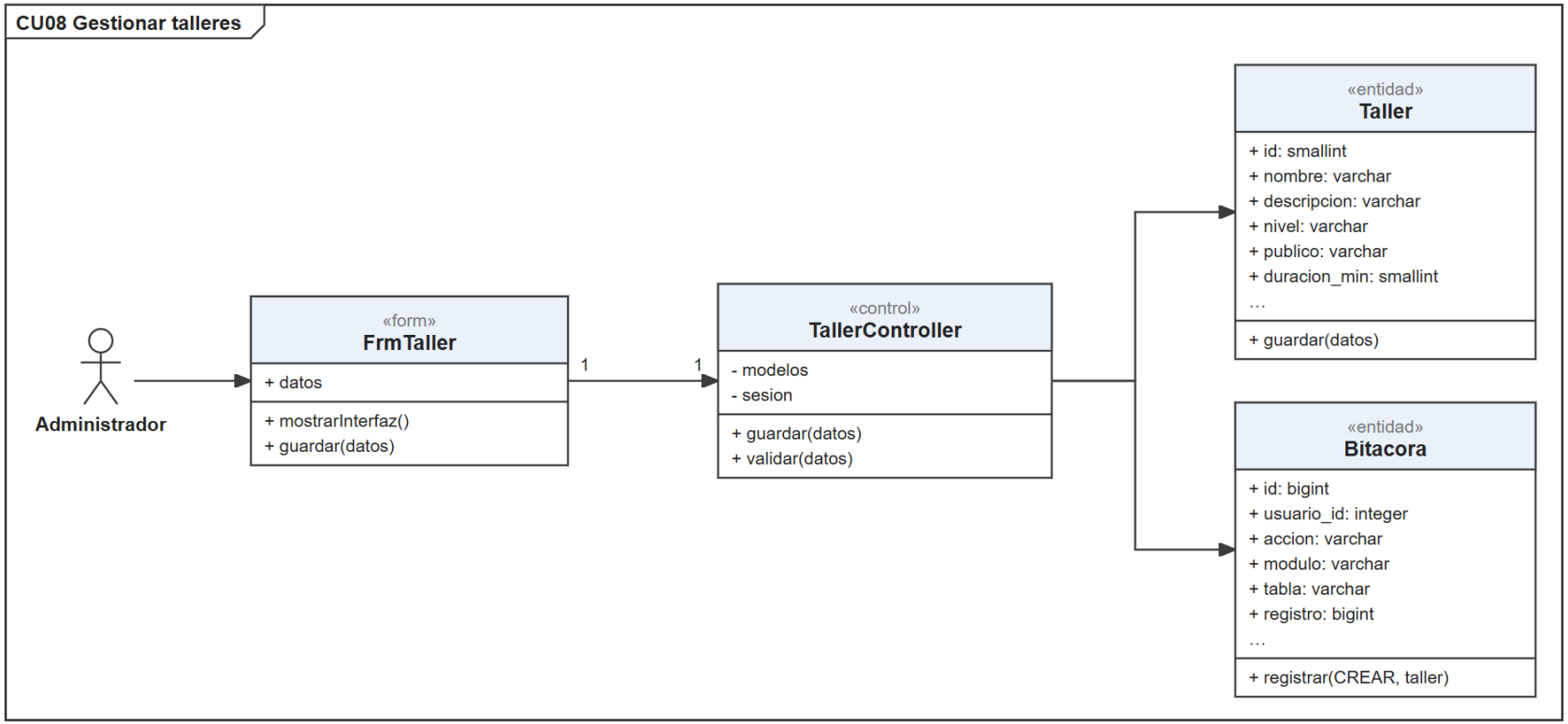
Nota. Elaboración propia.

Figura 119
Clases de análisis CU07 Gestionar maestras



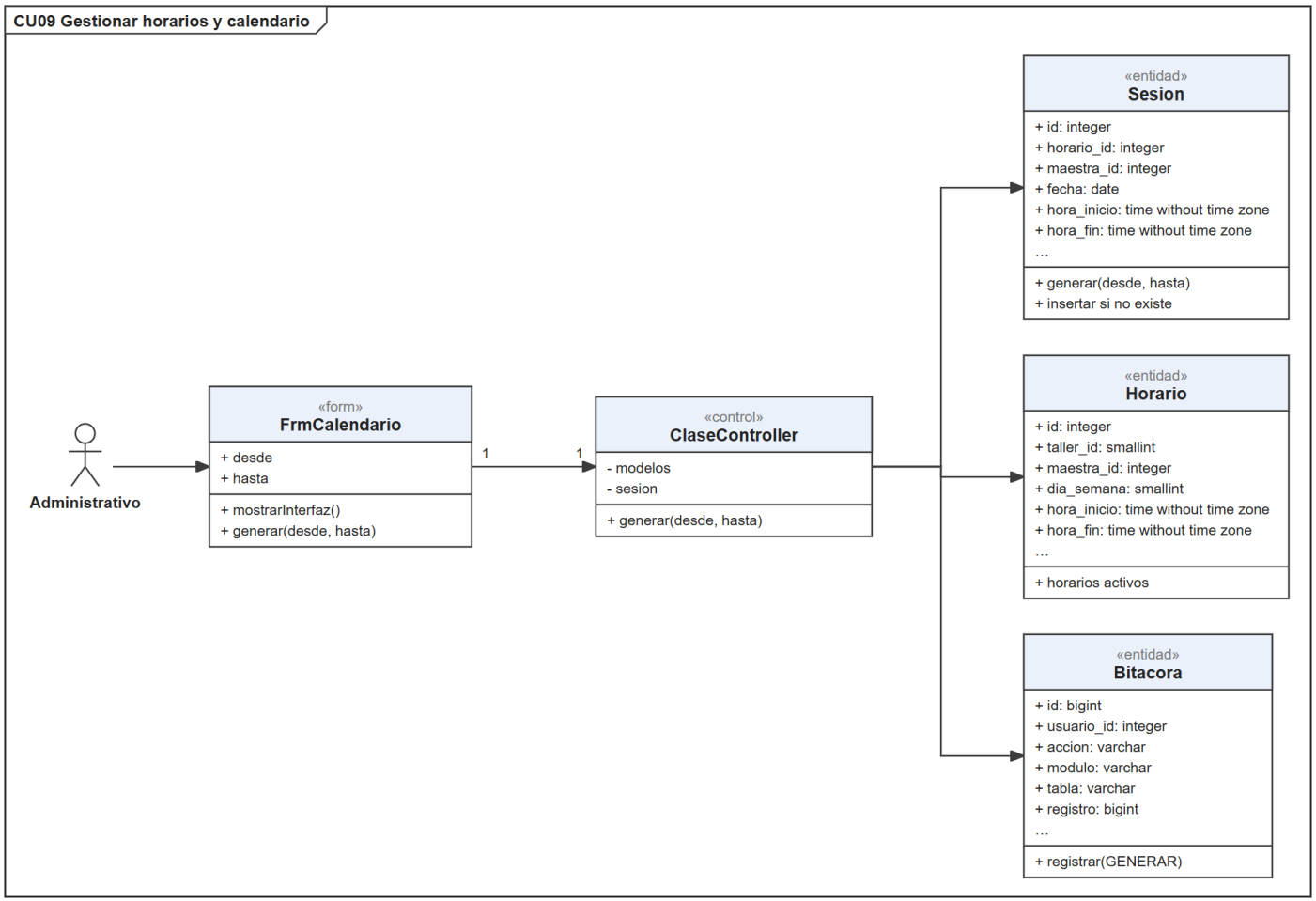
Nota. Elaboración propia.

Figura 120
Clases de análisis CU08 Gestionar talleres



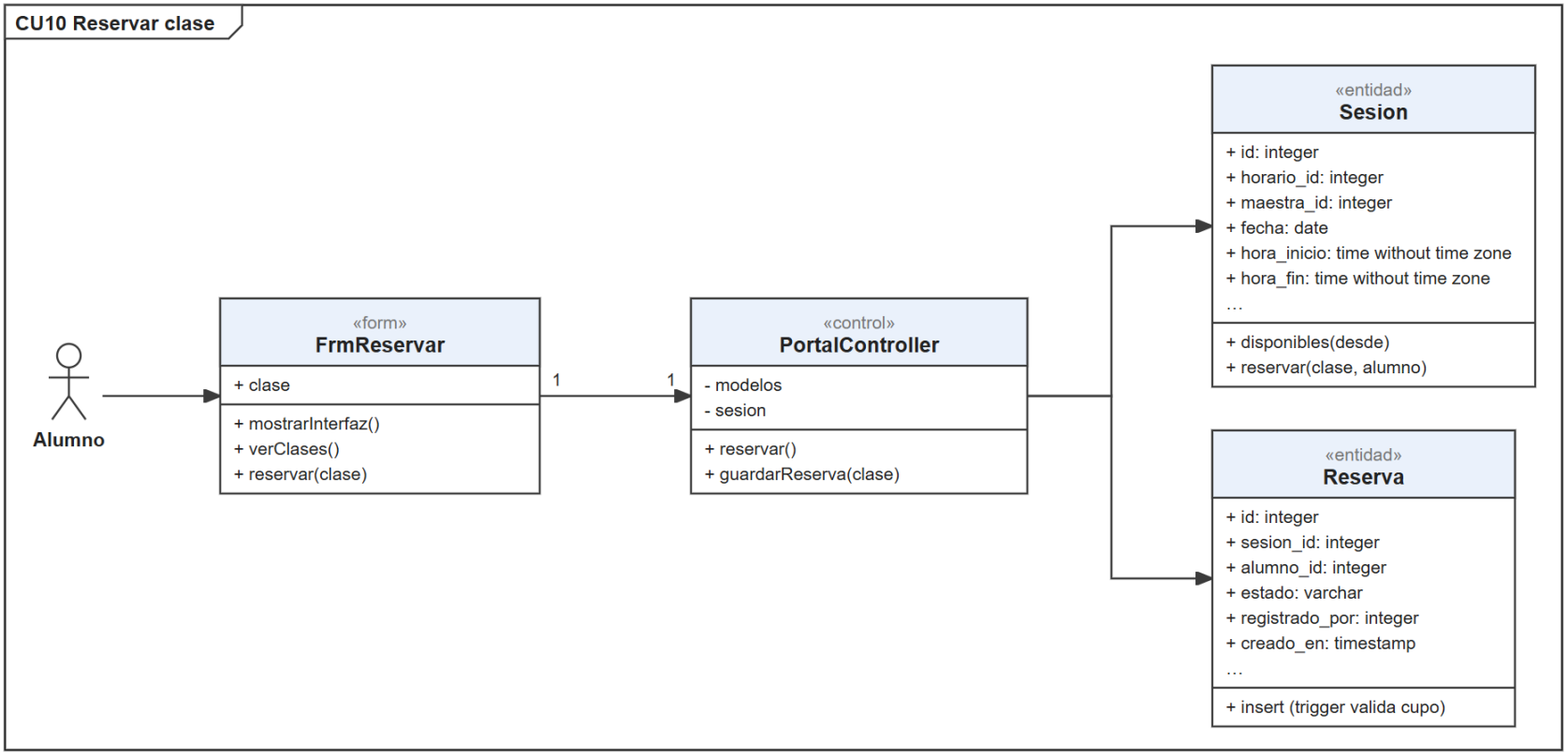
Nota. Elaboración propia.

Figura 121
Clases de análisis CU09 Gestionar horarios y calendario



Nota. Elaboración propia.

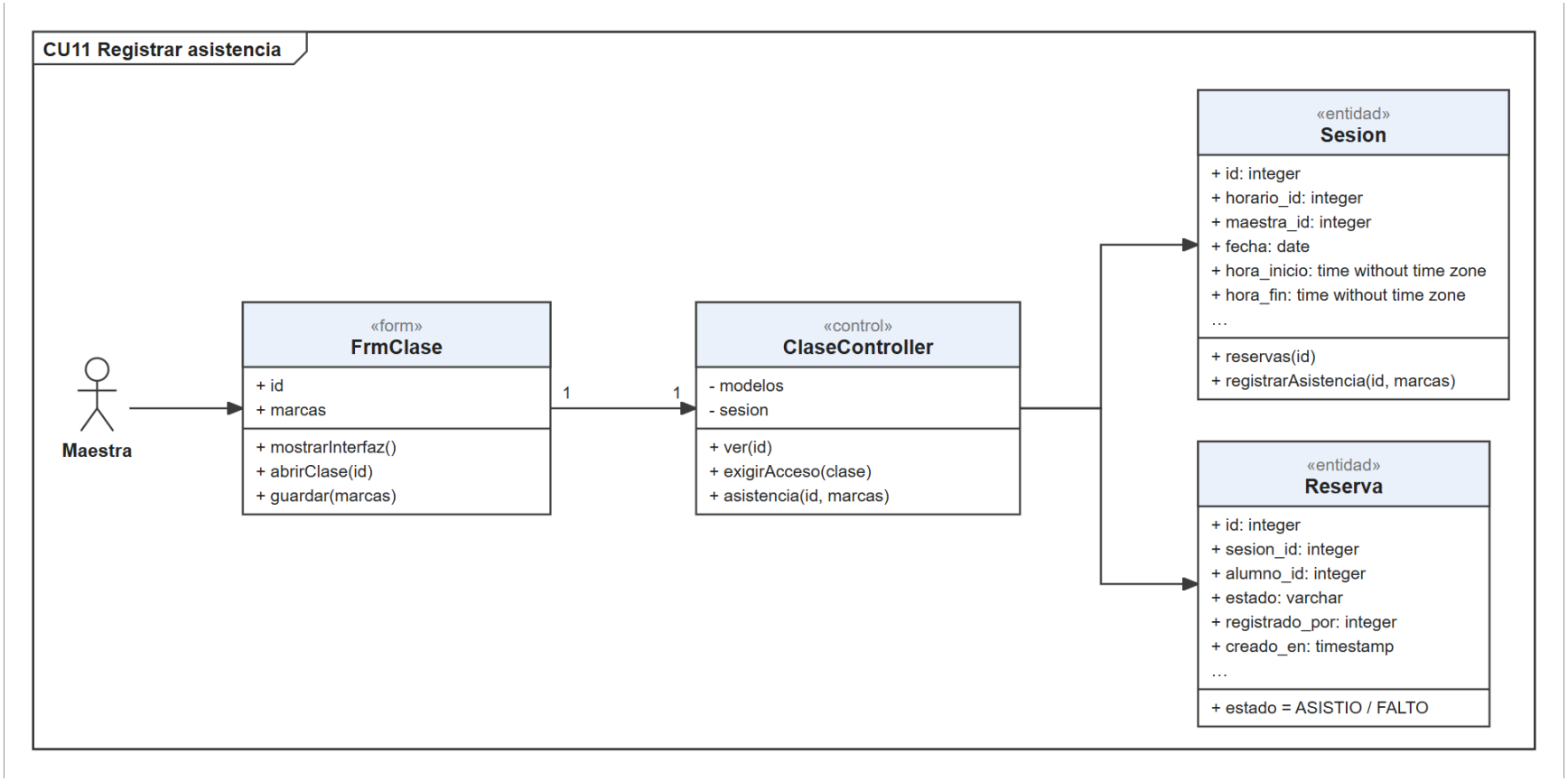
Figura 122
Clases de análisis CU10 Reservar clase



Nota. Elaboración propia.

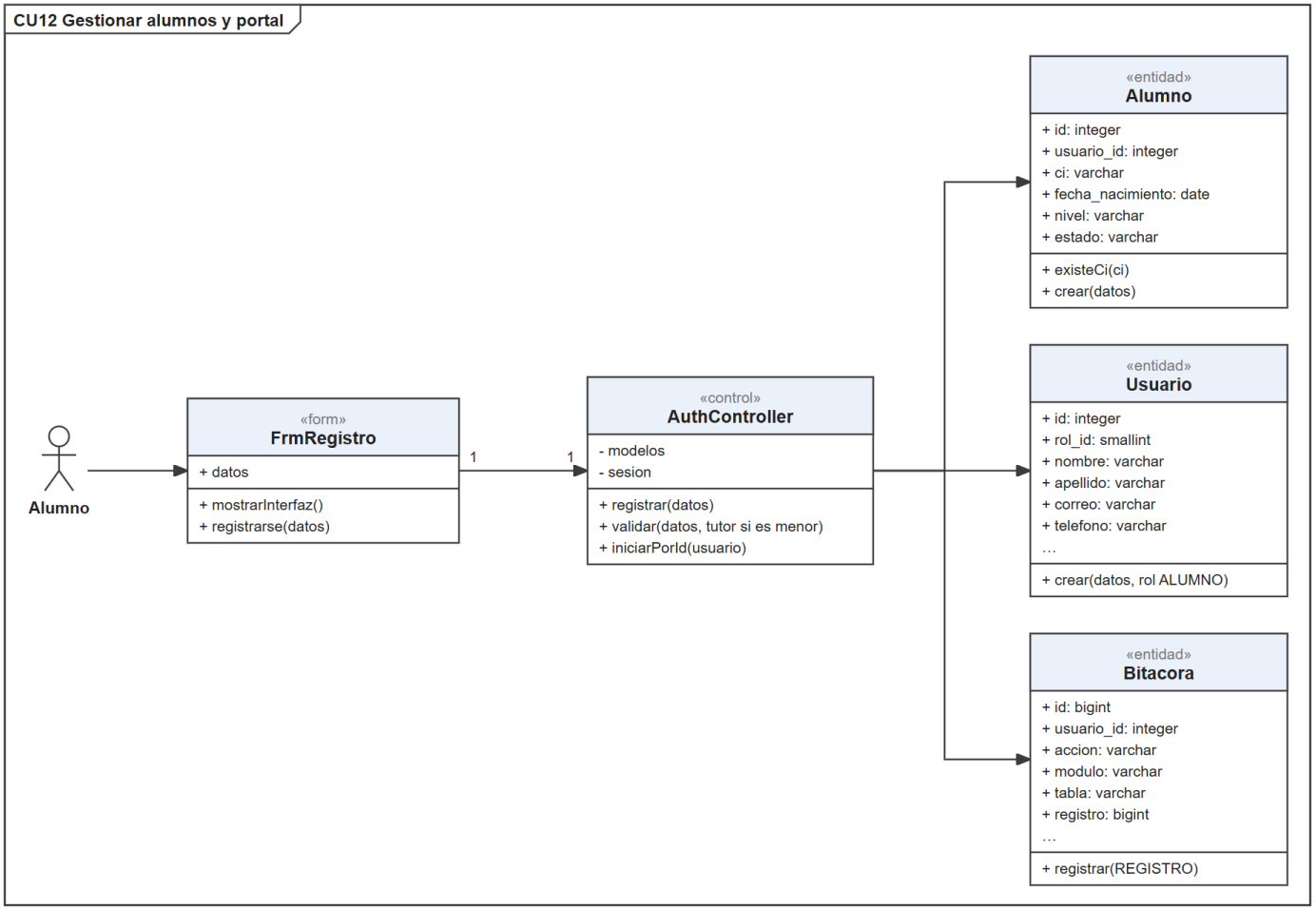
Figura 123

Clases de análisis CU11 Registrar asistencia



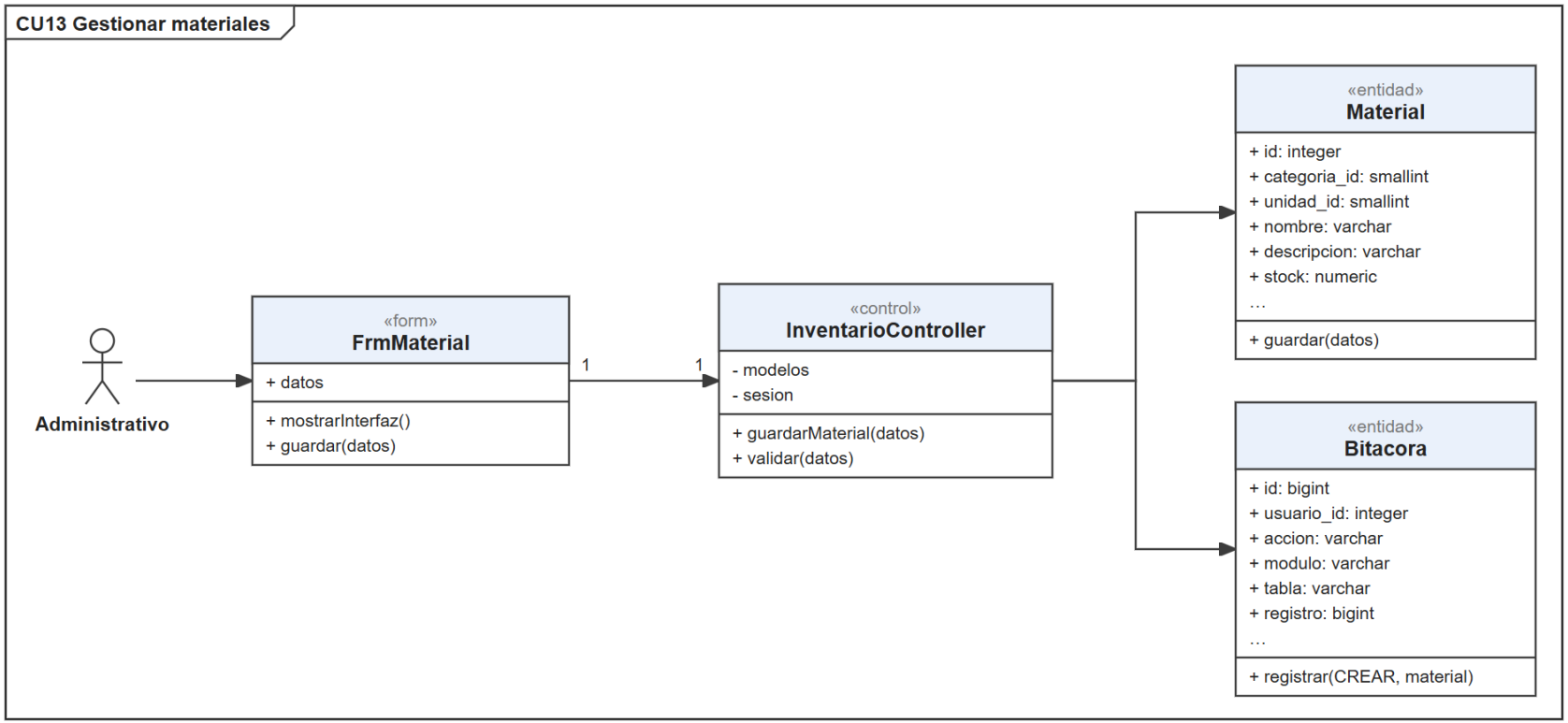
Nota. Elaboración propia.

Figura 124
Clases de análisis CU12 Gestionar alumnos y portal



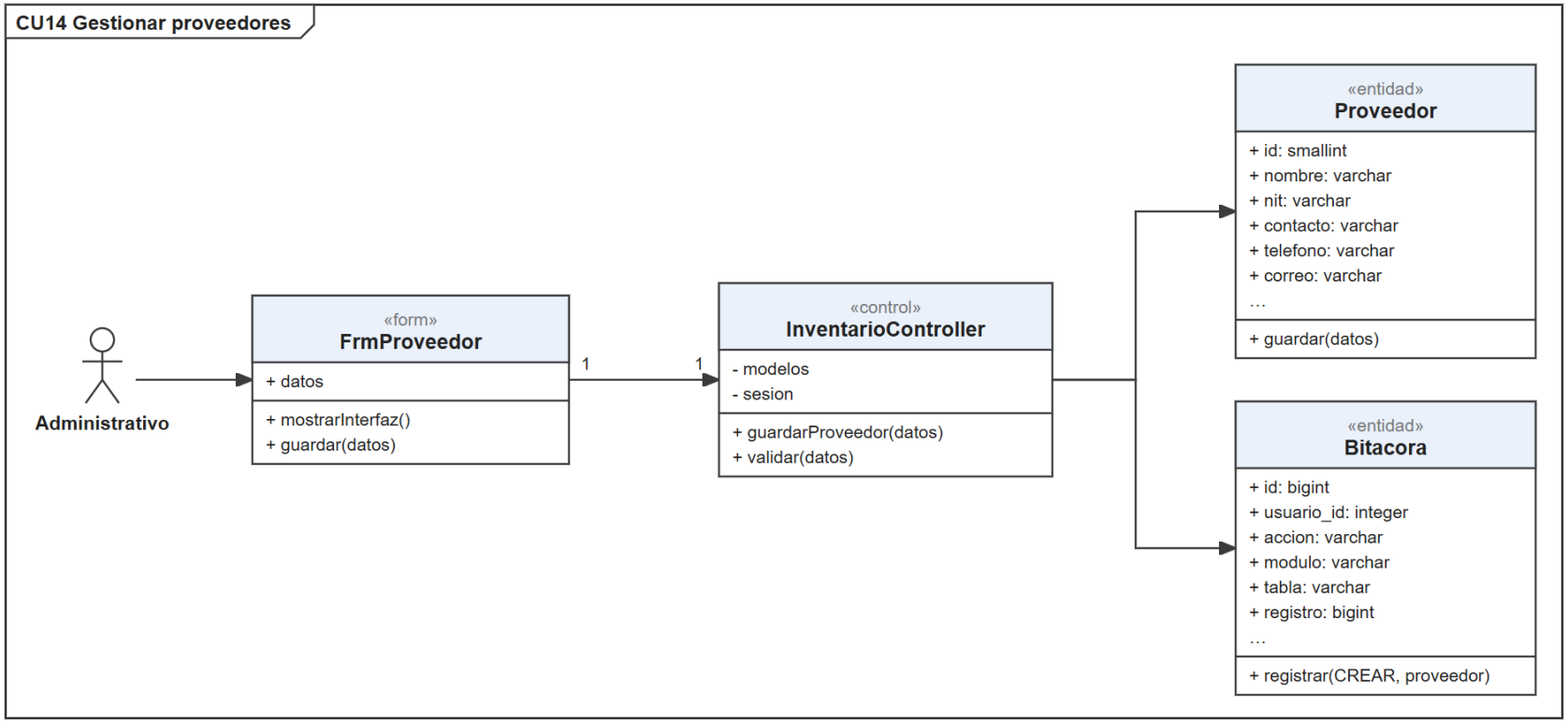
Nota. Elaboración propia.

Figura 125
Clases de análisis CU13 Gestionar materiales



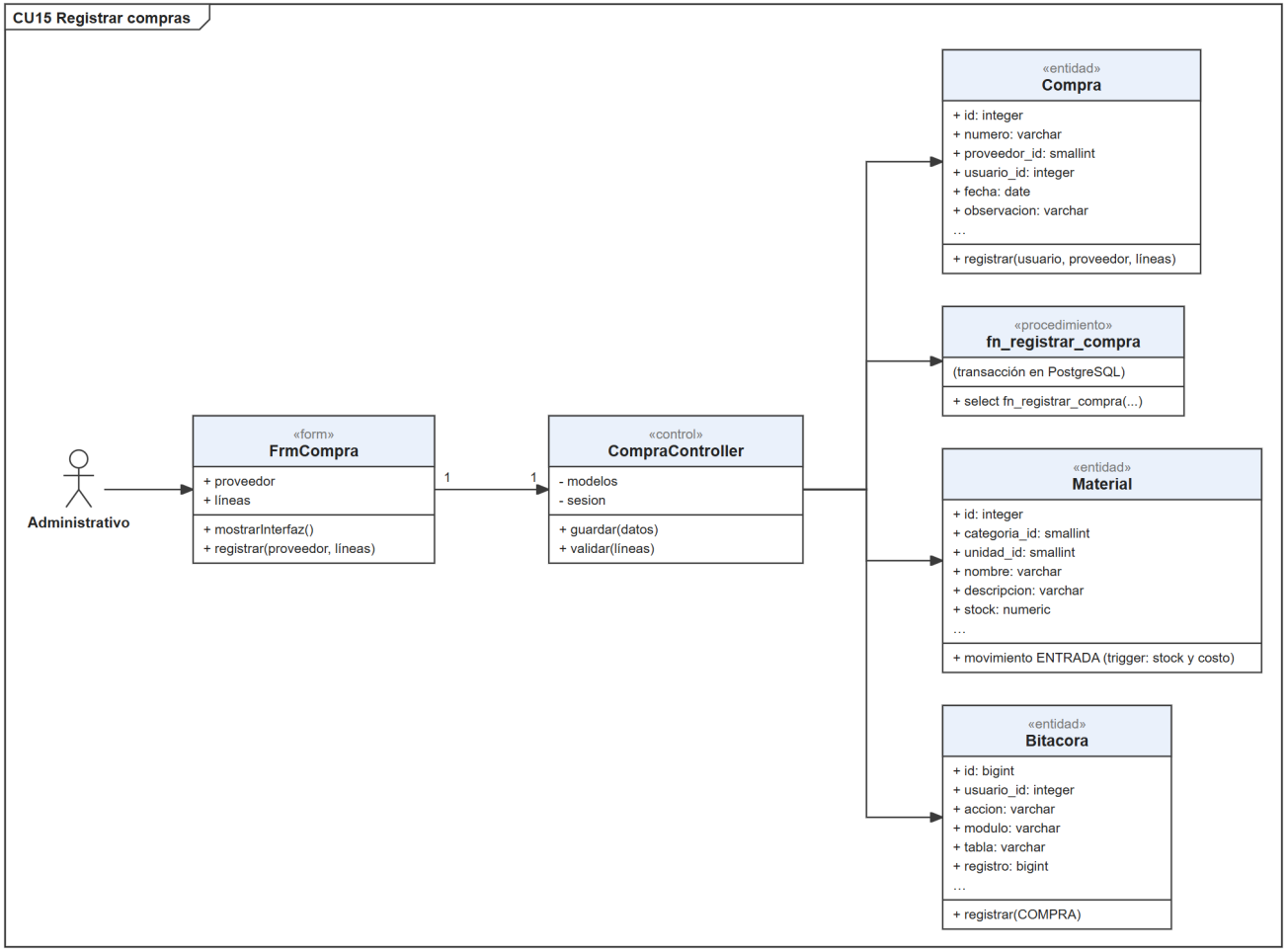
Nota. Elaboración propia.

Figura 126
Clases de análisis CU14 Gestionar proveedores



Nota. Elaboración propia.

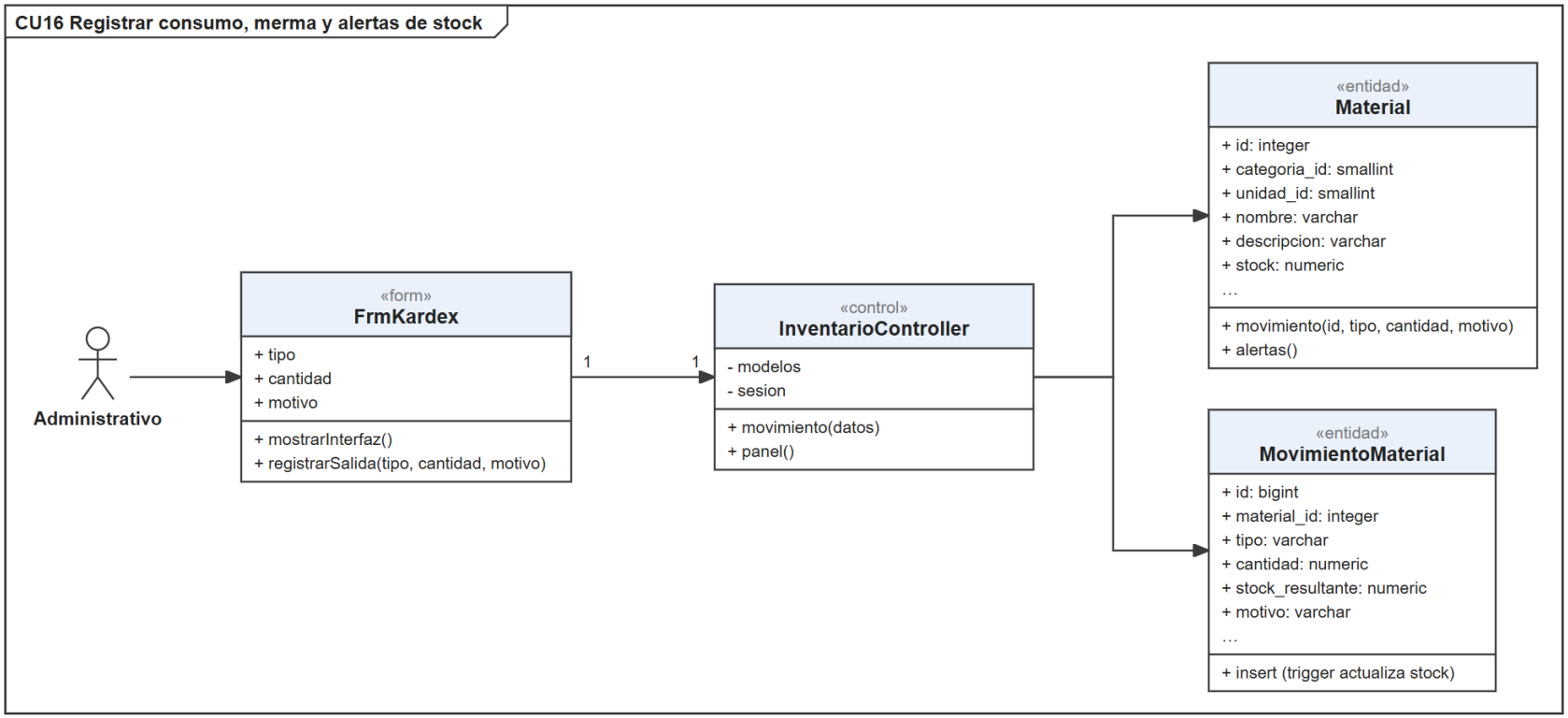
Figura 127
Clases de análisis CU15 Registrar compras



Nota. Elaboración propia.

Figura 128

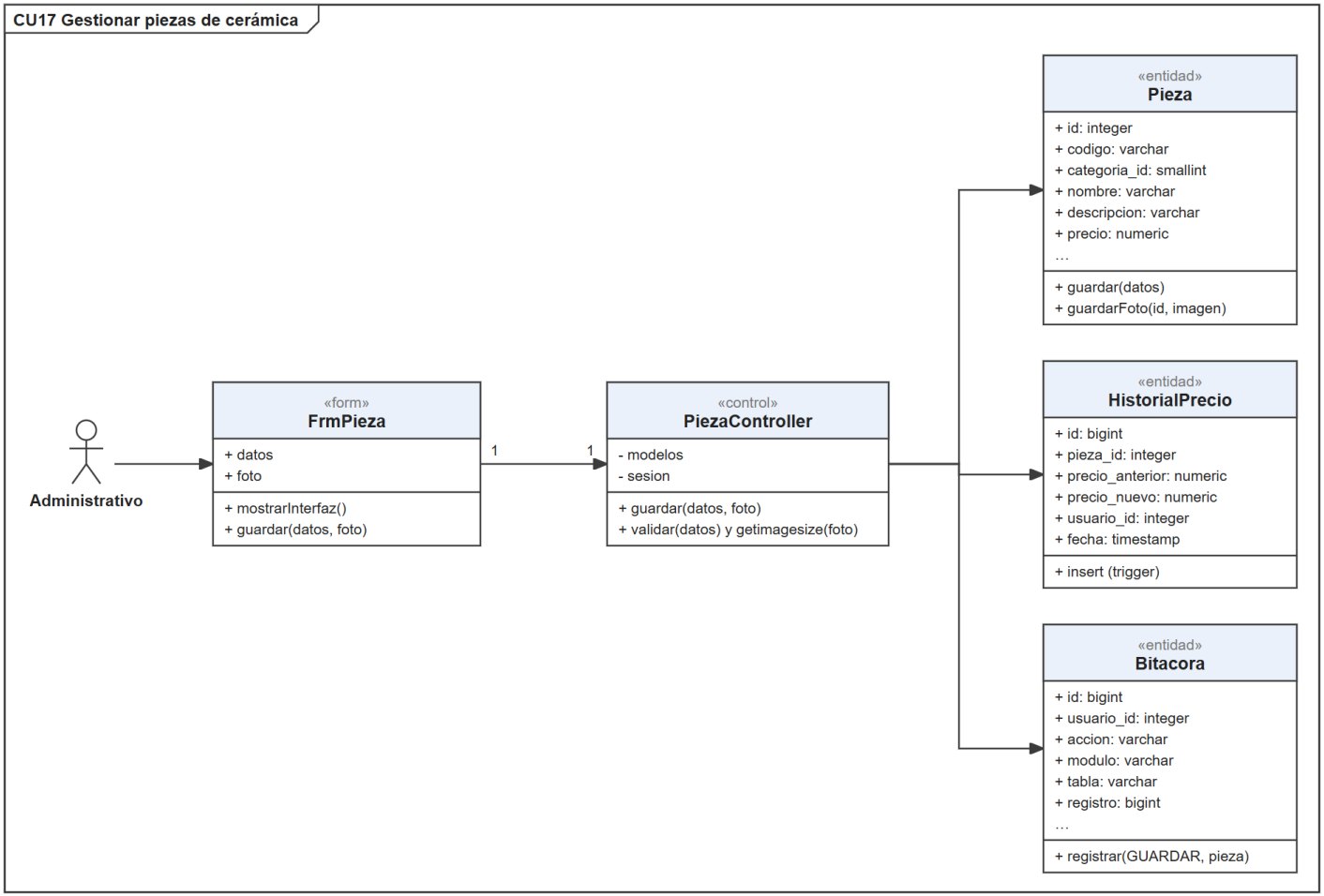
Clases de análisis CU16 Registrar consumo, merma y alertas de stock



Nota. Elaboración propia.

Figura 129

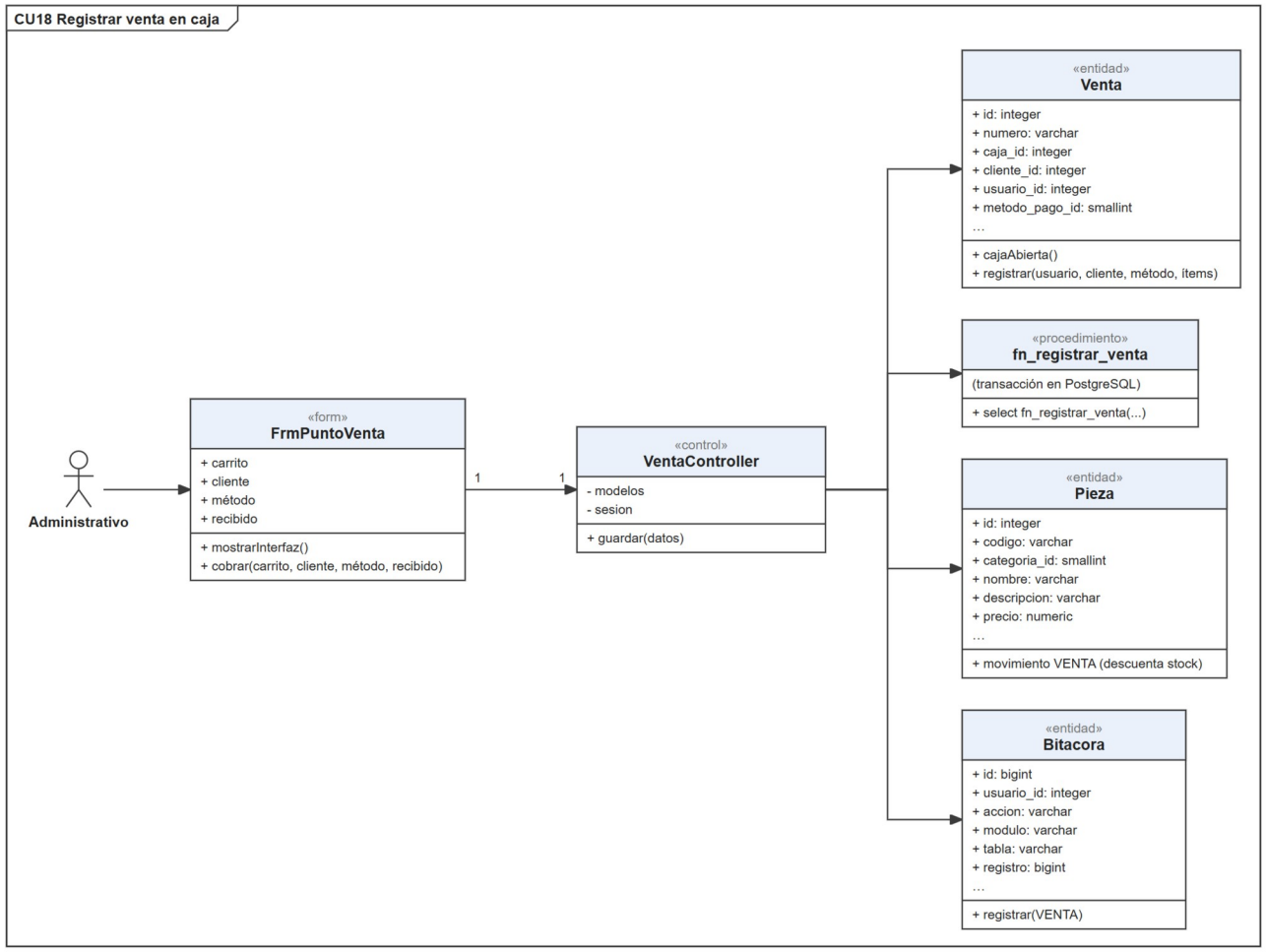
Clases de análisis CU17 Gestionar piezas de cerámica



Nota. Elaboración propia.

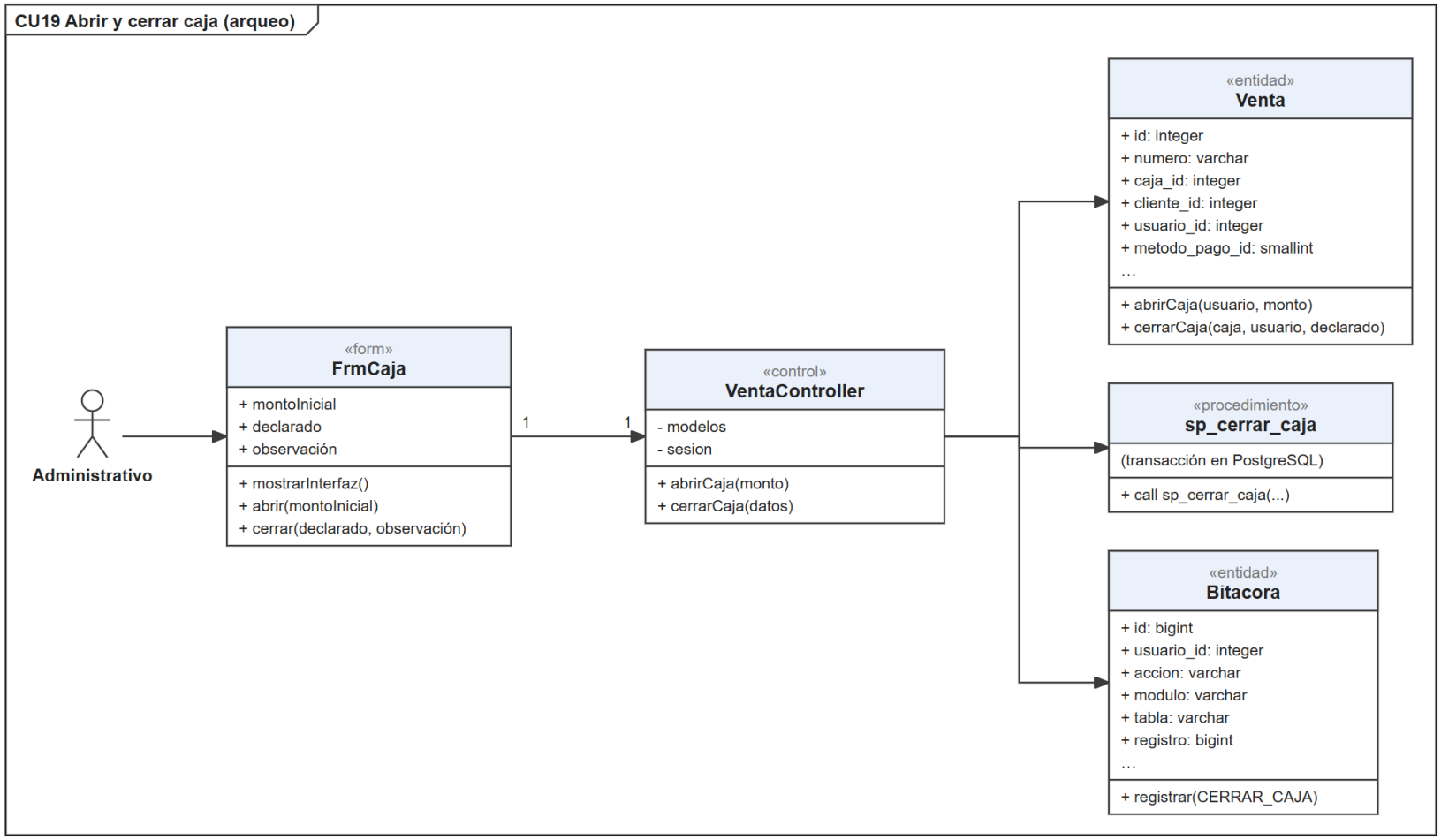
Figura 130

Clases de análisis CU18 Registrar venta en caja



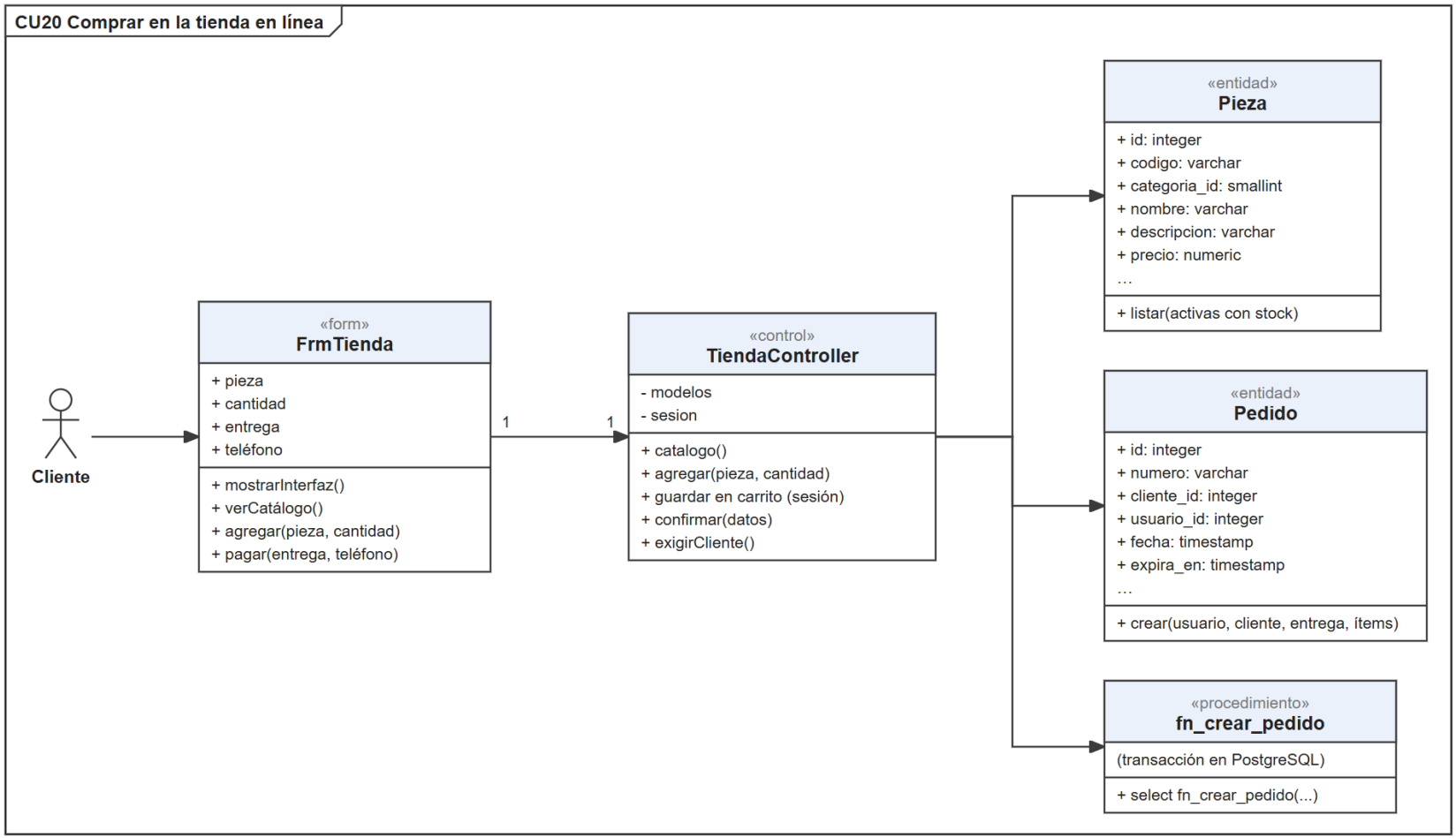
Nota. Elaboración propia.

Figura 131
Clases de análisis CU19 Abrir y cerrar caja (arqueo)



Nota. Elaboración propia.

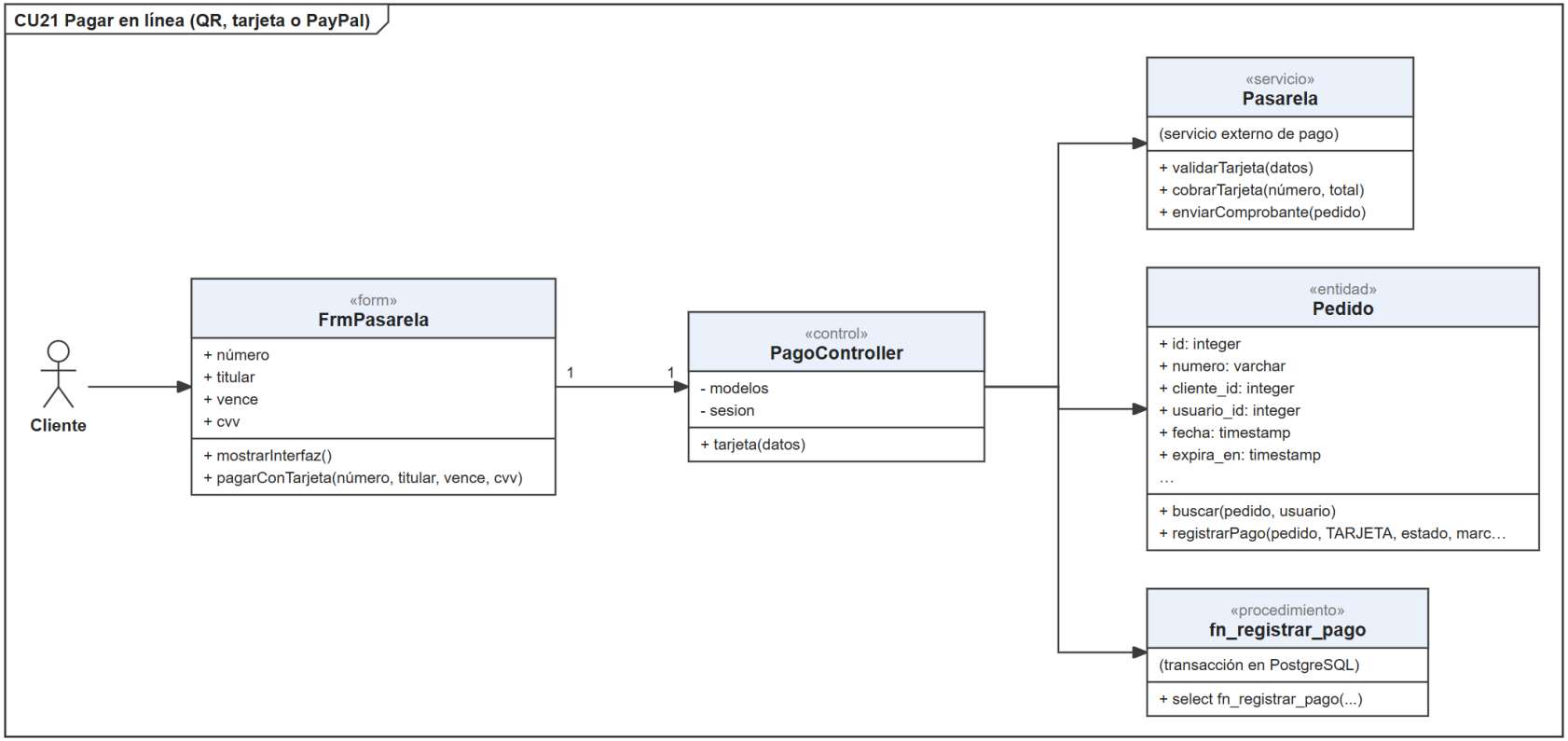
Figura 132
Clases de análisis CU20 Comprar en la tienda en línea



Nota. Elaboración propia.

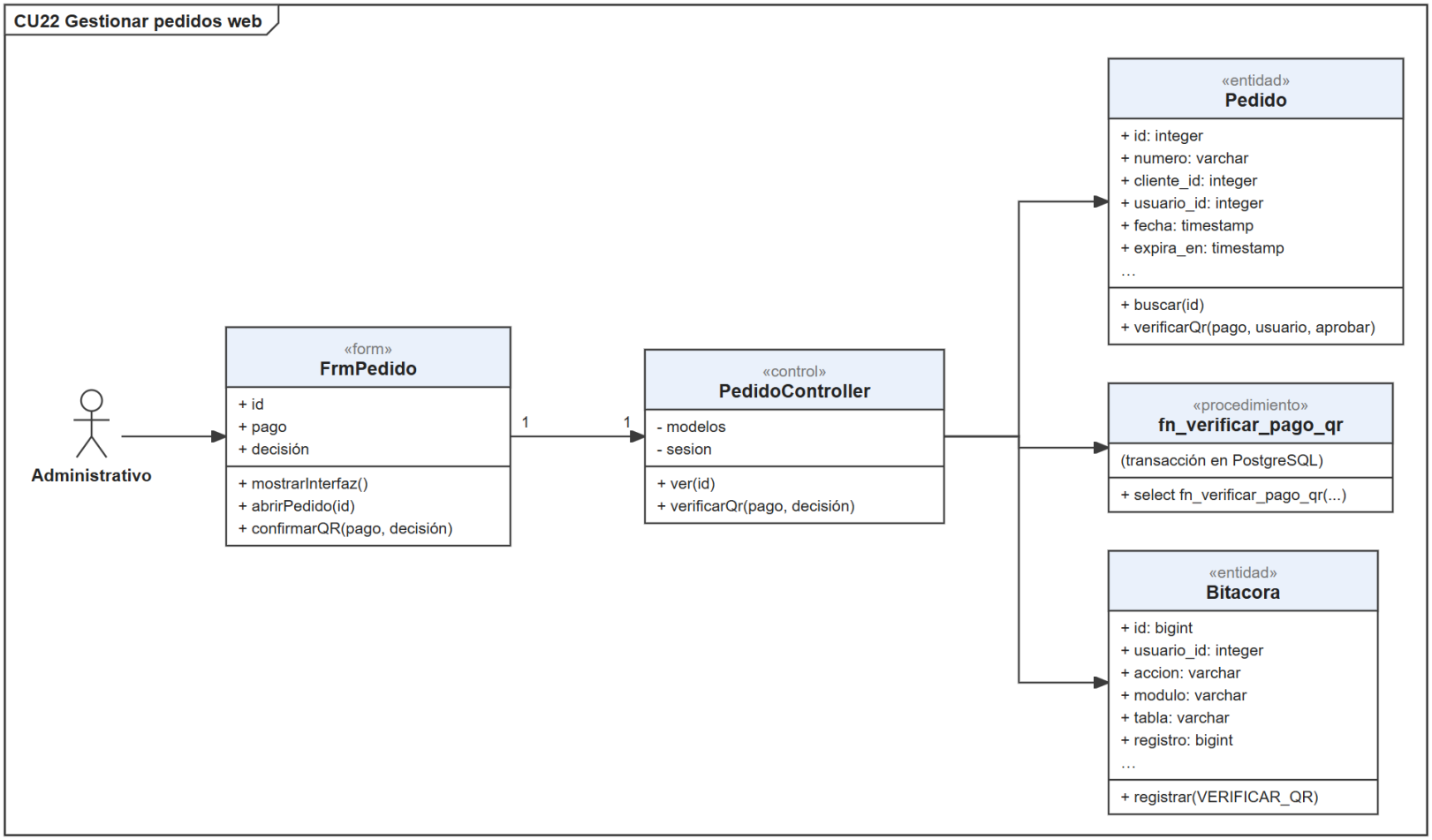
Figura 133

Clases de análisis CU21 Pagar en línea (QR, tarjeta o PayPal)



Nota. Elaboración propia.

Figura 134
Clases de análisis CU22 Gestionar pedidos web

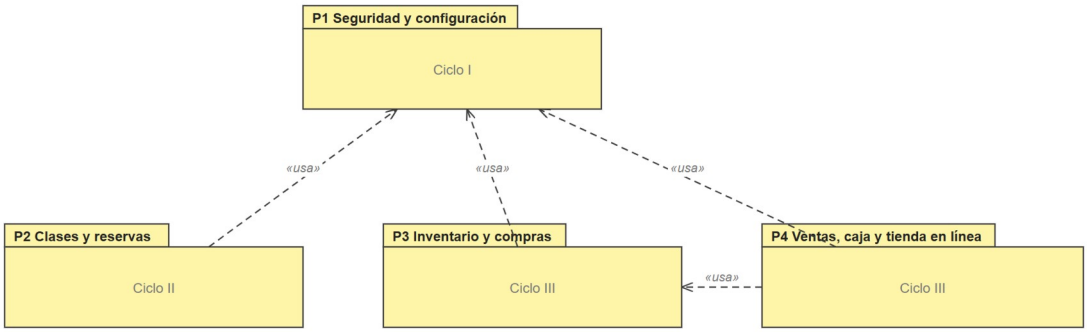


Nota. Elaboración propia.

3.4 Análisis de paquetes

Todos los paquetes dependen de P1, porque cada acción verifica la sesión y el permiso del usuario. P4 depende de P3 porque las piezas comparten categorías con el inventario. No hay dependencias en ciclo, por lo que cada paquete puede construirse y probarse en su propio ciclo.

Figura 135
Análisis de paquetes: dependencias



Nota. Elaboración propia.

Tabla 48
Contenido de cada paquete

Paquete	Interfaces	Controladores	Entidades
P1 Seguridad y configuración	FrmLogin, FrmRecuperar, FrmUsuario, FrmRoles, FrmBitacora, FrmConfiguracion	AuthController, RecuperacionController, UsuarioController, RolController, SistemaController	Usuario, Bitacora, Recuperacion, Rol, Configuracion
P2 Clases y reservas	FrmMaestra, FrmTaller, FrmCalendario, FrmReservar, FrmClase, FrmRegistro	MaestraController, TallerController, ClaseController, PortalController, AuthController	Maestra, Usuario, Bitacora, Taller, Sesion, Horario, Reserva, Alumno
P3 Inventario y compras	FrmMaterial, FrmProveedor, FrmCompra, FrmKardex	InventarioController, CompraController	Material, Bitacora, Proveedor, Compra, fn_registrar_compra, MovimientoMaterial
P4 Ventas, caja y tienda en línea	FrmPieza, FrmPuntoVenta, FrmCaja, FrmTienda, FrmPasarela, FrmPedido	PiezaController, VentaController, TiendaController, PagoController, PedidoController	Pieza, HistorialPrecio, Bitacora, Venta, fn_registrar_venta, sp_cerrar_caja, Pedido, fn_crear_pedido, Pasarela, fn_registrar_pago, fn_verificar_pago_qr

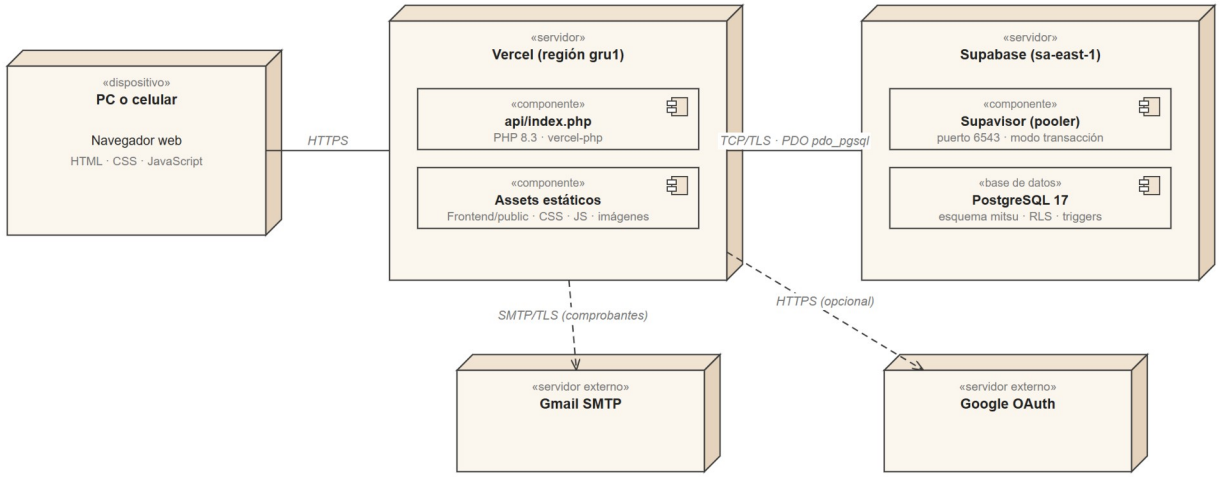
Capítulo 4. Flujo de trabajo: diseño

4.1 Diseño de arquitectura

4.1.1 Diseño de datos lógico (diagrama de despliegue)

El navegador del usuario se comunica por HTTPS con Vercel, donde corre la aplicación PHP. La aplicación se conecta a PostgreSQL en Supabase a través del pooler Supavisor en modo transacción (puerto 6543) con un usuario de mínimo privilegio. El correo SMTP y el ingreso con Google son servicios externos opcionales.

Figura 136
Diagrama de despliegue

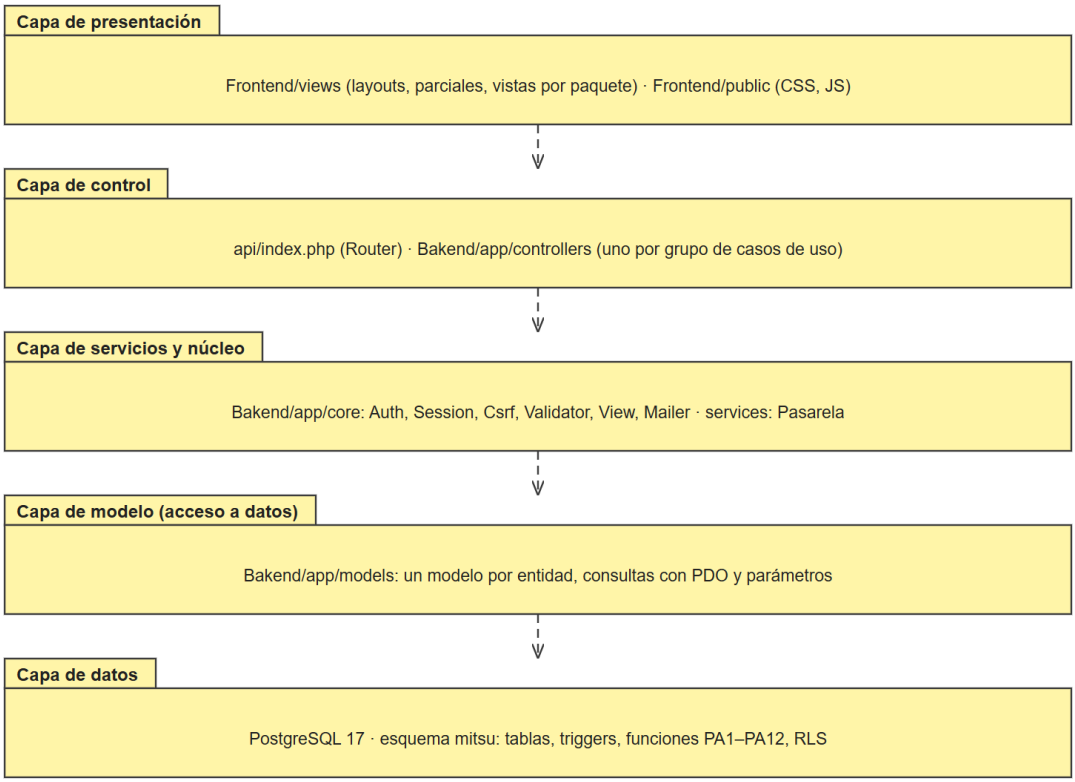


Nota. Elaboración propia.

4.1.2 Diseño de datos físico (diagrama organizado en capas)

La aplicación sigue el patrón MVC con cinco capas. Cada capa solo usa la de abajo: las vistas no consultan la base de datos y los modelos no generan HTML. Las reglas críticas se repiten en la capa de datos (triggers) como última defensa.

Figura 137
Arquitectura en capas



Nota. Elaboración propia.

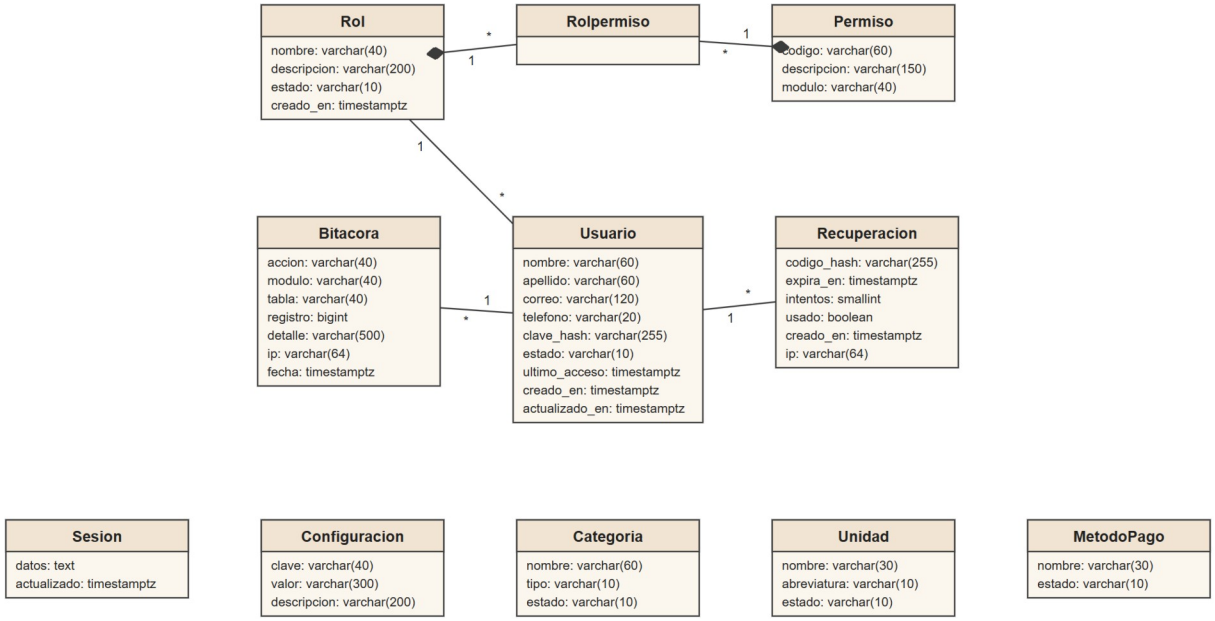
Capítulo 5. Diseño de datos

5.1 Diseño de datos lógico

Diagrama de clase

Las clases persistentes se derivan de las entidades de análisis. Se muestran por paquete; las llaves foráneas se representan como asociaciones y las composiciones (rombo negro) indican que la parte no existe sin el todo, como el detalle de una venta.

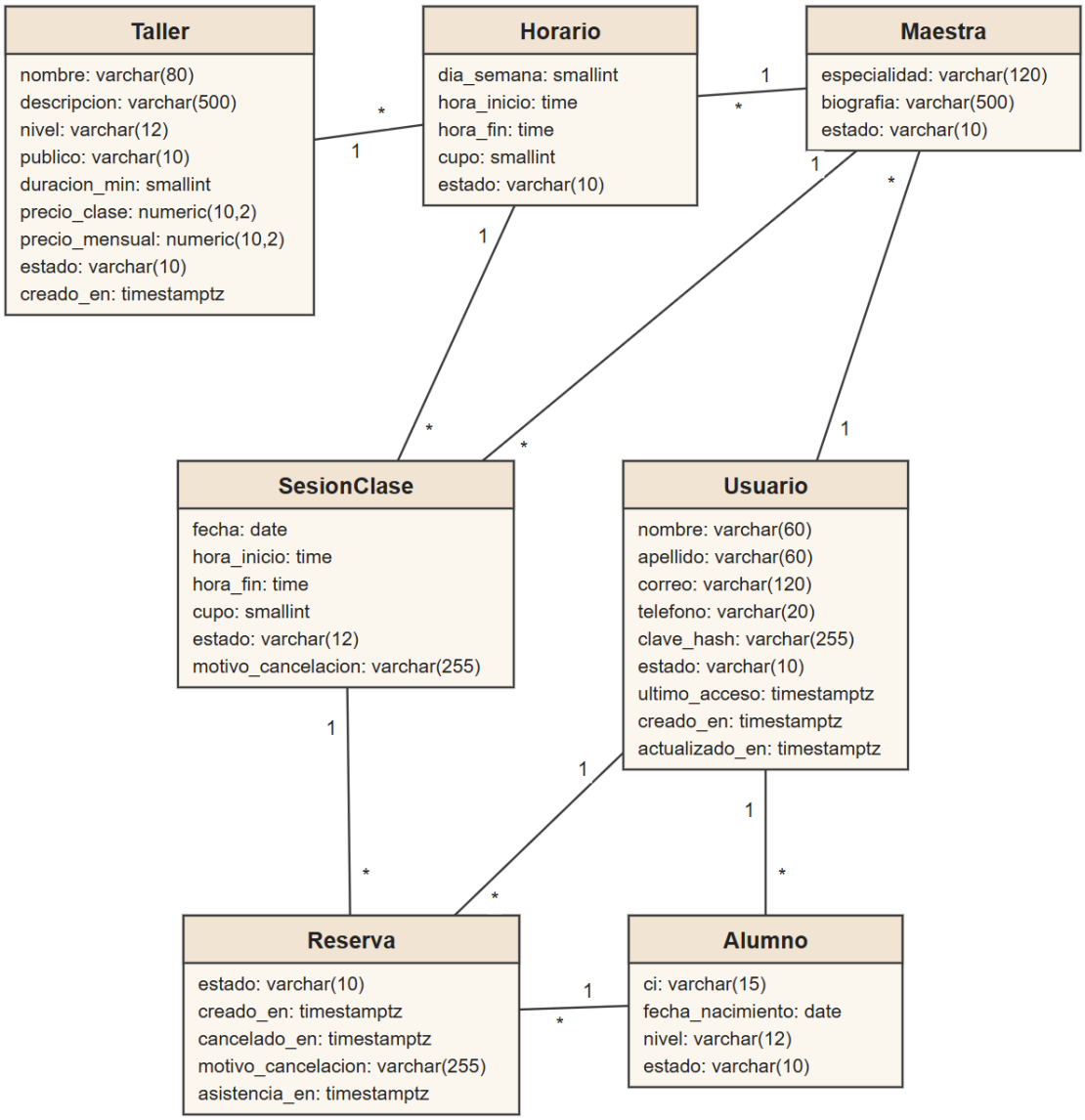
Figura 138
Clases persistentes de P1 Seguridad y configuración



Nota. Elaboración propia.

Figura 139

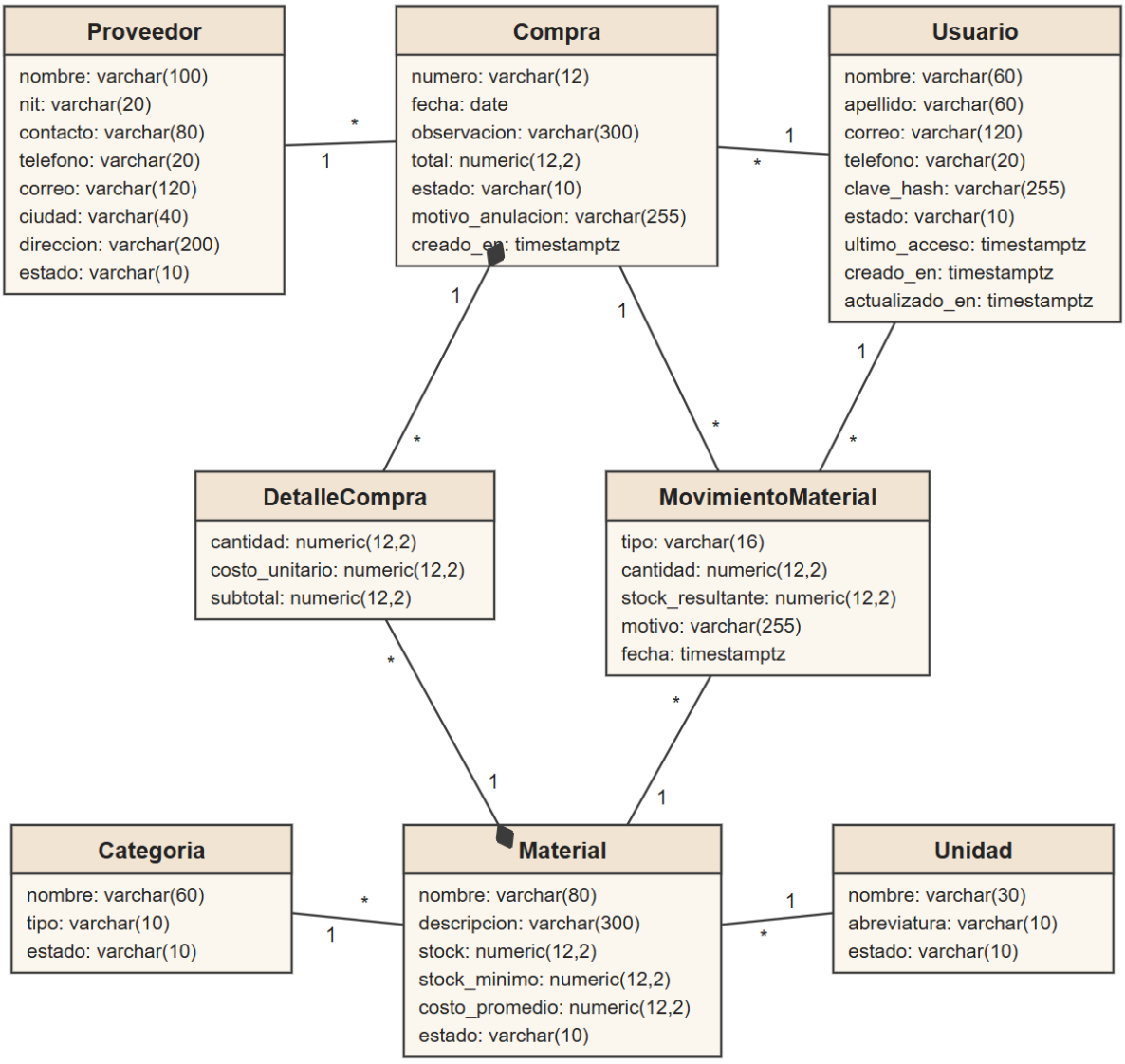
Clases persistentes de P2 Clases y reservas



Nota. Elaboración propia.

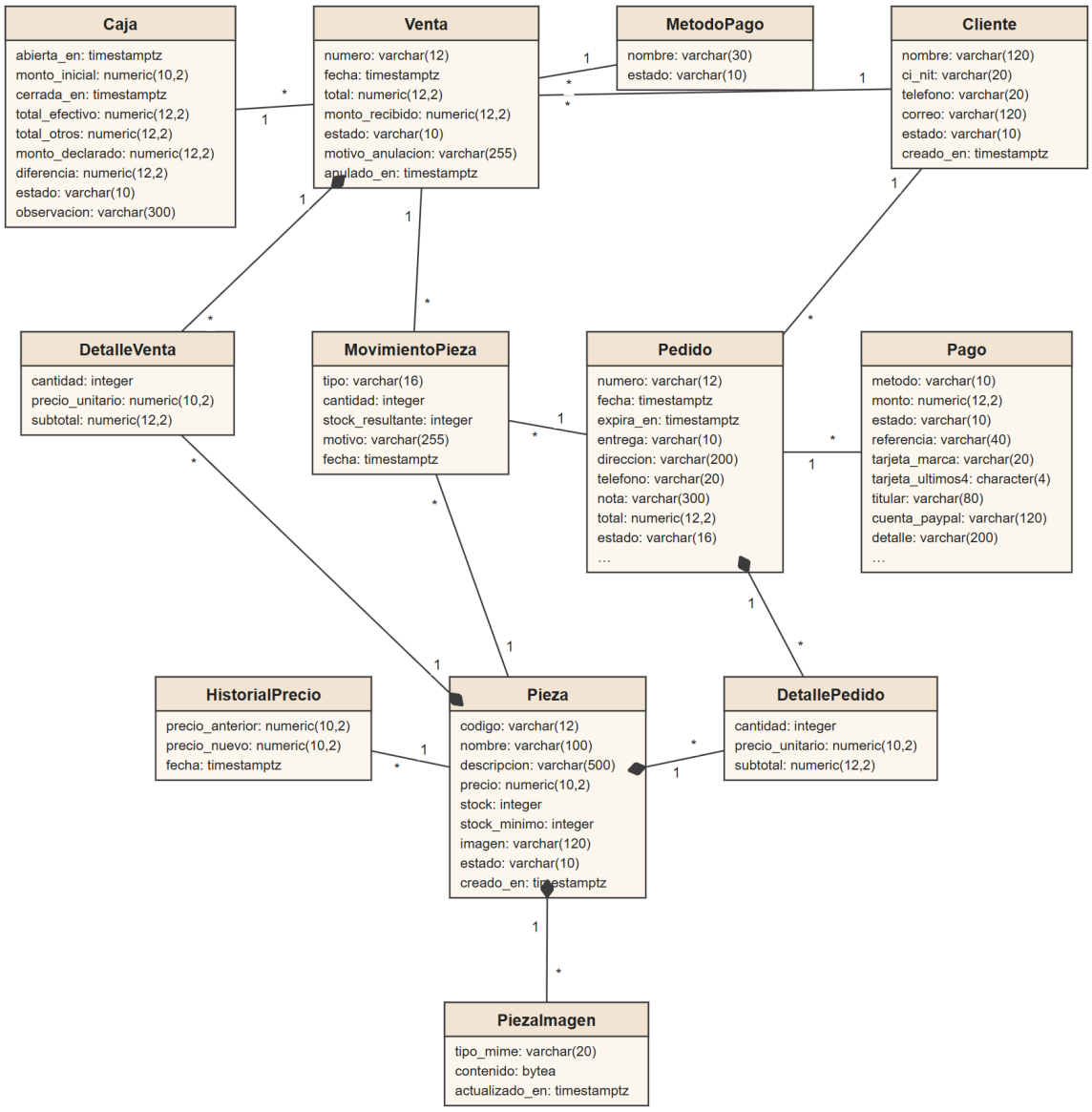
Figura 140

Clases persistentes de P3 Inventario y compras



Nota. Elaboración propia.

Figura 141
Clases persistentes de P4 Ventas, caja y tienda en línea



Nota. Elaboración propia.

Mapeo

Tabla 49
Mapeo de clases persistentes a tablas

Clase	Tabla	Llave primaria	Llaves foráneas → tabla
Alumno	`alumno`	`id`	`usuario_id → usuario`
Bitacora	`bitacora`	`id`	`usuario_id → usuario`
Caja	`caja`	`id`	`cerrada_por → usuario` `usuario_id → usuario`

Clase	Tabla	Llave primaria	Llaves foráneas → tabla
Categoria	categoria	id	
Cliente	cliente	id	usuario_id → usuario
Compra	compra	id	proveedor_id → proveedor usuario_id → usuario
Configuracion	configuracion	clave	
DetalleCompra	detalle_compra	id	compra_id → compra material_id → material
DetallePedido	detalle_pedido	id	pedido_id → pedido pieza_id → pieza
DetalleVenta	detalle_venta	id	pieza_id → pieza venta_id → venta
HistorialPrecio	historial_precio	id	pieza_id → pieza usuario_id → usuario
Horario	horario	id	maestra_id → maestra taller_id → taller
Maestra	maestra	id	usuario_id → usuario
Material	material	id	categoria_id → categoria unidad_id → unidad
MetodoPago	metodo_pago	id	
MovimientoMaterial	movimiento_material	id	compra_id → compra material_id → material usuario_id → usuario
MovimientoPieza	movimiento_pieza	id	pedido_id → pedido pieza_id → pieza usuario_id → usuario venta_id → venta
Pago	pago	id	pedido_id → pedido verificado_por → usuario
Pedido	pedido	id	cliente_id → cliente usuario_id → usuario
Permiso	permiso	id	
Pieza	pieza	id	categoria_id → categoria
PiezaImagen	pieza_imagen	pieza_id	pieza_id → pieza
Proveedor	proveedor	id	
Recuperacion	recuperacion	id	usuario_id → usuario
Reserva	reserva	id	alumno_id → alumno registrado_por → usuario sesion_id → sesion_clase
Rol	rol	id	
Rolpermiso	rolpermiso	rol_id,permiso_id	permiso_id → permiso rol_id → rol
Sesion	sesion	id	
SesionClase	sesion_clase	id	horario_id → horario maestra_id → maestra

Clase	Tabla	Llave primaria	Llaves foráneas → tabla
Taller	taller	id	
Unidad	unidad	id	
Usuario	usuario	id	rol_id → rol
Venta	venta	id	anulado_por → usuario caja_id → caja cliente_id → cliente metodo_pago_id → metodo_pago usuario_id → usuario

Nota. Generada desde el catálogo de PostgreSQL de la base publicada.

Normalización

Primera forma normal (1FN): cada columna guarda un solo valor. En la hoja de Excel del taller una fila decía "Martes y miércoles 10:00–12:00, Rosario y Elena". En el sistema cada horario es una fila de la tabla horario con un solo día, una sola hora de inicio y fin y una sola maestra.

Tabla 50
Ejemplo de 1FN: de la hoja de Excel a la tabla horario

Antes (Excel)	Después (tabla horario)
Modelado | Mar y Mié 10–12 | Rosario, Elena	taller=Modelado, dia_semana=2, 10:00–12:00, maestra=Rosario taller=Modelado, dia_semana=3, 10:00–12:00, maestra=Elena

Segunda forma normal (2FN): todo atributo depende de la llave completa. En rolpermiso, cuya llave es (rol_id, permiso_id), no se guarda el nombre del permiso, porque dependería solo de permiso_id; ese dato está en la tabla permiso. En detalle_venta la llave es id y precio_unitario depende del detalle (es el precio con que se vendió, copiado por un trigger), no de la pieza: es un dato histórico, no una redundancia.

Tercera forma normal (3FN): no hay dependencias transitivas. El material no guarda el nombre de su categoría ni de su unidad, sino categoria_id y unidad_id; así, si se renombra "kg" a

“kilogramo”, se cambia en un solo lugar. Lo mismo ocurre con el método de pago de una venta y con el rol de un usuario.

5.2 Diseño de datos físico

El diccionario de datos describe cada una de las 33 tablas de la base publicada: atributo, tipo de dato, amplitud, llave, si admite nulos y una descripción. Se generó desde el catálogo de PostgreSQL, por lo que coincide con la base real.

Tabla 51
Tabla alumno

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
usuario_id	integer		FK	No	Referencia a usuario
ci	varchar	15		Sí	Cédula de identidad
fecha_nacimiento	date			Sí	Fecha nacimiento
nivel	varchar	12		No	Nivel
estado	varchar	10		No	Estado del registro

Tabla 52
Tabla bitacora

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	bigint		PK	No	Identificador único
usuario_id	integer		FK	Sí	Referencia a usuario
accion	varchar	40		No	Acción realizada
modulo	varchar	40		No	Módulo del sistema
tabla	varchar	40		Sí	Tabla
registro	bigint			Sí	Registro
detalle	varchar	500		Sí	Detalle en formato JSON
ip	varchar	64		Sí	Dirección IP de origen
fecha	timestamptz			No	Fecha del registro

Tabla 53
Tabla caja

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
usuario_id	integer		FK	No	Referencia a usuario
abierta_en	timestamptz			No	Abierta en
monto_inicial	numeric	10,2		No	Monto inicial
cerrada_en	timestamptz			Sí	Cerrada en
cerrada_por	integer		FK	Sí	Cerrada por
total_efectivo	numeric	12,2		Sí	Total efectivo
total_otros	numeric	12,2		Sí	Total otros
monto_declarado	numeric	12,2		Sí	Monto declarado
diferencia	numeric	12,2		Sí	Diferencia
estado	varchar	10		No	Estado del registro
observacion	varchar	300		Sí	Observación

Tabla 54
Tabla categoria

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	smallint		PK	No	Identificador único
nombre	varchar	60		No	Nombre
tipo	varchar	10		No	Tipo o clase del registro
estado	varchar	10		No	Estado del registro

Tabla 55
Tabla cliente

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
nombre	varchar	120		No	Nombre
ci_nit	varchar	20		Sí	Ci nit
telefono	varchar	20		Sí	Teléfono de contacto
correo	varchar	120		Sí	Correo electrónico
estado	varchar	10		No	Estado del registro
creado_en	timestamptz			No	Fecha y hora de creación

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
usuario_id	integer		FK	Sí	Referencia a usuario

Tabla 56
Tabla compra

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
numero	varchar	12		No	Número correlativo del documento
proveedor_id	smallint		FK	No	Referencia a proveedor
usuario_id	integer		FK	No	Referencia a usuario
fecha	date			No	Fecha del registro
observacion	varchar	300		Sí	Observación
total	numeric	12,2		No	Importe total (Bs)
estado	varchar	10		No	Estado del registro
motivo_anulacion	varchar	255		Sí	Motivo anulacion
creado_en	timestamptz			No	Fecha y hora de creación

Tabla 57
Tabla configuracion

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
clave	varchar	40	PK	No	Nombre del parámetro
valor	varchar	300		No	Valor configurado
descripcion	varchar	200		No	Descripción

Tabla 58
Tabla detalle_compra

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
compra_id	integer		FK	No	Referencia a compra
material_id	integer		FK	No	Referencia a material
cantidad	numeric	12,2		No	Cantidad
costo_unitario	numeric	12,2		No	Costo unitario
subtotal	numeric	12,2		Sí	Subtotal de la línea (Bs)

Tabla 59
Tabla detalle_pedido

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
pedido_id	integer		FK	No	Referencia a pedido
pieza_id	integer		FK	No	Referencia a pieza
cantidad	integer			No	Cantidad
precio_unitario	numeric	10,2		No	Precio unitario aplicado (Bs)
subtotal	numeric	12,2		Sí	Subtotal de la línea (Bs)

Tabla 60
Tabla detalle_venta

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
venta_id	integer		FK	No	Referencia a venta
pieza_id	integer		FK	No	Referencia a pieza
cantidad	integer			No	Cantidad
precio_unitario	numeric	10,2		No	Precio unitario aplicado (Bs)
subtotal	numeric	12,2		Sí	Subtotal de la línea (Bs)

Tabla 61
Tabla historial_precio

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	bigint		PK	No	Identificador único
pieza_id	integer		FK	No	Referencia a pieza
precio_anterior	numeric	10,2		No	Precio anterior
precio_nuevo	numeric	10,2		No	Precio nuevo
usuario_id	integer		FK	Sí	Referencia a usuario
fecha	timestamptz			No	Fecha del registro

Tabla 62
Tabla horario

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
taller_id	smallint		FK	No	Referencia a taller
maestra_id	integer		FK	No	Referencia a maestra
dia_semana	smallint			No	Día de la semana (1 = lunes)
hora_inicio	time			No	Hora de inicio
hora_fin	time			No	Hora de fin
cupo	smallint			No	Cupo máximo de alumnos
estado	varchar	10		No	Estado del registro

Tabla 63

Tabla maestra

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
usuario_id	integer		FK	No	Referencia a usuario
especialidad	varchar	120		No	Especialidad
biografia	varchar	500		Sí	Biografia
estado	varchar	10		No	Estado del registro

Tabla 64

Tabla material

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
categoria_id	smallint		FK	No	Referencia a categoria
unidad_id	smallint		FK	No	Referencia a unidad
nombre	varchar	80		No	Nombre
descripcion	varchar	300		Sí	Descripción
stock	numeric	12,2		No	Existencia actual
stock_minimo	numeric	12,2		No	Existencia mínima antes de alertar
costo_promedio	numeric	12,2		No	Costo promedio ponderado (Bs)
estado	varchar	10		No	Estado del registro

Tabla 65
Tabla metodo_pago

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	smallint		PK	No	Identificador único
nombre	varchar	30		No	Nombre
estado	varchar	10		No	Estado del registro

Tabla 66
Tabla movimiento_material

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	bigint		PK	No	Identificador único
material_id	integer		FK	No	Referencia a material
tipo	varchar	16		No	Tipo o clase del registro
cantidad	numeric	12,2		No	Cantidad
stock_resultante	numeric	12,2		No	Stock resultante
motivo	varchar	255		No	Motivo del cambio o anulación
compra_id	integer		FK	Sí	Referencia a compra
usuario_id	integer		FK	No	Referencia a usuario
fecha	timestamptz			No	Fecha del registro

Tabla 67
Tabla movimiento_pieza

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	bigint		PK	No	Identificador único
pieza_id	integer		FK	No	Referencia a pieza
tipo	varchar	16		No	Tipo o clase del registro
cantidad	integer			No	Cantidad
stock_resultante	integer			No	Stock resultante
motivo	varchar	255		No	Motivo del cambio o anulación
venta_id	integer		FK	Sí	Referencia a venta
usuario_id	integer		FK	No	Referencia a usuario
fecha	timestamptz			No	Fecha del registro
pedido_id	integer		FK	Sí	Referencia a pedido

Tabla 68
Tabla pago

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
pedido_id	integer		FK	No	Referencia a pedido
metodo	varchar	10		No	Metodo
monto	numeric	12,2		No	Monto
estado	varchar	10		No	Estado del registro
referencia	varchar	40		No	Referencia
tarjeta_marca	varchar	20		Sí	Tarjeta marca
tarjeta_ultimos4	character	4		Sí	Tarjeta ultimos4
titular	varchar	80		Sí	Titular
cuenta_paypal	varchar	120		Sí	Cuenta paypal
detalle	varchar	200		Sí	Detalle en formato JSON
creado_en	timestamptz			No	Fecha y hora de creación
verificado_por	integer		FK	Sí	Verificado por
verificado_en	timestamptz			Sí	Verificado en

Tabla 69
Tabla pedido

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
numero	varchar	12		No	Número correlativo del documento
cliente_id	integer		FK	No	Referencia a cliente
usuario_id	integer		FK	No	Referencia a usuario
fecha	timestamptz			No	Fecha del registro
expira_en	timestamptz			No	Expira en
entrega	varchar	10		No	Entrega
direccion	varchar	200		Sí	Dirección
telefono	varchar	20		No	Teléfono de contacto
nota	varchar	300		Sí	Nota
total	numeric	12,2		No	Importe total (Bs)
estado	varchar	16		No	Estado del registro
motivo_cancelacion	varchar	255		Sí	Motivo cancelacion

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
pagado_en	timestamptz			Sí	Pagado en
entregado_en	timestamptz			Sí	Entregado en
comprobante_enviado	boolean			No	Comprobante enviado

Tabla 70
Tabla permiso

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	smallint		PK	No	Identificador único
codigo	varchar	60		No	Código
descripcion	varchar	150		No	Descripción
modulo	varchar	40		No	Módulo del sistema

Tabla 71
Tabla pieza

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
codigo	varchar	12		No	Código
categoria_id	smallint		FK	No	Referencia a categoria
nombre	varchar	100		No	Nombre
descripcion	varchar	500		Sí	Descripción
precio	numeric	10,2		No	Precio (Bs)
stock	integer			No	Existencia actual
stock_minimo	integer			No	Existencia mínima antes de alertar
imagen	varchar	120		Sí	Imagen
estado	varchar	10		No	Estado del registro
creado_en	timestamptz			No	Fecha y hora de creación

Tabla 72
Tabla pieza_imagen

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
pieza_id	integer		PK / FK	No	Referencia a pieza
tipo_mime	varchar	20		No	Tipo mime
contenido	bytea			No	Contenido

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
actualizado_en	timestamptz			No	Fecha y hora de la última modificación

Tabla 73
Tabla proveedor

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	smallint		PK	No	Identificador único
nombre	varchar	100		No	Nombre
nit	varchar	20		Sí	NIT
contacto	varchar	80		Sí	Contacto
telefono	varchar	20		Sí	Teléfono de contacto
correo	varchar	120		Sí	Correo electrónico
ciudad	varchar	40		No	Ciudad
direccion	varchar	200		Sí	Dirección
estado	varchar	10		No	Estado del registro

Tabla 74
Tabla recuperacion

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
usuario_id	integer		FK	No	Referencia a usuario
codigo_hash	varchar	255		No	Codigo hash
expira_en	timestamptz			No	Expira en
intentos	smallint			No	Intentos
usado	boolean			No	Usado
creado_en	timestamptz			No	Fecha y hora de creación
ip	varchar	64		Sí	Dirección IP de origen

Tabla 75
Tabla reserva

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
sesion_id	integer		FK	No	Referencia a sesion
alumno_id	integer		FK	No	Referencia a alumno

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
estado	varchar	10		No	Estado del registro
registrado_por	integer		FK	Sí	Registrado por
creado_en	timestamptz			No	Fecha y hora de creación
cancelado_en	timestamptz			Sí	Cancelado en
motivo_cancelacion	varchar	255		Sí	Motivo cancelacion
asistencia_en	timestamptz			Sí	Asistencia en

Tabla 76
Tabla rol

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	smallint		PK	No	Identificador único
nombre	varchar	40		No	Nombre
descripcion	varchar	200		Sí	Descripción
estado	varchar	10		No	Estado del registro
creado_en	timestamptz			No	Fecha y hora de creación

Tabla 77
Tabla rolpermiso

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
rol_id	smallint		PK / FK	No	Referencia a rol
permiso_id	smallint		PK / FK	No	Referencia a permiso

Tabla 78
Tabla sesion

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	varchar	128	PK	No	Identificador único
datos	text			No	Datos
actualizado	timestamptz			No	Actualizado

Tabla 79
Tabla sesion_clase

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
horario_id	integer		FK	No	Referencia a horario
maestra_id	integer		FK	No	Referencia a maestra
fecha	date			No	Fecha del registro
hora_inicio	time			No	Hora de inicio
hora_fin	time			No	Hora de fin
cupo	smallint			No	Cupo máximo de alumnos
estado	varchar	12		No	Estado del registro
motivo_cancelacion	varchar	255		Sí	Motivo cancelacion

Tabla 80

Tabla taller

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	smallint		PK	No	Identificador único
nombre	varchar	80		No	Nombre
descripcion	varchar	500		No	Descripción
nivel	varchar	12		No	Nivel
publico	varchar	10		No	Publico
duracion_min	smallint			No	Duracion min
precio_clase	numeric	10,2		No	Precio clase
precio_mensual	numeric	10,2		No	Precio mensual
estado	varchar	10		No	Estado del registro
creado_en	timestamptz			No	Fecha y hora de creación

Tabla 81

Tabla unidad

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	smallint		PK	No	Identificador único
nombre	varchar	30		No	Nombre
abreviatura	varchar	10		No	Abreviatura
estado	varchar	10		No	Estado del registro

Tabla 82
Tabla usuario

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
rol_id	smallint		FK	No	Referencia a rol
nombre	varchar	60		No	Nombre
apellido	varchar	60		No	Apellido
correo	varchar	120		No	Correo electrónico
telefono	varchar	20		Sí	Teléfono de contacto
clave_hash	varchar	255		No	Contraseña cifrada con bcrypt
estado	varchar	10		No	Estado del registro
ultimo_acceso	timestamptz			Sí	Último ingreso al sistema
creado_en	timestamptz			No	Fecha y hora de creación
actualizado_en	timestamptz			No	Fecha y hora de la última modificación

Tabla 83
Tabla venta

Atributo	Tipo de dato	Amplitud	Llave	Nulo	Descripción
id	integer		PK	No	Identificador único
numero	varchar	12		No	Número correlativo del documento
caja_id	integer		FK	No	Referencia a caja
cliente_id	integer		FK	No	Referencia a cliente
usuario_id	integer		FK	No	Referencia a usuario
metodo_pago_id	smallint		FK	No	Referencia a metodo pago
fecha	timestamptz			No	Fecha del registro
total	numeric	12,2		No	Importe total (Bs)
monto_recibido	numeric	12,2		Sí	Monto recibido
estado	varchar	10		No	Estado del registro
motivo_anulacion	varchar	255		Sí	Motivo anulacion
anulado_por	integer		FK	Sí	Anulado por
anulado_en	timestamptz			Sí	Anulado en

Tabla de volumen

Tabla 84
Volumen de datos por tabla

Tabla	Filas actuales	Tamaño actual (KB)	Crecimiento anual estimado (filas)
alumno	10	56	120
bitacora	98	88	6000
caja	15	40	300
categoria	9	40	menos de 10
cliente	7	72	250
compra	6	64	60
configuracion	6	32	menos de 10
detalle_compra	16	40	150
detalle_pedido	15	40	450
detalle_venta	41	40	1400
historial_precio	1	24	30
horario	13	48	menos de 10
maestra	4	48	menos de 10
material	11	72	10
metodo_pago	3	40	menos de 10
movimiento_material	25	40	400
movimiento_pieza	70	72	1800
pago	14	64	380
pedido	15	80	300
permiso	23	40	menos de 10
pieza	8	96	40
pieza_imagen	0	16	40
proveedor	3	64	menos de 10
recuperacion	0	16	40
reserva	250	96	1500
rol	5	40	menos de 10
rolpermiso	38	24	menos de 10
sesion	102	88	3000
sesion_clase	80	96	650
taller	7	48	menos de 10
unidad	4	40	menos de 10
usuario	17	80	150

Tabla	Filas actuales	Tamaño actual (KB)	Crecimiento anual estimado (filas)
venta	31	112	900

Nota. Filas y tamaño medidos en la base publicada con datos de demostración. El crecimiento anual es una estimación del equipo con base en 7 talleres, 13 horarios semanales y unas 3 ventas diarias.

Script

El script está dividido en siete archivos en la carpeta Bakend/database del repositorio: 01_esquema.sql (tablas, llaves, índices y restricciones), 02_reglas_negocio.sql (triggers RN01 a RN10), 03_datos_iniciales.sql (datos de demostración), 04_usuario_app.sql (usuario de mínimo privilegio y seguridad por filas), 05_consultas.sql (las 30 consultas), 06_procedimientos_triggers.sql (procedimientos PA1 a PA7) y 07_tienda_pasarela.sql (tienda en línea y pagos, PA8 a PA12). En total la base tiene 33 tablas y 20 triggers. A continuación se muestra un fragmento; el script completo está en el Anexo E.

```
create table horario (
    id          serial primary key,
    taller_id   smallint not null references taller (id),
    maestra_id  integer  not null references maestra (id),
    dia_semana  smallint not null check (dia_semana between 1 and 7),   -- 1 = lunes (ISO)
    hora_inicio time     not null,
    hora_fin    time     not null,
    cupo        smallint not null check (cupo between 1 and 30),
    estado      varchar(10) not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO')),
    check (hora_fin > hora_inicio),
    -- Una maestra no puede tener dos horarios activos que se crucen el mismo día
    constraint horario_sin_cruce_maestra exclude using gist (
        maestra_id with =,
        dia_semana with =,
        tsrange(date '2000-01-01' + hora_inicio, date '2000-01-01' + hora_fin) with &&
    ) where (estado = 'ACTIVO')
);
create index ix_horario_taller on horario (taller_id);

-- Sesión = clase concreta de un horario en una fecha
create table sesion_clase (
    id                 serial primary key,
    horario_id         integer     not null references horario (id),
    maestra_id         integer     not null references maestra (id),   -- puede reemplazarse
    fecha              date        not null,
    hora_inicio        time        not null,
    hora_fin           time        not null,
    cupo               smallint    not null check (cupo between 1 and 30),
    estado             varchar(12) not null default 'PROGRAMADA' check (estado in ('PROGRAMADA',
'REALIZADA', 'CANCELADA')),
    motivo_cancelacion varchar(255),
    check (hora_fin > hora_inicio),
    check (estado <> 'CANCELADA' or motivo_cancelacion is not null),
```

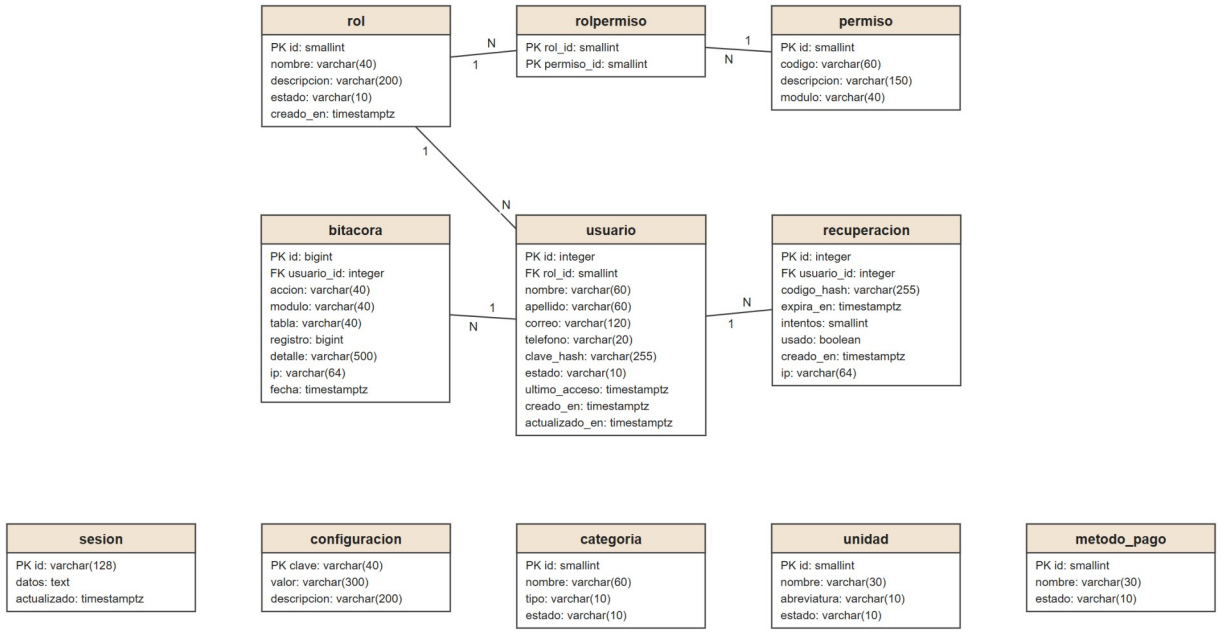
```
    unique (horario_id, fecha)
);
create index ix_sesion_fecha on sesion_clase (fecha);

create table reserva (
    id                 serial primary key,
    sesion_id          integer     not null references sesion_clase (id),
    alumno_id          integer     not null references alumno (id),
    estado             varchar(10) not null default 'RESERVADA' check (estado in ('RESERVADA',
'CANCELADA', 'ASISTIO', 'FALTO')),
    registrado_por     integer     references usuario (id),
    creado_en          timestamptz not null default now(),
    cancelado_en       timestamptz,
    motivo_cancelacion varchar(255),
    asistencia_en      timestamptz,
    check (estado <> 'CANCELADA' or motivo_cancelacion is not null)
);
```

Diagrama relacional

Figura 142

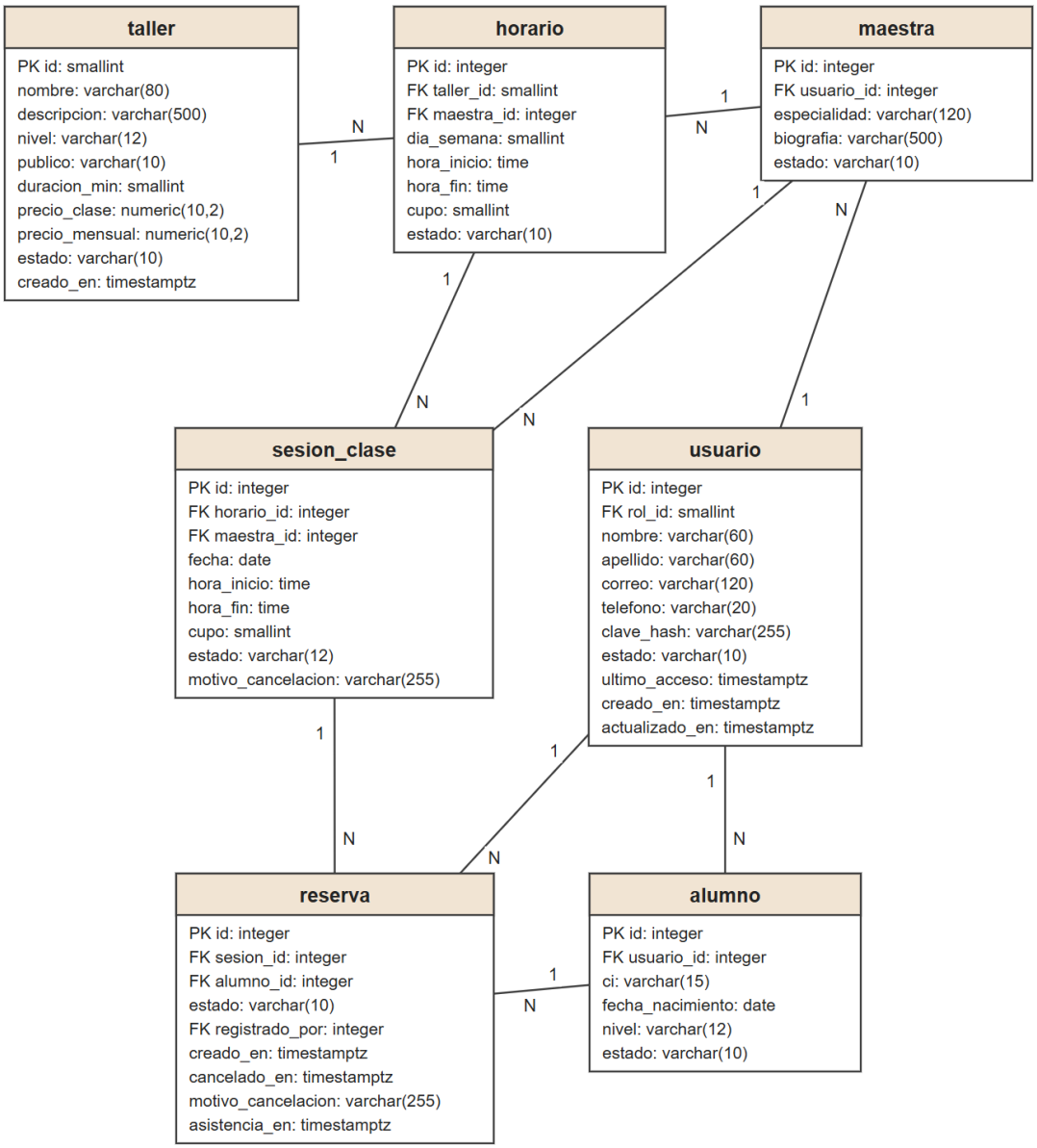
Diagrama relacional de P1 Seguridad y configuración



Nota. PK: llave primaria; FK: llave foránea. N y 1 indican la cardinalidad. Generado desde las tablas reales de la base publicada.

Figura 143

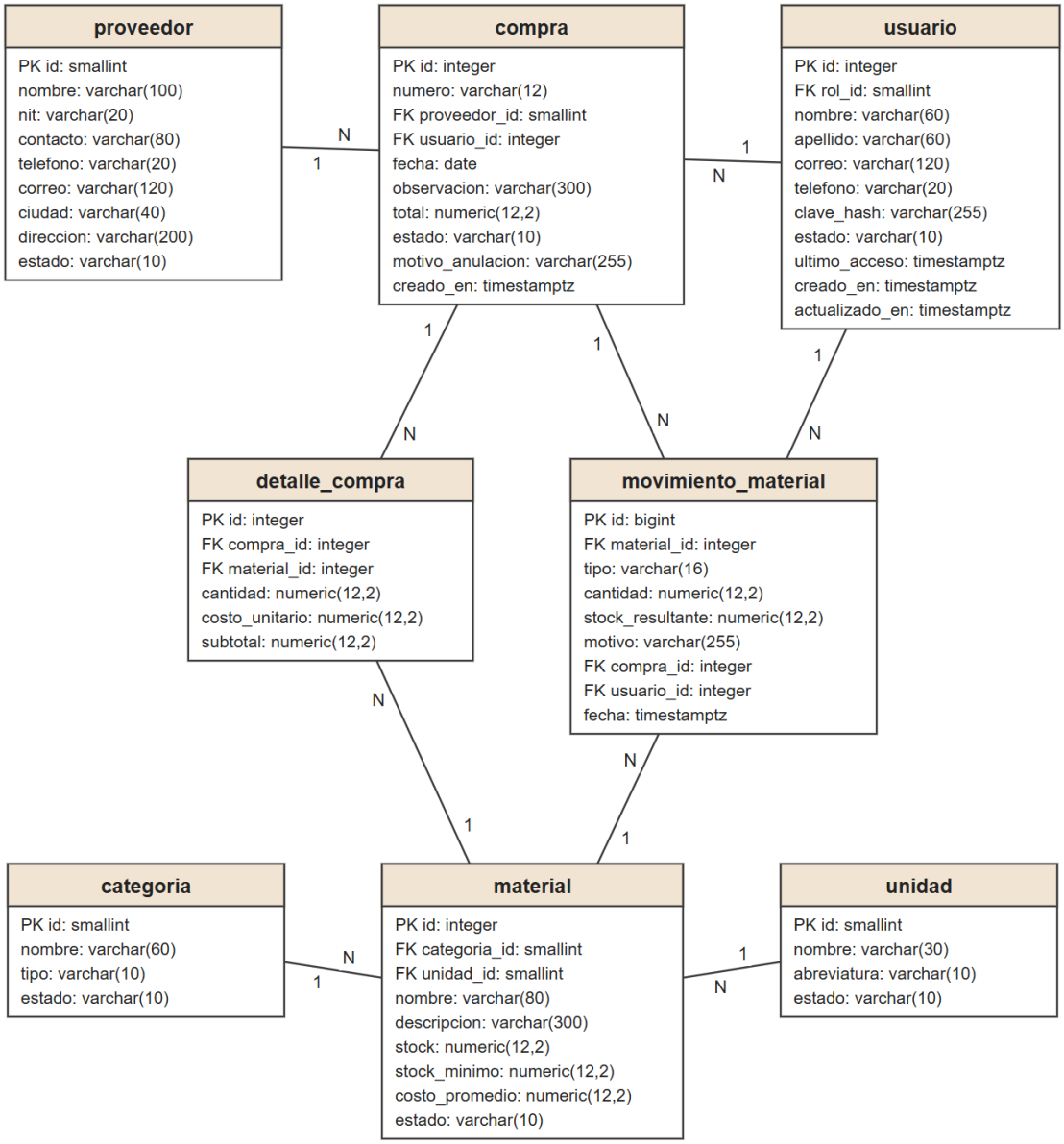
Diagrama relacional de P2 Clases y reservas



Nota. PK: llave primaria; FK: llave foránea. N y 1 indican la cardinalidad. Generado desde las tablas reales de la base publicada.

Figura 144

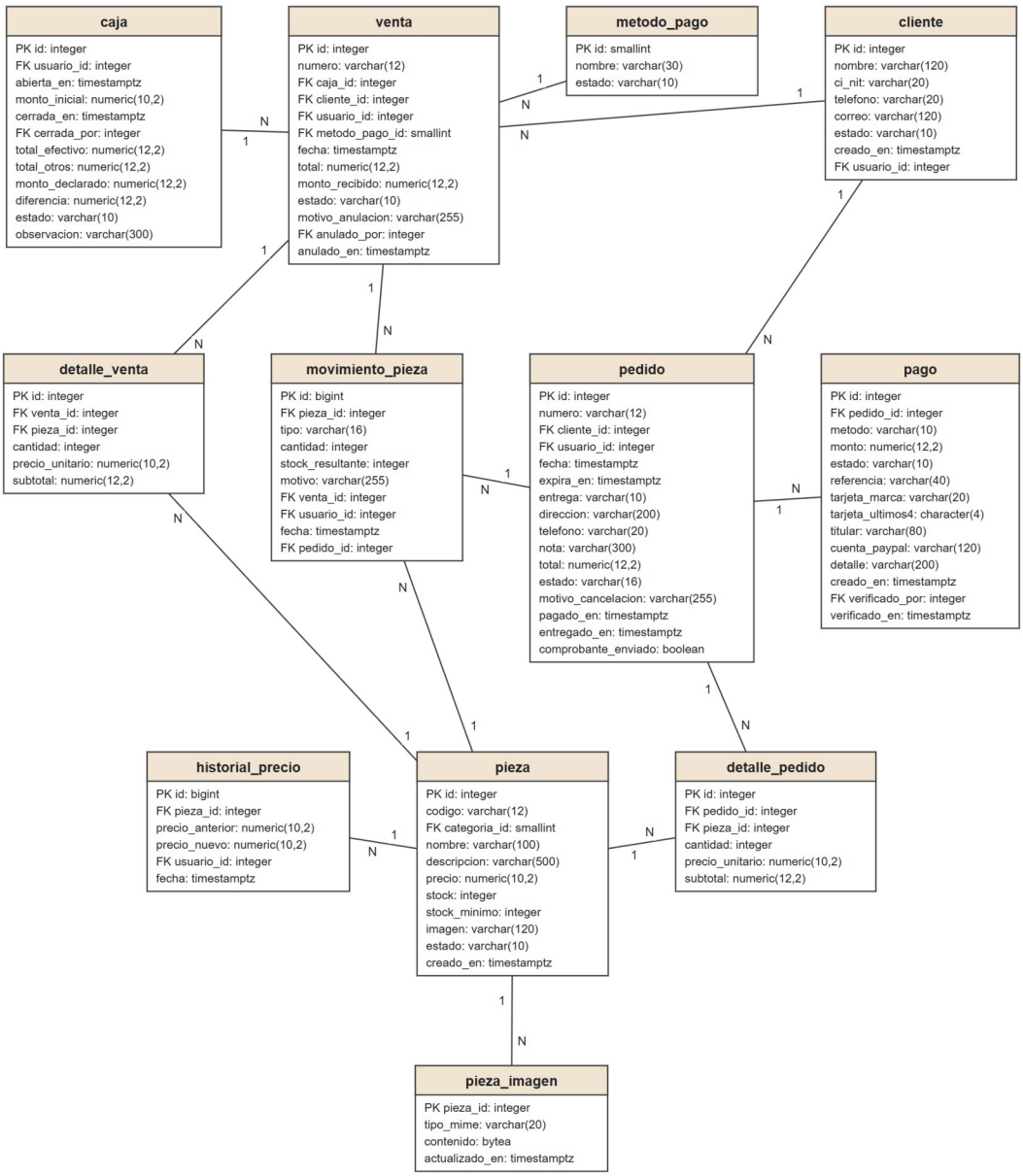
Diagrama relacional de P3 Inventario y compras



Nota. PK: llave primaria; FK: llave foránea. N y 1 indican la cardinalidad. Generado desde las tablas reales de la base publicada.

Figura 145

Diagrama relacional de P4 Ventas, caja y tienda en línea



Nota. PK: llave primaria; FK: llave foránea. N y 1 indican la cardinalidad. Generado desde las tablas reales de la base publicada.

Actualización de tuplas (población de datos)

La base se pobló con el script 03_datos_iniciales.sql: 5 roles, 23 permisos, 4 maestras, 10 alumnos, 7 talleres con 13 horarios (los de la entrevista), clases de tres semanas antes y después de la fecha actual, proveedores, materiales, compras, piezas con fotos del taller y ventas de 20

días. Los ejemplos siguientes se ejecutaron en la base publicada dentro de una transacción que luego se revirtió, para no alterar los datos.

Insert(): Insertar un proveedor nuevo

```
insert into proveedor (nombre, nit, contacto, telefono, ciudad)
values ('Arcillas del Chaco (demo)', '4056789014', 'Rosa Ayala', '46640001', 'Tarija') returning
id, nombre, ciudad;
```

Resultado: 1 fila(s) afectada(s).

id	nombre	ciudad
7	Arcillas del Chaco (demo)	Tarija

Update(): Subir 5 % el precio de las tazas (queda en historial_precio por trigger)

```
update pieza set precio = round(precio * 1.05, 2)
 where categoria_id = (select id from categoria where nombre = 'Tazas y vasos') returning codigo,
nombre, precio;
```

Resultado: 2 fila(s) afectada(s).

codigo	nombre	precio
TZ-002	Vaso Mejillas	99.75
TZ-001	Taza esmaltada Nube	99.75

Delete(): Borrar sesiones PHP vencidas (la única tabla donde se borra: no tiene historial)

```
delete from sesion where actualizado < now() - interval '2 hours' returning id;
```

Resultado: 12 fila(s) afectada(s).

id
1ov80s5v0q4kmk10gs6up4t93j
h9jvsd835fsjeejqi56bekai7s
t5esn09r49uk129g9upt398bl6
ctnsrh2f9rtmqba38o6566malp
8rn7uv1u0j969p871c3sobt8k1
ta9q8dtj413neg1sun9t55f0n8

Consultas (30 SQL)

Las 30 consultas se ejecutaron contra la base publicada con el usuario de la aplicación; se muestran hasta seis filas del resultado real de cada una.

Consultas simples

Q01. *Talleres activos ordenados por precio mensual*

```
select nombre, nivel, publico, precio_clase, precio_mensual from taller where estado = 'ACTIVO'
order by precio_mensual desc;
```

Resultado: 7 fila(s) (se muestran 6).

nombre	nivel	publico	precio_clase	precio_mensual
Modelado y torno	INTERMEDIO	ADULTOS	120.00	420.00
Torno alfarero	INTERMEDIO	ADULTOS	120.00	420.00
Modelado avanzado	AVANZADO	ADULTOS	110.00	400.00
Taller de sábado para adultos	TODOS	ADULTOS	100.00	360.00
Taller de modelado nocturno	TODOS	ADULTOS	90.00	320.00
Modelado a mano	INICIAL	ADULTOS	90.00	320.00

Q02. *Materiales en o bajo el stock mínimo (alerta de compra)*

```
select nombre, stock, stock_minimo from material where stock <= stock_minimo and estado =
'ACTIVO' order by stock;
```

Resultado: 4 fila(s).

nombre	stock	stock_minimo
Óxido de hierro	2.00	2.00
Esmalte transparente	3.00	5.00
Pinceles de cerámica	9.00	12.00
Arcilla roja	25.00	40.00

Q03. *Piezas a la venta con menos de 5 unidades*

```
select codigo, nombre, precio, stock from pieza where estado = 'ACTIVO' and stock < 5 order by
stock, nombre;
```

Resultado: 7 fila(s) (se muestran 6).

codigo	nombre	precio	stock
JR-001	Jarra Andina	350.00	0
VS-001	Vasija Inti	320.00	0
VJ-001	Set café: vaso y bandeja	160.00	1
VJ-003	Bowl esmaltado mediano	90.00	2
TZ-001	Taza esmaltada Nube	95.00	2
VS-002	Vasija Pajarito	290.00	2

Q04. *Clases programadas para hoy*

```
select fecha, hora_inicio, hora_fin, cupo, estado from sesion_clase where fecha = hoy_bo() order
by hora_inicio;
```

Resultado: 2 fila(s).

fecha	hora_inicio	hora_fin	cupo	estado
2026-10-29	10:00:00	12:00:00	8	PROGRAMADA
2026-10-29	10:00:00	12:00:00	6	PROGRAMADA

Q05. *Proveedores de Cochabamba o La Paz*

```
select nombre, ciudad, telefono from proveedor where ciudad in ('Cochabamba', 'La Paz') order by
ciudad;
```

Resultado: 2 fila(s).

nombre	ciudad	telefono
Minera Andina Cochabamba (demo)	Cochabamba	44250001
Minerales Illimani S.R.L. (demo)	La Paz	22450002

Q06. *Ventas anuladas con su motivo*

```
select numero, fecha::date as fecha, total, motivo_anulacion from venta where estado = 'ANULADA'
order by fecha desc;
```

Resultado: 3 fila(s).

numero	fecha	total	motivo_anulacion
NV-000036	2026-10-29	95.00	Prueba automática de anulación
NV-000033	2026-10-29	95.00	Prueba automática de anulación
NV-000026	2026-10-28	325.00	Pieza con fisura detectada al entregar (demostración)

Q07. *Cantidad de usuarios por estado*

```
select estado, count(*) as usuarios from usuario group by estado;
```

Resultado: 1 fila(s).

estado	usuarios
ACTIVO	17

Q08. *Compras registradas en los últimos 30 días*

```
select numero, fecha, total from compra where estado = 'REGISTRADA' and fecha >= hoy_bo() - 30
order by fecha desc;
```

Resultado: 6 fila(s).

numero	fecha	total
CP-000006	2026-10-29	415.00
CP-000007	2026-10-29	415.00
CP-000004	2026-10-23	736.00
CP-000003	2026-10-11	2460.00
CP-000002	2026-10-04	1225.00
CP-000001	2026-10-01	2295.00

Q09. *Intentos de inicio de sesión fallidos de la última semana*

```
select ip, count(*) as intentos from bitacora where accion = 'LOGIN_FALLIDO' and fecha > now() -
interval '7 days' group by ip;
```

Resultado: 0 fila(s).
(sin filas)

Q10. *Pedidos web por estado*

```
select estado, count(*) as pedidos, coalesce(sum(total), 0) as monto from pedido group by estado
order by estado;
```

Resultado: 5 fila(s).

estado	pedidos	monto
CANCELADO	2	190.00
EN_VERIFICACION	1	190.00
ENTREGADO	3	1050.00
PAGADO	7	1340.00
PENDIENTE_PAGO	2	255.00

Consultas múltiples

Q11. *Horario semanal: taller, maestra, día y hora*

```
select t.nombre as taller, u.nombre || ' ' || u.apellido as maestra, h.dia_semana, h.hora_inicio,
h.hora_fin, h.cupo
  from horario h join taller t on t.id = h.taller_id join maestra m on m.id = h.maestra_id join
usuario u on u.id = m.usuario_id
 where h.estado = 'ACTIVO' order by h.dia_semana, h.hora_inicio;
```

Resultado: 13 fila(s) (se muestran 6).

taller	maestra	dia_semana	hora_inicio	hora_fin	cupo
Modelado a mano	Rosario Gil	2	10:00:00	12:00:00	8
Modelado avanzado	Elena Rojas	2	10:00:00	12:00:00	6
Taller de modelado nocturno	Lucía Paz	2	18:00:00	20:00:00	8
Modelado a mano	Rosario Gil	3	10:00:00	12:00:00	8
Torno alfarero	Carla Méndez	3	10:00:00	12:00:00	6
Taller de modelado nocturno	Lucía Paz	3	18:00:00	20:00:00	8

Q12. *Ocupación de las clases de esta semana*

```
select s.fecha, t.nombre as taller, count(r.id) filter (where r.estado <> 'CANCELADA') as
reservas, s.cupo
  from sesion_clase s join horario h on h.id = s.horario_id join taller t on t.id = h.taller_id
  left join reserva r on r.sesion_id = s.id
 where s.fecha between date_trunc('week', hoy_bo())::date and date_trunc('week', hoy_bo())::date
+ 6
 group by s.id, s.fecha, t.nombre, s.cupo order by s.fecha, t.nombre;
```

Resultado: 13 fila(s) (se muestran 6).

fecha	taller	reservas	cupo
2026-10-27	Modelado a mano	4	8
2026-10-27	Modelado avanzado	4	6
2026-10-27	Taller de modelado nocturno	5	8
2026-10-28	Modelado a mano	3	8
2026-10-28	Taller de modelado nocturno	2	8
2026-10-28	Torno alfarero	3	6

Q13. *Porcentaje de asistencia por alumno*

```
select u.nombre || ' ' || u.apellido as alumno,
       count(*) filter (where r.estado = 'ASISTIO') as asistio, count(*) filter (where r.estado =
'FALTO') as falto,
       round(100.0 * count(*) filter (where r.estado = 'ASISTIO') / nullif(count(*) filter (where
r.estado in ('ASISTIO', 'FALTO')), 0)) as porcentaje
  from reserva r join alumno a on a.id = r.alumno_id join usuario u on u.id = a.usuario_id
 group by alumno order by porcentaje desc nulls last;
```

Resultado: 10 fila(s) (se muestran 6).

alumno	asistio	falto	porcentaje
Sofía Arteaga	2	0	100
Diego Fernández	17	0	100
Mateo Justiniano	2	0	100
Joaquín Salvatierra	14	1	93
Luis Peña	12	1	92
Camila Sosa	12	3	80

Q14. *Detalle de compras con proveedor y material*

```
select c.numero, p.nombre as proveedor, m.nombre as material, d.cantidad, d.costo_unitario,
d.subtotal
  from compra c join proveedor p on p.id = c.proveedor_id join detalle_compra d on d.compra_id =
c.id join material m on m.id = d.material_id
 order by c.fecha desc, m.nombre;
```

Resultado: 16 fila(s) (se muestran 6).

numero	proveedor	material	cantidad	costo_unitario	subtotal
CP-000007	Minera Andina Cochabamba (demo)	Arcilla gres blanca	20.00	9.50	190.00
CP-000006	Minera Andina Cochabamba (demo)	Arcilla gres blanca	20.00	9.50	190.00
CP-000007	Minera Andina Cochabamba (demo)	Óxido de cobalto	5.00	45.00	225.00
CP-000006	Minera Andina Cochabamba (demo)	Óxido de cobalto	5.00	45.00	225.00
CP-000004	Minera Andina Cochabamba (demo)	Arcilla gres blanca	80.00	9.20	736.00
CP-000003	Insumos Cerámicos del Oriente (demo)	Engobe blanco	5.00	60.00	300.00

Q15. *Ventas por método de pago (solo válidas)*

```
select mp.nombre as metodo, count(v.id) as ventas, sum(v.total) as total
  from venta v join metodo_pago mp on mp.id = v.metodo_pago_id where v.estado = 'VALIDA' group by
mp.nombre order by total desc;
```

Resultado: 3 fila(s).

metodo	ventas	total
Transferencia	11	2435.00
Efectivo	9	2110.00
QR	8	1665.00

Q16. *Piezas más vendidas en caja*

```
select p.codigo, p.nombre, sum(d.cantidad) as unidades, sum(d.subtotal) as ingresos
  from detalle_venta d join venta v on v.id = d.venta_id join pieza p on p.id = d.pieza_id
 where v.estado = 'VALIDA' group by p.codigo, p.nombre order by unidades desc;
```

Resultado: 5 fila(s).

codigo	nombre	unidades	ingresos
TZ-001	Taza esmaltada Nube	16	1520.00
TZ-002	Vaso Mejillas	14	1190.00
VJ-001	Set café: vaso y bandeja	10	1600.00
VJ-002	Plato llano de gres 28 cm	10	1200.00
JR-001	Jarra Andina	2	700.00

Q17. *Kardex de un material con el usuario que hizo cada movimiento*

```
select mm.fecha::date as fecha, mm.tipo, mm.cantidad, mm.stock_resultante, u.nombre as usuario
  from movimiento_material mm join usuario u on u.id = mm.usuario_id join material m on m.id =
mm.material_id
 where m.nombre = 'Arcilla roja' order by mm.fecha;
```

Resultado: 3 fila(s).

fecha	tipo	cantidad	stock_resultante	usuario
2026-10-01	ENTRADA	150.00	150.00	Valeria
2026-10-19	CONSUMO	95.00	25.00	Valeria
2026-10-27	CONSUMO	30.00	120.00	Valeria

Q18. *Cierres de caja con su responsable y diferencia del arqueo*

```
select c.abierta_en::date as dia, u.nombre || ' ' || u.apellido as responsable, c.total_efectivo,
c.total_otros, c.diferencia
  from caja c join usuario u on u.id = c.usuario_id where c.estado = 'CERRADA' order by
c.abierta_en desc;
```

Resultado: 14 fila(s) (se muestran 6).

dia	responsable	total_efectivo	total_otros	diferencia
2026-10-28	Valeria Suárez	0.00	755.00	0.00
2026-10-27	Valeria Suárez	320.00	510.00	0.00
2026-10-24	Valeria Suárez	525.00	85.00	0.00

dia	responsable	total_efectivo	total_otros	diferencia
2026-10-23	Valeria Suárez	0.00	400.00	0.00
2026-10-22	Valeria Suárez	0.00	290.00	0.00
2026-10-21	Valeria Suárez	0.00	850.00	0.00

Q19. *Permisos de cada rol*

```
select r.nombre as rol, count(rp.permiso_id) as permisos, string_agg(p.codigo, ', ' order by
p.codigo) as codigos
  from rol r left join rolpermiso rp on rp.rol_id = r.id left join permiso p on p.id =
rp.permiso_id group by r.nombre order by permisos desc;
```

Resultado: 5 fila(s).

rol	permisos	codigos
ADMINISTRADOR	20	alumnos.gestionar, asistencia.registrar, bitacora.ver, ca…
ADMINISTRATIVO	13	alumnos.gestionar, asistencia.registrar, caja.gestionar, …
ALUMNO	2	portal.alumno, tienda.comprar
MAESTRA	2	clases.propias, panel.ver
CLIENTE	1	tienda.comprar

Q20. *Pedidos web con cliente y método con que se pagaron*

```
select pe.numero, c.nombre as cliente, pe.total, pe.estado, pg.metodo, pg.estado as estado_pago
  from pedido pe join cliente c on c.id = pe.cliente_id left join pago pg on pg.pedido_id = pe.id
 order by pe.fecha desc;
```

Resultado: 18 fila(s) (se muestran 6).

numero	cliente	total	estado	metodo	estado_pago
PW-000015	Daniela Rivero	95.00	CANCELADO	null	null
PW-000014	Daniela Rivero	350.00	ENTREGADO	QR	APROBADO
PW-000013	Daniela Rivero	95.00	PAGADO	PAYPAL	APROBADO
PW-000012	Daniela Rivero	320.00	PAGADO	TARJETA	RECHAZADO
PW-000012	Daniela Rivero	320.00	PAGADO	TARJETA	APROBADO
PW-000011	Daniela Rivero	160.00	PENDIENTE_PAGO	null	null

Subconsultas

Q21. *Piezas con precio mayor al promedio de su categoría*

```
select p.codigo, p.nombre, p.precio from pieza p
 where p.precio > (select avg(x.precio) from pieza x where x.categoria_id = p.categoria_id) order
by p.precio desc;
```

Resultado: 2 fila(s).

codigo	nombre	precio
JR-001	Jarra Andina	350.00
VJ-001	Set café: vaso y bandeja	160.00

Q22. *Alumnos que nunca faltaron a una clase*

```
select u.nombre || ' ' || u.apellido as alumno from alumno a join usuario u on u.id =
a.usuario_id
 where exists (select 1 from reserva r where r.alumno_id = a.id and r.estado = 'ASISTIO')
   and not exists (select 1 from reserva r where r.alumno_id = a.id and r.estado = 'FALTO');
```

Resultado: 3 fila(s).

alumno
Diego Fernández
Mateo Justiniano
Sofía Arteaga

Q23. *Talleres con más reservas que el promedio*

```
select t.nombre, (select count(*) from reserva r join sesion_clase s on s.id = r.sesion_id join
horario h on h.id = s.horario_id
                   where h.taller_id = t.id and r.estado <> 'CANCELADA') as reservas
  from taller t
 where (select count(*) from reserva r join sesion_clase s on s.id = r.sesion_id join horario h
on h.id = s.horario_id
         where h.taller_id = t.id and r.estado <> 'CANCELADA')
     > (select avg(n) from (select count(*) as n from reserva r join sesion_clase s on s.id =
r.sesion_id join horario h on h.id = s.horario_id
                            where r.estado <> 'CANCELADA' group by h.taller_id) x)
 order by reservas desc;
```

Resultado: 3 fila(s).

nombre	reservas
Modelado a mano	85
Torno alfarero	55
Taller de modelado nocturno	39

Q24. *Materiales que nunca se compraron*

```
select nombre from material m where not exists (select 1 from detalle_compra d where
d.material_id = m.id);
```

Resultado: 0 fila(s).
(sin filas)

Q25. *Proveedor al que más se le compró*

```
select nombre, (select sum(total) from compra c where c.proveedor_id = p.id and c.estado =
'REGISTRADA') as comprado from proveedor p
 where p.id = (select proveedor_id from compra where estado = 'REGISTRADA' group by proveedor_id
order by sum(total) desc limit 1);
```

Resultado: 1 fila(s).

nombre	comprado
Minera Andina Cochabamba (demo)	3861.00

Q26. *Día con mayor venta del último mes*

```
select fecha::date as dia, sum(total) as total from venta where estado = 'VALIDA' group by
fecha::date
having sum(total) = (select max(t) from (select sum(total) as t from venta where estado =
'VALIDA' and fecha > now() - interval '30 days' group by fecha::date) x);
```

Resultado: 1 fila(s).

dia	total
2026-10-17	880.00

Q27. *Clases futuras que todavía tienen cupo libre*

```
select s.fecha, s.hora_inicio, (select t.nombre from horario h join taller t on t.id =
h.taller_id where h.id = s.horario_id) as taller,
       s.cupo - (select count(*) from reserva r where r.sesion_id = s.id and r.estado <>
'CANCELADA') as libres
  from sesion_clase s where s.fecha > hoy_bo() and s.estado = 'PROGRAMADA'
   and (select count(*) from reserva r where r.sesion_id = s.id and r.estado <> 'CANCELADA') <
s.cupo
 order by s.fecha, s.hora_inicio limit 10;
```

Resultado: 10 fila(s) (se muestran 6).

fecha	hora_inicio	taller	libres
2026-10-30	10:00:00	Modelado a mano	5
2026-10-30	10:00:00	Torno alfarero	2
2026-10-31	10:00:00	Taller de sábado para adultos	6
2026-10-31	16:00:00	Cerámica para niños	10
2026-10-31	18:00:00	Modelado y torno	1
2026-11-03	10:00:00	Modelado a mano	3

Q28. *Maestras con más alumnos atendidos que la maestra promedio*

```
select u.nombre || ' ' || u.apellido as maestra,
       (select count(*) from reserva r join sesion_clase s on s.id = r.sesion_id where
s.maestra_id = m.id and r.estado = 'ASISTIO') as atendidos
  from maestra m join usuario u on u.id = m.usuario_id
 where (select count(*) from reserva r join sesion_clase s on s.id = r.sesion_id where
s.maestra_id = m.id and r.estado = 'ASISTIO')
     >= (select avg(n) from (select count(*) as n from reserva r join sesion_clase s on s.id =
r.sesion_id where r.estado = 'ASISTIO' group by s.maestra_id) x)
 order by atendidos desc;
```

Resultado: 1 fila(s).

maestra	atendidos
Carla Méndez	34

Q29. *Clientes que compraron en caja y también en la tienda en línea*

```
select nombre from cliente c
 where c.id in (select cliente_id from venta where estado = 'VALIDA') and c.id in (select
cliente_id from pedido where estado in ('PAGADO', 'ENTREGADO'));
```

Resultado: 0 fila(s).
(sin filas)

Q30. *Valor del inventario de materiales frente al total comprado en el mes*

```
select (select sum(stock * costo_promedio) from material) as valor_inventario,
       (select coalesce(sum(total), 0) from compra where estado = 'REGISTRADA' and
date_trunc('month', fecha) = date_trunc('month', hoy_bo())) as comprado_mes;
```

Resultado: 1 fila(s).

valor_inventario	comprado_mes
4100.1700	7546.00

Procedimientos almacenados

Las operaciones que tocan varias tablas se programaron como funciones de PostgreSQL que se ejecutan en una sola transacción: si algo falla, no se guarda nada. Cada una se probó en la base publicada y luego se revirtió.

Tabla 85
Procedimientos almacenados

Código	Función	Qué hace
PA1	`fn_registrar_venta`	Registra la venta, su detalle y descuenta el stock
PA2	`fn_anular_venta`	Anula con motivo y devuelve el stock
PA3	`fn_registrar_compra`	Registra la compra y sube stock y costo promedio
PA4	`fn_anular_compra`	Anula la compra y revierte el stock
PA5	`fn_reservar_clase`	Reserva un lugar validando cupo y fecha
PA6	`fn_generar_sesiones`	Crea las clases del calendario sin duplicar
PA7	`sp_cerrar_caja`	Cierra la caja y calcula el arqueo
PA8	`fn_crear_pedido`	Crea el pedido en línea y reserva el stock 30 minutos
PA9	`fn_registrar_pago`	Registra el pago y marca el pedido como pagado
PA10	`fn_verificar_pago_qr`	El taller confirma o rechaza un pago por QR
PA11	`fn_cancelar_pedido`	Cancela y devuelve el stock
PA12	`fn_vencer_pedidos`	Cancela los pedidos no pagados a tiempo

El código de cada procedimiento se copia tal cual de los archivos del repositorio (Bakend/database). Debajo se muestra la prueba que se ejecutó en la base publicada y su resultado real.

PA1 fn_registrar_venta

Código (Bakend/database/06_procedimientos_triggers.sql):

```
create or replace function mitsu.fn_registrar_venta(
    p_usuario integer, p_cliente integer, p_metodo integer, p_recibido numeric, p_items jsonb)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_caja   integer;
    v_venta  integer;
    v_numero varchar;
    v_total  numeric(12,2);
    v_item   record;
    v_pieza  pieza%rowtype;
begin
    if p_items is null or jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0
then
        raise exception 'Agregue al menos una pieza a la venta.';
    end if;

    select id into v_caja from caja where estado = 'ABIERTA' for update;
    if v_caja is null then
        raise exception 'No hay una caja abierta. Abra la caja antes de vender.';
    end if;
    if not exists (select 1 from cliente where id = p_cliente and estado = 'ACTIVO') then
        raise exception 'El cliente no está activo.';
    end if;
    if not exists (select 1 from metodo_pago where id = p_metodo and estado = 'ACTIVO') then
        raise exception 'Método de pago no válido.';
    end if;

    insert into venta (caja_id, cliente_id, usuario_id, metodo_pago_id)
    values (v_caja, p_cliente, p_usuario, p_metodo)
    returning id, numero into v_venta, v_numero;

    -- Se recorren las piezas en orden de id para bloquearlas siempre en el mismo orden
    for v_item in
        select (e->>'pieza_id')::integer as pieza_id, sum((e->>'cantidad')::integer) as cantidad
          from jsonb_array_elements(p_items) e
         group by 1 order by 1
    loop
        if v_item.cantidad is null or v_item.cantidad <= 0 then
            raise exception 'La cantidad debe ser mayor a cero.';
        end if;
        select * into v_pieza from pieza where id = v_item.pieza_id for update;
        if not found then
            raise exception 'Una de las piezas no existe.';
        end if;
        if v_pieza.stock < v_item.cantidad then
            raise exception 'Stock insuficiente de "%": quedan % unidades.', v_pieza.nombre,
v_pieza.stock;
        end if;
        insert into detalle_venta (venta_id, pieza_id, cantidad, precio_unitario)
        values (v_venta, v_pieza.id, v_item.cantidad, v_pieza.precio);
        insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, venta_id, usuario_id)
        values (v_pieza.id, 'VENTA', v_item.cantidad, 'Venta ' || v_numero, v_venta, p_usuario);
    end loop;

    select total into v_total from venta where id = v_venta;
    if p_metodo = (select id from metodo_pago where normalizar(nombre) = 'efectivo')
       and coalesce(p_recibido, 0) < v_total then
        raise exception 'El monto recibido (Bs %) es menor al total (Bs %).',
coalesce(p_recibido, 0), v_total;
    end if;
```

```
    update venta set monto_recibido = coalesce(p_recibido, v_total) where id = v_venta;
    return v_venta;
end $$;
```

Prueba: Venta de 1 Vaso Mejillas en efectivo con Bs 200 recibidos.

```
select mitsu.fn_registrar_venta(2, 1, 1, 200, jsonb_build_array(jsonb_build_object('pieza_id',
(select id from mitsu.pieza where codigo = 'TZ-002'), 'cantidad', 1))) as venta_id;
      select numero, total, monto_recibido, (select stock from mitsu.pieza where codigo =
'TZ-002') as stock_restante from mitsu.venta order by id desc limit 1
```

numero	total	monto_recibido	stock_restante
NV-000045	95.00	200.00	2

PA2 fn_anular_venta

Código (Bakend/database/06_procedimientos_triggers.sql):

```
create or replace function mitsu.fn_anular_venta(p_venta integer, p_usuario integer, p_motivo
text)
returns void
language plpgsql
set search_path = mitsu, public
as $$
declare
    v venta%rowtype;
begin
    select * into v from venta where id = p_venta for update;
    if not found then
        raise exception 'La venta no existe.';
    end if;
    if v.estado = 'ANULADA' then
        raise exception 'La venta % ya estaba anulada.', v.numero;
    end if;
    if length(trim(coalesce(p_motivo, ''))) < 5 then
        raise exception 'Escriba el motivo de la anulación (mínimo 5 caracteres).';
    end if;

    insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, venta_id, usuario_id)
    select pieza_id, 'ANULACION', cantidad, 'Anulación de ' || v.numero, p_venta, p_usuario
      from detalle_venta where venta_id = p_venta order by pieza_id;

    update venta set estado = 'ANULADA', motivo_anulacion = trim(p_motivo), anulado_por =
p_usuario
     where id = p_venta;
end $$;
```

Prueba: Anular la última venta válida con motivo.

```
select mitsu.fn_anular_venta((select max(id) from mitsu.venta where estado = 'VALIDA'), 1,
'Cliente devolvió la pieza');
      select numero, estado, motivo_anulacion from mitsu.venta where anulado_en is not null order
by anulado_en desc limit 1
```

numero	estado	motivo_anulacion
NV-000029	ANULADA	Cliente devolvió la pieza

PA3 fn_registrar_compra

Código (Bakend/database/06_procedimientos_triggers.sql):

```
create or replace function mitsu.fn_registrar_compra(
    p_usuario integer, p_proveedor integer, p_fecha date, p_observacion text, p_items jsonb)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_compra integer;
    v_numero varchar;
    v_item   record;
    v_mat    material%rowtype;
begin
    if p_items is null or jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0
then
        raise exception 'Agregue al menos un material a la compra.';
    end if;
    if not exists (select 1 from proveedor where id = p_proveedor and estado = 'ACTIVO') then
        raise exception 'El proveedor no está activo.';
    end if;
    if p_fecha > hoy_bo() or p_fecha < hoy_bo() - 60 then
        raise exception 'La fecha de compra debe estar entre los últimos 60 días y hoy.';
    end if;

    insert into compra (proveedor_id, usuario_id, fecha, observacion)
    values (p_proveedor, p_usuario, p_fecha, nullif(trim(coalesce(p_observacion, '')), ''))
    returning id, numero into v_compra, v_numero;

    for v_item in
        select (e->>'material_id')::integer as material_id,
               (e->>'cantidad')::numeric as cantidad,
               (e->>'costo')::numeric as costo
          from jsonb_array_elements(p_items) e
         order by 1
    loop
        select * into v_mat from material where id = v_item.material_id for update;
        if not found or v_mat.estado <> 'ACTIVO' then
            raise exception 'Uno de los materiales no existe o está inactivo.';
        end if;
        if v_item.cantidad is null or v_item.cantidad <= 0 or v_item.costo is null or
v_item.costo < 0 then
            raise exception 'Cantidad y costo de "%" deben ser válidos.', v_mat.nombre;
        end if;

        insert into detalle_compra (compra_id, material_id, cantidad, costo_unitario)
        values (v_compra, v_mat.id, v_item.cantidad, v_item.costo);

        -- Costo promedio ponderado con el stock que había antes
        update material
           set costo_promedio = round((v_mat.stock * v_mat.costo_promedio + v_item.cantidad *
v_item.costo)
                                      / (v_mat.stock + v_item.cantidad), 2)
         where id = v_mat.id;
```

```
        insert into movimiento_material (material_id, tipo, cantidad, motivo, compra_id,
usuario_id)
        values (v_mat.id, 'ENTRADA', v_item.cantidad, 'Compra ' || v_numero, v_compra,
p_usuario);
    end loop;
    return v_compra;
end $$;
```

Prueba: Compra de 50 kg de arcilla roja a Bs 6,80.

```
select mitsu.fn_registrar_compra(2, 1, mitsu.hoy_bo(), 'Prueba',
jsonb_build_array(jsonb_build_object('material_id', (select id from mitsu.material where nombre =
'Arcilla roja'), 'cantidad', 50, 'costo', 6.8))) as compra_id;
      select nombre, stock, costo_promedio from mitsu.material where nombre = 'Arcilla roja'
```

nombre	stock	costo_promedio
Arcilla roja	75.00	6.70

PA4 fn_anular_compra

Código (Bakend/database/06_procedimientos_triggers.sql):

```
create or replace function mitsu.fn_anular_compra(p_compra integer, p_usuario integer, p_motivo
text)
returns void
language plpgsql
set search_path = mitsu, public
as $$
declare
    c compra%rowtype;
begin
    select * into c from compra where id = p_compra for update;
    if not found then
        raise exception 'La compra no existe.';
    end if;
    if c.estado = 'ANULADA' then
        raise exception 'La compra % ya estaba anulada.', c.numero;
    end if;
    if length(trim(coalesce(p_motivo, ''))) < 5 then
        raise exception 'Escriba el motivo de la anulación (mínimo 5 caracteres).';
    end if;
    -- Si el material ya se consumió, el trigger del kardex impide dejar stock negativo
    insert into movimiento_material (material_id, tipo, cantidad, motivo, compra_id, usuario_id)
    select material_id, 'AJUSTE_MENOS', cantidad, 'Anulación de ' || c.numero, p_compra,
p_usuario
      from detalle_compra where compra_id = p_compra order by material_id;
    update compra set estado = 'ANULADA', motivo_anulacion = trim(p_motivo) where id = p_compra;
end $$;
```

Prueba: Anular la última compra registrada.

```
select mitsu.fn_anular_compra((select max(id) from mitsu.compra where estado = 'REGISTRADA'), 1,
'Proveedor envió material equivocado');
      select numero, estado, motivo_anulacion from mitsu.compra where estado = 'ANULADA' order by
id desc limit 1
```

numero	estado	motivo_anulacion
CP-000007	ANULADA	Proveedor envió material equivocado

PA5 fn_reservar_clase

Código (Bakend/database/06_procedimientos_triggers.sql):

```
create or replace function mitsu.fn_reservar_clase(p_sesion integer, p_alumno integer, p_usuario
integer)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_id integer;
begin
    insert into reserva (sesion_id, alumno_id, registrado_por)
    values (p_sesion, p_alumno, p_usuario)
    returning id into v_id;
    return v_id;
end $$;
```

Prueba: Reservar la próxima clase con cupo para un alumno.

```
select mitsu.fn_reservar_clase((select s.id from mitsu.sesion_clase s where s.fecha >
mitsu.hoy_bo() and s.estado = 'PROGRAMADA'
               and not exists (select 1 from mitsu.reserva r where r.sesion_id = s.id and
r.alumno_id = 4) order by s.fecha limit 1), 4, 1) as reserva_id
```

reserva_id
265

PA6 fn_generar_sesiones

Código (Bakend/database/06_procedimientos_triggers.sql):

```
create or replace function mitsu.fn_generar_sesiones(p_desde date, p_hasta date)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_total integer;
begin
    if p_hasta < p_desde then
        raise exception 'La fecha final debe ser posterior a la inicial.';
    end if;
    if p_hasta - p_desde > 92 then
        raise exception 'Genere como máximo tres meses de clases por vez.';
    end if;
    insert into sesion_clase (horario_id, maestra_id, fecha, hora_inicio, hora_fin, cupo)
    select h.id, h.maestra_id, d::date, h.hora_inicio, h.hora_fin, h.cupo
      from horario h
      join taller t on t.id = h.taller_id and t.estado = 'ACTIVO'
     cross join generate_series(greatest(p_desde, hoy_bo()), p_hasta, interval '1 day') d
```

```
     where h.estado = 'ACTIVO' and extract(isodow from d) = h.dia_semana
    on conflict (horario_id, fecha) do nothing;
    get diagnostics v_total = row_count;
    return v_total;
end $$;
```

Prueba: Generar el calendario de las próximas 6 semanas.

```
select mitsu.fn_generar_sesiones(mitsu.hoy_bo(), mitsu.hoy_bo() + 42) as clases_creadas
```

clases_creadas
39

PA7 sp_cerrar_caja

Código (Bakend/database/06_procedimientos_triggers.sql):

```
create or replace procedure mitsu.sp_cerrar_caja(
    p_caja integer, p_usuario integer, p_declarado numeric, p_observacion text)
language plpgsql
set search_path = mitsu, public
as $$
declare
    c          caja%rowtype;
    v_efectivo numeric(12,2);
    v_otros    numeric(12,2);
begin
    select * into c from caja where id = p_caja for update;
    if not found or c.estado <> 'ABIERTA' then
        raise exception 'La caja no está abierta.';
    end if;
    if p_declarado is null or p_declarado < 0 then
        raise exception 'Ingrese el efectivo contado en caja.';
    end if;
    select coalesce(sum(v.total) filter (where normalizar(m.nombre) = 'efectivo'), 0),
           coalesce(sum(v.total) filter (where normalizar(m.nombre) <> 'efectivo'), 0)
      into v_efectivo, v_otros
      from venta v join metodo_pago m on m.id = v.metodo_pago_id
     where v.caja_id = p_caja and v.estado = 'VALIDA';

    update caja
       set total_efectivo  = v_efectivo,
           total_otros     = v_otros,
           monto_declarado = p_declarado,
           diferencia      = p_declarado - (c.monto_inicial + v_efectivo),
           cerrada_en      = now(),
           cerrada_por     = p_usuario,
           estado          = 'CERRADA',
           observacion     = nullif(trim(coalesce(p_observacion, '')), '')
     where id = p_caja;
end $$;
```

Prueba: Cerrar la caja abierta declarando Bs 500 en efectivo.

```
do $$ declare c integer := (select id from mitsu.caja where estado = 'ABIERTA'); begin call
mitsu.sp_cerrar_caja(c, 2, 500, 'Arqueo de prueba'); end $$;
      select estado, total_efectivo, monto_declarado, diferencia from mitsu.caja order by
cerrada_en desc nulls last limit 1
```

estado	total_efectivo	monto_declarado	diferencia
CERRADA	310.00	500.00	-10.00

PA8 fn_crear_pedido

Código (Bakend/database/07_tienda_pasarela.sql):

```
create or replace function mitsu.fn_crear_pedido(
    p_usuario integer, p_cliente integer, p_entrega text, p_direccion text, p_telefono text,
p_nota text, p_items jsonb)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_pedido integer;
    v_numero varchar;
    v_item   record;
    v_pieza  pieza%rowtype;
begin
    if p_items is null or jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0
then
        raise exception 'Tu carrito está vacío.';
    end if;
    insert into pedido (cliente_id, usuario_id, entrega, direccion, telefono, nota)
    values (p_cliente, p_usuario, p_entrega, nullif(trim(coalesce(p_direccion, '')), ''),
p_telefono, nullif(trim(coalesce(p_nota, '')), ''))
    returning id, numero into v_pedido, v_numero;

    for v_item in
        select (e->>'pieza_id')::integer as pieza_id, sum((e->>'cantidad')::integer) as cantidad
          from jsonb_array_elements(p_items) e group by 1 order by 1
    loop
        select * into v_pieza from pieza where id = v_item.pieza_id for update;
        if not found then
            raise exception 'Una de las piezas no existe.';
        end if;
        if v_pieza.stock < v_item.cantidad then
            raise exception 'Solo quedan % unidad(es) de "%".', v_pieza.stock, v_pieza.nombre;
        end if;
        insert into detalle_pedido (pedido_id, pieza_id, cantidad, precio_unitario) values
(v_pedido, v_pieza.id, v_item.cantidad, v_pieza.precio);
        insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, pedido_id, usuario_id)
        values (v_pieza.id, 'VENTA_WEB', v_item.cantidad, 'Pedido web ' || v_numero, v_pedido,
p_usuario);
    end loop;
    return v_pedido;
end $$;
```

Prueba: Pedido web de 1 Taza Nube con retiro en el taller.

```
select mitsu.fn_crear_pedido((select id from mitsu.usuario where correo = 'cliente@mitsu.test'),
(select c.id from mitsu.cliente c join mitsu.usuario u on u.id = c.usuario_id where u.correo =
'cliente@mitsu.test'), 'RETIRO', null, '72040001', null,
jsonb_build_array(jsonb_build_object('pieza_id', (select id from mitsu.pieza where codigo =
'TZ-001'), 'cantidad', 1))) as pedido_id;
      select numero, estado, total, expira_en::time(0) as reservado_hasta from mitsu.pedido order
by id desc limit 1
```

numero	estado	total	reservado_hasta
PW-000022	PENDIENTE_PAGO	95.00	19:15:14

PA9 fn_registrar_pago

Código (Bakend/database/07_tienda_pasarela.sql):

```
create or replace function mitsu.fn_registrar_pago(
    p_pedido integer, p_metodo text, p_estado text, p_referencia text,
    p_marca text, p_ultimos4 text, p_titular text, p_cuenta text, p_detalle text)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v pedido%rowtype;
    v_pago integer;
begin
    select * into v from pedido where id = p_pedido for update;
    if not found then
        raise exception 'El pedido no existe.';
    end if;
    if v.estado <> 'PENDIENTE_PAGO' then
        raise exception 'El pedido % no está esperando pago (estado: %).', v.numero,
lower(replace(v.estado, '_', ' '));
    end if;
    if v.expira_en < now() then
        raise exception 'El tiempo para pagar el pedido % venció. Vuelve a armar tu carrito.',
v.numero;
    end if;
    insert into pago (pedido_id, metodo, monto, estado, referencia, tarjeta_marca,
tarjeta_ultimos4, titular, cuenta_paypal, detalle)
    values (p_pedido, p_metodo, v.total, p_estado, p_referencia, p_marca, p_ultimos4, p_titular,
p_cuenta, p_detalle)
    returning id into v_pago;

    if p_estado = 'APROBADO' then
        update pedido set estado = 'PAGADO' where id = p_pedido;
    elsif p_estado = 'PENDIENTE' then
        update pedido set estado = 'EN_VERIFICACION' where id = p_pedido;
    end if;
    return v_pago;
end $$;
```

Prueba: Pedido web nuevo pagado con tarjeta (aprobado).

```
with p as (select mitsu.fn_crear_pedido((select id from mitsu.usuario where correo =
'cliente@mitsu.test'), (select c.id from mitsu.cliente c join mitsu.usuario u on u.id =
c.usuario_id where u.correo = 'cliente@mitsu.test'), 'RETIRO', null, '72040001', null,
jsonb_build_array(jsonb_build_object('pieza_id', (select id from mitsu.pieza where codigo =
'TZ-001'), 'cantidad', 1))) as id)
      select p.id as pedido_id, mitsu.fn_registrar_pago(p.id, 'TARJETA', 'APROBADO', 'AUT-
PRUEBA', 'Visa', '4242', 'DANIELA RIVERO', null, 'Aprobado') as pago_id from p;
      select numero, estado, pagado_en::time(0) as pagado from mitsu.pedido order by id desc
limit 1
```

numero	estado	pagado
PW-000023	PAGADO	18:45:14

PA10 fn_verificar_pago_qr

Código (Bakend/database/07_tienda_pasarela.sql):

```
create or replace function mitsu.fn_verificar_pago_qr(p_pago integer, p_usuario integer,
p_aprobar boolean, p_detalle text)
returns void
language plpgsql
set search_path = mitsu, public
as $$
declare
    pg pago%rowtype;
begin
    select * into pg from pago where id = p_pago for update;
    if not found or pg.metodo <> 'QR' or pg.estado <> 'PENDIENTE' then
        raise exception 'Ese pago no está pendiente de verificación.';
    end if;
    perform 1 from pedido where id = pg.pedido_id for update;
    update pago set estado = case when p_aprobar then 'APROBADO' else 'RECHAZADO' end,
                    verificado_por = p_usuario, verificado_en = now(),
                    detalle = coalesce(nullif(trim(coalesce(p_detalle, '')), ''), detalle)
     where id = p_pago;
    update pedido set estado = case when p_aprobar then 'PAGADO' else 'PENDIENTE_PAGO' end,
                      expira_en = case when p_aprobar then expira_en else now() + interval '30
minutes' end
     where id = pg.pedido_id;
end $$;
```

Se prueba en el flujo completo de la tienda (Anexo C): el personal confirma el pago por QR de un pedido en verificación.

PA11 fn_cancelar_pedido

Código (Bakend/database/07_tienda_pasarela.sql):

```
create or replace function mitsu.fn_cancelar_pedido(p_pedido integer, p_usuario integer, p_motivo
text)
returns void
language plpgsql
set search_path = mitsu, public
as $$
declare
    v pedido%rowtype;
begin
    select * into v from pedido where id = p_pedido for update;
    if not found then
        raise exception 'El pedido no existe.';
    end if;
    if v.estado in ('ENTREGADO', 'CANCELADO') then
        raise exception 'El pedido % ya está % y no se puede cancelar.', v.numero,
lower(v.estado);
    end if;
    if length(trim(coalesce(p_motivo, ''))) < 5 then
        raise exception 'Escribe el motivo de la cancelación (mínimo 5 caracteres).';
    end if;
```

```
    insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, pedido_id, usuario_id)
    select pieza_id, 'DEVOLUCION_WEB', cantidad, 'Cancelación de ' || v.numero, p_pedido,
p_usuario
      from detalle_pedido where pedido_id = p_pedido order by pieza_id;
    update pago set estado = 'RECHAZADO', detalle = 'Pedido cancelado' where pedido_id = p_pedido
and estado = 'PENDIENTE';
    update pedido set estado = 'CANCELADO', motivo_cancelacion = trim(p_motivo) where id =
p_pedido;
end $$;
```

Prueba: Cancelar el pedido en verificación de la demostración.

```
select mitsu.fn_cancelar_pedido((select id from mitsu.pedido where estado = 'EN_VERIFICACION'
order by id limit 1), 1, 'Cliente desistió'),
             (select count(*) from mitsu.movimiento_pieza where tipo = 'DEVOLUCION_WEB') as
devoluciones
```

fn_cancelar_pedido	devoluciones
	2

PA12 fn_vencer_pedidos

Código (Bakend/database/07_tienda_pasarela.sql):

```
create or replace function mitsu.fn_vencer_pedidos()
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v record;
    n integer := 0;
begin
    for v in select id, usuario_id from pedido where estado = 'PENDIENTE_PAGO' and expira_en <
now() for update skip locked loop
        perform fn_cancelar_pedido(v.id, v.usuario_id, 'No se pagó dentro de los 30 minutos');
        n := n + 1;
    end loop;
    return n;
end $$;
```

Se prueba en el flujo completo de la tienda (Anexo C): cancela los pedidos que no se pagaron en 30 minutos y devuelve el stock.

Triggers (disparadores)

Los triggers garantizan las reglas del negocio dentro de la base de datos, aunque alguien intente saltarse la aplicación. Cada prueba intenta violar una regla; el resultado esperado es que la base la rechace con un mensaje claro.

Tabla 86
Prueba de triggers y restricciones

Regla	Intento	Respuesta real de la base
RN04 tg_reserva_validar	Reservar una clase que ya pasó	La clase no está disponible para reservas.
RN04 tg_reserva_validar	Reservar dos veces la misma clase	El alumno ya tiene una reserva en esta clase.
RN06 tg_movimiento_material	Consumir más arcilla de la que hay	Stock insuficiente de "Arcilla roja": hay 25.00 y se pidió 9999.00.
RN07 tg_stock_protegido	Cambiar el stock de una pieza a mano	El stock solo cambia con compras, ventas o ajustes registrados.
RN08 tg_detalle_venta_precio	Intentar vender a otro precio (el trigger copia el real)	Aceptado: {"numero":"NV-000046","total":"95.00"}
RN09 tg_venta_reglas	Modificar una venta anulada	La venta NV-000036 ya está anulada y no se puede modificar.
RN02 tg_ultimo_admin	Desactivar al único administrador	No se puede desactivar ni cambiar de rol al único administrador activo.
RN03 horario_sin_cruce_maestra	Horario cruzado para la misma maestra	conflicting key value violates exclusion constraint "horario_sin_cruce_maestra"
ux_material_nombre (normalizar)	Material con el mismo nombre en mayúsculas y con tilde	duplicate key value violates unique constraint "ux_material_nombre"
tg_pedido_estado	Pasar un pedido entregado a pagado	El pedido PW-000005 ya está entregado y no puede cambiar.
Mínimo privilegio	El usuario de la aplicación intenta borrar una tabla	must be owner of table bitacora

Nota. Ejecutado contra la base publicada con el usuario de la aplicación; cada prueba se revirtió. En RN08 la venta se acepta pero con el precio real de la pieza (Bs 95), no con el enviado.

Ejemplo: trigger que valida una reserva (bloquea la clase y revisa cupo y fecha), copiado de Bakend/database/02_reglas_negocio.sql:

```
create or replace function mitsu.tg_reserva_validar()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare
    s       sesion_clase%rowtype;
    ocupado integer;
begin
    -- Se bloquea la fila de la sesión: dos reservas simultáneas no pueden pasar el cupo
    select * into s from sesion_clase where id = new.sesion_id for update;

    if tg_op = 'INSERT' then
        if s.estado <> 'PROGRAMADA' then
            raise exception 'La clase no está disponible para reservas.';
        end if;
        if s.fecha < hoy_bo() or (s.fecha = hoy_bo() and s.hora_inicio <= (now() at time zone
'America/La_Paz')::time) then
            raise exception 'No se puede reservar una clase que ya empezó o pasó.';
        end if;
        if not exists (select 1 from alumno a join usuario u on u.id = a.usuario_id
                       where a.id = new.alumno_id and a.estado = 'ACTIVO' and u.estado =
'ACTIVO') then
            raise exception 'El alumno no está activo.';
```

```
        end if;
        if exists (select 1 from reserva where sesion_id = new.sesion_id and alumno_id =
new.alumno_id and estado <> 'CANCELADA') then
            raise exception 'El alumno ya tiene una reserva en esta clase.';
        end if;
        select count(*) into ocupado from reserva where sesion_id = new.sesion_id and estado <>
'CANCELADA';
        if ocupado >= s.cupo then
            raise exception 'La clase ya no tiene cupos disponibles.';
        end if;
        new.estado := 'RESERVADA';
        return new;
    end if;

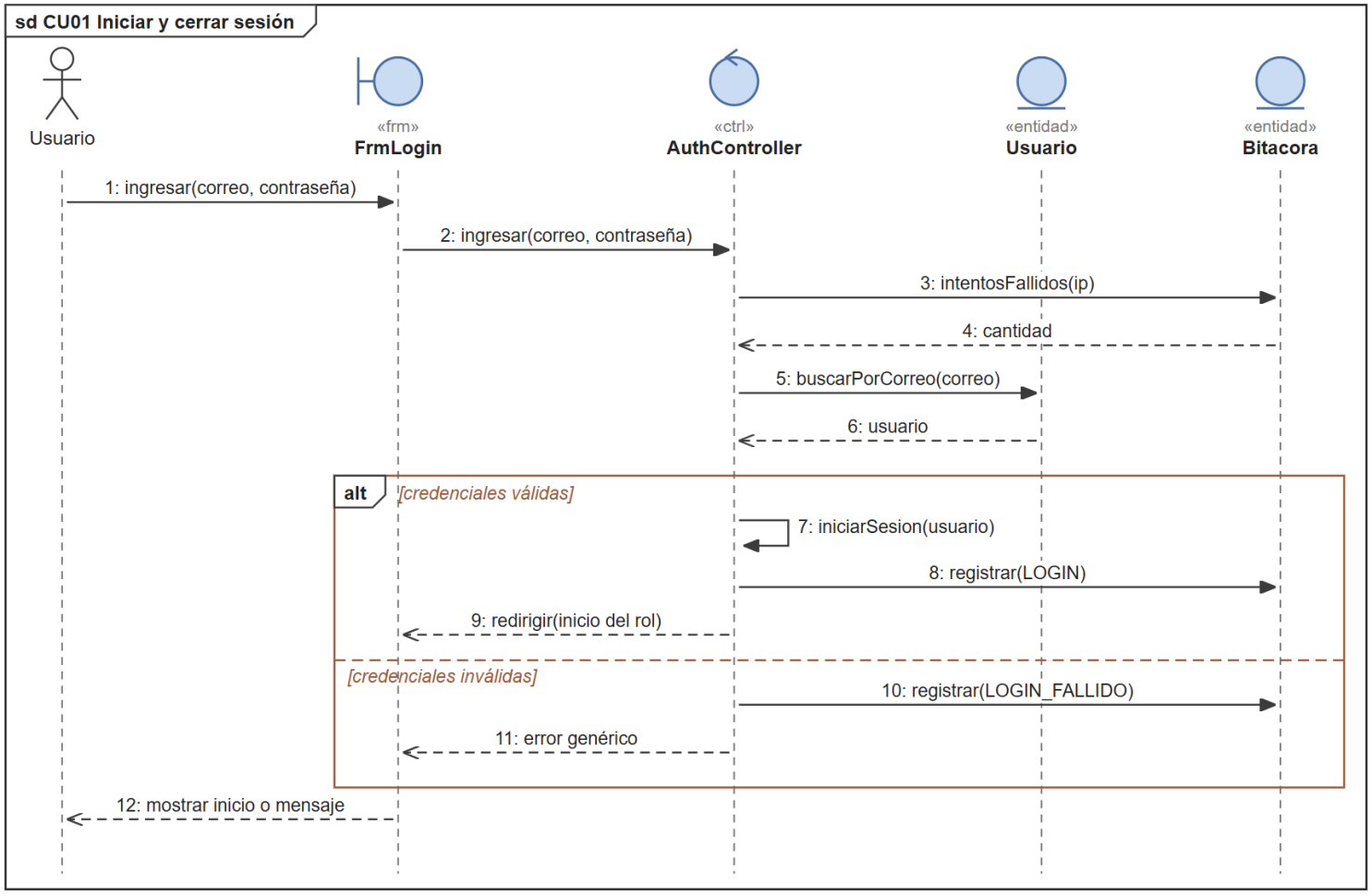
    -- UPDATE: transiciones permitidas
    if old.estado = new.estado then
        return new;
    end if;
    if old.estado = 'CANCELADA' then
        raise exception 'Una reserva cancelada no se puede modificar.';
    end if;
    if new.estado = 'CANCELADA' then
        if old.estado <> 'RESERVADA' then
            raise exception 'Solo se cancelan reservas pendientes, no las que ya tienen
asistencia.';
        end if;
        new.cancelado_en := now();
    elsif new.estado in ('ASISTIO', 'FALTO') then
        if s.fecha > hoy_bo() then
            raise exception 'La asistencia se registra el día de la clase o después.';
        end if;
        if s.estado = 'CANCELADA' then
            raise exception 'La clase fue cancelada.';
        end if;
        new.asistencia_en := now();
    elsif new.estado = 'RESERVADA' then
        raise exception 'No se puede volver la reserva a pendiente.';
    end if;
    return new;
end $$;
```

5.3 Diseñar casos de uso (diagrama de secuencia)

Los diagramas de secuencia muestran, en orden temporal, los mensajes entre el actor, la interfaz, el controlador y las entidades, con los fragmentos alt (alternativa), opt (opcional) y loop (repetición). Los nombres de controladores y métodos son los del código fuente.

Ciclo I: seguridad y configuración

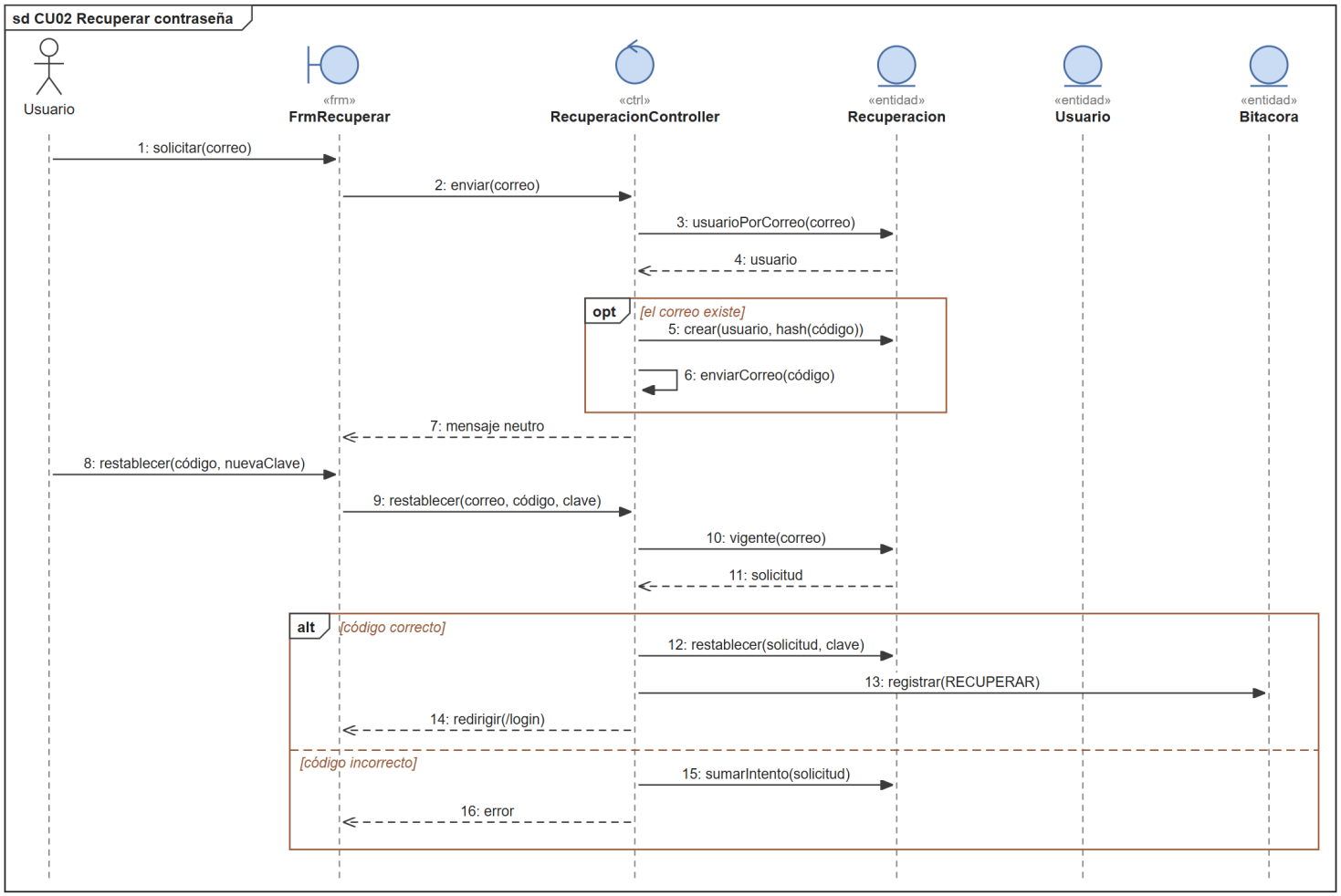
Figura 146
Diagrama de secuencia CU01 Iniciar y cerrar sesión



Nota. Elaboración propia.

Figura 147

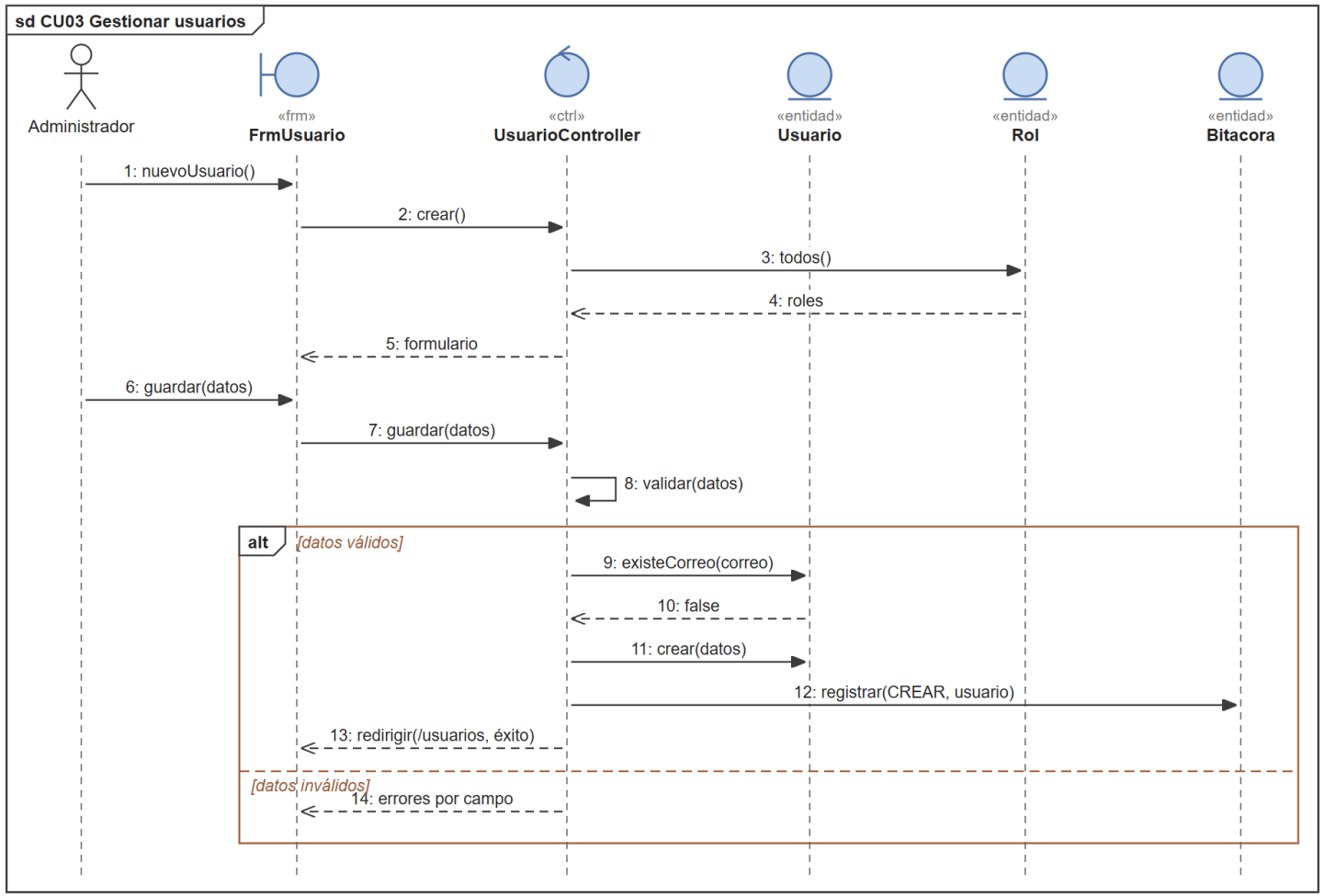
Diagrama de secuencia CU02 Recuperar contraseña



Nota. Elaboración propia.

Figura 148

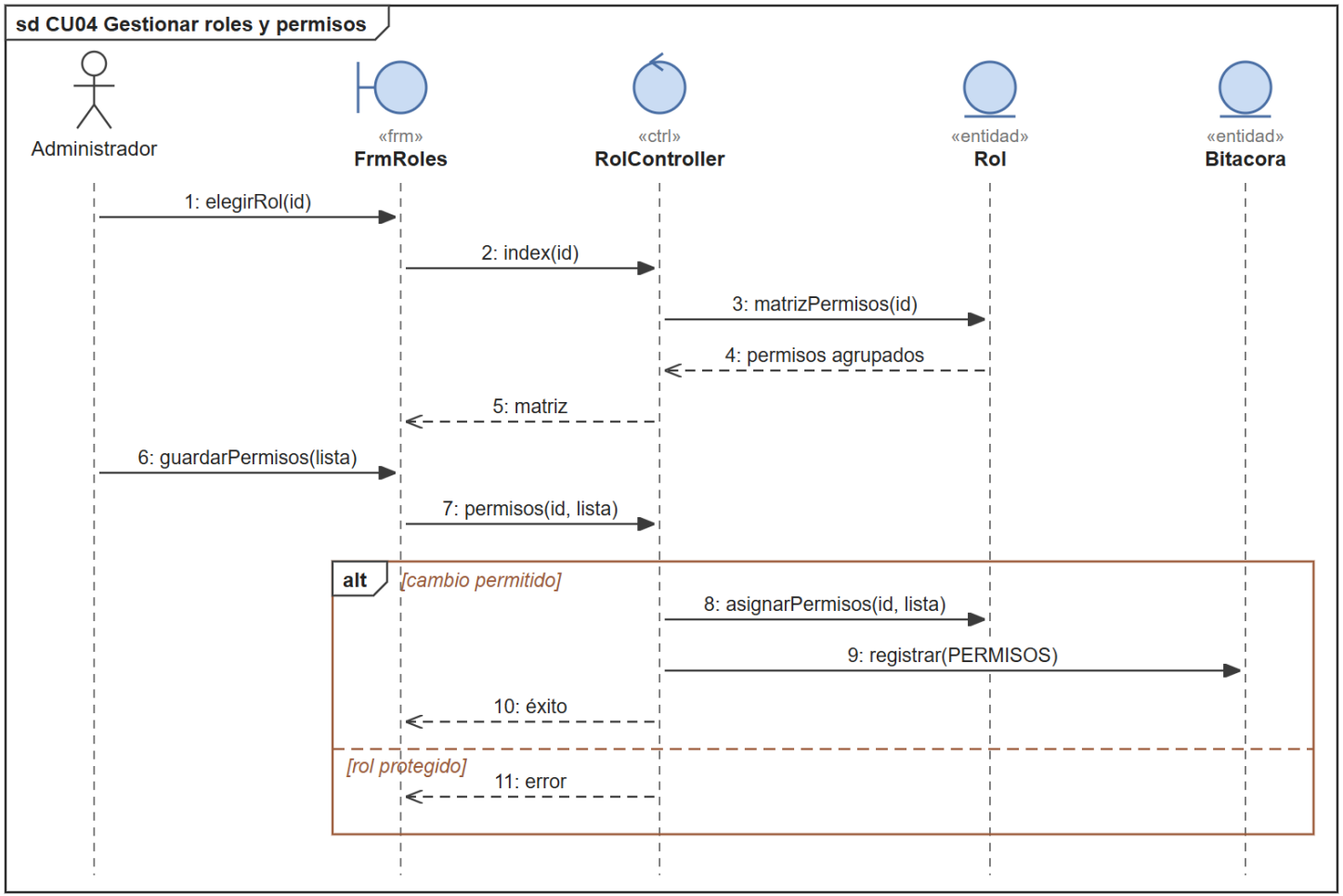
Diagrama de secuencia CU03 Gestionar usuarios



Nota. Elaboración propia.

Figura 149

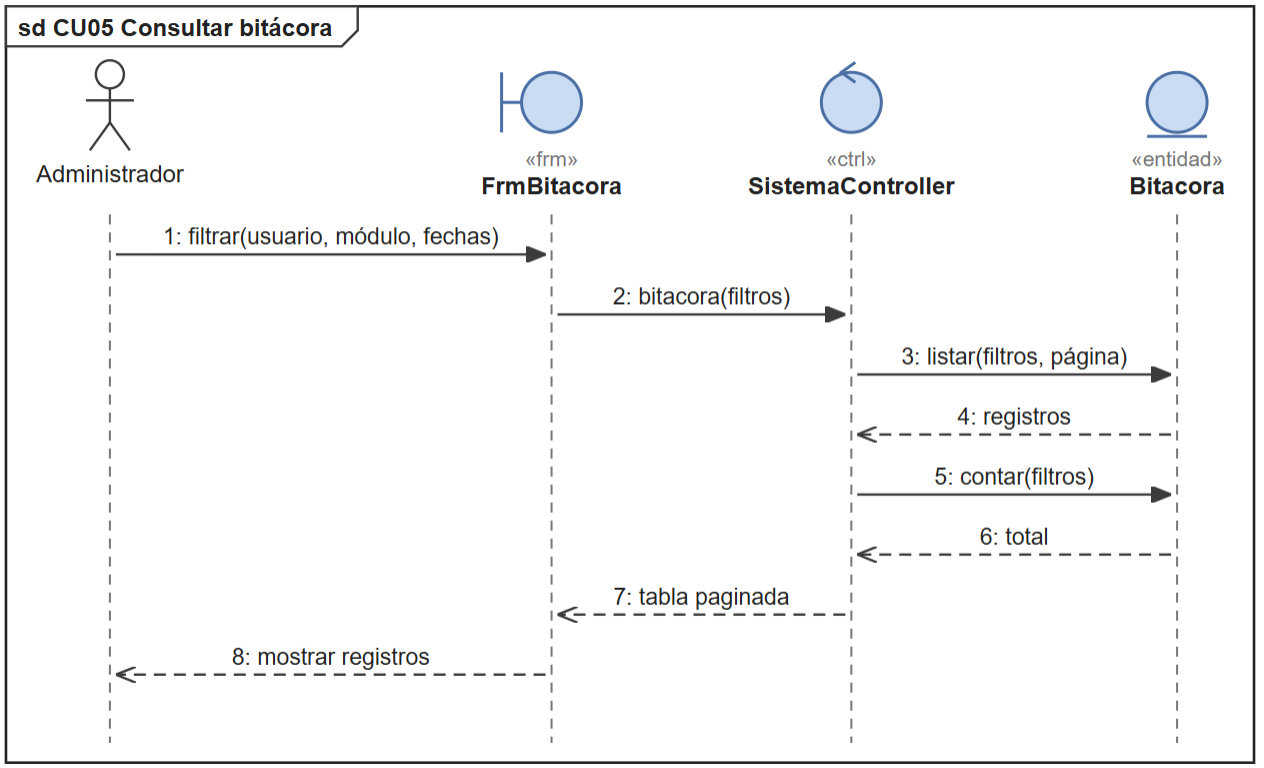
Diagrama de secuencia CU04 Gestionar roles y permisos



Nota. Elaboración propia.

Figura 150

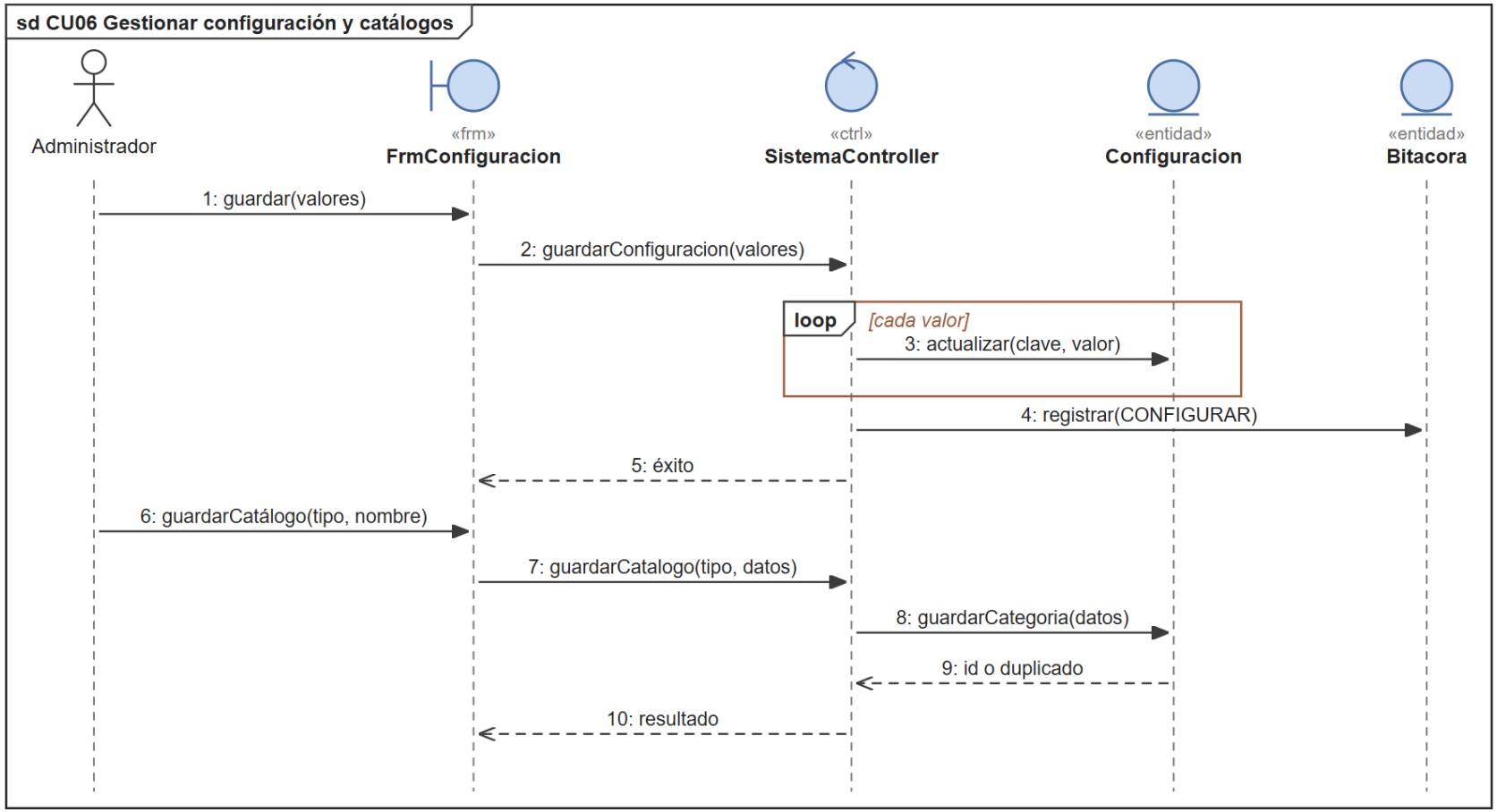
Diagrama de secuencia CU05 Consultar bitácora



Nota. Elaboración propia.

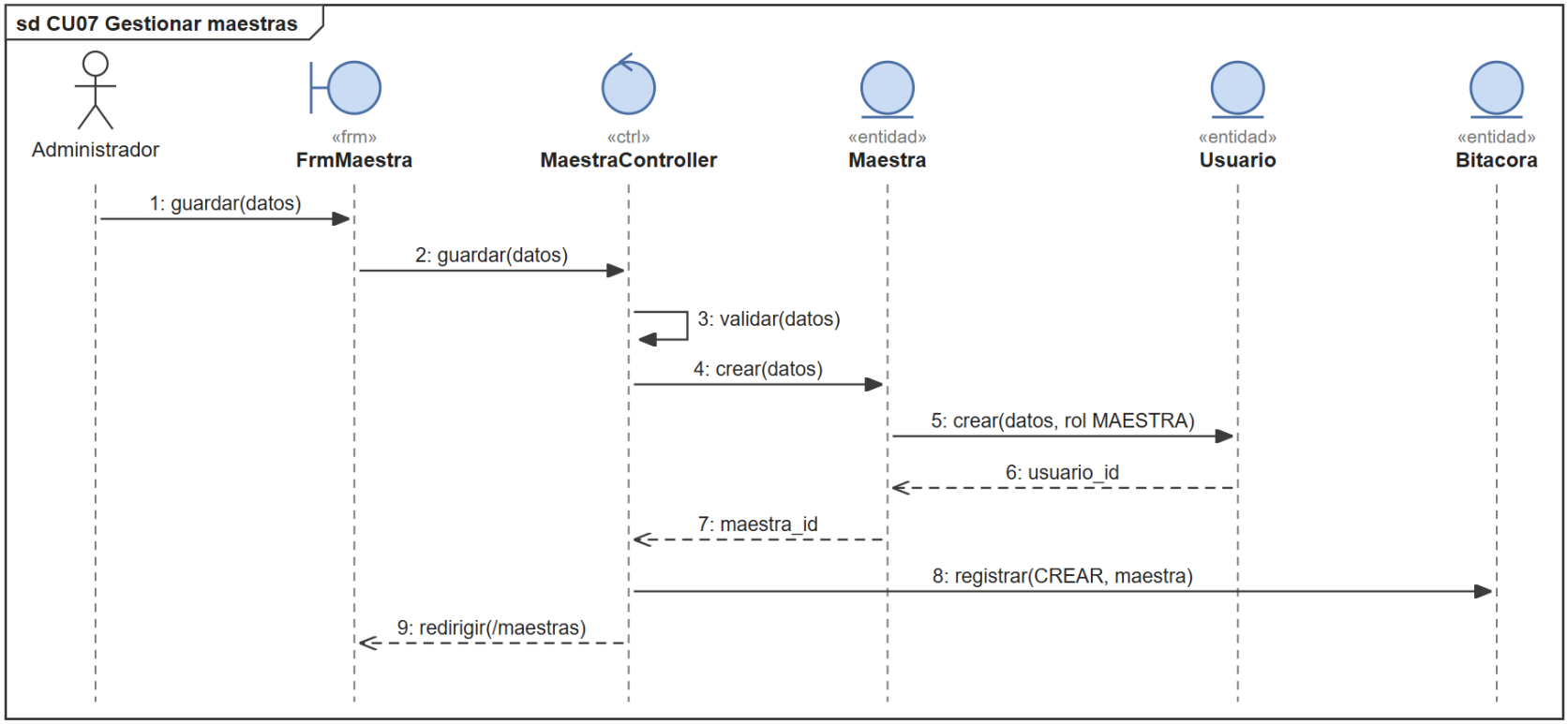
Figura 151

Diagrama de secuencia CU06 Gestionar configuración y catálogos



Nota. Elaboración propia.

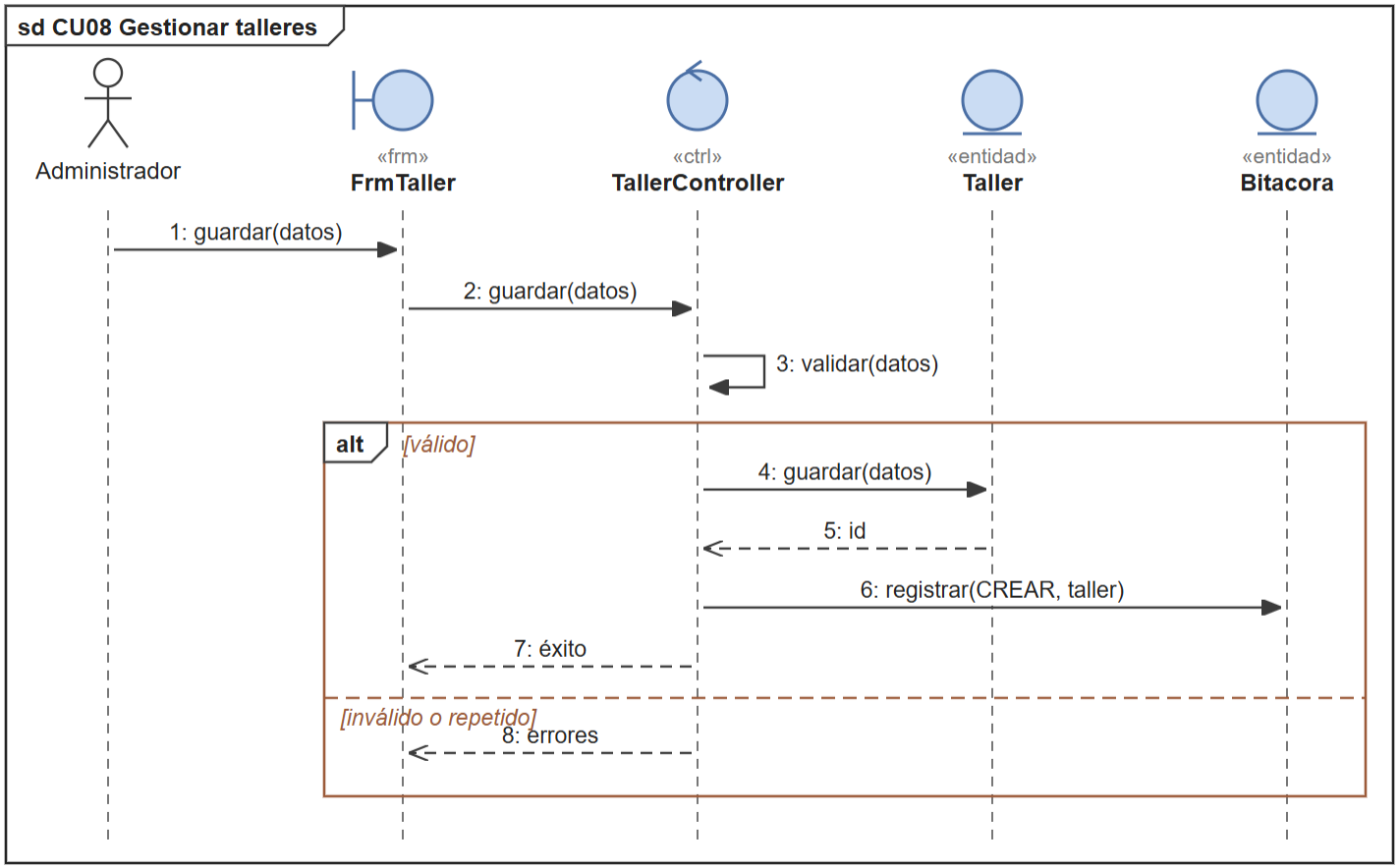
Figura 152
Diagrama de secuencia CU07 Gestionar maestras



Nota. Elaboración propia.

Figura 153

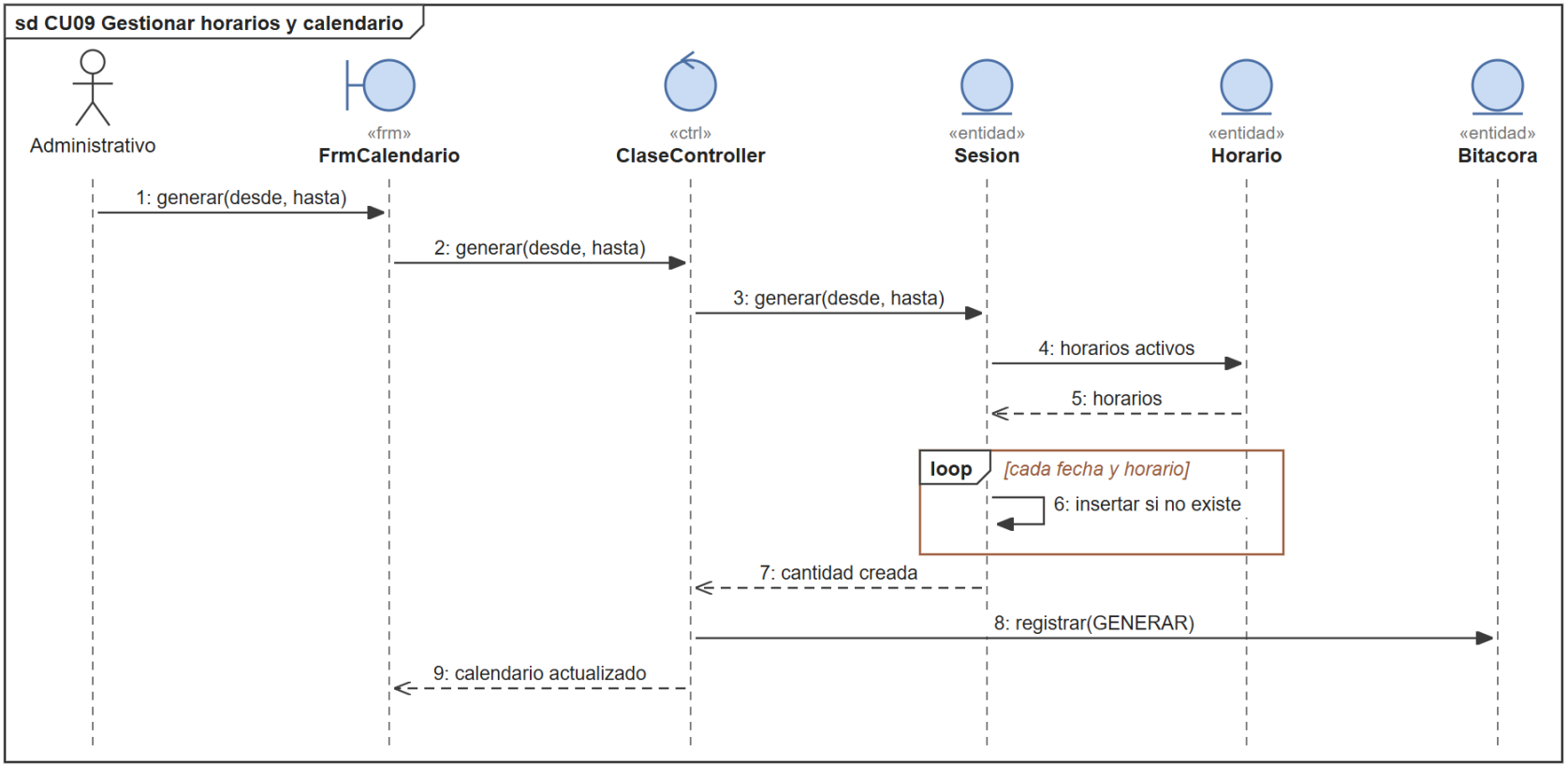
Diagrama de secuencia CU08 Gestionar talleres



Nota. Elaboración propia.

Figura 154

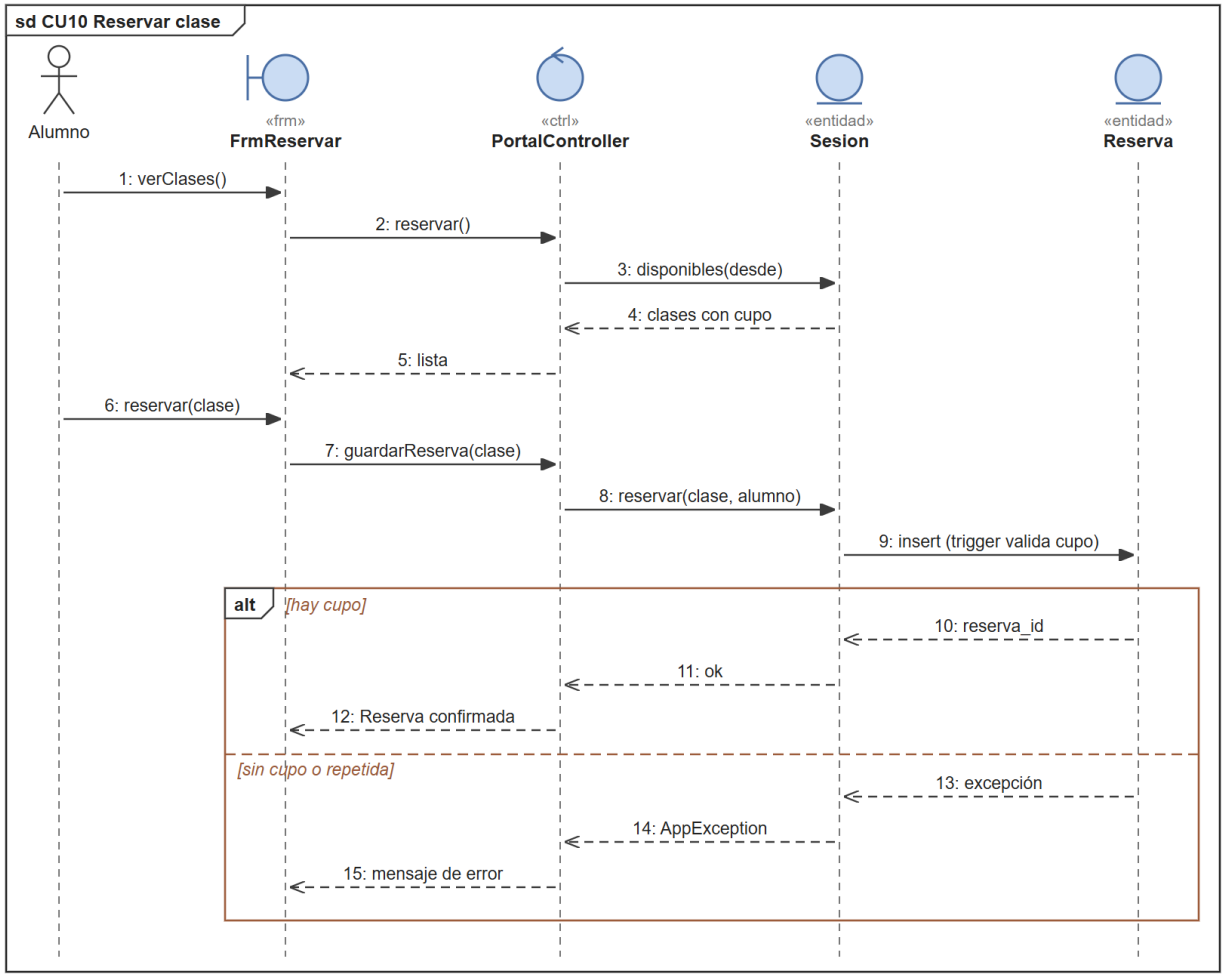
Diagrama de secuencia CU09 Gestionar horarios y calendario



Nota. Elaboración propia.

Figura 155

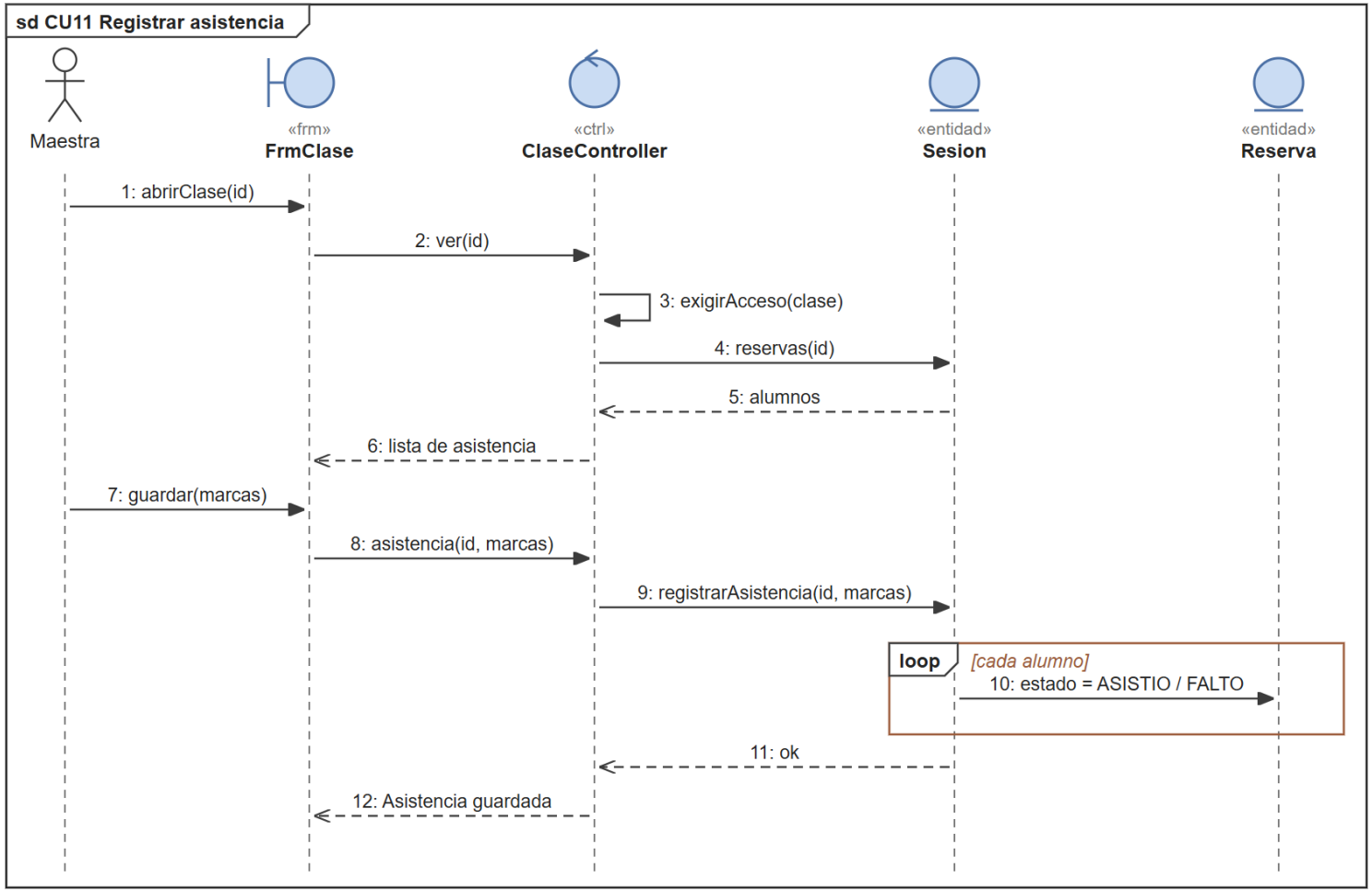
Diagrama de secuencia CU10 Reservar clase



Nota. Elaboración propia.

Figura 156

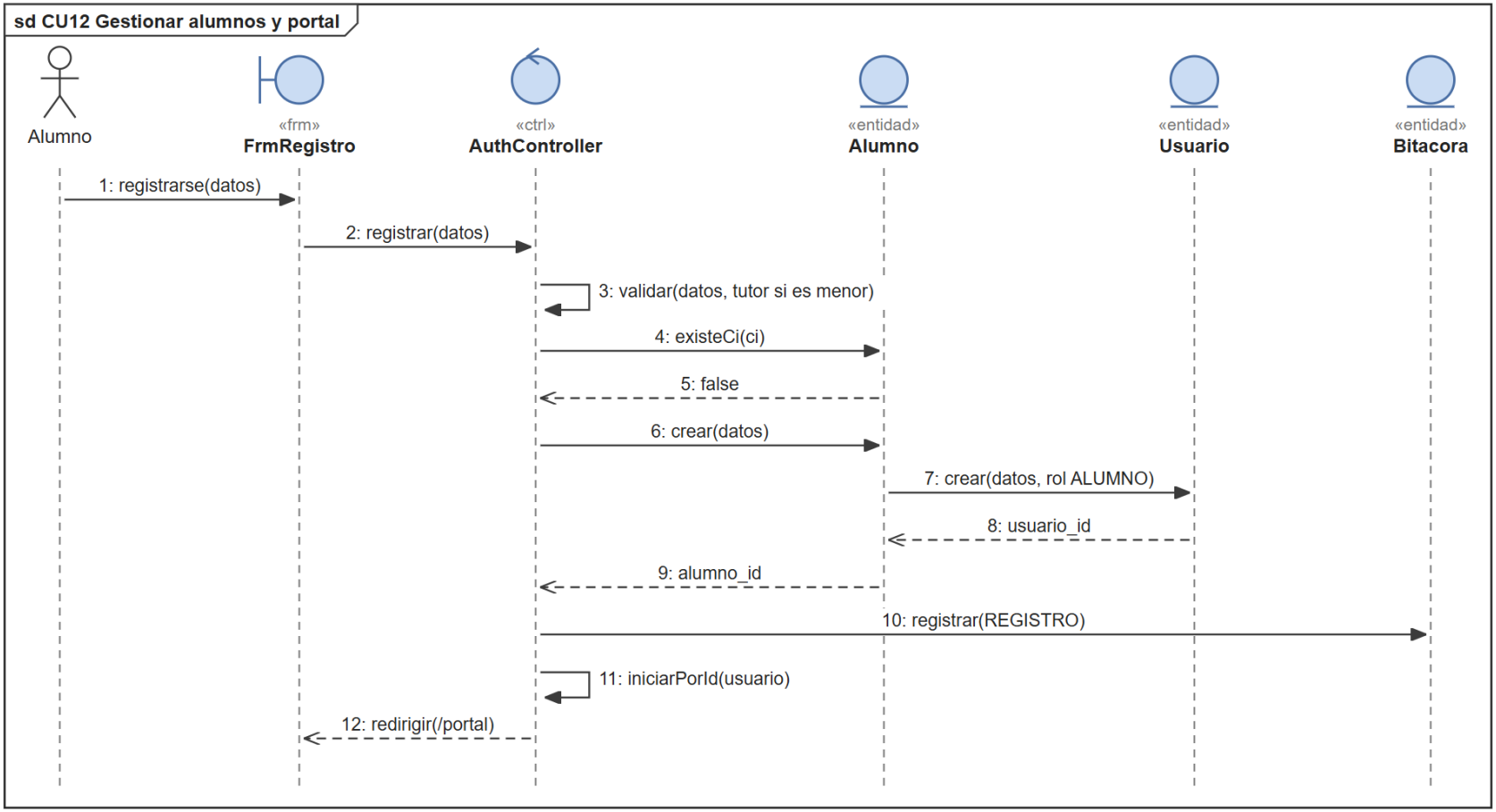
Diagrama de secuencia CU11 Registrar asistencia



Nota. Elaboración propia.

Figura 157

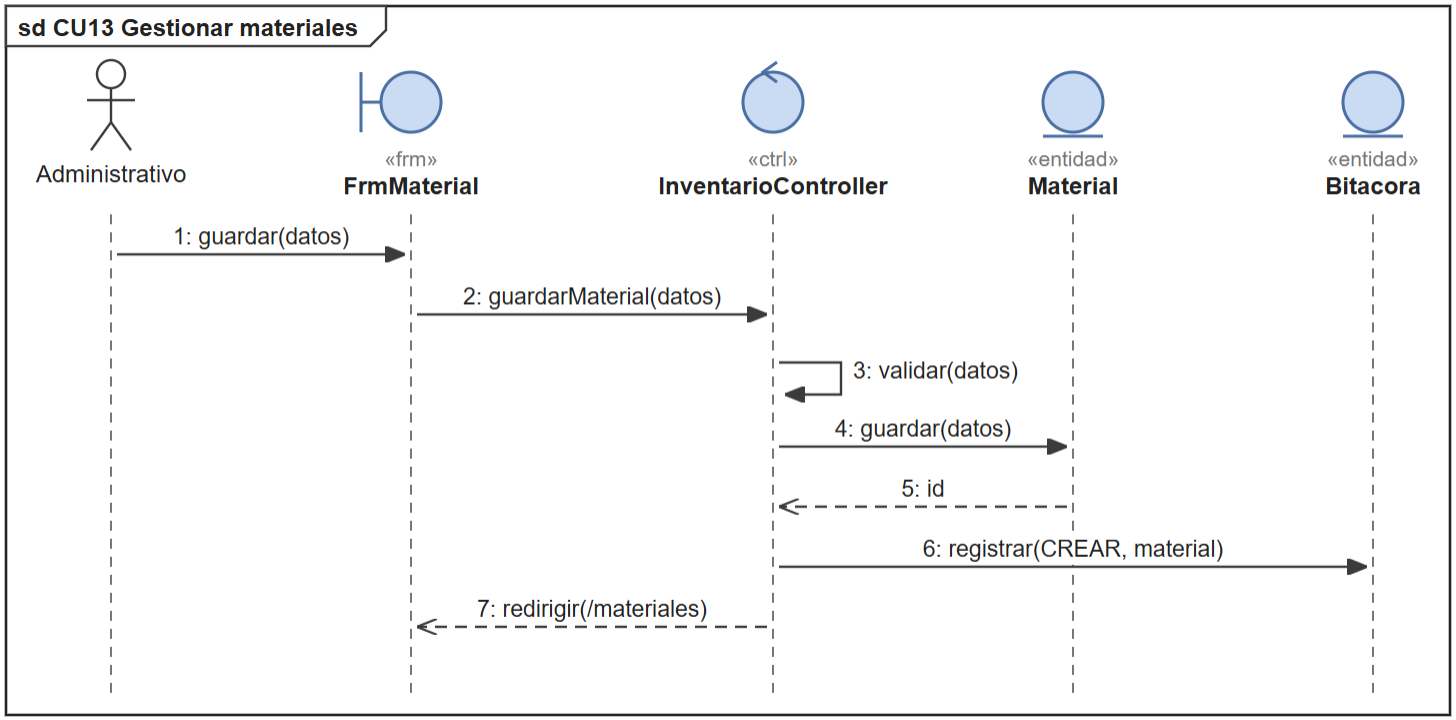
Diagrama de secuencia CU12 Gestionar alumnos y portal



Nota. Elaboración propia.

Ciclo III: inventario, ventas y tienda en línea

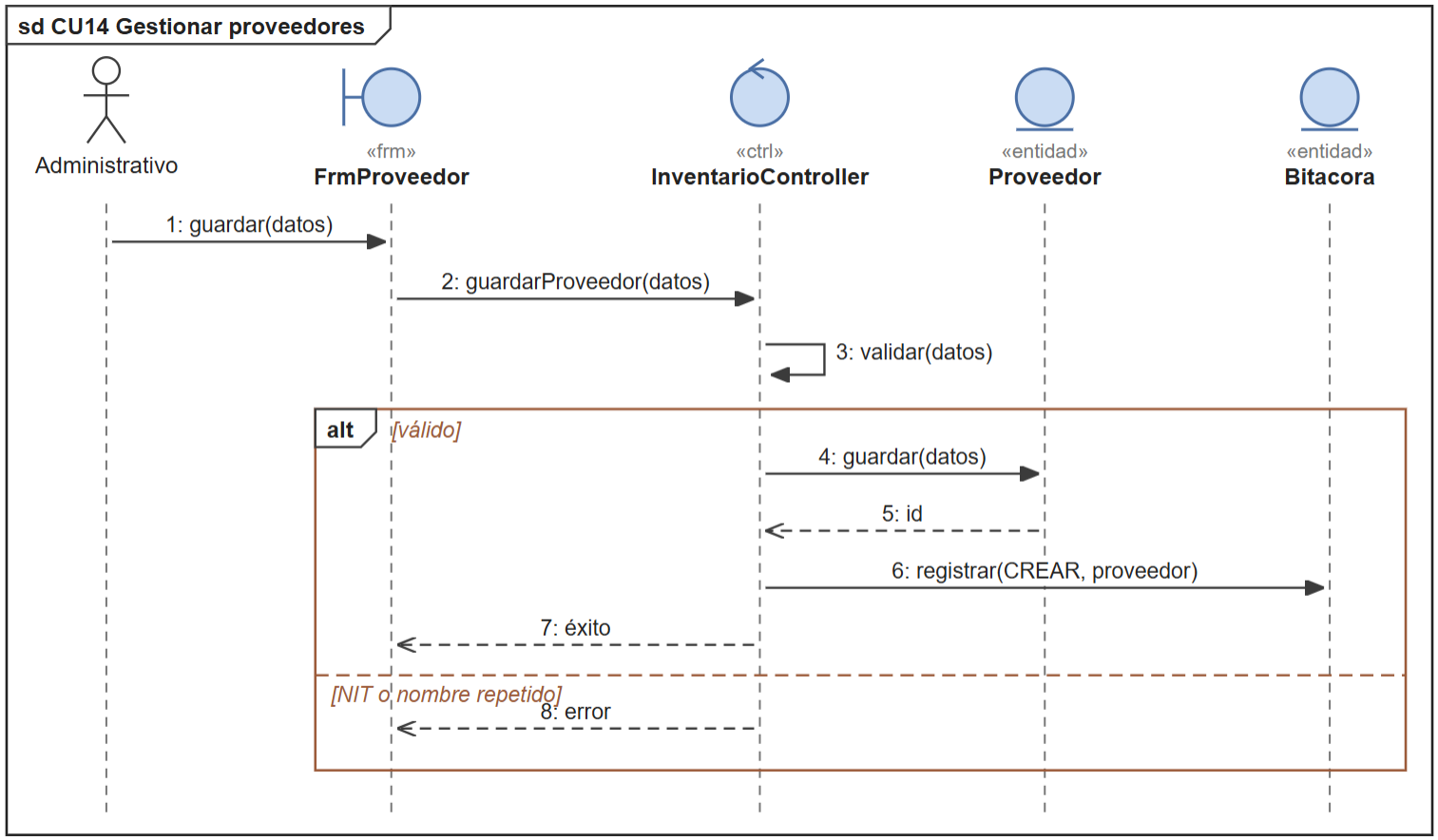
Figura 158
Diagrama de secuencia CU13 Gestionar materiales



Nota. Elaboración propia.

Figura 159

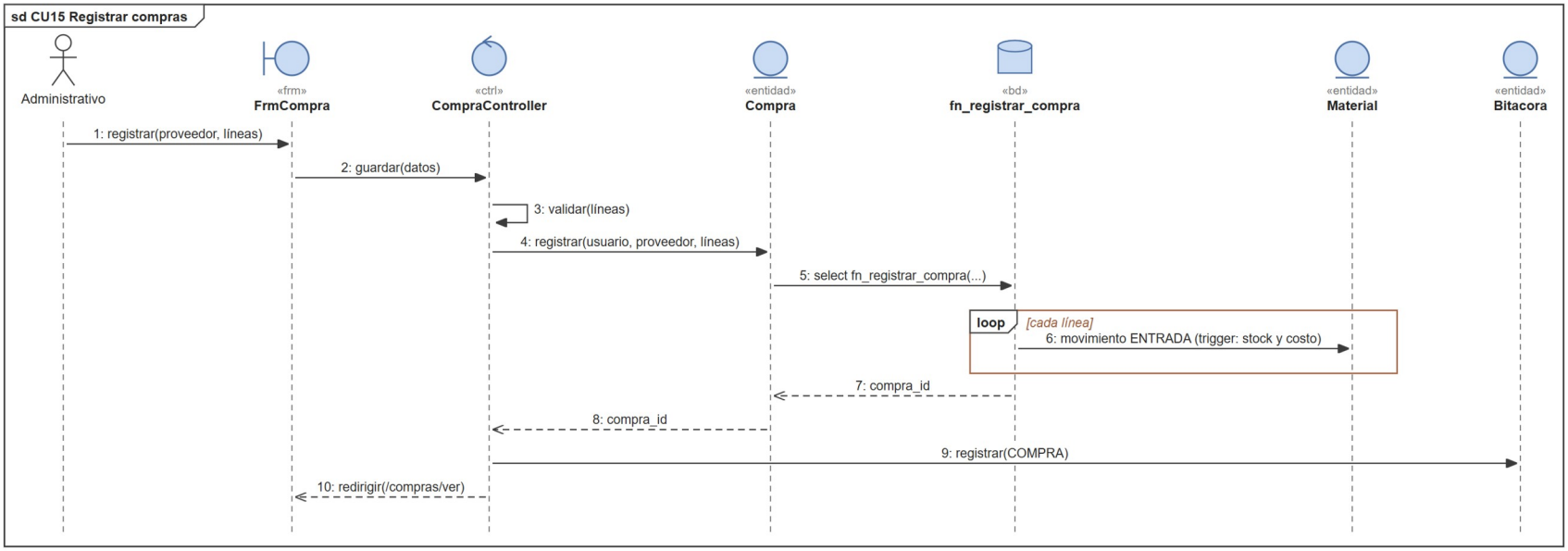
Diagrama de secuencia CU14 Gestionar proveedores



Nota. Elaboración propia.

Figura 160

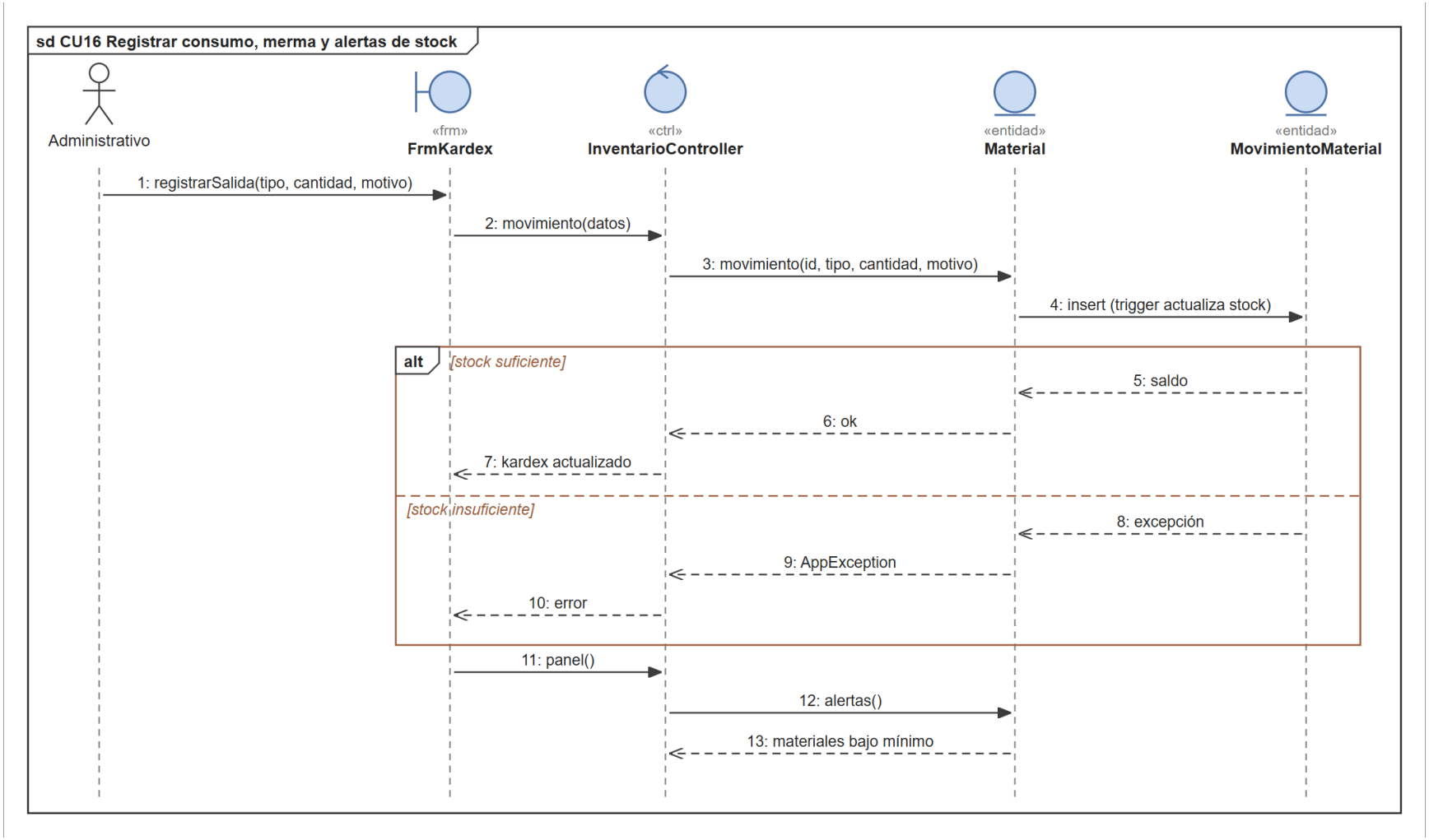
Diagrama de secuencia CU15 Registrar compras



Nota. Elaboración propia.

Figura 161

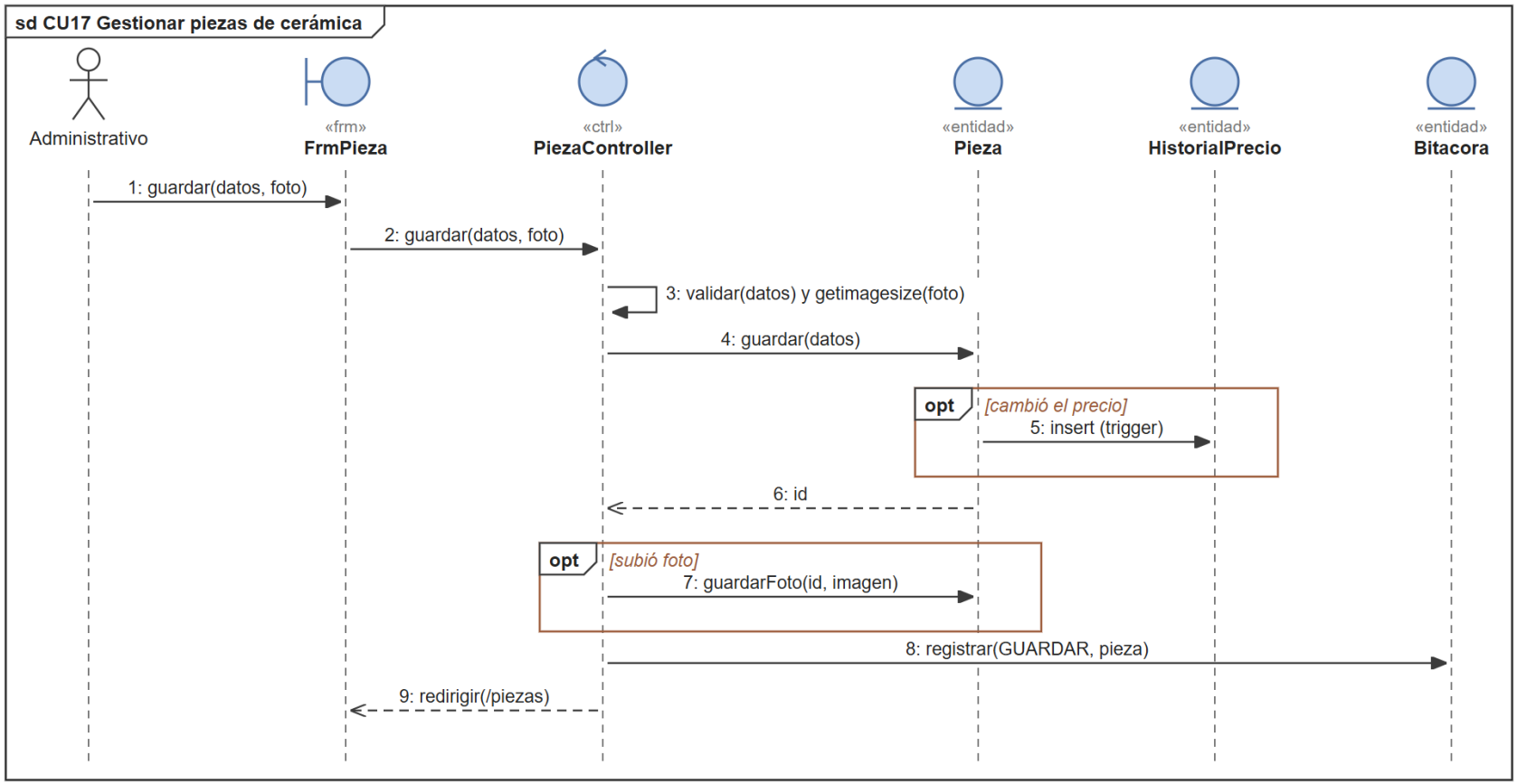
Diagrama de secuencia CU16 Registrar consumo, merma y alertas de stock



Nota. Elaboración propia.

Figura 162

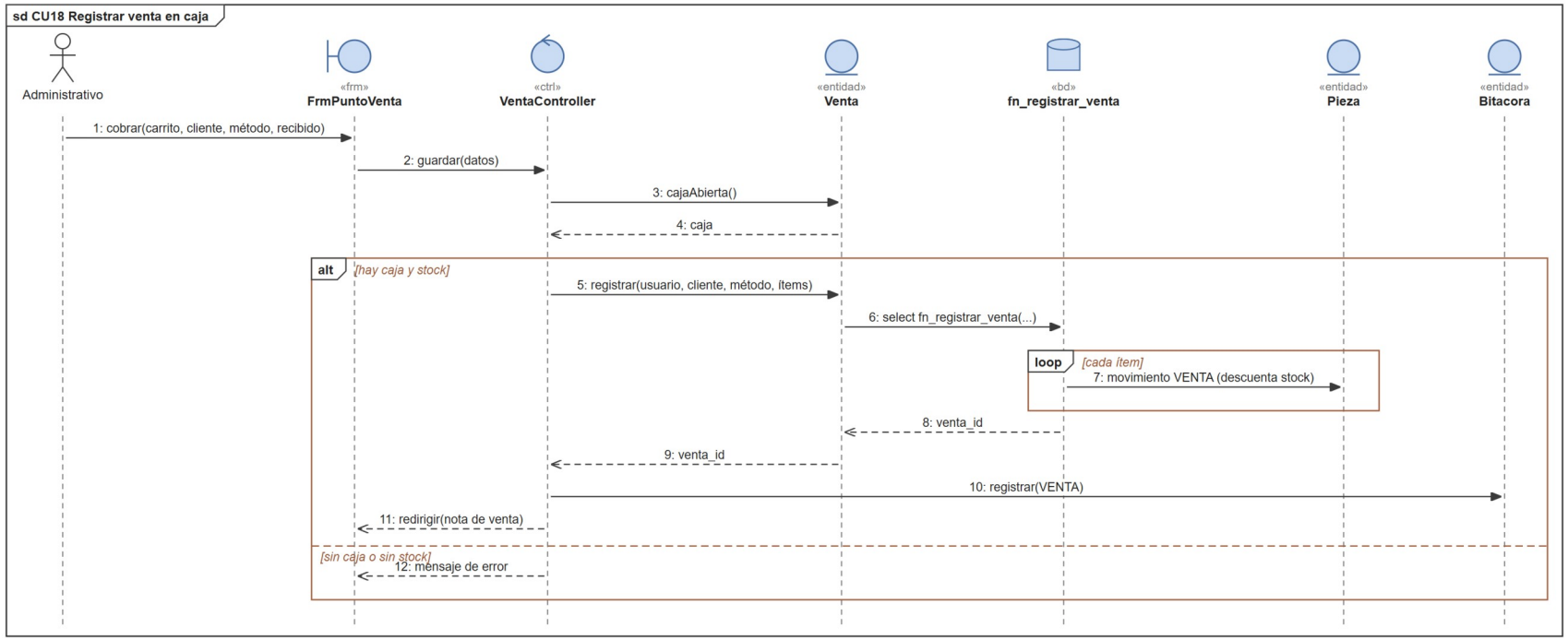
Diagrama de secuencia CU17 Gestionar piezas de cerámica



Nota. Elaboración propia.

Figura 163

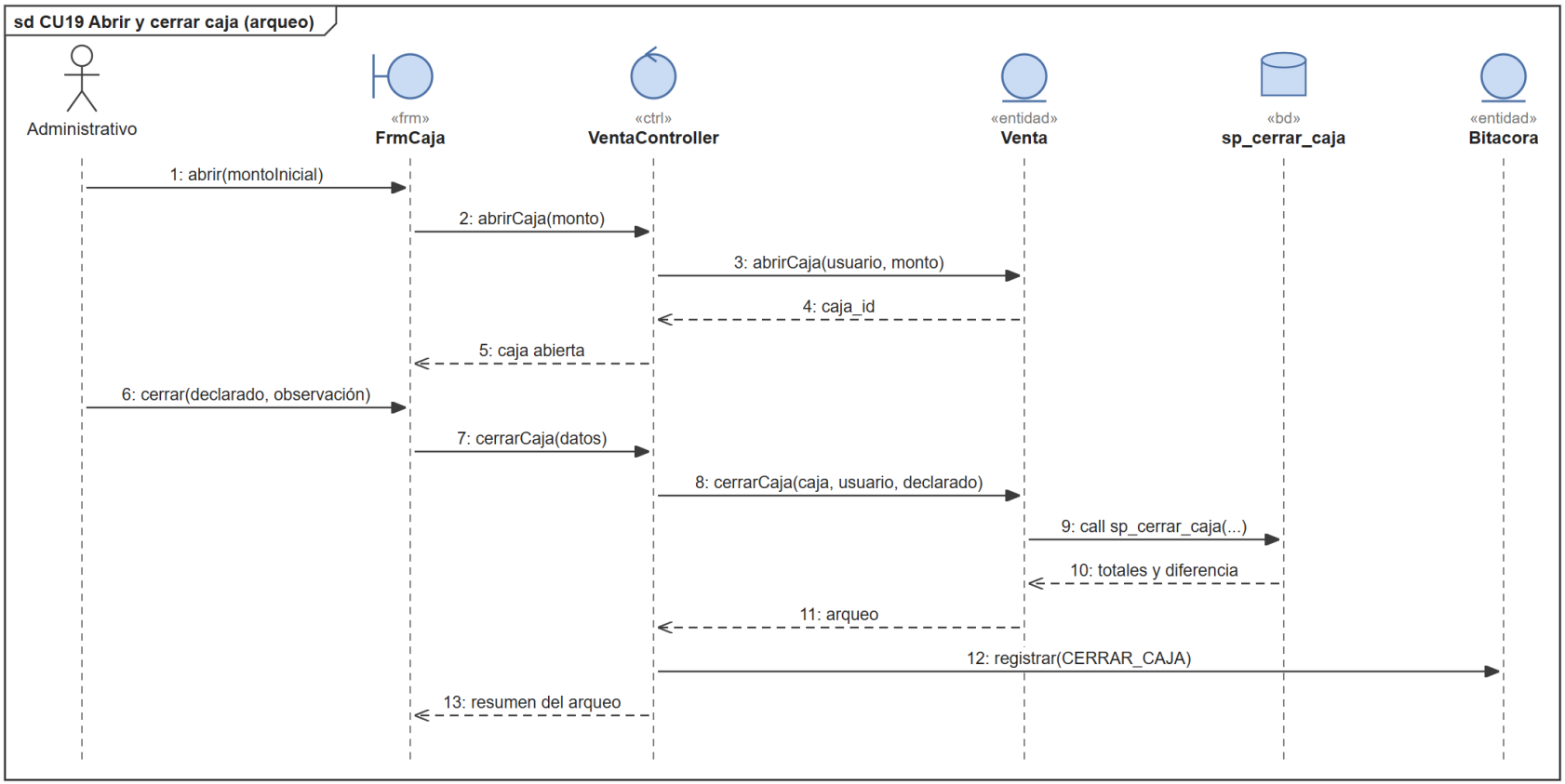
Diagrama de secuencia CU18 Registrar venta en caja



Nota. Elaboración propia.

Figura 164

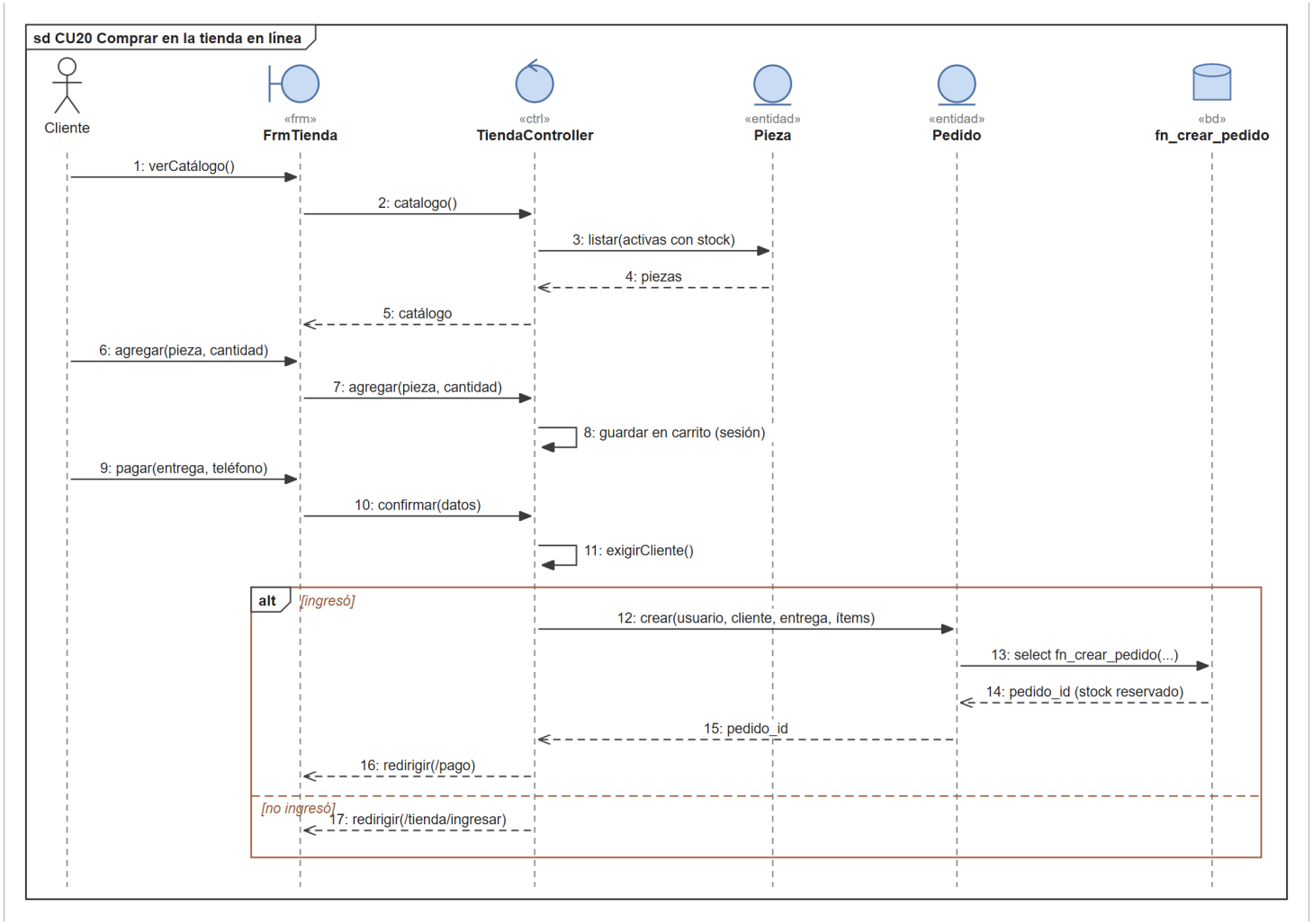
Diagrama de secuencia CU19 Abrir y cerrar caja (arqueo)



Nota. Elaboración propia.

Figura 165

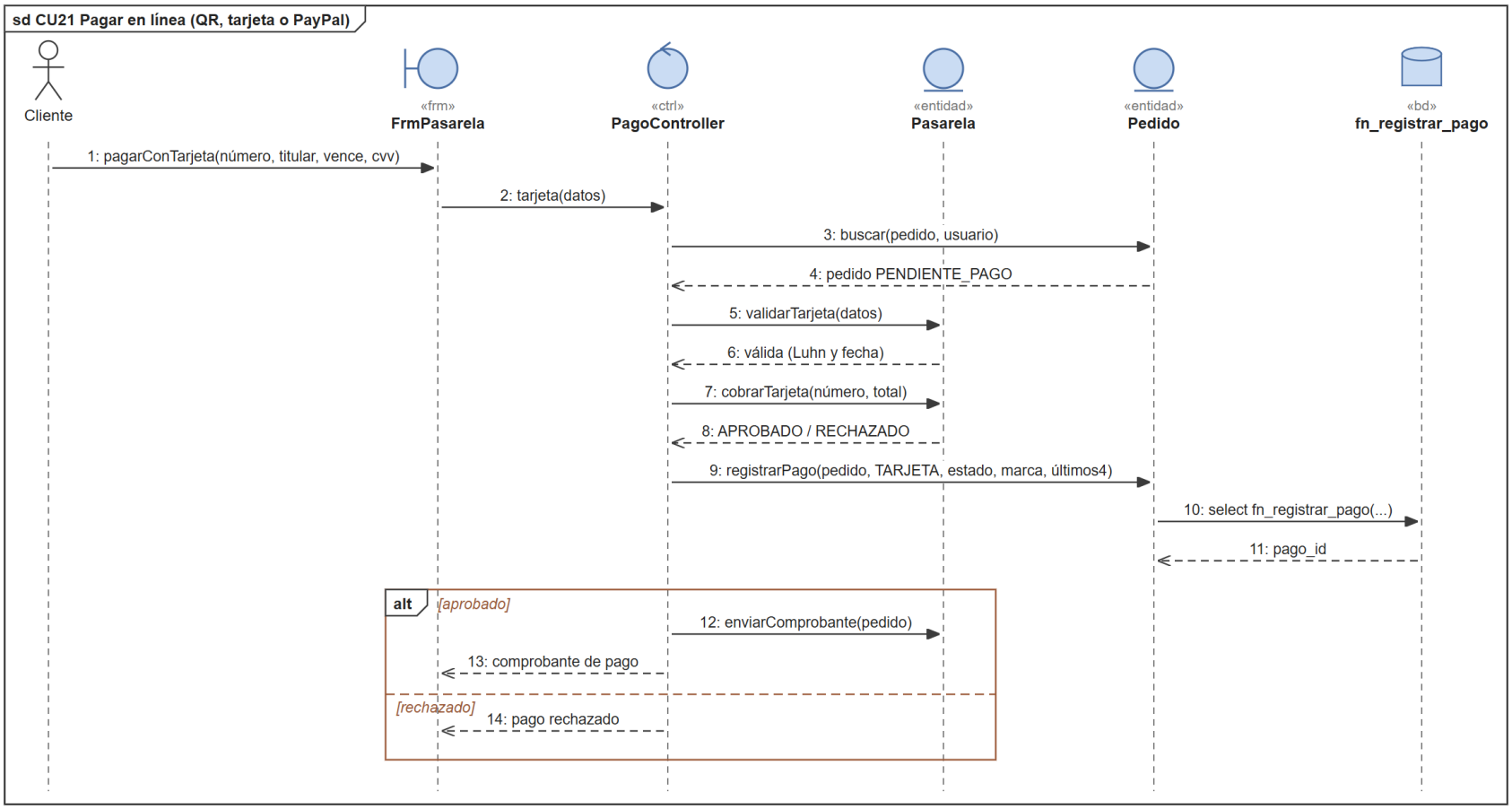
Diagrama de secuencia CU20 Comprar en la tienda en línea



Nota. Elaboración propia.

Figura 166

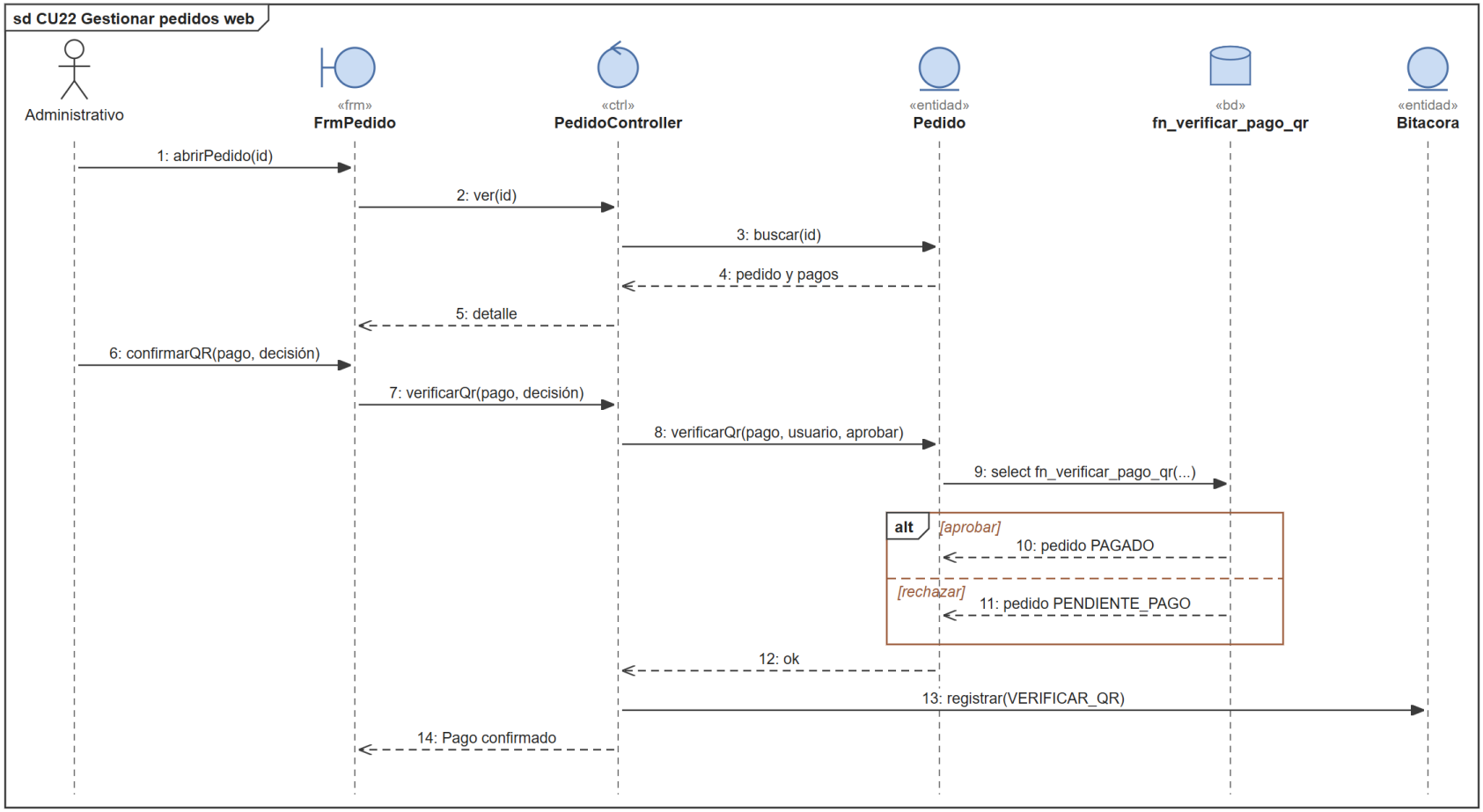
Diagrama de secuencia CU21 Pagar en línea (QR, tarjeta o PayPal)



Nota. Elaboración propia.

Figura 167

Diagrama de secuencia CU22 Gestionar pedidos web



Nota. Elaboración propia.

Capítulo 6. Flujo de trabajo: implementación

6.1 Elección de la plataforma de desarrollo del software

Lenguaje de programación

PHP 8.3 sin framework, con un MVC propio. Es conocido por el equipo, gratuito y soportado en Vercel; al no usar framework se ve y se explica cada capa. La interfaz usa HTML5, CSS3 y JavaScript propio, adaptable al celular y con modo claro y oscuro.

Base de datos

PostgreSQL 17 en Supabase. Permite triggers, funciones PL/pgSQL, restricciones EXCLUDE para que no se crucen los horarios y seguridad por filas.

Sistemas operativos

En producción, **Linux** en la nube (Vercel y Supabase), por lo que no hay servidores que administrar. En desarrollo, **Windows 11** con el servidor integrado de PHP.

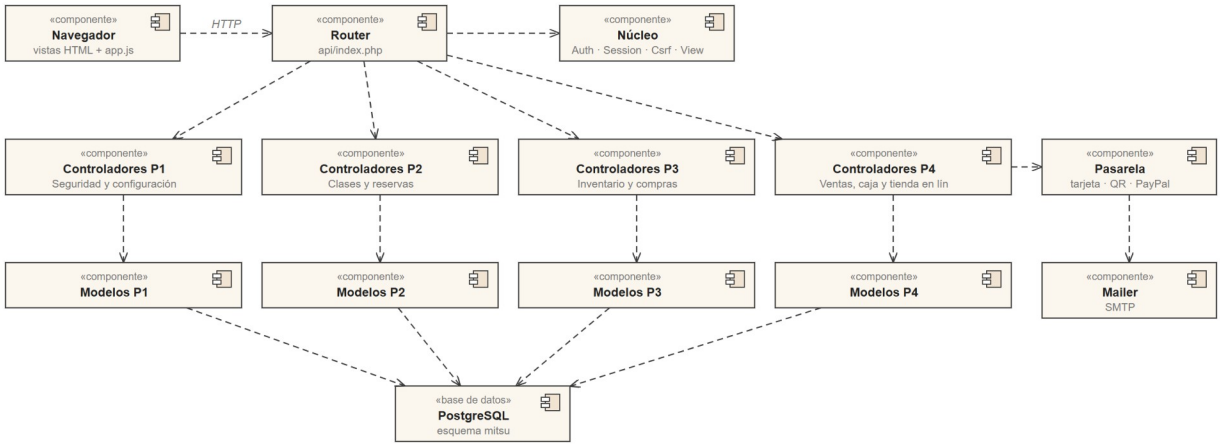
Otros

Git y GitHub para el control de versiones, PDO con pdo_pgsql para el acceso seguro a datos, Puppeteer para las pruebas y capturas, y Visual Studio Code como editor.

6.2 Implementación de la arquitectura del sistema

Diagrama de componentes principal del sistema

Figura 168
Diagrama de componentes principal



Nota. Elaboración propia.

Estructura de carpetas

Árbol generado con git ls-files: muestra las carpetas del repositorio hasta tres niveles y los archivos de la raíz y de api/.

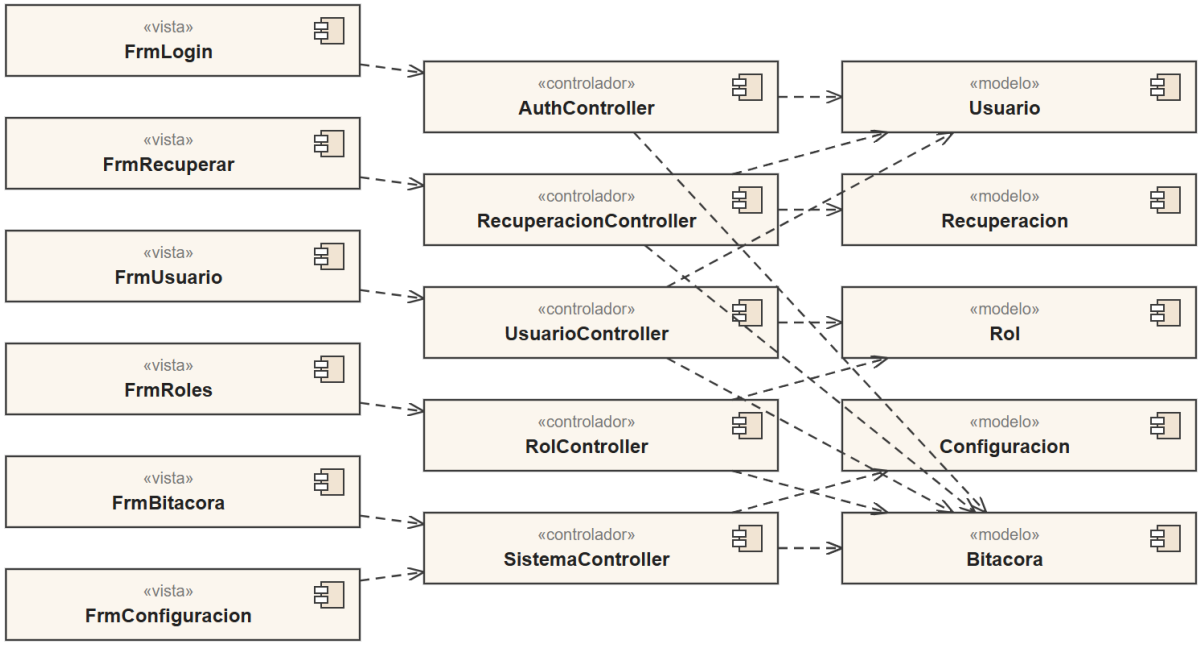
```
Taller-Mitsu_Proyecto Final/
├── api/                              Vercel exige esta carpeta
│   └── index.php                     Controlador frontal: entrada de todas las rutas
├── Bakend/                           Lógica del sistema y base de datos
│   ├── app/                          bootstrap.php: arranque de la aplicación
│   │   ├── controllers/              Un controlador por grupo de casos de uso
│   │   ├── core/                     Núcleo: Router, Auth, Validator, Mailer
│   │   ├── models/                   Un modelo por entidad (consultas con parámetros)
│   │   └── services/                 Pasarela de pago
│   └── database/                     Scripts SQL 01 a 07 y ejecutar.php
├── Docs/                             Documentación del proyecto
│   ├── diagramas/                    Diagramas UML (Python) en svg y png
│   │   ├── png/
│   │   └── svg/
│   ├── informe/                      Este informe y sus generadores
│   │   ├── capturas/
│   │   ├── datos/
│   │   └── recursos/
│   └── pruebas/                      Pruebas automáticas
├── Frontend/                         Lo que ve el usuario
│   ├── public/                       Archivos que descarga el navegador
│   │   └── assets/                   CSS, JavaScript e imágenes
│   └── views/                        Pantallas por paquete
│       ├── errors/
│       ├── layouts/
│       ├── P1_Seguridad/
```

```
│       ├── P2_Clases/
│       ├── P3_Inventario/
│       ├── P4_Ventas/
│       ├── partials/
│       └── Tienda/
├── .env.example
├── .gitignore
├── .htaccess                          Apache (Laragon o XAMPP)
├── .vercelignore
├── README.md
├── router.php                         Servidor local de PHP
└── vercel.json                        Configuración del despliegue
```

6.3 Implementación de la arquitectura del subsistema

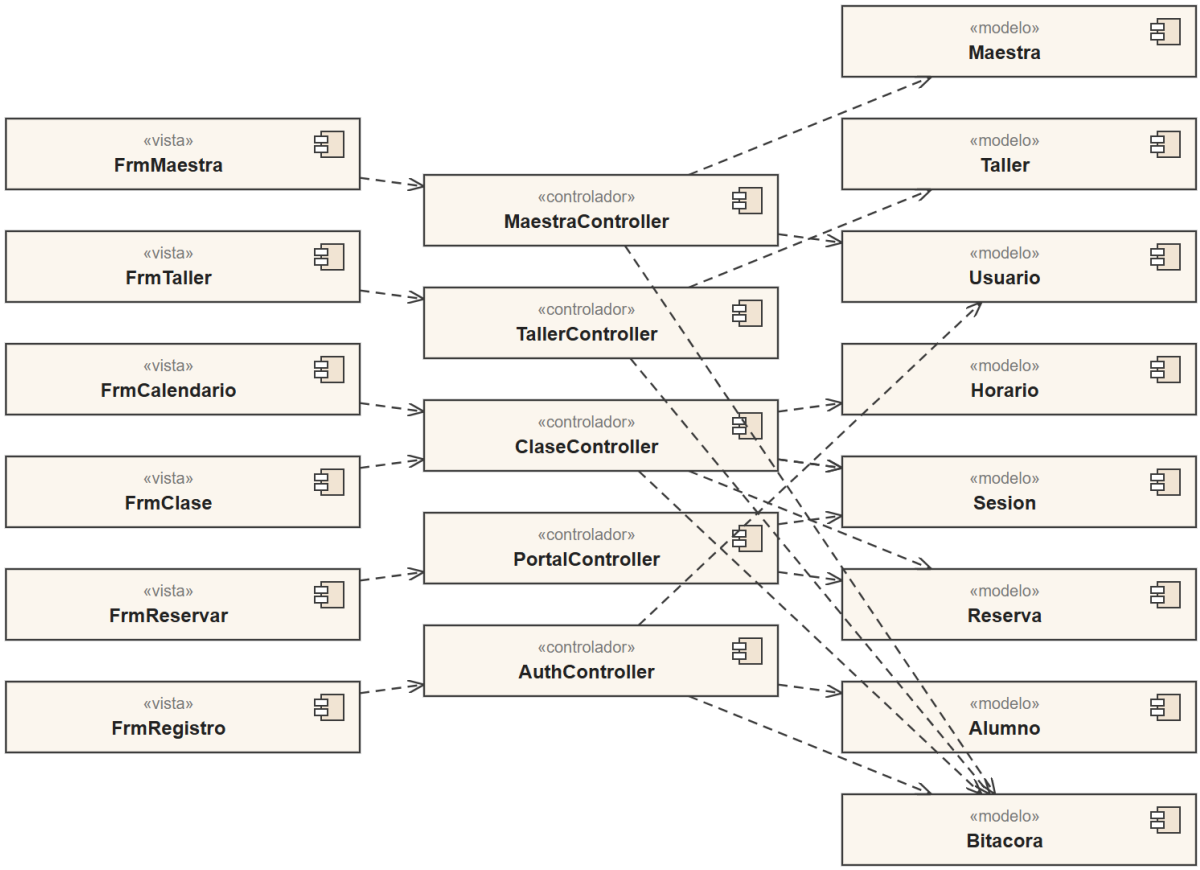
Diagrama de componentes de cada paquete

Figura 169
Componentes de P1 Seguridad y configuración



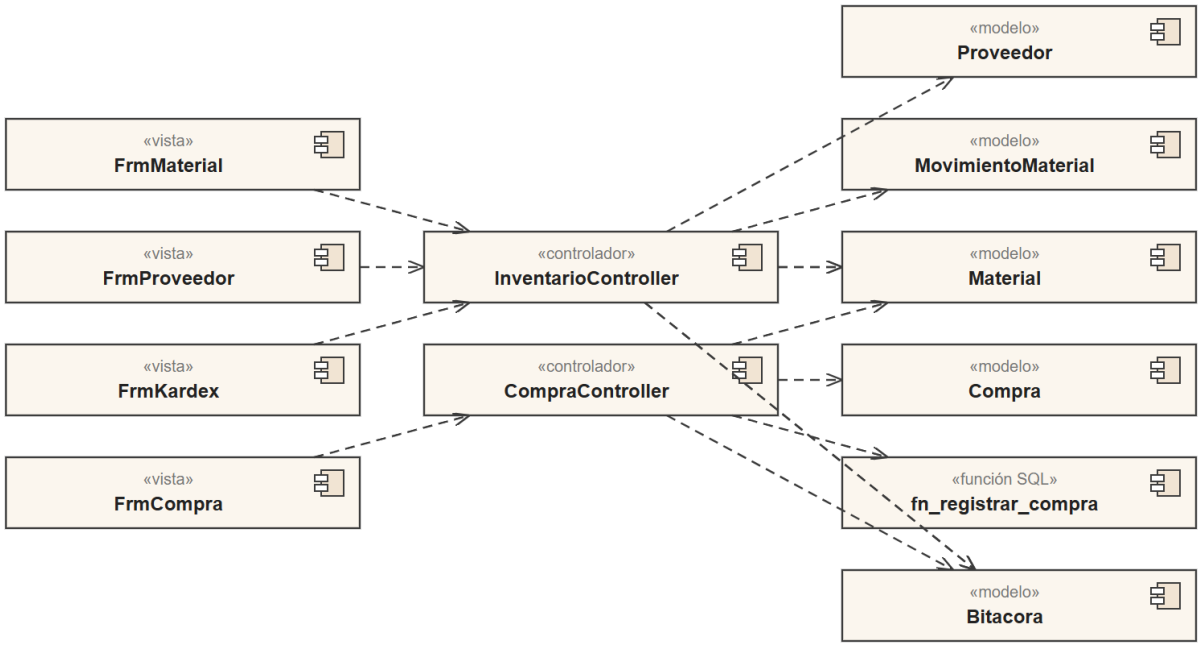
Nota. Elaboración propia.

Figura 170
Componentes de P2 Clases y reservas



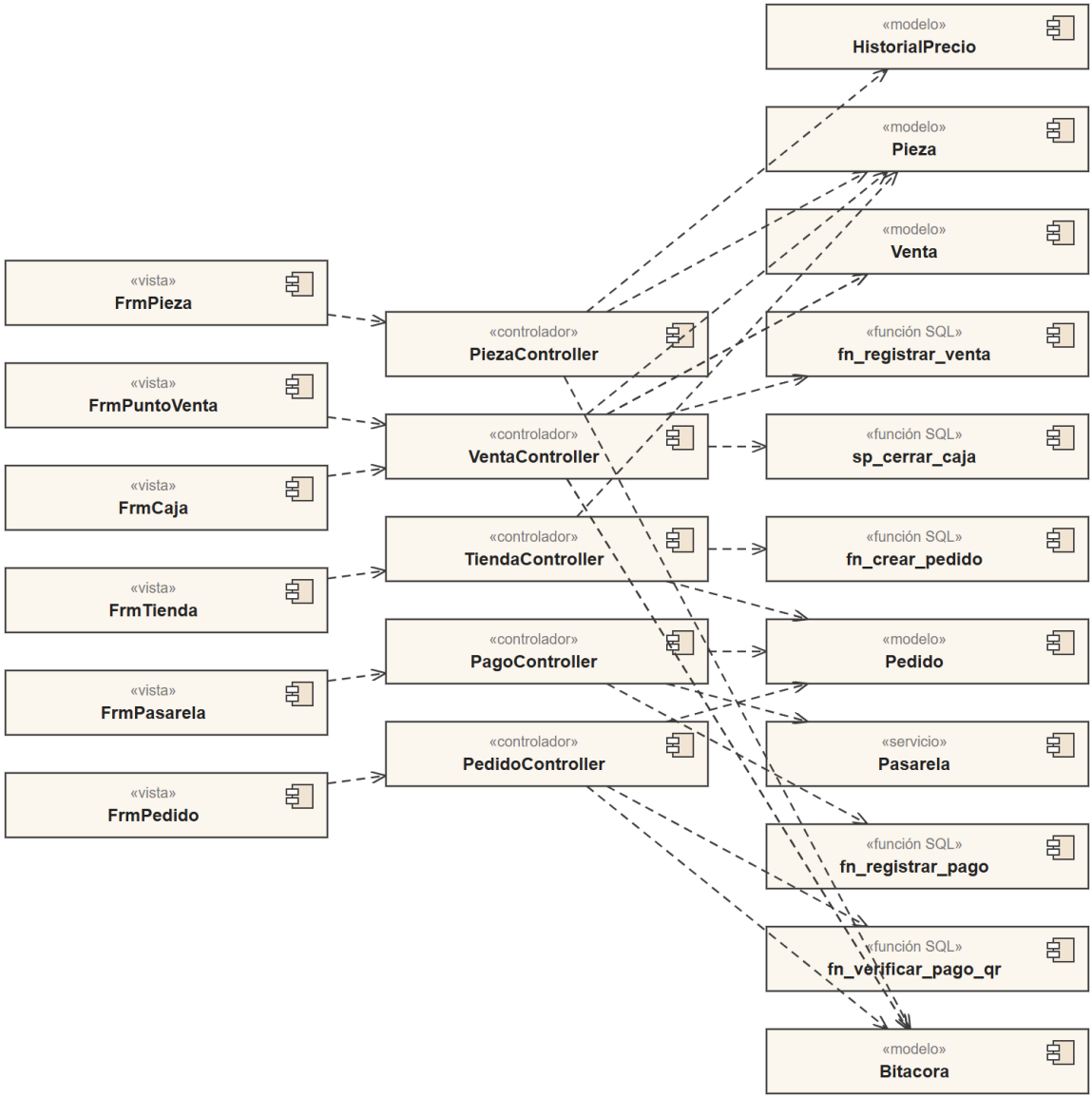
Nota. Elaboración propia.

Figura 171
Componentes de P3 Inventario y compras



Nota. Elaboración propia.

Figura 172
Componentes de P4 Ventas, caja y tienda en línea



Nota. Elaboración propia.

Seguridad implementada

- Contraseñas cifradas con bcrypt; bloqueo de 10 minutos tras 5 intentos fallidos desde la misma IP.
- Sesiones guardadas en la base de datos, con cookie HttpOnly, Secure y SameSite.
- Token CSRF en todos los formularios y escape de toda salida HTML.

- Cada ruta exige un permiso que se verifica en el servidor; una maestra no puede abrir la clase de otra (403).
- Consultas siempre con parámetros (sin concatenar datos del usuario).
- Usuario de base de datos de mínimo privilegio, seguridad por filas y triggers como última defensa.
- De las tarjetas solo se guardan marca y últimos 4 dígitos; nunca el número completo ni el CVV.
- Las claves reales están en variables de entorno de Vercel y en un archivo .env ignorado por Git.

7. Conclusiones

- Se capturaron los requisitos del taller con una entrevista, un análisis FODA y el método de Ishikawa, y se tradujeron en 22 casos de uso distribuidos en tres ciclos del PUDS.
- El modelo de análisis (comunicación y clases de análisis) y el de diseño (paquetes, capas, despliegue, secuencia y componentes) se mantuvieron consistentes con el código: los controladores y métodos de los diagramas existen en el sistema.
- La base de datos tiene 33 tablas normalizadas hasta 3FN, con reglas críticas en triggers y procedimientos que se probaron contra la base publicada: las 30 consultas, 10 procedimientos y 11 pruebas de reglas dieron el resultado esperado.
- El sistema está publicado en internet y funciona en computadora y celular: la tienda se ve primero, el cliente ingresa solo para pagar y puede hacerlo con QR, tarjeta o PayPal en modo de demostración.
- Los problemas P1 a P7 de la lista final quedan atendidos por los ciclos I a III; la cafetería, las mensualidades y los reportes se abordarán en el ciclo IV.

8. Recomendaciones

- Pasar la base a un plan pagado de Supabase antes de usarla en el taller: el plan gratuito se pausa tras 7 días sin uso.
- Configurar el correo SMTP de Gmail con verificación en dos pasos y contraseña de aplicación para enviar comprobantes y códigos de recuperación.
- Antes de cobrar de verdad, contratar una pasarela autorizada en Bolivia (QR bancario con conciliación automática) o una cuenta comercial de PayPal.
- Capacitar a recepción y maestras con el manual del Anexo D y cambiar todas las contraseñas de demostración.
- Implementar el ciclo IV (cafetería, mensualidades y reportes) y medir los beneficios estimados en el capítulo 5 con datos reales.

9. Bibliografía

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Booch, G., Rumbaugh, J., y Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2.ª ed.). Pearson Educación.

Elmasri, R., y Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Ishikawa, K. (1985). *What is total quality control? The Japanese way*. Prentice Hall.

Jacobson, I., Booch, G., y Rumbaugh, J. (2000). *El proceso unificado de desarrollo de software*. Addison Wesley.

Larman, C. (2004). *UML y patrones: Una introducción al análisis y diseño orientado a objetos y al proceso unificado* (2.ª ed.). Pearson Educación.

Laudon, K. C., y Laudon, J. P. (2020). *Management information systems: Managing the digital firm* (16th ed.). Pearson.

Luhn, H. P. (1960). *Computer for verifying numbers* (U.S. Patent No. 2,950,048). U.S. Patent and Trademark Office.

Mell, P., y Grance, T. (2011). *The NIST definition of cloud computing* (NIST Special Publication 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

PCI Security Standards Council. (2022). *Payment Card Industry Data Security Standard* (v4.0). https://www.pcisecuritystandards.org/

PHP Group. (2024). *PHP manual*. https://www.php.net/manual/es/

PostgreSQL Global Development Group. (2024). *PostgreSQL 17 documentation*. https://www.postgresql.org/docs/17/

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Supabase. (2025). *Supabase documentation*. https://supabase.com/docs

Vercel. (2025). *Vercel documentation*. https://vercel.com/docs

Anexos

Anexo A. Entrevista a la gerente

Tabla 87
Datos de la entrevista

Campo	Dato
Lugar	Taller de Cerámica Mitsü, Santa Cruz de la Sierra
Duración	22 minutos
Fecha	No consta en el registro original del grupo
Entrevistada	Alessandra Civardi Soria, gerente
Entrevistador	Mauricio Miranda Soria
Objetivo	Conocer los procesos actuales y las necesidades del taller

Nota. Transcripción tomada del informe de perfil anterior del grupo.

1. ¿Cuáles fueron los inicios del taller?

Luego de terminar sus estudios en Argentina, y con la ayuda de su amiga Daniela, surgió la idea de crear un taller de cerámica para poner en práctica lo aprendido en la carrera de Artes Plásticas. Al ver lo difícil que era conseguir arcilla localmente y realizar los procesos de fabricación, decidieron fundar el taller para cubrir esa necesidad.

2. ¿Cuáles fueron sus inspiraciones para la estética de sus piezas?

Lo aprendido en la universidad en Argentina sobre diseños contemporáneos, fusionado con la esencia de los pueblos originarios de Bolivia (región andina de La Paz y Amazonía boliviana). No se limitan a estas temáticas; buscan expandir sus horizontes con cada proyecto, apostando por una fabricación artesanal con materiales originarios del país.

3. ¿En qué productos se enfocan más al momento de crear?

En vajillas, tazas y platos, con un enfoque utilitario o más artístico según los pedidos de los clientes.

4. ¿Cuántas maestras dan clases y qué horarios manejan?

Son 4 maestras. Martes: modelado a mano y avanzado (10:00–12:00) y modelado (18:00 –20:00). Miércoles: modelado a mano y torno (10:00–12:00) y taller de modelado (18:00–20:00).

Jueves y viernes: modelado a mano y torno (10:00–12:00). Sábado: adultos (10:00–12:00), niños (16:00–18:00) y modelado y torno (18:00–20:00).

5. ¿Qué es lo más solicitado, las clases o las piezas terminadas?

Las clases, porque las piezas terminadas tienen un costo más elevado por el trabajo artesanal.

6. ¿Piensan ampliar el personal del taller?

Por el momento no, por la situación económica del país. A futuro sería ideal contratar personal dedicado solo a la producción.

7. ¿Cómo consiguen la arcilla y los materiales?

Los compran a empresas mineras de Cochabamba y La Paz, que son sus únicos proveedores de arcilla y minerales como el cuarzo.

8. ¿Cómo organizan sus clases y la administración? ¿Usan algún sistema?

Usan hojas de cálculo de Excel para los horarios y un pequeño software de gestión para sus finanzas.

9. ¿Ofrecen otro servicio aparte de las clases y la venta?

Sí, una cafetería para alumnos y visitantes, con personal propio dedicado a la atención gastronómica.

10. ¿Qué ingredientes usan en la cafetería?

Mayormente naturales y de cultivo propio, para un menú saludable y delicioso.

Anexo B. Cuentas de prueba

Tabla 88

Cuentas de demostración

Rol	Correo	Contraseña
Administrador	`admin@mitsu.test`	`Admin2026!`
Administrativo	`ventas@mitsu.test`	`Ventas2026!`
Maestra	`maestra@mitsu.test`	`Maestra2026!`

Rol	Correo	Contraseña
Alumno	`alumno@mitsu.test`	`Alumno2026!`
Cliente	`cliente@mitsu.test`	`Cliente2026!`

Nota. Contraseñas inventadas solo para la demostración. Tarjetas de prueba: 4242 4242 4242 4242 (aprobada) y 4000 0000 0000 0002 (rechazada), con cualquier fecha futura y CVV de 3 dígitos.

Anexo C. Pruebas del sistema

Tabla 89
Pruebas automáticas

Script	Qué prueba	Resultado
`Docs/pruebas/probar_rutas.php`	55 pantallas con cada rol; 20 accesos prohibidos (403); página inexistente (404); formulario sin token (419).	Todo correcto
`Docs/pruebas/probar_flujos.php`	20 operaciones de escritura: usuarios, roles, talleres, reservas, asistencia, compras, ventas, anulaciones y caja.	Todo correcto
`Docs/pruebas/probar_tienda.php`	34 verificaciones de la tienda: carrito sin cuenta, ingreso que vuelve al pago, Luhn, tarjeta vencida, rechazada y aprobada, cobro doble, PayPal, QR simulado (pago realizado y no realizado), QR verificado por el taller, entrega y cancelación con devolución de stock, seguridad del cliente y recuperación de contraseña con el código en pantalla.	Todo correcto
`Docs/informe/resultados_bd.php`	30 consultas, 10 procedimientos y 11 pruebas de reglas contra la base publicada.	0 errores

Nota. Los scripts se ejecutaron en local y contra el sistema publicado durante el desarrollo de los ciclos I a III.

Anexo D. Manual de uso

Visitante y cliente

1. Entre a la página principal: verá las piezas y las clases.
2. Abra una pieza y presione “Agregar al carrito”.
3. En el carrito presione “Pagar”. Si no tiene cuenta, créela con su correo; el sistema lo devuelve al carrito.
4. Elija retiro en el taller o envío, escriba su teléfono y confirme: las piezas quedan reservadas 30 minutos.
5. Pague con el QR del taller (en modo demostración, “Pago realizado” simula la respuesta del banco), con tarjeta o con PayPal. Verá su comprobante en “Mis pedidos”.

Alumno

1. Cree su cuenta en “Crea tu cuenta de alumno” (si es menor, con datos del tutor).
2. En “Reservar clase” elija una clase con lugares libres y presione “Reservar mi lugar”.
3. Puede cancelar una reserva hasta 12 horas antes desde su inicio.

Maestra

1. Ingrese con su correo: verá sus clases de la semana.
2. Abra la clase del día, marque “Presente” o “Falta” en cada alumno y guarde.

Administrativo

1. Abra la caja al iniciar el día con el monto inicial.
2. En “Vender” toque las piezas, elija cliente y método de pago, escriba el monto recibido y presione “Cobrar y emitir nota”.
3. Registre compras en “Compras → Nueva compra”; el stock sube solo.
4. Registre consumo o merma desde el kardex de cada material.
5. En “Pedidos web” confirme los pagos por QR y marque los pedidos entregados.
6. Al cerrar, cuente el efectivo y cierre la caja: el sistema muestra la diferencia.

Administrador

1. Cree usuarios y asígneles un rol en “Usuarios”.
2. Ajuste qué puede hacer cada rol en “Roles y permisos”.
3. Registre maestras, talleres y horarios; genere el calendario.
4. Revise la bitácora y anule ventas con motivo cuando corresponda.

Anexo E. Script completo de la base de datos

Se incluyen los siete archivos de la carpeta Bakend/database, copiados tal cual están en el repositorio y en el orden de instalación indicado en el README.

01_esquema.sql

```
-- ======================================================================
-- Taller de Cerámica Mitsü · Sistema de Información
```

```
-- 01_esquema.sql · Estructura de la base de datos (PostgreSQL 17)
-- ------------------------------------------------------------------------
-- Esquema propio "mitsu" (no se usa "public").
-- Normalizado hasta 3FN. Ningún registro con historial se borra:
-- se desactiva (estado ACTIVO / INACTIVO) o se anula con motivo.
--
--   Ciclo I   · P1 Seguridad y configuración
--   Ciclo II  · P2 Clases y reservas
--   Ciclo III · P3 Inventario y compras · P4 Ventas y caja
-- ========================================================================


-- Extensiones: unaccent (comparar sin tildes), pgcrypto (claves de los datos
-- de prueba) y btree_gist (restricción EXCLUDE de horarios sin cruce).
-- Se crean solo si faltan, así el script también lo puede correr el dueño del esquema.
do $$
begin
    if not exists (select 1 from pg_namespace where nspname = 'extensions') then
        create schema extensions;
    end if;
    if not exists (select 1 from pg_extension where extname = 'unaccent') then
        create extension unaccent with schema extensions;
    end if;
    if not exists (select 1 from pg_extension where extname = 'pgcrypto') then
        create extension pgcrypto with schema extensions;
    end if;
    if not exists (select 1 from pg_extension where extname = 'btree_gist') then
        create extension btree_gist with schema extensions;
    end if;
    if not exists (select 1 from pg_namespace where nspname = 'mitsu') then
        create schema mitsu;
    end if;
end $$;

set search_path = mitsu, extensions, public;


-- Nombre normalizado: sin mayúsculas ni tildes ("Cerámica" = "ceramica").
-- Se usa en índices únicos para impedir nombres repetidos.
create or replace function mitsu.normalizar(texto text)
returns text
language sql immutable parallel safe
as $$ select lower(extensions.unaccent('extensions.unaccent'::regdictionary, trim(coalesce(texto, '')))) $$;


-- Fecha y hora de Bolivia (los servidores trabajan en UTC)
create or replace function mitsu.hoy_bo()
returns date
language sql stable
as $$ select (now() at time zone 'America/La_Paz')::date $$;


-- ========================================================================
-- CICLO I · P1 SEGURIDAD Y CONFIGURACIÓN
-- ========================================================================


create table rol (
    id          smallserial primary key,
    nombre      varchar(40)  not null,
    descripcion varchar(200),
    estado      varchar(10)  not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO')),
    creado_en   timestamptz  not null default now()
);
create unique index ux_rol_nombre on rol (normalizar(nombre));


create table permiso (
    id          smallserial primary key,
    codigo      varchar(60)  not null unique check (codigo ~ '^[a-z_]+\.[a-z_]+$'),
    descripcion varchar(150) not null,
    modulo      varchar(40)  not null
);


create table rolpermiso (
```

```
    rol_id     smallint not null references rol (id),
    permiso_id smallint not null references permiso (id),
    primary key (rol_id, permiso_id)
);

create table usuario (
    id             serial primary key,
    rol_id         smallint     not null references rol (id),
    nombre         varchar(60)  not null check (length(trim(nombre)) >= 2),
    apellido       varchar(60)  not null check (length(trim(apellido)) >= 2),
    correo         varchar(120) not null check (correo ~* '^[^@\s]+@[^@\s]+\.[a-z]{2,}$'),
    telefono       varchar(20)  check (telefono ~ '^[0-9+\- ]{7,20}$'),
    clave_hash     varchar(255) not null,
    estado         varchar(10)  not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO')),
    ultimo_acceso  timestamptz,
    creado_en      timestamptz  not null default now(),
    actualizado_en timestamptz  not null default now()
);
create unique index ux_usuario_correo on usuario (lower(correo));
create index ix_usuario_rol on usuario (rol_id);
create index ix_usuario_nombre on usuario (normalizar(nombre || ' ' || apellido));

-- Sesiones PHP: en Vercel cada petición puede caer en otra instancia
create table sesion (
    id          varchar(128) primary key,
    datos       text         not null default '',
    actualizado timestamptz  not null default now()
);
create index ix_sesion_actualizado on sesion (actualizado);

create table bitacora (
    id         bigserial primary key,
    usuario_id integer      references usuario (id),
    accion     varchar(40)  not null,
    modulo     varchar(40)  not null,
    tabla      varchar(40),
    registro   bigint,
    detalle    varchar(500),
    ip         varchar(64),
    fecha      timestamptz  not null default now()
);
create index ix_bitacora_fecha on bitacora (fecha desc);
create index ix_bitacora_login on bitacora (ip, fecha) where accion = 'LOGIN_FALLIDO';

-- Recuperación de contraseña: código de 6 dígitos guardado con hash
create table recuperacion (
    id          serial primary key,
    usuario_id  integer     not null references usuario (id),
    codigo_hash varchar(255) not null,
    expira_en   timestamptz not null,
    intentos    smallint    not null default 0 check (intentos between 0 and 5),
    usado       boolean     not null default false,
    ip          varchar(64),
    creado_en   timestamptz not null default now(),
    check (expira_en > creado_en)
);
create index ix_recuperacion_usuario on recuperacion (usuario_id, creado_en desc);

create table configuracion (
    clave       varchar(40)  primary key check (clave ~ '^[A-Z_]+$'),
    valor       varchar(300) not null,
    descripcion varchar(200) not null
);

create table categoria (
    id     smallserial primary key,
    nombre varchar(60) not null,
    tipo   varchar(10) not null check (tipo in ('PIEZA', 'MATERIAL')),
    estado varchar(10) not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO'))
);
```

```
create unique index ux_categoria_nombre on categoria (tipo, normalizar(nombre));


create table unidad (
    id          smallserial primary key,
    nombre      varchar(30) not null,
    abreviatura varchar(10) not null,
    estado      varchar(10) not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO'))
);
create unique index ux_unidad_nombre on unidad (normalizar(nombre));


create table metodo_pago (
    id     smallserial primary key,
    nombre varchar(30) not null,
    estado varchar(10) not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO'))
);
create unique index ux_metodo_pago_nombre on metodo_pago (normalizar(nombre));


-- ========================================================================
-- CICLO II · P2 CLASES Y RESERVAS
-- ========================================================================


create table maestra (
    id           serial primary key,
    usuario_id   integer      not null unique references usuario (id),
    especialidad varchar(120) not null,
    biografia    varchar(500),
    estado       varchar(10)  not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO'))
);


create table alumno (
    id               serial primary key,
    usuario_id       integer     not null unique references usuario (id),
    ci               varchar(15) check (ci ~ '^[0-9]{5,12}(-[0-9A-Z]{1,3})?$'),
    fecha_nacimiento date        check (fecha_nacimiento between date '1920-01-01' and current_date),
    nivel            varchar(12) not null default 'INICIAL' check (nivel in ('INICIAL', 'INTERMEDIO',
'AVANZADO')),
    estado           varchar(10) not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO'))
);
create unique index ux_alumno_ci on alumno (ci) where ci is not null;


-- Taller = curso que ofrece el estudio (modelado, torno, niños…)
create table taller (
    id             smallserial primary key,
    nombre         varchar(80)   not null,
    descripcion    varchar(500)  not null,
    nivel          varchar(12)   not null check (nivel in ('INICIAL', 'INTERMEDIO', 'AVANZADO', 'TODOS')),
    publico        varchar(10)   not null default 'ADULTOS' check (publico in ('ADULTOS', 'NINOS', 'TODOS')),
    duracion_min   smallint      not null default 120 check (duracion_min between 30 and 300),
    precio_clase   numeric(10,2) not null check (precio_clase > 0),
    precio_mensual numeric(10,2) not null check (precio_mensual > 0),
    estado         varchar(10)   not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO')),
    creado_en      timestamptz   not null default now()
);
create unique index ux_taller_nombre on taller (normalizar(nombre));


-- Horario semanal fijo de un taller con su maestra
create table horario (
    id          serial primary key,
    taller_id   smallint not null references taller (id),
    maestra_id  integer  not null references maestra (id),
    dia_semana  smallint not null check (dia_semana between 1 and 7),   -- 1 = lunes (ISO)
    hora_inicio time     not null,
    hora_fin    time     not null,
    cupo        smallint not null check (cupo between 1 and 30),
    estado      varchar(10) not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO')),
    check (hora_fin > hora_inicio),
    -- Una maestra no puede tener dos horarios activos que se crucen el mismo día
    constraint horario_sin_cruce_maestra exclude using gist (
        maestra_id with =,
```

```
        dia_semana with =,
        tsrange(date '2000-01-01' + hora_inicio, date '2000-01-01' + hora_fin) with &&
    ) where (estado = 'ACTIVO')
);
create index ix_horario_taller on horario (taller_id);


-- Sesión = clase concreta de un horario en una fecha
create table sesion_clase (
    id                serial primary key,
    horario_id        integer     not null references horario (id),
    maestra_id        integer     not null references maestra (id),   -- puede reemplazarse
    fecha             date        not null,
    hora_inicio       time        not null,
    hora_fin          time        not null,
    cupo              smallint    not null check (cupo between 1 and 30),
    estado            varchar(12) not null default 'PROGRAMADA' check (estado in ('PROGRAMADA', 'REALIZADA',
'CANCELADA')),
    motivo_cancelacion varchar(255),
    check (hora_fin > hora_inicio),
    check (estado <> 'CANCELADA' or motivo_cancelacion is not null),
    unique (horario_id, fecha)
);
create index ix_sesion_fecha on sesion_clase (fecha);


create table reserva (
    id                serial primary key,
    sesion_id         integer     not null references sesion_clase (id),
    alumno_id         integer     not null references alumno (id),
    estado            varchar(10) not null default 'RESERVADA' check (estado in ('RESERVADA', 'CANCELADA',
'ASISTIO', 'FALTO')),
    registrado_por    integer     references usuario (id),
    creado_en         timestamptz not null default now(),
    cancelado_en      timestamptz,
    motivo_cancelacion varchar(255),
    asistencia_en     timestamptz,
    check (estado <> 'CANCELADA' or motivo_cancelacion is not null)
);
-- Un alumno no se inscribe dos veces a la misma clase
create unique index ux_reserva_activa on reserva (sesion_id, alumno_id) where estado <> 'CANCELADA';
create index ix_reserva_alumno on reserva (alumno_id);


-- ======================================================================
-- CICLO III · P3 INVENTARIO Y COMPRAS
-- ======================================================================


create table proveedor (
    id        smallserial primary key,
    nombre    varchar(100) not null,
    nit       varchar(20)  check (nit ~ '^[0-9]{5,15}$'),
    contacto  varchar(80),
    telefono  varchar(20)  check (telefono ~ '^[0-9+\- ]{7,20}$'),
    correo    varchar(120) check (correo ~* '^[^@\s]+@[^@\s]+\.[a-z]{2,}$'),
    ciudad    varchar(40)  not null,
    direccion varchar(200),
    estado    varchar(10)  not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO'))
);
create unique index ux_proveedor_nombre on proveedor (normalizar(nombre));
create unique index ux_proveedor_nit on proveedor (nit) where nit is not null;


-- Insumo para producir (arcilla, cuarzo, esmaltes, óxidos…)
create table material (
    id             serial primary key,
    categoria_id   smallint      not null references categoria (id),
    unidad_id      smallint      not null references unidad (id),
    nombre         varchar(80)   not null,
    descripcion    varchar(300),
    stock          numeric(12,2) not null default 0 check (stock >= 0),
    stock_minimo   numeric(12,2) not null default 0 check (stock_minimo >= 0),
    costo_promedio numeric(12,2) not null default 0 check (costo_promedio >= 0),
    estado         varchar(10)   not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO'))
```

```
);
create unique index ux_material_nombre on material (normalizar(nombre));


create sequence compra_numero_seq;
create table compra (
    id              serial primary key,
    numero          varchar(12)   not null unique default 'CP-' ||
lpad(nextval('mitsu.compra_numero_seq')::text, 6, '0'),
    proveedor_id    smallint      not null references proveedor (id),
    usuario_id      integer       not null references usuario (id),
    fecha           date          not null default mitsu.hoy_bo(),
    observacion     varchar(300),
    total           numeric(12,2) not null default 0 check (total >= 0),
    estado          varchar(10)   not null default 'REGISTRADA' check (estado in ('REGISTRADA', 'ANULADA')),
    motivo_anulacion varchar(255),
    creado_en       timestamptz   not null default now(),
    check (estado <> 'ANULADA' or motivo_anulacion is not null)
);
create index ix_compra_fecha on compra (fecha desc);


create table detalle_compra (
    id             serial primary key,
    compra_id      integer       not null references compra (id),
    material_id    integer       not null references material (id),
    cantidad       numeric(12,2) not null check (cantidad > 0),
    costo_unitario numeric(12,2) not null check (costo_unitario >= 0),
    subtotal       numeric(12,2) generated always as (round(cantidad * costo_unitario, 2)) stored,
    unique (compra_id, material_id)
);

-- Kardex de materiales: todo cambio de stock queda registrado aquí
create table movimiento_material (
    id              bigserial primary key,
    material_id     integer       not null references material (id),
    tipo            varchar(16)   not null check (tipo in ('ENTRADA', 'CONSUMO', 'MERMA', 'AJUSTE_MAS',
'AJUSTE_MENOS')),
    cantidad        numeric(12,2) not null check (cantidad > 0),
    stock_resultante numeric(12,2) not null default 0 check (stock_resultante >= 0),
    motivo          varchar(255)  not null,
    compra_id       integer       references compra (id),
    usuario_id      integer       not null references usuario (id),
    fecha           timestamptz   not null default now()
);
create index ix_movmat_material on movimiento_material (material_id, fecha desc);


-- ========================================================================
-- CICLO III · P4 VENTAS Y CAJA
-- ========================================================================


-- Pieza de cerámica terminada que se vende (tazas, vasijas, vajilla…)
create table pieza (
    id           serial primary key,
    codigo       varchar(12)   not null unique check (codigo ~ '^[A-Z]{2,4}-[0-9]{3,6}$'),
    categoria_id smallint      not null references categoria (id),
    nombre       varchar(100)  not null,
    descripcion  varchar(500),
    precio       numeric(10,2) not null check (precio > 0),
    stock        integer       not null default 0 check (stock >= 0),
    stock_minimo integer       not null default 1 check (stock_minimo >= 0),
    imagen       varchar(120),                       -- imagen estática de /assets
    estado       varchar(10)   not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO')),
    creado_en    timestamptz   not null default now()
);
create unique index ux_pieza_nombre on pieza (normalizar(nombre));


-- Foto subida desde el sistema (Vercel no guarda archivos en disco)
create table pieza_imagen (
    pieza_id       integer     primary key references pieza (id),
    tipo_mime      varchar(20) not null check (tipo_mime in ('image/jpeg', 'image/png', 'image/webp')),
    contenido      bytea       not null check (octet_length(contenido) <= 2097152),
```

```
    actualizado_en timestamptz not null default now()
);


create table historial_precio (
    id              bigserial primary key,
    pieza_id        integer       not null references pieza (id),
    precio_anterior numeric(10,2) not null,
    precio_nuevo    numeric(10,2) not null,
    usuario_id      integer       references usuario (id),
    fecha           timestamptz   not null default now()
);


create table cliente (
    id        serial primary key,
    nombre    varchar(120) not null,
    ci_nit    varchar(20)  check (ci_nit ~ '^[0-9]{4,15}(-[0-9A-Z]{1,3})?$'),
    telefono  varchar(20)  check (telefono ~ '^[0-9+\- ]{7,20}$'),
    correo    varchar(120) check (correo ~* '^[^@\s]+@[^@\s]+\.[a-z]{2,}$'),
    estado    varchar(10)  not null default 'ACTIVO' check (estado in ('ACTIVO', 'INACTIVO')),
    creado_en timestamptz  not null default now()
);
create unique index ux_cliente_ci_nit on cliente (ci_nit) where ci_nit is not null;
create index ix_cliente_nombre on cliente (normalizar(nombre));


-- Solo puede haber una caja abierta a la vez en el taller
create table caja (
    id              serial primary key,
    usuario_id      integer       not null references usuario (id),
    abierta_en      timestamptz   not null default now(),
    monto_inicial   numeric(10,2) not null check (monto_inicial >= 0),
    cerrada_en      timestamptz,
    cerrada_por     integer       references usuario (id),
    total_efectivo  numeric(12,2),
    total_otros     numeric(12,2),
    monto_declarado numeric(12,2) check (monto_declarado >= 0),
    diferencia      numeric(12,2),
    estado          varchar(10)   not null default 'ABIERTA' check (estado in ('ABIERTA', 'CERRADA')),
    observacion     varchar(300),
    check (estado = 'ABIERTA' or (cerrada_en is not null and monto_declarado is not null))
);
create unique index ux_caja_unica_abierta on caja (estado) where estado = 'ABIERTA';


create sequence venta_numero_seq;
create table venta (
    id               serial primary key,
    numero           varchar(12)   not null unique default 'NV-' ||
lpad(nextval('mitsu.venta_numero_seq')::text, 6, '0'),
    caja_id          integer       not null references caja (id),
    cliente_id       integer       not null references cliente (id),
    usuario_id       integer       not null references usuario (id),
    metodo_pago_id   smallint      not null references metodo_pago (id),
    fecha            timestamptz   not null default now(),
    total            numeric(12,2) not null default 0 check (total >= 0),
    monto_recibido   numeric(12,2) check (monto_recibido >= 0),
    estado           varchar(10)   not null default 'VALIDA' check (estado in ('VALIDA', 'ANULADA')),
    motivo_anulacion varchar(255),
    anulado_por      integer       references usuario (id),
    anulado_en       timestamptz,
    check (estado <> 'ANULADA' or (motivo_anulacion is not null and anulado_por is not null))
);
create index ix_venta_fecha on venta (fecha desc);
create index ix_venta_caja on venta (caja_id);


create table detalle_venta (
    id              serial primary key,
    venta_id        integer       not null references venta (id),
    pieza_id        integer       not null references pieza (id),
    cantidad        integer       not null check (cantidad > 0),
    precio_unitario numeric(10,2) not null check (precio_unitario > 0),   -- copiado al vender
    subtotal        numeric(12,2) generated always as (cantidad * precio_unitario) stored,
```

```
    unique (venta_id, pieza_id)
);

-- Kardex de piezas
create table movimiento_pieza (
    id               bigserial primary key,
    pieza_id         integer     not null references pieza (id),
    tipo             varchar(16) not null check (tipo in ('PRODUCCION', 'VENTA', 'ANULACION', 'AJUSTE_MAS',
'AJUSTE_MENOS')),
    cantidad         integer     not null check (cantidad > 0),
    stock_resultante integer     not null default 0 check (stock_resultante >= 0),
    motivo           varchar(255) not null,
    venta_id         integer     references venta (id),
    usuario_id       integer     not null references usuario (id),
    fecha            timestamptz not null default now()
);
create index ix_movpieza_pieza on movimiento_pieza (pieza_id, fecha desc);
```

02_reglas_negocio.sql

```
-- ========================================================================
-- Taller de Cerámica Mitsü
-- 02_reglas_negocio.sql · Triggers (última línea de defensa)
-- ------------------------------------------------------------------------
-- PHP ya valida todo, pero estas reglas se cumplen aunque alguien
-- escriba directamente en la base de datos.
-- ========================================================================

set search_path = mitsu, extensions, public;

-- Usuario que hace la operación (PHP lo fija con set_config dentro de la transacción)
create or replace function mitsu.usuario_actual()
returns integer
language sql stable
as $$ select nullif(current_setting('mitsu.usuario_id', true), '')::integer $$;


-- ------------------------------------------------------------------------
-- RN01 · usuario.actualizado_en se mantiene solo
-- ------------------------------------------------------------------------
create or replace function mitsu.tg_tocar_actualizado()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    new.actualizado_en := now();
    return new;
end $$;

create trigger tg_usuario_actualizado
    before update on usuario
    for each row execute function tg_tocar_actualizado();


-- ------------------------------------------------------------------------
-- RN02 · Siempre debe quedar al menos un administrador activo
-- ------------------------------------------------------------------------
create or replace function mitsu.tg_ultimo_admin()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare
    v_admin smallint := (select id from rol where nombre = 'ADMINISTRADOR');
begin
    if old.rol_id = v_admin and old.estado = 'ACTIVO'
       and (new.estado <> 'ACTIVO' or new.rol_id <> v_admin)
       and not exists (select 1 from usuario where rol_id = v_admin and estado = 'ACTIVO' and id <> old.id)
then
        raise exception 'No se puede desactivar ni cambiar de rol al único administrador activo.';
    end if;
    return new;
```

```
end $$;


create trigger tg_usuario_ultimo_admin
    before update of estado, rol_id on usuario
    for each row execute function tg_ultimo_admin();


-- ---------------------------------------------------------------------
-- RN03 · Un horario solo se asigna a una maestra y un taller activos
-- ---------------------------------------------------------------------
create or replace function mitsu.tg_horario_valido()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    if new.estado = 'ACTIVO' then
        if not exists (select 1 from maestra m join usuario u on u.id = m.usuario_id
                       where m.id = new.maestra_id and m.estado = 'ACTIVO' and u.estado = 'ACTIVO') then
            raise exception 'La maestra seleccionada no está activa.';
        end if;
        if not exists (select 1 from taller where id = new.taller_id and estado = 'ACTIVO') then
            raise exception 'El taller seleccionado no está activo.';
        end if;
    end if;
    return new;
end $$;


create trigger tg_horario_valido
    before insert or update on horario
    for each row execute function tg_horario_valido();


-- ---------------------------------------------------------------------
-- RN04 · Reservas: con cupo, sin repetir, solo clases futuras y programadas
-- ---------------------------------------------------------------------
create or replace function mitsu.tg_reserva_validar()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare
    s       sesion_clase%rowtype;
    ocupado integer;
begin
    -- Se bloquea la fila de la sesión: dos reservas simultáneas no pueden pasar el cupo
    select * into s from sesion_clase where id = new.sesion_id for update;


    if tg_op = 'INSERT' then
        if s.estado <> 'PROGRAMADA' then
            raise exception 'La clase no está disponible para reservas.';
        end if;
        if s.fecha < hoy_bo() or (s.fecha = hoy_bo() and s.hora_inicio <= (now() at time zone
'America/La_Paz')::time) then
            raise exception 'No se puede reservar una clase que ya empezó o pasó.';
        end if;
        if not exists (select 1 from alumno a join usuario u on u.id = a.usuario_id
                       where a.id = new.alumno_id and a.estado = 'ACTIVO' and u.estado = 'ACTIVO') then
            raise exception 'El alumno no está activo.';
        end if;
        if exists (select 1 from reserva where sesion_id = new.sesion_id and alumno_id = new.alumno_id and
estado <> 'CANCELADA') then
            raise exception 'El alumno ya tiene una reserva en esta clase.';
        end if;
        select count(*) into ocupado from reserva where sesion_id = new.sesion_id and estado <> 'CANCELADA';
        if ocupado >= s.cupo then
            raise exception 'La clase ya no tiene cupos disponibles.';
        end if;
        new.estado := 'RESERVADA';
        return new;
    end if;


    -- UPDATE: transiciones permitidas
    if old.estado = new.estado then
```

```
        return new;
    end if;
    if old.estado = 'CANCELADA' then
        raise exception 'Una reserva cancelada no se puede modificar.';
    end if;
    if new.estado = 'CANCELADA' then
        if old.estado <> 'RESERVADA' then
            raise exception 'Solo se cancelan reservas pendientes, no las que ya tienen asistencia.';
        end if;
        new.cancelado_en := now();
    elsif new.estado in ('ASISTIO', 'FALTO') then
        if s.fecha > hoy_bo() then
            raise exception 'La asistencia se registra el día de la clase o después.';
        end if;
        if s.estado = 'CANCELADA' then
            raise exception 'La clase fue cancelada.';
        end if;
        new.asistencia_en := now();
    elsif new.estado = 'RESERVADA' then
        raise exception 'No se puede volver la reserva a pendiente.';
    end if;
    return new;
end $$;

create trigger tg_reserva_validar
    before insert or update on reserva
    for each row execute function tg_reserva_validar();

-- ------------------------------------------------------------------------
-- RN05 · Si se cancela una clase, se cancelan sus reservas pendientes
-- ------------------------------------------------------------------------
create or replace function mitsu.tg_sesion_cancelada()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    if new.estado = 'CANCELADA' and old.estado <> 'CANCELADA' then
        update reserva
           set estado = 'CANCELADA', motivo_cancelacion = 'Clase cancelada: ' || new.motivo_cancelacion
         where sesion_id = new.id and estado = 'RESERVADA';
    end if;
    if new.cupo < (select count(*) from reserva where sesion_id = new.id and estado <> 'CANCELADA') then
        raise exception 'El cupo no puede ser menor que las reservas ya hechas.';
    end if;
    return new;
end $$;

create trigger tg_sesion_cancelada
    after update on sesion_clase
    for each row execute function tg_sesion_cancelada();

-- ------------------------------------------------------------------------
-- RN06 · Kardex de materiales: actualiza el stock y nunca lo deja negativo
-- ------------------------------------------------------------------------
create or replace function mitsu.tg_movimiento_material()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare
    v_stock numeric(12,2);
    v_nombre varchar;
begin
    select stock, nombre into v_stock, v_nombre from material where id = new.material_id for update;
    if new.tipo in ('ENTRADA', 'AJUSTE_MAS') then
        v_stock := v_stock + new.cantidad;
    else
        v_stock := v_stock - new.cantidad;
        if v_stock < 0 then
            raise exception 'Stock insuficiente de "%": hay % y se pidió %.', v_nombre, v_stock +
new.cantidad, new.cantidad;
        end if;
```

```
    end if;
    update material set stock = v_stock where id = new.material_id;
    new.stock_resultante := v_stock;
    return new;
end $$;


create trigger tg_movimiento_material
    before insert on movimiento_material
    for each row execute function tg_movimiento_material();


-- ---------------------------------------------------------------------
-- RN07 · Kardex de piezas: actualiza el stock y nunca lo deja negativo
-- ---------------------------------------------------------------------
create or replace function mitsu.tg_movimiento_pieza()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare
    v_stock integer;
    v_nombre varchar;
begin
    select stock, nombre into v_stock, v_nombre from pieza where id = new.pieza_id for update;
    if new.tipo in ('PRODUCCION', 'ANULACION', 'AJUSTE_MAS') then
        v_stock := v_stock + new.cantidad;
    else
        v_stock := v_stock - new.cantidad;
        if v_stock < 0 then
            raise exception 'Stock insuficiente de "%": quedan % unidades.', v_nombre, v_stock + new.cantidad;
        end if;
    end if;
    update pieza set stock = v_stock where id = new.pieza_id;
    new.stock_resultante := v_stock;
    return new;
end $$;


create trigger tg_movimiento_pieza
    before insert on movimiento_pieza
    for each row execute function tg_movimiento_pieza();


-- El stock solo cambia mediante movimientos (no se edita a mano)
create or replace function mitsu.tg_stock_protegido()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    if new.stock <> old.stock and pg_trigger_depth() = 1 then
        raise exception 'El stock solo cambia con compras, ventas o ajustes registrados.';
    end if;
    return new;
end $$;


create trigger tg_material_stock_protegido before update of stock on material
    for each row execute function tg_stock_protegido();
create trigger tg_pieza_stock_protegido before update of stock on pieza
    for each row execute function tg_stock_protegido();


-- ---------------------------------------------------------------------
-- RN08 · El precio de venta se copia de la pieza y el total se calcula
-- ---------------------------------------------------------------------
create or replace function mitsu.tg_detalle_venta_precio()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare
    p pieza%rowtype;
begin
    select * into p from pieza where id = new.pieza_id;
    if p.estado <> 'ACTIVO' then
        raise exception 'La pieza "%" no está a la venta.', p.nombre;
    end if;
```

```
    new.precio_unitario := p.precio;
    return new;
end $$;


create trigger tg_detalle_venta_precio
    before insert on detalle_venta
    for each row execute function tg_detalle_venta_precio();


create or replace function mitsu.tg_total_venta()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    update venta set total = (select coalesce(sum(subtotal), 0) from detalle_venta where venta_id =
new.venta_id)
     where id = new.venta_id;
    return null;
end $$;


create trigger tg_total_venta
    after insert on detalle_venta
    for each row execute function tg_total_venta();


create or replace function mitsu.tg_total_compra()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    update compra set total = (select coalesce(sum(subtotal), 0) from detalle_compra where compra_id =
new.compra_id)
     where id = new.compra_id;
    return null;
end $$;


create trigger tg_total_compra
    after insert on detalle_compra
    for each row execute function tg_total_compra();


-- ------------------------------------------------------------------------
-- RN09 · Ventas: solo con caja abierta; una venta anulada no se modifica
-- ------------------------------------------------------------------------
create or replace function mitsu.tg_venta_reglas()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    if tg_op = 'INSERT' then
        if not exists (select 1 from caja where id = new.caja_id and estado = 'ABIERTA') then
            raise exception 'No hay una caja abierta. Abra la caja antes de vender.';
        end if;
        return new;
    end if;
    if old.estado = 'ANULADA' then
        raise exception 'La venta % ya está anulada y no se puede modificar.', old.numero;
    end if;
    if new.estado = 'ANULADA' then
        new.anulado_en := now();
    end if;
    return new;
end $$;


create trigger tg_venta_reglas
    before insert or update on venta
    for each row execute function tg_venta_reglas();


-- ------------------------------------------------------------------------
-- RN10 · Historial de precios de las piezas
-- ------------------------------------------------------------------------
create or replace function mitsu.tg_historial_precio()
```

```
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    if new.precio <> old.precio then
        insert into historial_precio (pieza_id, precio_anterior, precio_nuevo, usuario_id)
        values (new.id, old.precio, new.precio, usuario_actual());
    end if;
    return new;
end $$;

create trigger tg_historial_precio
    after update of precio on pieza
    for each row execute function tg_historial_precio();
```

03_datos_iniciales.sql

```
-- ======================================================================
-- Taller de Cerámica Mitsü
-- 03_datos_iniciales.sql · Catálogos, permisos y DATOS DE DEMOSTRACIÓN
-- ----------------------------------------------------------------------
-- Los horarios salen de la entrevista a la gerencia. Los precios, stocks,
-- proveedores, alumnos y ventas son ESTIMADOS / FICTICIOS para demostrar
-- el sistema. Las fechas se calculan desde HOY, así siempre hay datos
-- de "hoy", de días pasados y de días futuros.
--
-- Cuentas de prueba (contraseñas inventadas, solo para la demostración):
--   admin@mitsu.test    Admin2026!     Administrador (gerencia)
--   ventas@mitsu.test   Ventas2026!    Personal administrativo
--   maestra@mitsu.test  Maestra2026!   Maestra
--   alumno@mitsu.test   Alumno2026!    Alumno
-- ======================================================================

set search_path = mitsu, extensions, public;

-- ----------------------------------------------------------------------
-- Roles y permisos
-- ----------------------------------------------------------------------
insert into rol (nombre, descripcion) values
    ('ADMINISTRADOR',  'Gerencia del taller: acceso completo y configuración'),
    ('ADMINISTRATIVO', 'Atención al cliente, ventas, caja, inventario y reservas'),
    ('MAESTRA',        'Imparte talleres, ve sus clases y registra asistencia'),
    ('ALUMNO',         'Reserva clases desde su portal');

insert into permiso (codigo, descripcion, modulo) values
    ('panel.ver',              'Ver el panel de indicadores',                'P1 Seguridad'),
    ('usuarios.gestionar',     'Registrar, editar y desactivar usuarios',    'P1 Seguridad'),
    ('roles.gestionar',        'Crear roles y asignar permisos',             'P1 Seguridad'),
    ('bitacora.ver',           'Consultar la bitácora del sistema',          'P1 Seguridad'),
    ('configuracion.gestionar','Editar datos del taller y catálogos',        'P1 Seguridad'),
    ('maestras.gestionar',     'Registrar y editar maestras',                'P2 Clases'),
    ('talleres.gestionar',     'Registrar talleres, precios y horarios',     'P2 Clases'),
    ('sesiones.gestionar',     'Generar, cancelar y ajustar clases del calendario', 'P2 Clases'),
    ('alumnos.gestionar',      'Registrar y editar alumnos',                 'P2 Clases'),
    ('reservas.gestionar',     'Reservar o cancelar clases para un alumno',  'P2 Clases'),
    ('asistencia.registrar',   'Registrar asistencia de cualquier clase',    'P2 Clases'),
    ('clases.propias',         'Ver sus clases y registrar su asistencia',   'P2 Clases'),
    ('portal.alumno',          'Usar el portal del alumno',                  'P2 Clases'),
    ('materiales.gestionar',   'Registrar materiales y su consumo o merma',  'P3 Inventario'),
    ('proveedores.gestionar',  'Registrar proveedores',                      'P3 Inventario'),
    ('compras.gestionar',      'Registrar y anular compras',                 'P3 Inventario'),
    ('piezas.gestionar',       'Registrar piezas, fotos, precios y producción','P4 Ventas'),
    ('clientes.gestionar',     'Registrar clientes',                         'P4 Ventas'),
    ('ventas.registrar',       'Vender en caja y emitir nota de venta',      'P4 Ventas'),
    ('ventas.anular',          'Anular ventas con motivo',                   'P4 Ventas'),
    ('caja.gestionar',         'Abrir y cerrar caja',                        'P4 Ventas');

insert into rolpermiso (rol_id, permiso_id)
select r.id, p.id from rol r cross join permiso p
```

```
 where (r.nombre = 'ADMINISTRADOR' and p.codigo not in ('portal.alumno', 'clases.propias'))
    or (r.nombre = 'ADMINISTRATIVO' and p.codigo in ('panel.ver', 'alumnos.gestionar', 'reservas.gestionar',
            'asistencia.registrar', 'sesiones.gestionar', 'materiales.gestionar', 'proveedores.gestionar',
            'compras.gestionar', 'piezas.gestionar', 'clientes.gestionar', 'ventas.registrar',
'caja.gestionar'))
    or (r.nombre = 'MAESTRA' and p.codigo in ('panel.ver', 'clases.propias'))
    or (r.nombre = 'ALUMNO' and p.codigo = 'portal.alumno');


-- ------------------------------------------------------------------
-- Configuración y catálogos
-- ------------------------------------------------------------------
insert into configuracion (clave, valor, descripcion) values
    ('NOMBRE_TALLER',      'Taller de Cerámica Mitsü',            'Nombre que aparece en el sistema y en las
notas de venta'),
    ('CIUDAD',             'Santa Cruz de la Sierra, Bolivia',    'Ciudad del taller'),
    ('TELEFONO',           '70000000',                            'Teléfono de contacto (dato de
demostración)'),
    ('CORREO',             'contacto@mitsu.test',                 'Correo de contacto (dato de
demostración)'),
    ('HORAS_CANCELACION',  '12',                                  'Horas mínimas de anticipación para que el
alumno cancele una reserva'),
    ('SEMANAS_CALENDARIO', '4',                                   'Semanas hacia adelante que se generan
clases en el calendario');


insert into unidad (nombre, abreviatura) values
    ('Kilogramo', 'kg'), ('Litro', 'L'), ('Unidad', 'u'), ('Gramo', 'g');


insert into metodo_pago (nombre) values ('Efectivo'), ('QR'), ('Transferencia');


insert into categoria (nombre, tipo) values
    ('Arcillas', 'MATERIAL'), ('Minerales', 'MATERIAL'), ('Esmaltes y engobes', 'MATERIAL'),
    ('Óxidos y pigmentos', 'MATERIAL'), ('Herramientas y consumibles', 'MATERIAL'),
    ('Tazas y vasos', 'PIEZA'), ('Vasijas y jarras', 'PIEZA'), ('Vajilla', 'PIEZA'), ('Decorativas', 'PIEZA');


-- ------------------------------------------------------------------
-- Usuarios (contraseñas con bcrypt, igual que password_hash() de PHP)
-- ------------------------------------------------------------------
insert into usuario (rol_id, nombre, apellido, correo, telefono, clave_hash)
select (select id from rol where nombre = u.rol), u.nombre, u.apellido, u.correo, u.telefono,
       crypt(u.clave, gen_salt('bf', 10))
from (values
    ('ADMINISTRADOR',  'Alessandra', 'Civardi',     'admin@mitsu.test',            '70010001', 'Admin2026!'),
    ('ADMINISTRATIVO', 'Valeria',    'Suárez',      'ventas@mitsu.test',           '70010002', 'Ventas2026!'),
    ('MAESTRA',        'Rosario',    'Gil',         'maestra@mitsu.test',          '70010003',
'Maestra2026!'),
    ('MAESTRA',        'Elena',      'Rojas',       'elena.rojas@mitsu.test',      '70010004',
'Maestra2026!'),
    ('MAESTRA',        'Lucía',      'Paz',         'lucia.paz@mitsu.test',        '70010005',
'Maestra2026!'),
    ('MAESTRA',        'Carla',      'Méndez',      'carla.mendez@mitsu.test',     '70010006',
'Maestra2026!'),
    ('ALUMNO',         'Camila',     'Sosa',        'alumno@mitsu.test',           '71020001', 'Alumno2026!'),
    ('ALUMNO',         'Diego',      'Fernández',   'diego.fernandez@mitsu.test',  '71020002', 'Alumno2026!'),
    ('ALUMNO',         'Rosa',       'Mamani',      'rosa.mamani@mitsu.test',      '71020003', 'Alumno2026!'),
    ('ALUMNO',         'Luis',       'Peña',        'luis.pena@mitsu.test',        '71020004', 'Alumno2026!'),
    ('ALUMNO',         'Andrea',     'Vaca',        'andrea.vaca@mitsu.test',      '71020005', 'Alumno2026!'),
    ('ALUMNO',         'Mateo',      'Justiniano',  'mateo.justiniano@mitsu.test', '71020006', 'Alumno2026!'),
    ('ALUMNO',         'Sofía',      'Arteaga',     'sofia.arteaga@mitsu.test',    '71020007', 'Alumno2026!'),
    ('ALUMNO',         'Gabriel',    'Roca',        'gabriel.roca@mitsu.test',     '71020008', 'Alumno2026!'),
    ('ALUMNO',         'Valentina',  'Cuéllar',     'valentina.cuellar@mitsu.test','71020009', 'Alumno2026!'),
    ('ALUMNO',         'Joaquín',    'Salvatierra', 'joaquin.salvatierra@mitsu.test','71020010','Alumno2026!')
) as u (rol, nombre, apellido, correo, telefono, clave);


insert into maestra (usuario_id, especialidad, biografia)
select u.id, m.especialidad, m.bio from usuario u join (values
    ('maestra@mitsu.test',        'Modelado a mano',             'Técnicas de pellizco, churro y placa.'),
    ('elena.rojas@mitsu.test',    'Modelado avanzado y esmaltes','Esmaltado de alta temperatura y diseño de
superficies.'),
    ('lucia.paz@mitsu.test',      'Talleres y cerámica infantil','Talleres nocturnos y clases para niños.'),
```

```
    ('carla.mendez@mitsu.test',   'Torno alfarero',              'Centrado, levantado y retorneado en torno
eléctrico.')
) as m (correo, especialidad, bio) on u.correo = m.correo;


insert into alumno (usuario_id, ci, fecha_nacimiento, nivel)
select u.id, a.ci, a.nac::date, a.nivel from usuario u join (values
    ('alumno@mitsu.test',             '8123451', '1998-04-12', 'INTERMEDIO'),
    ('diego.fernandez@mitsu.test',    '8123452', '1995-09-30', 'INICIAL'),
    ('rosa.mamani@mitsu.test',        '8123453', '2001-01-21', 'AVANZADO'),
    ('luis.pena@mitsu.test',          '8123454', '1990-06-02', 'INICIAL'),
    ('andrea.vaca@mitsu.test',        '8123455', '2003-11-17', 'INTERMEDIO'),
    ('mateo.justiniano@mitsu.test',   '8123456', '2015-03-08', 'INICIAL'),
    ('sofia.arteaga@mitsu.test',      '8123457', '2014-08-25', 'INICIAL'),
    ('gabriel.roca@mitsu.test',       '8123458', '1987-12-01', 'AVANZADO'),
    ('valentina.cuellar@mitsu.test',  '8123459', '1999-02-14', 'INTERMEDIO'),
    ('joaquin.salvatierra@mitsu.test','8123460', '1993-07-19', 'INICIAL')
) as a (correo, ci, nac, nivel) on u.correo = a.correo;


-- ---------------------------------------------------------------------
-- Talleres y horarios (horarios según la entrevista; precios estimados)
-- ---------------------------------------------------------------------
insert into taller (nombre, descripcion, nivel, publico, precio_clase, precio_mensual) values
    ('Modelado a mano',        'Pellizco, churro y placa para crear piezas utilitarias desde cero.',
'INICIAL',    'ADULTOS', 90,  320),
    ('Modelado avanzado',      'Piezas de mayor tamaño, tapas, asas y acabados con engobes.',
'AVANZADO',   'ADULTOS', 110, 400),
    ('Torno alfarero',         'Centrado, levantado de paredes y retorneado en torno eléctrico.',
'INTERMEDIO', 'ADULTOS', 120, 420),
    ('Taller de modelado nocturno', 'Taller libre después del trabajo, con guía de la maestra.',
'TODOS',      'ADULTOS', 90,  320),
    ('Taller de sábado para adultos', 'Encuentro de fin de semana para crear y esmaltar piezas propias.',
'TODOS',      'ADULTOS', 100, 360),
    ('Cerámica para niños',    'Juego con arcilla para niños de 6 a 12 años: figuras, cuencos y texturas.',
'INICIAL',    'NINOS',   70,  250),
    ('Modelado y torno',       'Combina mano y torno para proyectos personales.',
'INTERMEDIO', 'ADULTOS', 120, 420);


insert into horario (taller_id, maestra_id, dia_semana, hora_inicio, hora_fin, cupo)
select t.id, m.id, h.dia, h.ini::time, h.fin::time, h.cupo
from (values
    ('Modelado a mano',               'maestra@mitsu.test',      2, '10:00', '12:00', 8),
    ('Modelado avanzado',             'elena.rojas@mitsu.test',  2, '10:00', '12:00', 6),
    ('Taller de modelado nocturno',   'lucia.paz@mitsu.test',    2, '18:00', '20:00', 8),
    ('Modelado a mano',               'maestra@mitsu.test',      3, '10:00', '12:00', 8),
    ('Torno alfarero',                'carla.mendez@mitsu.test', 3, '10:00', '12:00', 6),
    ('Taller de modelado nocturno',   'lucia.paz@mitsu.test',    3, '18:00', '20:00', 8),
    ('Modelado a mano',               'maestra@mitsu.test',      4, '10:00', '12:00', 8),
    ('Torno alfarero',                'carla.mendez@mitsu.test', 4, '10:00', '12:00', 6),
    ('Modelado a mano',               'elena.rojas@mitsu.test',  5, '10:00', '12:00', 8),
    ('Torno alfarero',                'carla.mendez@mitsu.test', 5, '10:00', '12:00', 6),
    ('Taller de sábado para adultos', 'elena.rojas@mitsu.test',  6, '10:00', '12:00', 10),
    ('Cerámica para niños',           'lucia.paz@mitsu.test',    6, '16:00', '18:00', 10),
    ('Modelado y torno',              'carla.mendez@mitsu.test', 6, '18:00', '20:00', 6)
) as h (taller, correo, dia, ini, fin, cupo)
join taller t on t.nombre = h.taller
join usuario u on u.correo = h.correo
join maestra m on m.usuario_id = u.id;


-- Clases del calendario: desde hace 3 semanas hasta dentro de 3 semanas
insert into sesion_clase (horario_id, maestra_id, fecha, hora_inicio, hora_fin, cupo, estado)
select h.id, h.maestra_id, d::date, h.hora_inicio, h.hora_fin, h.cupo,
       case when d::date < hoy_bo() then 'REALIZADA' else 'PROGRAMADA' end
from horario h
cross join generate_series(hoy_bo() - 21, hoy_bo() + 21, interval '1 day') d
where extract(isodow from d) = h.dia_semana;


-- Una clase futura cancelada (para mostrar el caso)
update sesion_clase set estado = 'CANCELADA', motivo_cancelacion = 'Mantenimiento del horno (dato de
demostración)'
 where id = (select id from sesion_clase where fecha > hoy_bo() + 7 order by fecha, hora_inicio limit 1);
```

```
-- Reservas: las clases pasadas ya tienen asistencia registrada. Este trigger impide
-- reservar fechas pasadas, por eso se desactiva solo mientras se cargan los datos históricos.
alter table reserva disable trigger tg_reserva_validar;

insert into reserva (sesion_id, alumno_id, estado, registrado_por, creado_en, asistencia_en)
select x.sesion_id, x.alumno_id,
       case when x.fecha >= hoy_bo() then 'RESERVADA'
            when (abs(hashtext(x.sesion_id || 'a' || x.alumno_id)) % 6) = 0 then 'FALTO'
            else 'ASISTIO' end,
       x.usuario_id,
       (x.fecha - 3)::timestamp + time '15:00',
       case when x.fecha < hoy_bo() then x.fecha::timestamp + x.hora_fin end
from (
    select s.id as sesion_id, a.id as alumno_id, s.fecha, s.hora_fin, a.usuario_id, s.cupo,
           row_number() over (partition by s.id order by abs(hashtext(s.id || '-' || a.id))) as n
    from sesion_clase s
    join horario h on h.id = s.horario_id
    join taller t on t.id = h.taller_id
    cross join alumno a
    where s.estado <> 'CANCELADA'
      and (t.publico = 'NINOS') = (a.fecha_nacimiento > date '2012-01-01')   -- los niños van al taller
infantil
      and (abs(hashtext(a.id || '/' || s.id)) % 10) < 4
) x
where x.n < x.cupo;   -- siempre queda al menos un cupo libre

alter table reserva enable trigger tg_reserva_validar;

-- ---------------------------------------------------------------------
-- Inventario: proveedores, materiales y compras (datos estimados)
-- ---------------------------------------------------------------------
insert into proveedor (nombre, nit, contacto, telefono, correo, ciudad, direccion) values
    ('Minera Andina Cochabamba (demo)',      '1023456011', 'Jorge Quiroga', '44250001', 'ventas@andina.test',
'Cochabamba', 'Zona industrial, km 5'),
    ('Minerales Illimani S.R.L. (demo)',     '2034567012', 'Marta Choque',  '22450002',
'pedidos@illimani.test','La Paz',     'Av. Montes 100'),
    ('Insumos Cerámicos del Oriente (demo)', '3045678013', 'Pablo Rivero',  '33450003', 'info@oriente.test',
'Santa Cruz', '4to anillo, calle 7');

insert into material (categoria_id, unidad_id, nombre, descripcion, stock_minimo)
select c.id, u.id, m.nombre, m.descr, m.minimo
from (values
    ('Arcillas',                  'kg', 'Arcilla roja',          'Arcilla de baja temperatura para modelado',
40),
    ('Arcillas',                  'kg', 'Arcilla gres blanca',   'Gres de alta temperatura para torno',
40),
    ('Arcillas',                  'kg', 'Caolín',                'Arcilla blanca para pastas y esmaltes',
10),
    ('Minerales',                 'kg', 'Cuarzo molido',         'Sílice para esmaltes',
10),
    ('Minerales',                 'kg', 'Feldespato potásico',   'Fundente para esmaltes',
10),
    ('Esmaltes y engobes',        'L',  'Esmalte transparente',  'Esmalte brillante de alta temperatura',
5),
    ('Esmaltes y engobes',        'L',  'Esmalte celeste nube',  'Esmalte de efecto escurrido',
4),
    ('Esmaltes y engobes',        'L',  'Engobe blanco',         'Engobe para decorar en cuero',
3),
    ('Óxidos y pigmentos',        'kg', 'Óxido de cobalto',      'Pigmento azul',
2),
    ('Óxidos y pigmentos',        'kg', 'Óxido de hierro',       'Pigmento rojizo',
2),
    ('Herramientas y consumibles', 'u',  'Pinceles de cerámica',  'Pinceles de pelo suave',
12)
) as m (categoria, unidad, nombre, descr, minimo)
join categoria c on c.nombre = m.categoria and c.tipo = 'MATERIAL'
join unidad u on u.abreviatura = m.unidad;

do $$
```

```
declare
    v_admin  integer := (select id from usuario where correo = 'admin@mitsu.test');
    v_ventas integer := (select id from usuario where correo = 'ventas@mitsu.test');
    v_compra integer;
    c record;
    d record;
begin
    for c in
        select * from (values
            (1, 'Minera Andina Cochabamba (demo)',      28, 'Pedido mensual de arcillas'),
            (2, 'Minerales Illimani S.R.L. (demo)',     25, 'Minerales para esmaltes'),
            (3, 'Insumos Cerámicos del Oriente (demo)', 18, 'Esmaltes y pinceles'),
            (4, 'Minera Andina Cochabamba (demo)',      6,  'Reposición de arcilla gres')
        ) as t (n, proveedor, dias, obs)
    loop
        insert into compra (proveedor_id, usuario_id, fecha, observacion, creado_en)
        values ((select id from proveedor where nombre = c.proveedor), v_ventas, hoy_bo() - c.dias, c.obs,
                (hoy_bo() - c.dias)::timestamp + time '10:30')
        returning id into v_compra;


        for d in
            select m.id, x.cant, x.costo from (values
                (1, 'Arcilla roja', 150, 6.50), (1, 'Arcilla gres blanca', 120, 9.00), (1, 'Caolín', 30,
8.00),
                (2, 'Cuarzo molido', 40, 7.00), (2, 'Feldespato potásico', 30, 7.50),
                (2, 'Óxido de cobalto', 3, 180.00), (2, 'Óxido de hierro', 4, 45.00),
                (3, 'Esmalte transparente', 12, 95.00), (3, 'Esmalte celeste nube', 6, 120.00),
                (3, 'Engobe blanco', 5, 60.00), (3, 'Pinceles de cerámica', 20, 15.00),
                (4, 'Arcilla gres blanca', 80, 9.20)
            ) as x (n, material, cant, costo)
            join material m on m.nombre = x.material
            where x.n = c.n
        loop
            insert into detalle_compra (compra_id, material_id, cantidad, costo_unitario) values (v_compra,
d.id, d.cant, d.costo);
            insert into movimiento_material (material_id, tipo, cantidad, motivo, compra_id, usuario_id,
fecha)
            values (d.id, 'ENTRADA', d.cant, 'Compra ' || (select numero from compra where id = v_compra),
v_compra, v_ventas,
                    (hoy_bo() - c.dias)::timestamp + time '10:30');
            update material set costo_promedio = d.costo where id = d.id;
        end loop;
    end loop;


    -- Consumo en clases y mermas: algunos materiales quedan bajo el mínimo (alertas)
    insert into movimiento_material (material_id, tipo, cantidad, motivo, usuario_id, fecha)
    select m.id, x.tipo, x.cant, x.motivo, v_ventas, now() - make_interval(days => x.dias)
    from (values
        ('Arcilla roja',          'CONSUMO', 95,  'Consumo en clases de modelado', 10),
        ('Arcilla roja',          'CONSUMO', 30,  'Consumo en clases de modelado', 2),
        ('Arcilla gres blanca',   'CONSUMO', 110, 'Consumo en clases de torno',    9),
        ('Cuarzo molido',         'CONSUMO', 25,  'Preparación de esmaltes',       12),
        ('Esmalte transparente',  'CONSUMO', 9,   'Esmaltado de piezas del mes',   5),
        ('Óxido de cobalto',      'CONSUMO', 2,   'Decoración de vajilla',         4),
        ('Pinceles de cerámica',  'MERMA',   11,  'Pinceles dañados',              3)
    ) as x (material, tipo, cant, motivo, dias)
    join material m on m.nombre = x.material;
end $$;


-- ---------------------------------------------------------------------
-- Piezas a la venta (fotos reales del taller; precios estimados)
-- ---------------------------------------------------------------------
insert into pieza (codigo, categoria_id, nombre, descripcion, precio, stock_minimo, imagen)
select p.codigo, c.id, p.nombre, p.descr, p.precio, p.minimo, p.imagen
from (values
    ('TZ-001', 'Tazas y vasos',    'Taza esmaltada Nube',          'Taza con relieve tallado y esmalte celeste
escurrido.',           95,  3, 'assets/img/piezas/taza-celeste.jpg'),
    ('TZ-002', 'Tazas y vasos',    'Vaso Mejillas',                'Vaso de gres con rostro, mejillas rojas y
hoja verde.',           85,  3, 'assets/img/piezas/vaso-rostro.jpg'),
    ('VS-001', 'Vasijas y jarras', 'Vasija Inti',                  'Vasija de dos asas con rostro y sol
pintado a mano.',              320, 1, 'assets/img/piezas/vasija-sol.jpg'),
```

```
    ('VS-002', 'Vasijas y jarras', 'Vasija Pajarito',              'Vasija con rostro, aros y pajarito
esmaltado en el borde.',        290, 2, 'assets/img/piezas/vasija-pajarito.jpg'),
    ('JR-001', 'Vasijas y jarras', 'Jarra Andina',                 'Jarra con rostro y friso de felinos
inspirado en culturas andinas.', 350, 1, 'assets/img/piezas/jarra-andina.jpg'),
    ('VJ-001', 'Vajilla',          'Set café: vaso y bandeja',     'Vaso esmaltado con bandeja ovalada, el
mismo de la cafetería.',    160, 2, 'assets/img/piezas/set-cafe.jpg'),
    ('VJ-002', 'Vajilla',          'Plato llano de gres 28 cm',    'Plato utilitario apto para microondas.',
120, 4, null),
    ('VJ-003', 'Vajilla',          'Bowl esmaltado mediano',       'Bowl de 15 cm para sopas y ensaladas.',
90,  3, null)
) as p (codigo, categoria, nombre, descr, precio, minimo, imagen)
join categoria c on c.nombre = p.categoria and c.tipo = 'PIEZA';


-- Producción inicial (hace 30 días)
insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, usuario_id, fecha)
select p.id, 'PRODUCCION', x.cant, 'Lote de producción inicial',
       (select id from usuario where correo = 'admin@mitsu.test'), now() - interval '30 days'
from (values ('TZ-001', 20), ('TZ-002', 22), ('VS-001', 3), ('VS-002', 2), ('JR-001', 5),
             ('VJ-001', 12), ('VJ-002', 20), ('VJ-003', 2)) as x (codigo, cant)
join pieza p on p.codigo = x.codigo;


insert into cliente (nombre, ci_nit, telefono, correo) values
    ('Cliente general',      null,        null,       null),
    ('María José Antelo',    '6543210',   '72030001', 'mj.antelo@correo.test'),
    ('Café del Parque (demo)','1029384756','33470002', 'compras@cafeparque.test'),
    ('Fernando Rivero',      '7788990',   '72030003', null),
    ('Paola Céspedes',       '5566778',   '72030004', 'paola.c@correo.test'),
    ('Hotel Los Tajibos (demo)', '1234509876', '33420005', null);


-- ---------------------------------------------------------------------
-- Ventas de los últimos 20 días: una caja por día; la de hoy queda abierta
-- ---------------------------------------------------------------------
do $$
declare
    v_ventas  integer := (select id from usuario where correo = 'ventas@mitsu.test');
    v_admin   integer := (select id from usuario where correo = 'admin@mitsu.test');
    v_caja    integer;
    v_venta   integer;
    v_dia     date;
    v_ventas_dia integer;
    v_pieza   record;
    v_cant    integer;
    i         integer;
    j         integer;
begin
    for v_dia in select generate_series(hoy_bo() - 20, hoy_bo(), interval '1 day')::date loop
        continue when extract(isodow from v_dia) in (1, 7);     -- lunes y domingo cerrado

        insert into caja (usuario_id, abierta_en, monto_inicial)
        values (v_ventas, v_dia + time '09:00', 200) returning id into v_caja;

        v_ventas_dia := 1 + abs(hashtext(v_dia::text)) % 3;
        for i in 1 .. v_ventas_dia loop
            insert into venta (caja_id, cliente_id, usuario_id, metodo_pago_id, fecha)
            values (v_caja,
                    (select id from cliente order by abs(hashtext(id || v_dia::text || i)) limit 1),
                    v_ventas,
                    1 + abs(hashtext(v_dia::text || i)) % 3,
                    v_dia + time '10:00' + make_interval(hours => i * 2))
            returning id into v_venta;

            j := 0;
            for v_pieza in
                select id, codigo from pieza
                 where codigo in ('TZ-001', 'TZ-002', 'VJ-001', 'VJ-002', 'JR-001') and stock > 3
                 order by abs(hashtext(id || '/' || v_venta)) limit 1 + abs(hashtext(v_venta::text)) % 2
            loop
                v_cant := case when v_pieza.codigo = 'JR-001' then 1 else 1 + abs(hashtext(v_pieza.id ||
v_dia::text)) % 2 end;
```

```
                insert into detalle_venta (venta_id, pieza_id, cantidad, precio_unitario) values (v_venta,
v_pieza.id, v_cant, 1);
                insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, venta_id, usuario_id, fecha)
                values (v_pieza.id, 'VENTA', v_cant, 'Venta ' || (select numero from venta where id =
v_venta), v_venta, v_ventas,
                        v_dia + time '10:00' + make_interval(hours => i * 2));
                j := j + 1;
            end loop;


            update venta set monto_recibido = ceil(total / 10) * 10 where id = v_venta;
        end loop;


        if v_dia < hoy_bo() then
            update caja c
               set total_efectivo  = coalesce((select sum(total) from venta where caja_id = c.id and estado =
'VALIDA' and metodo_pago_id = 1), 0),
                   total_otros     = coalesce((select sum(total) from venta where caja_id = c.id and estado =
'VALIDA' and metodo_pago_id <> 1), 0),
                   monto_declarado = c.monto_inicial + coalesce((select sum(total) from venta where caja_id =
c.id and estado = 'VALIDA' and metodo_pago_id = 1), 0),
                   diferencia      = 0,
                   cerrada_en      = v_dia + time '20:30',
                   cerrada_por     = v_ventas,
                   estado          = 'CERRADA'
             where id = v_caja;
        end if;
    end loop;


    -- Una venta anulada con motivo (devuelve el stock)
    select id into v_venta from venta where fecha::date = hoy_bo() - 3 order by id limit 1;
    if v_venta is null then
        select id into v_venta from venta where fecha::date < hoy_bo() order by id desc limit 1;
    end if;
    insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, venta_id, usuario_id, fecha)
    select pieza_id, 'ANULACION', cantidad, 'Anulación de venta (demostración)', venta_id, v_admin, now() -
interval '3 days'
      from detalle_venta where venta_id = v_venta;
    update venta set estado = 'ANULADA', motivo_anulacion = 'Pieza con fisura detectada al entregar
(demostración)',
                     anulado_por = v_admin
     where id = v_venta;
end $$;


-- Un cambio de precio para el historial
select set_config('mitsu.usuario_id', (select id::text from usuario where correo = 'admin@mitsu.test'),
false);
update pieza set precio = 95 where codigo = 'TZ-002' and precio = 85;
select set_config('mitsu.usuario_id', '', false);


-- Bitácora inicial
insert into bitacora (usuario_id, accion, modulo, detalle, ip, fecha)
select id, 'INICIAR_SESION', 'SEGURIDAD', 'Inicio de sesión (dato de demostración)', '127.0.0.1', now() -
interval '1 day'
  from usuario where correo in ('admin@mitsu.test', 'ventas@mitsu.test', 'maestra@mitsu.test');
```

04_usuario_app.sql

```
-- ======================================================================
-- Taller de Cerámica Mitsü
-- 04_usuario_app.sql · Usuario de base de datos de MÍNIMO PRIVILEGIO
-- ----------------------------------------------------------------------
-- La aplicación PHP NO se conecta como "postgres": usa "mitsu_app", que
-- solo puede leer/escribir datos y ejecutar procedimientos del esquema
-- mitsu. No puede crear, alterar ni borrar tablas.
--
-- La contraseña NO se guarda en este archivo ni en GitHub. Se ejecuta con:
--   php Bakend/database/ejecutar.php 04_usuario_app.sql  (toma APP_DB_PASSWORD del .env)
-- y la misma clave va en DATABASE_URL (.env local y variables de Vercel).
-- ======================================================================
```

```
-- Crear el rol y fijar su clave requiere privilegio CREATEROLE (usuario postgres).
-- Si este script lo corre el dueño del esquema, esa parte se omite.
do $$
declare
    puede_crear boolean := (select rolcreaterole or rolsuper from pg_roles where rolname = current_user);
begin
    if puede_crear then
        if not exists (select 1 from pg_roles where rolname = 'mitsu_app') then
            create role mitsu_app login noinherit;
        end if;
        execute format('alter role mitsu_app with password %L', :'clave_app');
        -- Zona horaria de Bolivia y esquema por defecto para cada conexión
        alter role mitsu_app set timezone = 'America/La_Paz';
        alter role mitsu_app set search_path = mitsu, extensions, public;
        alter role mitsu_app set statement_timeout = '15s';
    end if;
end $$;


grant usage on schema mitsu to mitsu_app;
grant select, insert, update, delete on all tables in schema mitsu to mitsu_app;
grant usage, select on all sequences in schema mitsu to mitsu_app;
grant execute on all functions in schema mitsu to mitsu_app;
grant execute on all procedures in schema mitsu to mitsu_app;


-- Las tablas que no deben cambiar desde la aplicación quedan de solo lectura
revoke insert, update, delete on permiso, bitacora, historial_precio,
       movimiento_material, movimiento_pieza from mitsu_app;
grant insert on bitacora, movimiento_material, movimiento_pieza, historial_precio to mitsu_app;


-- ---------------------------------------------------------------------
-- RLS (seguridad a nivel de fila) activada en todo el esquema.
-- Las claves públicas de Supabase (anon / authenticated) no tienen
-- permisos ni políticas: no pueden ver nada de este esquema.
-- ---------------------------------------------------------------------
do $$
declare t record;
begin
    for t in select tablename from pg_tables where schemaname = 'mitsu' loop
        execute format('alter table mitsu.%I enable row level security', t.tablename);
        execute format('drop policy if exists app_acceso on mitsu.%I', t.tablename);
        execute format('create policy app_acceso on mitsu.%I for all to mitsu_app using (true) with check
(true)', t.tablename);
    end loop;
end $$;


revoke all on schema mitsu from public;
```

05_consultas.sql

```
-- ======================================================================
-- Taller de Cerámica Mitsü
-- 05_consultas.sql · 30 consultas: 10 simples, 10 múltiples (JOIN) y
-- 10 subconsultas, más ejemplos de actualización de tuplas.
-- Cada consulta empieza con "-- Qnn ·" para que el informe la ubique.
-- ======================================================================


set search_path = mitsu, extensions, public;


-- ---------------------------------------------------------------------
-- Actualización de tuplas (se ejecutan en una transacción que se revierte)
-- ---------------------------------------------------------------------
-- I1 · Insertar un proveedor nuevo
insert into proveedor (nombre, nit, contacto, telefono, ciudad)
values ('Arcillas del Chaco (demo)', '4056789014', 'Rosa Ayala', '46640001', 'Tarija') returning id, nombre,
ciudad;
-- U1 · Subir 5 % el precio de las tazas (queda en historial_precio por trigger)
update pieza set precio = round(precio * 1.05, 2)
```

```
 where categoria_id = (select id from categoria where nombre = 'Tazas y vasos') returning codigo, nombre,
precio;
-- D1 · Borrar sesiones PHP vencidas (la única tabla donde se borra: no tiene historial)
delete from sesion where actualizado < now() - interval '2 hours' returning id;


-- ------------------------------------------------------------------------
-- CONSULTAS SIMPLES (una tabla)
-- ------------------------------------------------------------------------
-- Q01 · Talleres activos ordenados por precio mensual
select nombre, nivel, publico, precio_clase, precio_mensual from taller where estado = 'ACTIVO' order by
precio_mensual desc;
-- Q02 · Materiales en o bajo el stock mínimo (alerta de compra)
select nombre, stock, stock_minimo from material where stock <= stock_minimo and estado = 'ACTIVO' order by
stock;
-- Q03 · Piezas a la venta con menos de 5 unidades
select codigo, nombre, precio, stock from pieza where estado = 'ACTIVO' and stock < 5 order by stock, nombre;
-- Q04 · Clases programadas para hoy
select fecha, hora_inicio, hora_fin, cupo, estado from sesion_clase where fecha = hoy_bo() order by
hora_inicio;
-- Q05 · Proveedores de Cochabamba o La Paz
select nombre, ciudad, telefono from proveedor where ciudad in ('Cochabamba', 'La Paz') order by ciudad;
-- Q06 · Ventas anuladas con su motivo
select numero, fecha::date as fecha, total, motivo_anulacion from venta where estado = 'ANULADA' order by
fecha desc;
-- Q07 · Cantidad de usuarios por estado
select estado, count(*) as usuarios from usuario group by estado;
-- Q08 · Compras registradas en los últimos 30 días
select numero, fecha, total from compra where estado = 'REGISTRADA' and fecha >= hoy_bo() - 30 order by fecha
desc;
-- Q09 · Intentos de inicio de sesión fallidos de la última semana
select ip, count(*) as intentos from bitacora where accion = 'LOGIN_FALLIDO' and fecha > now() - interval '7
days' group by ip;
-- Q10 · Pedidos web por estado
select estado, count(*) as pedidos, coalesce(sum(total), 0) as monto from pedido group by estado order by
estado;


-- ------------------------------------------------------------------------
-- CONSULTAS MÚLTIPLES (JOIN de varias tablas)
-- ------------------------------------------------------------------------
-- Q11 · Horario semanal: taller, maestra, día y hora
select t.nombre as taller, u.nombre || ' ' || u.apellido as maestra, h.dia_semana, h.hora_inicio, h.hora_fin,
h.cupo
  from horario h join taller t on t.id = h.taller_id join maestra m on m.id = h.maestra_id join usuario u on
u.id = m.usuario_id
 where h.estado = 'ACTIVO' order by h.dia_semana, h.hora_inicio;
-- Q12 · Ocupación de las clases de esta semana
select s.fecha, t.nombre as taller, count(r.id) filter (where r.estado <> 'CANCELADA') as reservas, s.cupo
  from sesion_clase s join horario h on h.id = s.horario_id join taller t on t.id = h.taller_id
  left join reserva r on r.sesion_id = s.id
 where s.fecha between date_trunc('week', hoy_bo())::date and date_trunc('week', hoy_bo())::date + 6
 group by s.id, s.fecha, t.nombre, s.cupo order by s.fecha, t.nombre;
-- Q13 · Porcentaje de asistencia por alumno
select u.nombre || ' ' || u.apellido as alumno,
       count(*) filter (where r.estado = 'ASISTIO') as asistio, count(*) filter (where r.estado = 'FALTO') as
falto,
       round(100.0 * count(*) filter (where r.estado = 'ASISTIO') / nullif(count(*) filter (where r.estado in
('ASISTIO', 'FALTO')), 0)) as porcentaje
  from reserva r join alumno a on a.id = r.alumno_id join usuario u on u.id = a.usuario_id
 group by alumno order by porcentaje desc nulls last;
-- Q14 · Detalle de compras con proveedor y material
select c.numero, p.nombre as proveedor, m.nombre as material, d.cantidad, d.costo_unitario, d.subtotal
  from compra c join proveedor p on p.id = c.proveedor_id join detalle_compra d on d.compra_id = c.id join
material m on m.id = d.material_id
 order by c.fecha desc, m.nombre;
-- Q15 · Ventas por método de pago (solo válidas)
select mp.nombre as metodo, count(v.id) as ventas, sum(v.total) as total
  from venta v join metodo_pago mp on mp.id = v.metodo_pago_id where v.estado = 'VALIDA' group by mp.nombre
order by total desc;
-- Q16 · Piezas más vendidas en caja
select p.codigo, p.nombre, sum(d.cantidad) as unidades, sum(d.subtotal) as ingresos
  from detalle_venta d join venta v on v.id = d.venta_id join pieza p on p.id = d.pieza_id
 where v.estado = 'VALIDA' group by p.codigo, p.nombre order by unidades desc;
```

```
-- Q17 · Kardex de un material con el usuario que hizo cada movimiento
select mm.fecha::date as fecha, mm.tipo, mm.cantidad, mm.stock_resultante, u.nombre as usuario
  from movimiento_material mm join usuario u on u.id = mm.usuario_id join material m on m.id = mm.material_id
 where m.nombre = 'Arcilla roja' order by mm.fecha;
-- Q18 · Cierres de caja con su responsable y diferencia del arqueo
select c.abierta_en::date as dia, u.nombre || ' ' || u.apellido as responsable, c.total_efectivo,
c.total_otros, c.diferencia
  from caja c join usuario u on u.id = c.usuario_id where c.estado = 'CERRADA' order by c.abierta_en desc;
-- Q19 · Permisos de cada rol
select r.nombre as rol, count(rp.permiso_id) as permisos, string_agg(p.codigo, ', ' order by p.codigo) as
codigos
  from rol r left join rolpermiso rp on rp.rol_id = r.id left join permiso p on p.id = rp.permiso_id group by
r.nombre order by permisos desc;
-- Q20 · Pedidos web con cliente y método con que se pagaron
select pe.numero, c.nombre as cliente, pe.total, pe.estado, pg.metodo, pg.estado as estado_pago
  from pedido pe join cliente c on c.id = pe.cliente_id left join pago pg on pg.pedido_id = pe.id
 order by pe.fecha desc;


-- ---------------------------------------------------------------------
-- SUBCONSULTAS
-- ---------------------------------------------------------------------
-- Q21 · Piezas con precio mayor al promedio de su categoría
select p.codigo, p.nombre, p.precio from pieza p
 where p.precio > (select avg(x.precio) from pieza x where x.categoria_id = p.categoria_id) order by p.precio
desc;
-- Q22 · Alumnos que nunca faltaron a una clase
select u.nombre || ' ' || u.apellido as alumno from alumno a join usuario u on u.id = a.usuario_id
 where exists (select 1 from reserva r where r.alumno_id = a.id and r.estado = 'ASISTIO')
   and not exists (select 1 from reserva r where r.alumno_id = a.id and r.estado = 'FALTO');
-- Q23 · Talleres con más reservas que el promedio
select t.nombre, (select count(*) from reserva r join sesion_clase s on s.id = r.sesion_id join horario h on
h.id = s.horario_id
                  where h.taller_id = t.id and r.estado <> 'CANCELADA') as reservas
  from taller t
 where (select count(*) from reserva r join sesion_clase s on s.id = r.sesion_id join horario h on h.id =
s.horario_id
         where h.taller_id = t.id and r.estado <> 'CANCELADA')
     > (select avg(n) from (select count(*) as n from reserva r join sesion_clase s on s.id = r.sesion_id join
horario h on h.id = s.horario_id
                            where r.estado <> 'CANCELADA' group by h.taller_id) x)
 order by reservas desc;
-- Q24 · Materiales que nunca se compraron
select nombre from material m where not exists (select 1 from detalle_compra d where d.material_id = m.id);
-- Q25 · Proveedor al que más se le compró
select nombre, (select sum(total) from compra c where c.proveedor_id = p.id and c.estado = 'REGISTRADA') as
comprado from proveedor p
 where p.id = (select proveedor_id from compra where estado = 'REGISTRADA' group by proveedor_id order by
sum(total) desc limit 1);
-- Q26 · Día con mayor venta del último mes
select fecha::date as dia, sum(total) as total from venta where estado = 'VALIDA' group by fecha::date
having sum(total) = (select max(t) from (select sum(total) as t from venta where estado = 'VALIDA' and fecha >
now() - interval '30 days' group by fecha::date) x);
-- Q27 · Clases futuras que todavía tienen cupo libre
select s.fecha, s.hora_inicio, (select t.nombre from horario h join taller t on t.id = h.taller_id where h.id
= s.horario_id) as taller,
       s.cupo - (select count(*) from reserva r where r.sesion_id = s.id and r.estado <> 'CANCELADA') as
libres
  from sesion_clase s where s.fecha > hoy_bo() and s.estado = 'PROGRAMADA'
   and (select count(*) from reserva r where r.sesion_id = s.id and r.estado <> 'CANCELADA') < s.cupo
 order by s.fecha, s.hora_inicio limit 10;
-- Q28 · Maestras con más alumnos atendidos que la maestra promedio
select u.nombre || ' ' || u.apellido as maestra,
       (select count(*) from reserva r join sesion_clase s on s.id = r.sesion_id where s.maestra_id = m.id and
r.estado = 'ASISTIO') as atendidos
  from maestra m join usuario u on u.id = m.usuario_id
 where (select count(*) from reserva r join sesion_clase s on s.id = r.sesion_id where s.maestra_id = m.id and
r.estado = 'ASISTIO')
     >= (select avg(n) from (select count(*) as n from reserva r join sesion_clase s on s.id = r.sesion_id
where r.estado = 'ASISTIO' group by s.maestra_id) x)
 order by atendidos desc;
-- Q29 · Clientes que compraron en caja y también en la tienda en línea
select nombre from cliente c
```

```
 where c.id in (select cliente_id from venta where estado = 'VALIDA') and c.id in (select cliente_id from
pedido where estado in ('PAGADO', 'ENTREGADO'));
-- Q30 · Valor del inventario de materiales frente al total comprado en el mes
select (select sum(stock * costo_promedio) from material) as valor_inventario,
       (select coalesce(sum(total), 0) from compra where estado = 'REGISTRADA' and date_trunc('month', fecha)
= date_trunc('month', hoy_bo())) as comprado_mes;
```

06_procedimientos_triggers.sql

```
-- ========================================================================
-- Taller de Cerámica Mitsü
-- 06_procedimientos_triggers.sql · Procedimientos almacenados (PL/pgSQL)
-- ------------------------------------------------------------------------
-- Cada procedimiento hace una operación completa dentro de una sola
-- transacción: si un paso falla, no queda nada a medias.
-- Las reglas de stock, cupo y caja las vuelven a revisar los triggers
-- de 02_reglas_negocio.sql.
-- ========================================================================

set search_path = mitsu, extensions, public;


-- ------------------------------------------------------------------------
-- PA1 · Registrar una venta en caja (CU18)
-- p_items: [{"pieza_id": 1, "cantidad": 2}, ...]
-- ------------------------------------------------------------------------
create or replace function mitsu.fn_registrar_venta(
    p_usuario integer, p_cliente integer, p_metodo integer, p_recibido numeric, p_items jsonb)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_caja   integer;
    v_venta  integer;
    v_numero varchar;
    v_total  numeric(12,2);
    v_item   record;
    v_pieza  pieza%rowtype;
begin
    if p_items is null or jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0 then
        raise exception 'Agregue al menos una pieza a la venta.';
    end if;


    select id into v_caja from caja where estado = 'ABIERTA' for update;
    if v_caja is null then
        raise exception 'No hay una caja abierta. Abra la caja antes de vender.';
    end if;
    if not exists (select 1 from cliente where id = p_cliente and estado = 'ACTIVO') then
        raise exception 'El cliente no está activo.';
    end if;
    if not exists (select 1 from metodo_pago where id = p_metodo and estado = 'ACTIVO') then
        raise exception 'Método de pago no válido.';
    end if;


    insert into venta (caja_id, cliente_id, usuario_id, metodo_pago_id)
    values (v_caja, p_cliente, p_usuario, p_metodo)
    returning id, numero into v_venta, v_numero;


    -- Se recorren las piezas en orden de id para bloquearlas siempre en el mismo orden
    for v_item in
        select (e->>'pieza_id')::integer as pieza_id, sum((e->>'cantidad')::integer) as cantidad
          from jsonb_array_elements(p_items) e
         group by 1 order by 1
    loop
        if v_item.cantidad is null or v_item.cantidad <= 0 then
            raise exception 'La cantidad debe ser mayor a cero.';
        end if;
        select * into v_pieza from pieza where id = v_item.pieza_id for update;
        if not found then
```

```
            raise exception 'Una de las piezas no existe.';
        end if;
        if v_pieza.stock < v_item.cantidad then
            raise exception 'Stock insuficiente de "%": quedan % unidades.', v_pieza.nombre, v_pieza.stock;
        end if;
        insert into detalle_venta (venta_id, pieza_id, cantidad, precio_unitario)
        values (v_venta, v_pieza.id, v_item.cantidad, v_pieza.precio);
        insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, venta_id, usuario_id)
        values (v_pieza.id, 'VENTA', v_item.cantidad, 'Venta ' || v_numero, v_venta, p_usuario);
    end loop;


    select total into v_total from venta where id = v_venta;
    if p_metodo = (select id from metodo_pago where normalizar(nombre) = 'efectivo')
       and coalesce(p_recibido, 0) < v_total then
        raise exception 'El monto recibido (Bs %) es menor al total (Bs %).', coalesce(p_recibido, 0),
v_total;
    end if;
    update venta set monto_recibido = coalesce(p_recibido, v_total) where id = v_venta;
    return v_venta;
end $$;


-- ------------------------------------------------------------------------
-- PA2 · Anular una venta con motivo: devuelve el stock (CU18)
-- ------------------------------------------------------------------------
create or replace function mitsu.fn_anular_venta(p_venta integer, p_usuario integer, p_motivo text)
returns void
language plpgsql
set search_path = mitsu, public
as $$
declare
    v venta%rowtype;
begin
    select * into v from venta where id = p_venta for update;
    if not found then
        raise exception 'La venta no existe.';
    end if;
    if v.estado = 'ANULADA' then
        raise exception 'La venta % ya estaba anulada.', v.numero;
    end if;
    if length(trim(coalesce(p_motivo, ''))) < 5 then
        raise exception 'Escriba el motivo de la anulación (mínimo 5 caracteres).';
    end if;


    insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, venta_id, usuario_id)
    select pieza_id, 'ANULACION', cantidad, 'Anulación de ' || v.numero, p_venta, p_usuario
      from detalle_venta where venta_id = p_venta order by pieza_id;


    update venta set estado = 'ANULADA', motivo_anulacion = trim(p_motivo), anulado_por = p_usuario
     where id = p_venta;
end $$;


-- ------------------------------------------------------------------------
-- PA3 · Registrar una compra: suma stock y recalcula el costo promedio (CU15)
-- p_items: [{"material_id": 1, "cantidad": 50, "costo": 6.5}, ...]
-- ------------------------------------------------------------------------
create or replace function mitsu.fn_registrar_compra(
    p_usuario integer, p_proveedor integer, p_fecha date, p_observacion text, p_items jsonb)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_compra integer;
    v_numero varchar;
    v_item   record;
    v_mat    material%rowtype;
begin
    if p_items is null or jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0 then
        raise exception 'Agregue al menos un material a la compra.';
    end if;
```

```
    if not exists (select 1 from proveedor where id = p_proveedor and estado = 'ACTIVO') then
        raise exception 'El proveedor no está activo.';
    end if;
    if p_fecha > hoy_bo() or p_fecha < hoy_bo() - 60 then
        raise exception 'La fecha de compra debe estar entre los últimos 60 días y hoy.';
    end if;

    insert into compra (proveedor_id, usuario_id, fecha, observacion)
    values (p_proveedor, p_usuario, p_fecha, nullif(trim(coalesce(p_observacion, '')), ''))
    returning id, numero into v_compra, v_numero;

    for v_item in
        select (e->>'material_id')::integer as material_id,
               (e->>'cantidad')::numeric as cantidad,
               (e->>'costo')::numeric as costo
          from jsonb_array_elements(p_items) e
         order by 1
    loop
        select * into v_mat from material where id = v_item.material_id for update;
        if not found or v_mat.estado <> 'ACTIVO' then
            raise exception 'Uno de los materiales no existe o está inactivo.';
        end if;
        if v_item.cantidad is null or v_item.cantidad <= 0 or v_item.costo is null or v_item.costo < 0 then
            raise exception 'Cantidad y costo de "%" deben ser válidos.', v_mat.nombre;
        end if;

        insert into detalle_compra (compra_id, material_id, cantidad, costo_unitario)
        values (v_compra, v_mat.id, v_item.cantidad, v_item.costo);

        -- Costo promedio ponderado con el stock que había antes
        update material
           set costo_promedio = round((v_mat.stock * v_mat.costo_promedio + v_item.cantidad * v_item.costo)
                                      / (v_mat.stock + v_item.cantidad), 2)
         where id = v_mat.id;

        insert into movimiento_material (material_id, tipo, cantidad, motivo, compra_id, usuario_id)
        values (v_mat.id, 'ENTRADA', v_item.cantidad, 'Compra ' || v_numero, v_compra, p_usuario);
    end loop;
    return v_compra;
end $$;

-- ---------------------------------------------------------------------
-- PA4 · Anular una compra: descuenta lo que había ingresado (CU15)
-- ---------------------------------------------------------------------
create or replace function mitsu.fn_anular_compra(p_compra integer, p_usuario integer, p_motivo text)
returns void
language plpgsql
set search_path = mitsu, public
as $$
declare
    c compra%rowtype;
begin
    select * into c from compra where id = p_compra for update;
    if not found then
        raise exception 'La compra no existe.';
    end if;
    if c.estado = 'ANULADA' then
        raise exception 'La compra % ya estaba anulada.', c.numero;
    end if;
    if length(trim(coalesce(p_motivo, ''))) < 5 then
        raise exception 'Escriba el motivo de la anulación (mínimo 5 caracteres).';
    end if;
    -- Si el material ya se consumió, el trigger del kardex impide dejar stock negativo
    insert into movimiento_material (material_id, tipo, cantidad, motivo, compra_id, usuario_id)
    select material_id, 'AJUSTE_MENOS', cantidad, 'Anulación de ' || c.numero, p_compra, p_usuario
      from detalle_compra where compra_id = p_compra order by material_id;
    update compra set estado = 'ANULADA', motivo_anulacion = trim(p_motivo) where id = p_compra;
end $$;
```

```
-- ------------------------------------------------------------------------
-- PA5 · Reservar una clase (CU10): el trigger valida cupo y fecha
-- ------------------------------------------------------------------------
create or replace function mitsu.fn_reservar_clase(p_sesion integer, p_alumno integer, p_usuario integer)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_id integer;
begin
    insert into reserva (sesion_id, alumno_id, registrado_por)
    values (p_sesion, p_alumno, p_usuario)
    returning id into v_id;
    return v_id;
end $$;


-- ------------------------------------------------------------------------
-- PA6 · Generar las clases del calendario a partir de los horarios (CU09)
-- No duplica: si la clase de ese horario y fecha ya existe, la salta.
-- ------------------------------------------------------------------------
create or replace function mitsu.fn_generar_sesiones(p_desde date, p_hasta date)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_total integer;
begin
    if p_hasta < p_desde then
        raise exception 'La fecha final debe ser posterior a la inicial.';
    end if;
    if p_hasta - p_desde > 92 then
        raise exception 'Genere como máximo tres meses de clases por vez.';
    end if;
    insert into sesion_clase (horario_id, maestra_id, fecha, hora_inicio, hora_fin, cupo)
    select h.id, h.maestra_id, d::date, h.hora_inicio, h.hora_fin, h.cupo
      from horario h
      join taller t on t.id = h.taller_id and t.estado = 'ACTIVO'
     cross join generate_series(greatest(p_desde, hoy_bo()), p_hasta, interval '1 day') d
     where h.estado = 'ACTIVO' and extract(isodow from d) = h.dia_semana
    on conflict (horario_id, fecha) do nothing;
    get diagnostics v_total = row_count;
    return v_total;
end $$;


-- ------------------------------------------------------------------------
-- PA7 · Cerrar la caja con arqueo (CU19) — procedimiento (CALL)
-- ------------------------------------------------------------------------
create or replace procedure mitsu.sp_cerrar_caja(
    p_caja integer, p_usuario integer, p_declarado numeric, p_observacion text)
language plpgsql
set search_path = mitsu, public
as $$
declare
    c          caja%rowtype;
    v_efectivo numeric(12,2);
    v_otros    numeric(12,2);
begin
    select * into c from caja where id = p_caja for update;
    if not found or c.estado <> 'ABIERTA' then
        raise exception 'La caja no está abierta.';
    end if;
    if p_declarado is null or p_declarado < 0 then
        raise exception 'Ingrese el efectivo contado en caja.';
    end if;
    select coalesce(sum(v.total) filter (where normalizar(m.nombre) = 'efectivo'), 0),
           coalesce(sum(v.total) filter (where normalizar(m.nombre) <> 'efectivo'), 0)
      into v_efectivo, v_otros
      from venta v join metodo_pago m on m.id = v.metodo_pago_id
     where v.caja_id = p_caja and v.estado = 'VALIDA';
```

```
    update caja
       set total_efectivo  = v_efectivo,
           total_otros     = v_otros,
           monto_declarado = p_declarado,
           diferencia      = p_declarado - (c.monto_inicial + v_efectivo),
           cerrada_en      = now(),
           cerrada_por     = p_usuario,
           estado          = 'CERRADA',
           observacion     = nullif(trim(coalesce(p_observacion, '')), '')
     where id = p_caja;
end $$;


-- ------------------------------------------------------------------
-- Trigger de auditoría: toda anulación queda en la bitácora
-- aunque se haga directamente en la base de datos
-- ------------------------------------------------------------------
create or replace function mitsu.tg_auditar_anulacion()
returns trigger
language plpgsql
set search_path = mitsu, public
as $$
begin
    if new.estado = 'ANULADA' and old.estado <> 'ANULADA' then
        insert into bitacora (usuario_id, accion, modulo, tabla, registro, detalle)
        values (coalesce(usuario_actual(), (to_jsonb(new) ->> 'anulado_por')::integer),
                'ANULAR', case when tg_table_name = 'venta' then 'VENTAS' else 'INVENTARIO' end,
                tg_table_name, new.id, left(new.numero || ': ' || new.motivo_anulacion, 500));
    end if;
    return null;
end $$;

create trigger tg_auditar_venta  after update of estado on venta  for each row execute function
tg_auditar_anulacion();
create trigger tg_auditar_compra after update of estado on compra for each row execute function
tg_auditar_anulacion();
```

07_tienda_pasarela.sql

```
-- ======================================================================
-- Taller de Cerámica Mitsü
-- 07_tienda_pasarela.sql · Ciclo III (ampliación): tienda en línea,
-- pedidos web y pasarela de pago (QR, tarjeta y PayPal)
-- ------------------------------------------------------------------
-- CU20 Comprar en la tienda en línea · CU21 Pagar en línea ·
-- CU22 Gestionar pedidos en línea
--
-- La pasarela trabaja en MODO DE DEMOSTRACIÓN: no se cobra dinero real.
-- De la tarjeta solo se guardan la marca y los 4 últimos dígitos
-- (nunca el número completo ni el código de seguridad).
-- ======================================================================

set search_path = mitsu, extensions, public;


-- ------------------------------------------------------------------
-- Rol CLIENTE y permisos nuevos
-- ------------------------------------------------------------------
insert into rol (nombre, descripcion)
select 'CLIENTE', 'Compra piezas en la tienda en línea y paga con QR, tarjeta o PayPal'
where not exists (select 1 from rol where nombre = 'CLIENTE');


insert into permiso (codigo, descripcion, modulo) values
    ('tienda.comprar',    'Comprar en la tienda en línea y ver sus pedidos', 'P4 Ventas'),
    ('pedidos.gestionar', 'Confirmar pagos QR, entregar y cancelar pedidos web', 'P4 Ventas')
on conflict (codigo) do nothing;


insert into rolpermiso (rol_id, permiso_id)
select r.id, p.id from rol r join permiso p on
```

```
      (r.nombre in ('CLIENTE', 'ALUMNO') and p.codigo = 'tienda.comprar')
   or (r.nombre in ('ADMINISTRADOR', 'ADMINISTRATIVO') and p.codigo = 'pedidos.gestionar')
on conflict do nothing;


-- El cliente de la tienda tiene cuenta de usuario (los de mostrador no)
alter table cliente add column if not exists usuario_id integer unique references usuario (id);


-- ------------------------------------------------------------------------
-- Pedido web
-- ------------------------------------------------------------------------
create sequence if not exists pedido_numero_seq;
create table if not exists pedido (
    id                  serial primary key,
    numero              varchar(12)   not null unique default 'PW-' ||
lpad(nextval('mitsu.pedido_numero_seq')::text, 6, '0'),
    cliente_id          integer       not null references cliente (id),
    usuario_id          integer       not null references usuario (id),
    fecha               timestamptz   not null default now(),
    expira_en           timestamptz   not null default now() + interval '30 minutes',
    entrega             varchar(10)   not null default 'RETIRO' check (entrega in ('RETIRO', 'ENVIO')),
    direccion           varchar(200),
    telefono            varchar(20)   not null check (telefono ~ '^[0-9+\- ]{7,20}$'),
    nota                varchar(300),
    total               numeric(12,2) not null default 0 check (total >= 0),
    estado              varchar(16)   not null default 'PENDIENTE_PAGO'
                        check (estado in ('PENDIENTE_PAGO', 'EN_VERIFICACION', 'PAGADO', 'ENTREGADO',
'CANCELADO')),
    motivo_cancelacion  varchar(255),
    pagado_en           timestamptz,
    entregado_en        timestamptz,
    comprobante_enviado boolean       not null default false,
    check (entrega = 'RETIRO' or direccion is not null),
    check (estado <> 'CANCELADO' or motivo_cancelacion is not null)
);
create index if not exists ix_pedido_usuario on pedido (usuario_id, fecha desc);
create index if not exists ix_pedido_estado on pedido (estado);


create table if not exists detalle_pedido (
    id              serial primary key,
    pedido_id       integer       not null references pedido (id),
    pieza_id        integer       not null references pieza (id),
    cantidad        integer       not null check (cantidad between 1 and 20),
    precio_unitario numeric(10,2) not null check (precio_unitario > 0),
    subtotal        numeric(12,2) generated always as (cantidad * precio_unitario) stored,
    unique (pedido_id, pieza_id)
);


-- Intento de pago (puede haber varios por pedido: uno rechazado y luego otro aprobado)
create table if not exists pago (
    id               serial primary key,
    pedido_id        integer       not null references pedido (id),
    metodo           varchar(10)   not null check (metodo in ('QR', 'TARJETA', 'PAYPAL')),
    monto            numeric(12,2) not null check (monto > 0),
    estado           varchar(10)   not null check (estado in ('PENDIENTE', 'APROBADO', 'RECHAZADO')),
    referencia       varchar(40)   not null,
    tarjeta_marca    varchar(20),
    tarjeta_ultimos4 char(4)       check (tarjeta_ultimos4 ~ '^[0-9]{4}$'),
    titular          varchar(80),
    cuenta_paypal    varchar(120),
    detalle          varchar(200),
    creado_en        timestamptz   not null default now(),
    verificado_por   integer       references usuario (id),
    verificado_en    timestamptz,
    check (metodo <> 'TARJETA' or tarjeta_ultimos4 is not null)
);
create index if not exists ix_pago_pedido on pago (pedido_id);
create unique index if not exists ux_pago_aprobado on pago (pedido_id) where estado = 'APROBADO';


-- El kardex de piezas registra también las ventas web
alter table movimiento_pieza add column if not exists pedido_id integer references pedido (id);
```

```
alter table movimiento_pieza alter column tipo type varchar(16);
alter table movimiento_pieza drop constraint if exists movimiento_pieza_tipo_check;
alter table movimiento_pieza add constraint movimiento_pieza_tipo_check
    check (tipo in ('PRODUCCION', 'VENTA', 'ANULACION', 'AJUSTE_MAS', 'AJUSTE_MENOS', 'VENTA_WEB',
'DEVOLUCION_WEB'));


create or replace function mitsu.tg_movimiento_pieza()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare
    v_stock integer;
    v_nombre varchar;
begin
    select stock, nombre into v_stock, v_nombre from pieza where id = new.pieza_id for update;
    if new.tipo in ('PRODUCCION', 'ANULACION', 'AJUSTE_MAS', 'DEVOLUCION_WEB') then
        v_stock := v_stock + new.cantidad;
    else
        v_stock := v_stock - new.cantidad;
        if v_stock < 0 then
            raise exception 'Stock insuficiente de "%": quedan % unidades.', v_nombre, v_stock + new.cantidad;
        end if;
    end if;
    update pieza set stock = v_stock where id = new.pieza_id;
    new.stock_resultante := v_stock;
    return new;
end $$;


-- ---------------------------------------------------------------------
-- Triggers del pedido: precio copiado, total calculado y estados válidos
-- ---------------------------------------------------------------------
create or replace function mitsu.tg_detalle_pedido_precio()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare p pieza%rowtype;
begin
    select * into p from pieza where id = new.pieza_id;
    if p.estado <> 'ACTIVO' then
        raise exception 'La pieza "%" ya no está a la venta.', p.nombre;
    end if;
    new.precio_unitario := p.precio;
    return new;
end $$;


drop trigger if exists tg_detalle_pedido_precio on detalle_pedido;
create trigger tg_detalle_pedido_precio before insert on detalle_pedido
    for each row execute function tg_detalle_pedido_precio();


create or replace function mitsu.tg_total_pedido()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
    update pedido set total = (select coalesce(sum(subtotal), 0) from detalle_pedido where pedido_id =
new.pedido_id)
     where id = new.pedido_id;
    return null;
end $$;


drop trigger if exists tg_total_pedido on detalle_pedido;
create trigger tg_total_pedido after insert on detalle_pedido
    for each row execute function tg_total_pedido();


-- Ciclo de vida: PENDIENTE_PAGO → EN_VERIFICACION → PAGADO → ENTREGADO, o CANCELADO
create or replace function mitsu.tg_pedido_estado()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
begin
```

```
    if old.estado = new.estado then
        return new;
    end if;
    if old.estado in ('ENTREGADO', 'CANCELADO') then
        raise exception 'El pedido % ya está % y no puede cambiar.', old.numero, lower(old.estado);
    end if;
    if not ((old.estado = 'PENDIENTE_PAGO'  and new.estado in ('EN_VERIFICACION', 'PAGADO', 'CANCELADO'))
         or (old.estado = 'EN_VERIFICACION' and new.estado in ('PENDIENTE_PAGO', 'PAGADO', 'CANCELADO'))
         or (old.estado = 'PAGADO'          and new.estado in ('ENTREGADO', 'CANCELADO'))) then
        raise exception 'Cambio de estado no permitido: % → %.', old.estado, new.estado;
    end if;
    if new.estado = 'PAGADO' then
        new.pagado_en := now();
    elsif new.estado = 'ENTREGADO' then
        new.entregado_en := now();
    end if;
    return new;
end $$;


drop trigger if exists tg_pedido_estado on pedido;
create trigger tg_pedido_estado before update of estado on pedido
    for each row execute function tg_pedido_estado();


-- Un pago aprobado debe cubrir exactamente el total del pedido
create or replace function mitsu.tg_pago_validar()
returns trigger language plpgsql
set search_path = mitsu, public
as $$
declare v_total numeric(12,2);
begin
    select total into v_total from pedido where id = new.pedido_id;
    if new.monto <> v_total then
        raise exception 'El monto del pago (Bs %) no coincide con el total del pedido (Bs %).', new.monto,
v_total;
    end if;
    return new;
end $$;


drop trigger if exists tg_pago_validar on pago;
create trigger tg_pago_validar before insert on pago
    for each row execute function tg_pago_validar();


-- ---------------------------------------------------------------------
-- PA8 · Crear pedido web: reserva el stock (VENTA_WEB) por 30 minutos
-- ---------------------------------------------------------------------
create or replace function mitsu.fn_crear_pedido(
    p_usuario integer, p_cliente integer, p_entrega text, p_direccion text, p_telefono text, p_nota text,
p_items jsonb)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v_pedido integer;
    v_numero varchar;
    v_item   record;
    v_pieza  pieza%rowtype;
begin
    if p_items is null or jsonb_typeof(p_items) <> 'array' or jsonb_array_length(p_items) = 0 then
        raise exception 'Tu carrito está vacío.';
    end if;
    insert into pedido (cliente_id, usuario_id, entrega, direccion, telefono, nota)
    values (p_cliente, p_usuario, p_entrega, nullif(trim(coalesce(p_direccion, '')), ''), p_telefono,
nullif(trim(coalesce(p_nota, '')), ''))
    returning id, numero into v_pedido, v_numero;


    for v_item in
        select (e->>'pieza_id')::integer as pieza_id, sum((e->>'cantidad')::integer) as cantidad
          from jsonb_array_elements(p_items) e group by 1 order by 1
    loop
```

```
        select * into v_pieza from pieza where id = v_item.pieza_id for update;
        if not found then
            raise exception 'Una de las piezas no existe.';
        end if;
        if v_pieza.stock < v_item.cantidad then
            raise exception 'Solo quedan % unidad(es) de "%".', v_pieza.stock, v_pieza.nombre;
        end if;
        insert into detalle_pedido (pedido_id, pieza_id, cantidad, precio_unitario) values (v_pedido,
v_pieza.id, v_item.cantidad, v_pieza.precio);
        insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, pedido_id, usuario_id)
        values (v_pieza.id, 'VENTA_WEB', v_item.cantidad, 'Pedido web ' || v_numero, v_pedido, p_usuario);
    end loop;
    return v_pedido;
end $$;


-- ---------------------------------------------------------------------
-- PA9 · Registrar un intento de pago (CU21)
--   TARJETA / PAYPAL aprobado → pedido PAGADO
--   QR pendiente              → pedido EN_VERIFICACION (lo confirma el taller)
--   rechazado                 → el pedido sigue esperando otro intento
-- ---------------------------------------------------------------------
create or replace function mitsu.fn_registrar_pago(
    p_pedido integer, p_metodo text, p_estado text, p_referencia text,
    p_marca text, p_ultimos4 text, p_titular text, p_cuenta text, p_detalle text)
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v pedido%rowtype;
    v_pago integer;
begin
    select * into v from pedido where id = p_pedido for update;
    if not found then
        raise exception 'El pedido no existe.';
    end if;
    if v.estado <> 'PENDIENTE_PAGO' then
        raise exception 'El pedido % no está esperando pago (estado: %).', v.numero, lower(replace(v.estado,
'_', ' '));
    end if;
    if v.expira_en < now() then
        raise exception 'El tiempo para pagar el pedido % venció. Vuelve a armar tu carrito.', v.numero;
    end if;
    insert into pago (pedido_id, metodo, monto, estado, referencia, tarjeta_marca, tarjeta_ultimos4, titular,
cuenta_paypal, detalle)
    values (p_pedido, p_metodo, v.total, p_estado, p_referencia, p_marca, p_ultimos4, p_titular, p_cuenta,
p_detalle)
    returning id into v_pago;


    if p_estado = 'APROBADO' then
        update pedido set estado = 'PAGADO' where id = p_pedido;
    elsif p_estado = 'PENDIENTE' then
        update pedido set estado = 'EN_VERIFICACION' where id = p_pedido;
    end if;
    return v_pago;
end $$;


-- ---------------------------------------------------------------------
-- PA10 · El taller confirma o rechaza un pago por QR (CU22)
-- ---------------------------------------------------------------------
create or replace function mitsu.fn_verificar_pago_qr(p_pago integer, p_usuario integer, p_aprobar boolean,
p_detalle text)
returns void
language plpgsql
set search_path = mitsu, public
as $$
declare
    pg pago%rowtype;
begin
    select * into pg from pago where id = p_pago for update;
    if not found or pg.metodo <> 'QR' or pg.estado <> 'PENDIENTE' then
```

```
        raise exception 'Ese pago no está pendiente de verificación.';
    end if;
    perform 1 from pedido where id = pg.pedido_id for update;
    update pago set estado = case when p_aprobar then 'APROBADO' else 'RECHAZADO' end,
                    verificado_por = p_usuario, verificado_en = now(),
                    detalle = coalesce(nullif(trim(coalesce(p_detalle, '')), ''), detalle)
     where id = p_pago;
    update pedido set estado = case when p_aprobar then 'PAGADO' else 'PENDIENTE_PAGO' end,
                      expira_en = case when p_aprobar then expira_en else now() + interval '30 minutes' end
     where id = pg.pedido_id;
end $$;


-- ---------------------------------------------------------------------
-- PA11 · Cancelar un pedido: devuelve el stock reservado
-- ---------------------------------------------------------------------
create or replace function mitsu.fn_cancelar_pedido(p_pedido integer, p_usuario integer, p_motivo text)
returns void
language plpgsql
set search_path = mitsu, public
as $$
declare
    v pedido%rowtype;
begin
    select * into v from pedido where id = p_pedido for update;
    if not found then
        raise exception 'El pedido no existe.';
    end if;
    if v.estado in ('ENTREGADO', 'CANCELADO') then
        raise exception 'El pedido % ya está % y no se puede cancelar.', v.numero, lower(v.estado);
    end if;
    if length(trim(coalesce(p_motivo, ''))) < 5 then
        raise exception 'Escribe el motivo de la cancelación (mínimo 5 caracteres).';
    end if;
    insert into movimiento_pieza (pieza_id, tipo, cantidad, motivo, pedido_id, usuario_id)
    select pieza_id, 'DEVOLUCION_WEB', cantidad, 'Cancelación de ' || v.numero, p_pedido, p_usuario
      from detalle_pedido where pedido_id = p_pedido order by pieza_id;
    update pago set estado = 'RECHAZADO', detalle = 'Pedido cancelado' where pedido_id = p_pedido and estado =
'PENDIENTE';
    update pedido set estado = 'CANCELADO', motivo_cancelacion = trim(p_motivo) where id = p_pedido;
end $$;


-- ---------------------------------------------------------------------
-- PA12 · Libera el stock de los pedidos que no se pagaron a tiempo
-- ---------------------------------------------------------------------
create or replace function mitsu.fn_vencer_pedidos()
returns integer
language plpgsql
set search_path = mitsu, public
as $$
declare
    v record;
    n integer := 0;
begin
    for v in select id, usuario_id from pedido where estado = 'PENDIENTE_PAGO' and expira_en < now() for
update skip locked loop
        perform fn_cancelar_pedido(v.id, v.usuario_id, 'No se pagó dentro de los 30 minutos');
        n := n + 1;
    end loop;
    return n;
end $$;


-- ---------------------------------------------------------------------
-- Permisos y RLS de las tablas nuevas para el usuario de la aplicación
-- ---------------------------------------------------------------------
grant select, insert, update on pedido, detalle_pedido, pago to mitsu_app;
grant usage, select on all sequences in schema mitsu to mitsu_app;
grant execute on all functions in schema mitsu to mitsu_app;
do $$
declare t text;
begin
    foreach t in array array['pedido', 'detalle_pedido', 'pago'] loop
```

```
        execute format('alter table mitsu.%I enable row level security', t);
        execute format('drop policy if exists app_acceso on mitsu.%I', t);
        execute format('create policy app_acceso on mitsu.%I for all to mitsu_app using (true) with check
(true)', t);
    end loop;
end $$;


-- ---------------------------------------------------------------------
-- Datos de demostración: una clienta con cuenta y dos pedidos web
--   cliente@mitsu.test / Cliente2026!   (contraseña inventada)
-- ---------------------------------------------------------------------
do $$
declare
    v_usuario integer;
    v_cliente integer;
    v_pedido  integer;
begin
    if exists (select 1 from usuario where correo = 'cliente@mitsu.test') then
        return;
    end if;
    insert into usuario (rol_id, nombre, apellido, correo, telefono, clave_hash)
    values ((select id from rol where nombre = 'CLIENTE'), 'Daniela', 'Rivero', 'cliente@mitsu.test',
'72040001',
            crypt('Cliente2026!', gen_salt('bf', 10)))
    returning id into v_usuario;
    insert into cliente (nombre, ci_nit, telefono, correo, usuario_id)
    values ('Daniela Rivero', '7012345', '72040001', 'cliente@mitsu.test', v_usuario)
    returning id into v_cliente;


    -- Pedido pagado con tarjeta (listo para retirar)
    v_pedido := fn_crear_pedido(v_usuario, v_cliente, 'RETIRO', null, '72040001', 'Para regalo, por favor',
                                '[{"pieza_id": 1, "cantidad": 1}]'::jsonb);
    perform fn_registrar_pago(v_pedido, 'TARJETA', 'APROBADO', 'AUT-' || lpad((v_pedido * 7919 %
1000000)::text, 6, '0'),
                              'Visa', '4242', 'DANIELA RIVERO', null, 'Aprobado (modo demostración)');


    -- Pedido pagado por QR, esperando que el taller confirme el depósito
    v_pedido := fn_crear_pedido(v_usuario, v_cliente, 'ENVIO', 'Av. Banzer, 4to anillo, calle 3 #120',
'72040001', null,
                                '[{"pieza_id": 2, "cantidad": 2}]'::jsonb);
    perform fn_registrar_pago(v_pedido, 'QR', 'PENDIENTE', 'QR-' || lpad((v_pedido * 104729 % 1000000)::text,
6, '0'),
                              null, null, null, null, 'El cliente indicó que ya pagó con QR');
end $$;
```

Anexo F. Enlaces del proyecto

Los siguientes códigos QR permiten abrir el sistema publicado y el repositorio desde un celular. Basta con escanearlos con la cámara.

Sistema en línea

https://taller-mitsu-ceramica.vercel.app

Abre la tienda pública del taller. Desde «Ingresar» se accede al panel con las cuentas del Anexo B.

Repositorio en GitHub

https://github.com/Kato-001/taller-mitsu-ceramica

Código fuente, scripts SQL, diagramas, pruebas y este informe, con la estructura de carpetas del punto 6.2.

Todas las contraseñas de las cuentas de prueba son inventadas para la demostración; las claves reales de la base de datos están en variables de entorno y no aparecen en el repositorio.